O'REILLY®

华章IT

Quarkus 实战

专为Kubernetes而优化的Java解决方案

Quarkus Cookbook

[西班牙] Alex Soto Bueno
[美] Jason Porter 著
张晓宇 刘岩 宋净超 译

机械工业出版社
China Machine Press

O'Reilly精品图书系列

Quarkus实战：专为Kubernetes而优化的Java解决方案

Quarkus Cookbook: Kubernetes-Optimized Java Solutions

（西）亚历克斯·索托·布埃诺（Alex Soto Bueno）　（美）詹森·波特（Jason Porter）　著

张晓宇　刘岩　宋净超　译

ISBN：978-7-111-67763-5

本书纸版由机械工业出版社于2021年出版，电子版由华章分社（北京华章图文信息有限公司，北京奥维博世图书发行有限公司）在中华人民共和国境内（不包括中国香港、澳门特别行政区及中国台湾地区）制作与发行。

版权所有，侵权必究

客服热线：+ 86-10-68995265

客服信箱：service@bbbvip.com

官方网址：www.hzmedia.com.cn

新浪微博 @华章数媒

微信公众号 华章电子书（微信号：hzebook）

O’Reilly Media, Inc.介绍

O’Reilly以“分享创新知识，改变世界”为己任。40多年来我们一直向企业、个人提供成功所需之技能及思想，激励他们创新并做得更好。

O’Reilly业务的核心是独特的专家及创新者网络，众多专家及创新者通过我们分享知识。我们的在线学习（Online Learning）平台提供独家的直播培训、图书及视频，使客户更容易获取业务成功所需的专业知识。几十年来O’Reilly图书一直被视为学习开创未来之技术的权威资料。我们每年举办的诸多会议是活跃的技术聚会场所，来自各领域的专业人士在此建立联系，讨论最佳实践并发现可能影响技术行业未来的新趋势。

我们的客户渴望做出推动世界前进的创新之举，我们希望能助他们一臂之力。

业界评论

“O'Reilly Radar博客有口皆碑。”

——Wired

“O'Reilly凭借一系列非凡想法（真希望当初我也想到了）建立了数百万美元的业务。”

——Business 2.0

“O'Reilly Conference是聚集关键思想领袖的绝对典范。”

——CRN

“一本O'Reilly的书就代表一个有用、有前途、需要学习的主题。”

——Irish Times

“Tim是位特立独行的商人，他不光放眼于最长远、最广阔的领域，并且切实地按照Yogi Berra的建议——如果你在路上遇到岔路口，那就走小路——去做了。回顾过去，Tim似乎每一次都选择了小路，而且有几次都是一闪即逝的机会，尽管大路也不错。”

——Linux Journal

译者序

Quarkus是一款有别于传统Java架构的新技术框架。它建立在我们熟知的技术栈上，使用了诸多成熟的技术，如JPA、JAX-RS、Eclipse Vert.x、Eclipse MicroProfile和CDI等，并将这些技术与Kubernetes紧密融合在一起。用户可以借助Kubernetes高效的调度运维能力，最大限度地节约资源。

云原生的星星之火，自开源社区的Kubernetes爆红之后，变成燎原之势，随后云原生相关的技术如雨后春笋般涌现。刘岩、宋净超和我都是云原生社区的成员，也热衷于布道各种相关技术，是这一领域的狂热爱好者。我们的共同爱好之一就是时刻关注最新出版的介绍好的国外技术或者成熟技术的优秀书籍。

在这一过程中，我们机缘巧合地发现了本书，恰好本书在国内刚刚引进，满怀热情的我们就此踏上了研究Quarkus之旅。

本书采用十分简单的问题-解决方案-讨论的结构，细致透彻地描绘了Quarkus的技术点。通过本书，用户可以借助Quarkus提高Java相关的研发效率，从而在快节奏的微服务构建和基于云的应用程序开发领域立于不败之地。

在整个翻译过程中，我们得到机械工业出版社华章公司李忠明编辑的大力帮助，在此表示衷心感谢。

最后，感谢大家有缘阅读本书，希望我们三人的绵薄之力可以帮助到崇尚云原生技术的你。在Quarkus的技术道路上，希望我们能享受同样的快乐。

张晓宇

序言

自21世纪初以来，Red Hat和之前的JBoss（后被Red Hat收购）一直位于企业级Java发展的最前沿阵营。这两家公司强调开源和社区，与其他供应商、咨询公司、个人和标准组织合作，开发围绕Java的流行框架，以应对新的技术浪潮。作为流行的编程语言之一，Java被广泛用于物联网、网络服务、医疗保健和金融服务等领域。然而，云计算的出现带来了一些令传统的Java框架甚至Java虚拟机难以适应的问题。

Red Hat是第一个通过OpenShift将Java企业版（Java EE）应用服务器与云结合起来的供应商。它的所有Java项目和产品组合都延续了这一趋势。然而，当世界转向Linux Containers和Kubernetes时，无论何种编程语言的应用开发模式都朝着不可逆的方向发展。Java一直致力于实现动态应用的开发，多年来基于Java构建的框架也利用了这方面的优势。为了确保Java仍然是云开发者的首选，我们需要改变思维方式，创造新工具和新框架。

当2018年年初开始研究Quarkus时，我们放下成见，考虑了所有可行的选择。通过与包括Red Hat OpenJDK团队在内的很多组织合作，我们看到了以GraalVM为代表的基于Java创建不可变原生镜像的机会，最

终诞生了Quarkus。在这几年里，Quarkus在Java社区掀起了一场风暴，其发展速度和受欢迎程度超出了我们的想象。我们坚信，它代表了企业级Java在Kubernetes中的未来。我们重新架构了许多流行的Java框架，从头开始编写了一些新框架，所有这些都是为了确保Quarkus成为Java开发领域卓越的Kubernetes原生框架，包括用于Serverless和Knative等新领域。

本书的作者一直积极参与Quarkus的开发且在团队中工作过相当长的时间。他们对使用Quarkus开发应用程序有独到的见解，这是目前很少有人能做到的。他们写作本书是为了用自己的经验帮助读者构建Quarkus应用程序。无论你是刚接触Quarkus还是已经熟悉了Quarkus，都会发现本书正是你所需要的。本书涵盖了很多内容，包括：基本的Quarkus应用程序构建，测试应用程序，使用Eclipse MicroProfile添加高级概念，响应式编程（这是Quarkus的核心部分），使用Spring进行构建，等等。如果你对Quarkus感兴趣，那么本书就是你增长知识、提高技能的绝佳指南。同时希望你加入Quarkus社区，与大家共同进步。

Mark Little博士，JBoss CTO，工程副总裁

前言

与传统的Java框架不同，Quarkus建立在我们已熟知的技术之上。它使用JPA、JAX-RS、Eclipse Vert.x、Eclipse MicroProfile和CDI等一些熟悉的技术，并将这些技术组合到一个紧凑、易于部署、针对OpenJDK Hotspot或GraalVM的Kubernetes优化的容器中。你可以高效地使用Kubernetes集群，在应用程序需要扩展时，最大限度地利用机器上的资源。无论你处于Kubernetes的什么阶段，都会发现它的用处，本书将为你提供掌握Quarkus所需的工具和资源。

本书目标读者

显然，我们希望每个人都能阅读本书！不过，我们的目标读者需要：

- 熟悉Java和应用程序开发。
- 了解传统的软件开发。
- 经常将服务部署到机器集群或云中。

写作原因

Quarkus是一个相对较新的框架，又处于新的领域（原生Java和GraalVM）。我们想深入研究一些互联网上找不到的例子和实践。本书中没有需要理解的大型应用，所有的例子都是自包含的，可以随时使用。我们希望本书能够作为Quarkus开发的参考！

阅读指南

本书的章节组织相对松散，基本顺序如下：

· 第1章和第2章介绍Quarkus和项目的建立。

· 第3～6章介绍Quarkus的核心部分——使用CDI和Eclipse MicroProfile构建RESTful应用程序。这些章节还将展示如何打包应用程序。

· 第7～14章将更加深入地介绍一些重要的概念，如容错、持久性、安全性，以及与其他服务的交互。你还将了解Quarkus与Kubernetes的集成。

· 第15章和第16章讲述使用Quarkus进行响应式编程（reactive programming），以及框架的一些附加功能，如模板、调度和OpenAPI。

排版约定

本书中使用以下排版约定：

斜体（*Italic*）

表示新的术语、URL、电子邮件地址、文件名和文件扩展名。

等宽字体（Constant width）

用于程序清单，以及段落中的程序元素，例如变量名、函数名、数据库、数据类型、环境变量、语句以及关键字。

等宽粗体（Constant width bold）

表示应由用户直接输入的命令或其他文本。

等宽斜体（*Constant width italic*）

表示应由用户提供的值或由上下文确定的值替换的文本。

该图示表示提示或建议。

该图示表示一般性说明。

该图示表示警告或注意。

该图示表示需要记住的要点。

示例代码

可以从https://oreil.ly/quarkus-cookbook-code下载补充材料（示例代码、练习等）。

这里的代码是为了帮助你更好地理解本书的内容。通常，可以在程序或文档中使用本书中的代码，而不需要联系O'Reilly获得许可，除非需要大段地复制代码。例如，使用本书中所提供的几个代码片段来编写一个程序不需要得到我们的许可，但销售或发布O'Reilly的示例代码则需要获得许可。引用本书的示例代码来回答问题也不需要许可，将本书中的很大一部分示例代码放到自己的产品文档中则需要获得许可。

非常欢迎读者使用本书中的代码，希望（但不强制）注明出处。注明出处的形式包含书名、作者、出版社和ISBN，例如：

Quarkus Cookbook，作者Alex Soto Bueno和Jason Porter，由O'Reilly出版，书号978-1-492-06265-3

如果读者觉得对示例代码的使用超出了上面所给出的许可范围，欢迎通过permissions@oreilly.com联系我们。

O'Reilly在线学习平台（O'Reilly Online Learning）

40多年来，O'Reilly Media致力于提供技术和商业培训、知识和卓越见解，来帮助众多公司取得成功。

我们拥有独一无二的专家和革新者组成的庞大网络，他们通过图书、文章、会议和我们的在线学习平台分享他们的知识和经验。O'Reilly的在线学习平台允许你按需访问现场培训课程、深入的学习路径、交互式编程环境，以及O'Reilly和200多家其他出版商提供的大量文本和视频资源。有关的更多信息，请访问http://oreilly.com。

如何联系我们

对于本书，如果有任何意见或疑问，请按照以下地址联系本书出版商。

美国：

O'Reilly Media，Inc.

1005 Gravenstein Highway North

Sebastopol，CA 95472

中国：

北京市西城区西直门南大街2号成铭大厦C座807室（100035）

奥莱利技术咨询（北京）有限公司

要询问技术问题或对本书提出建议，请发送电子邮件至bookquestions@oreilly.com。

本书配套网站https://oreil.ly/quarkus-cookbook上列出了勘误表、示例以及其他信息。

关于书籍、课程、会议和新闻的更多信息，请访问我们的网站http://www.oreilly.com。

我们在Facebook上的地址：http://facebook.com/oreilly

我们在Twitter上的地址：http://twitter.com/oreillymedia

我们在YouTube上的地址：http://www.youtube.com/oreillymedia

致谢

Jason Porter：在疫情隔离期间做什么？当然是写书了！感谢所有奋战在医疗一线的勇士。我要感谢Quarkus和GraalVM背后的团队提

供了一个神奇的工具和有趣的开发体验。我从事软件开发已经有20多年了，Quarkus给我带来了像刚开始学习软件开发时的乐趣。非常感谢Georgios Andrianakis和Daniel Hinojosa对本书的技术审阅，使得本书更易理解、更加实用，希望学习Quarkus的人能够喜欢。还要感谢Red Hat公司让我有机会写作本书。感谢Alex再次邀请我一起写书。最后，感谢我的5个孩子（Kaili、Emily、Zackary、Nicolas和Rebecca）和妻子Tessie。

Alex Soto Bueno：本书是在COVID-19流行期间完成的，所以首先我要感谢所有的医护人员。还要感谢Red Hat开发者团队，特别是Burr Sutter，让我有机会写作本书。能和Jason一起写书是我的荣幸。最后，感谢我的父母、妻子Jessica，还有女儿Ada和Alexandra。

第1章　Quarkus概述

Kubernetes已成为用于部署企业应用的事实平台。随着企业应用容器化和加速向Kubernetes平台迁移，原有的基于Java的应用程序在编码、部署和维护方式等方面发生了很大变化。如果不对Java应用程序做任何变更就直接容器化并运行，那么会遇到很多麻烦。由容器组成的Pod（Kubernetes中的一个术语）是Kubernetes中的基本调度单元，因此，为了避免入坑，深入理解如何正确地容器化Java应用程序是非常重要的。

Quarkus是一个集成了Kubernetes的开源云原生框架，基于Apache License 2.0发布，用于创建适配GraalVM和OpenJDK HotSpot的Kubernetes原生应用，基于Hibernate、Eclipse MicroProfile、Kubernetes、Apache Camel和Eclipse Vert.x等流行库和技术构建。

Quarkus的优点包括可以与Docker和Kubernetes轻松集成，启动速度快，驻留集大小（RSS）内存消耗低，可提高开发者的生产力。本章概述Quarkus是什么，解决什么问题，如何与Kubernetes集成，为什么它会得到开发人员的喜爱，以及最值得关注的特性。

1.1 开发者友好

Quarkus可以提高Java开发者的工作效率，让你在快节奏的微服务构建和基于云的应用程序开发领域立于不败之地。

Quarkus应用程序更易于扩容，可以更有效地使用Kubernetes集群的资源，充分利用Java开源领域几十年的工作成果。

开始Quarkus开发，不需要学习新的技术，只要熟悉依赖注入、JAX-RS、Hibernate和Eclipse MicroProfile等概念就已足够。你职业生涯中积累的所有知识都可以直接用到Quarkus中。学习其他框架可能需要数周的时间，而上手Quarkus只需要几天甚至几个小时。

Quarkus是专为下一代应用程序开发与部署而设计和优化的。它支持应用程序的全生命周期管理，从搭建应用脚手架和开发模式（save-and-refresh工作流）下的实时重载，到部署到基于云的Kubernetes集群中。Quarkus让开发者专注于解决问题而非细枝末节，从而提高生产力。

1.2 与Kubernetes集成

前面提到，Quarkus是要在Kubernetes上运行的。这听起来不错，但我们知道，很多应用都可以运行在Kubernetes上。应用只要能放到Docker容器中就能在Kubernetes上运行。即便如此，还必须对应用做很多裁剪、优化、配置才能在Kubernetes上高效运行，还需要编写多个YAML文件——实际上没人愿意写YAML。

Quarkus通过一系列增强功能，让我们在部署应用程序和使用Kubernetes时省去了大量工作。Quarkus应用启动时，附带了一些Dockerfile文件，用于生成Docker容器，这是很好的开始。这些文件经过优化，可以在OpenJDK JVM上运行，或者在GraalVM上作为原生可执行文件运行，其中包含了应用程序运行所必需的内容，以尽可能地避免出现臃肿的容器镜像。

其次，当使用Kubernetes扩展时，Quarkus可以为普通的Kubernetes或OpenShift部署生成资源（YAML文件）！不再需要编写YAML文件或检查缩进。毕竟，比起调试YAML文件中的格式错误，开发者更愿意把精力放在代码上。部署到Kubernetes集群之前，Quarkus还可以将镜像推送到镜像仓库中。所有这些应用镜像都可以通过Quarkus应用配置做进一步增强和定制，详见第4章。例如，在Quarkus 1.4和

更高版本中，ConfigMap和Secret可以从API服务器中读取——不需要挂载Pod中的任何文件！

1.3 内存和第一响应时间

Quarkus被称为“超音速、亚原子”（supersonic，subatomic）的Java框架。你可能觉得这样描述有过度营销的嫌疑，但充分了解Quarkus的作用后，你会发现Quarkus确实是一个非常小巧、快速和高效的执行框架。通过Quarkus，你可以部署一个经过优化的Kubernetes原生应用。例如，假设你想部署一个Kubernetes原生应用，容器镜像约为200 MB或更小。在Quarkus中，这个应用将在几分之一秒内启动好并准备接受请求，只占用不到50 MB的内存。

在Kubernetes集群中部署应用程序时，你希望部署尽可能多的实例，尽可能多地利用集群资源，便于扩展应用以满足意外的负载。当扩展时，你希望新应用实例能够快速启动和运行——这就是原生可执行文件的优势所在。Quarkus在原生可执行文件的构建过程中，尽可能地对应用程序和框架进行预启动。这可以使应用程序快速启动，并为服务请求做好准备，而无须像JVM那样做额外的类加载、运行时扫描或其他预热工作。

当然，内存资源是有限的。了解应用程序到底使用了多少内存，在试图保持低内存使用量的同时不使JVM内存不足，是决定部署密度的关键。Quarkus可以成功实现这一点，它的原生可执行文件体积小，内存效率高。

1.4 Quarkus基本工作流程

通过本书，你将了解Quarkus生态系统，了解扩展、集成和设计决策，还会了解整个过程中使用的基本工作流程，帮助你提高工作效率。简而言之，工作流程如下：

1. 搭建脚手架（scaffold）

2. 启动开发模式

3. 编码

4. 测试

5. 打包

6. 部署

为应用程序搭建脚手架，或将一个扩展添加到现有启动，这为你构建应用程序打下了坚实的基础，你将在第2章中熟悉这一点。搭建完脚手架之后，在开发模式下运行应用程序，这也将在第2章中介绍。然后，你将了解应用程序的典型任务：创建RESTful服务，完成基本编程模型，以及执行应用程序配置。开发模式将给你提供近乎实时的反馈，而不需要你熟悉编译、打包和部署这些烦琐的流程。在第5章中，

你将看到如何测试针对JVM和原生可执行文件的Quarkus应用程序，确保应用程序符合预期标准并正确运行。第6章将介绍如何创建最终可交付文件，以及如何为特定部署策略打包应用程序。该工作流程的最后一块（即部署）将在第10章中介绍。之后，你将学习如何使应用程序更具容错性，如何与各种持久化引擎交互，以及如何与外部服务通信。我们还将解释额外的集成，以帮助你利用其他库和编程范式的现有知识。我们将指导你设置应用程序所需的Kubernetes优化，构建Kubernetes资源，并将其全部推送上线。

第2章　搭建脚手架

在本章中，你将学习如何创建Quarkus的项目结构。Quarkus自带了一些不同的方法来搭建项目的脚手架。

本章涵盖以下主题：

- 从Maven到VSCode IDE，用不同的方式来构建项目的脚手架
- 通过实时重载改善开发者的体验
- 使用Quarkus提供静态资源

2.1 用Maven搭建Quarkus项目的脚手架

问题

如何生成一个简单的项目来快速启动Quarkus？

解决方案

使用Quarkus Maven插件。

讨论

使用Quarkus Maven插件可以创建一个简单的项目，该项目可以随时部署，包含以下内容：

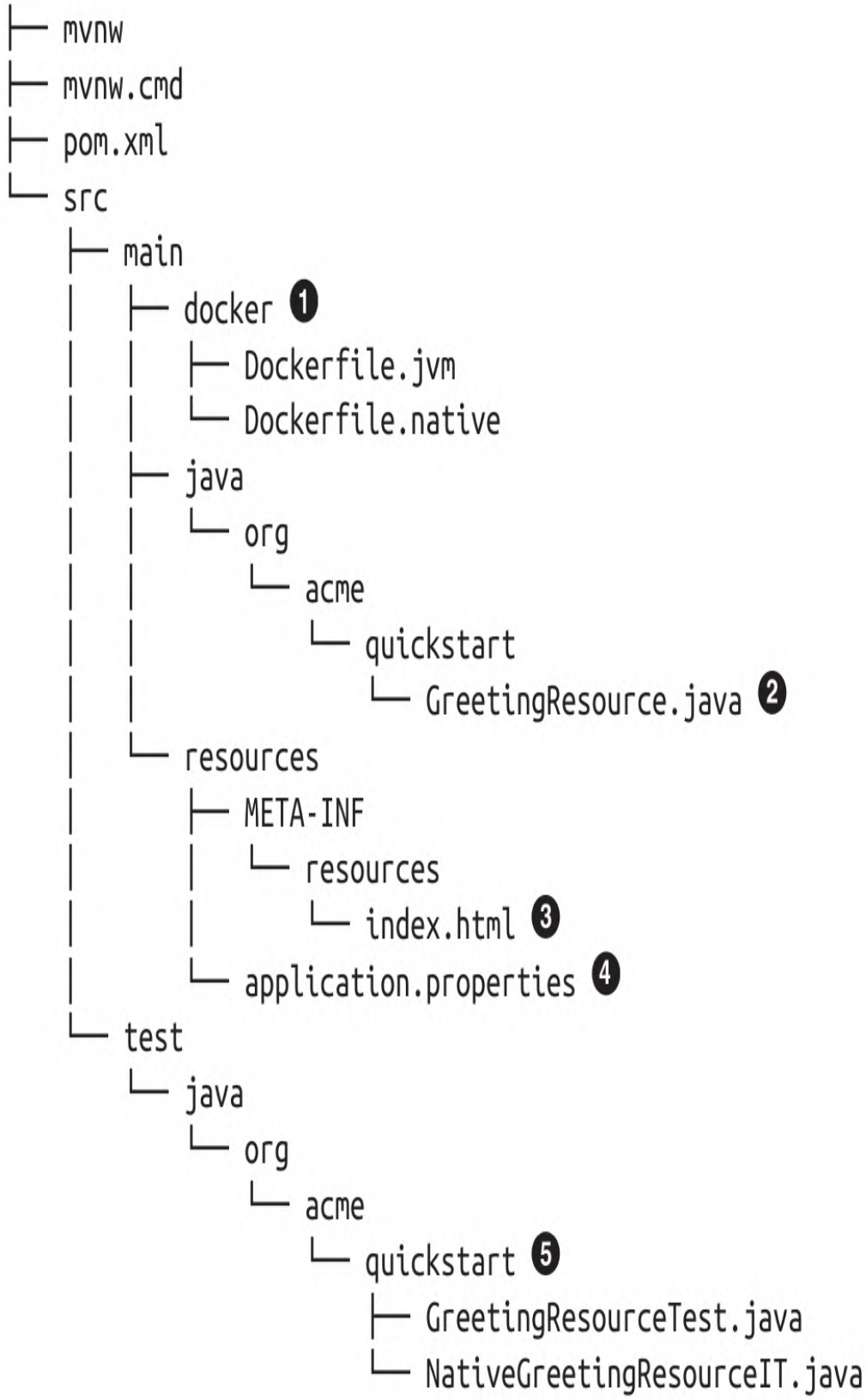
- 一个包含最小Quarkus依赖的pom.xml文件
- 一个简单的JAX-RS资源
- 一个JAX-RS资源的测试
- 原生测试
- 用于生成容器的Dockerfile

· 一个空的配置文件

我们假设你已经安装了Apache Maven（http://maven.apache.org）。打开终端，执行以下命令：

```
mvn io.quarkus:quarkus-maven-plugin:1.4.1.Final:create \
    -DprojectGroupId=org.acme \
    -DprojectArtifactId=getting-started \
    -DclassName="org.acme.quickstart.GreetingResource" \
    -Dpath="/hello"
```

该项目有如下结构：


mvnw
mvnw.cmd
pom.xml
src
main
docker ❶
Dockerfile.jvm
Dockerfile.native
java
org
acme
quickstart
GreetingResource.java ❷
resources
META-INF
resources
index.html ❸
application.properties ❹
test
java
org
acme
quickstart ❺
GreetingResourceTest.java
NativeGreetingResourceIT.java

❶Dockerfile

❷JAX-RS资源

❸静态资源

❹配置文件

❻为JAX-RS资源自动生成的测试

2.2 用Gradle搭建Quarkus项目的脚手架

问题

对于Gradle用户，如何生成一个简单的项目来快速启动Quarkus？

解决方案

使用Quarkus Maven插件。

讨论

你可以使用Quarkus Maven插件来构建一个简单的Quarkus项目。你只需将输出设置为Gradle项目，由此产生的项目就可以部署了，它包含以下内容：

- 一个包含最小Quarkus依赖的build.gradle文件
- 一个简单的JAX-RS资源
- 一个JAX-RS资源的测试
- 原生测试

· 用于生成容器的Dockerfile

· 一个空的配置文件

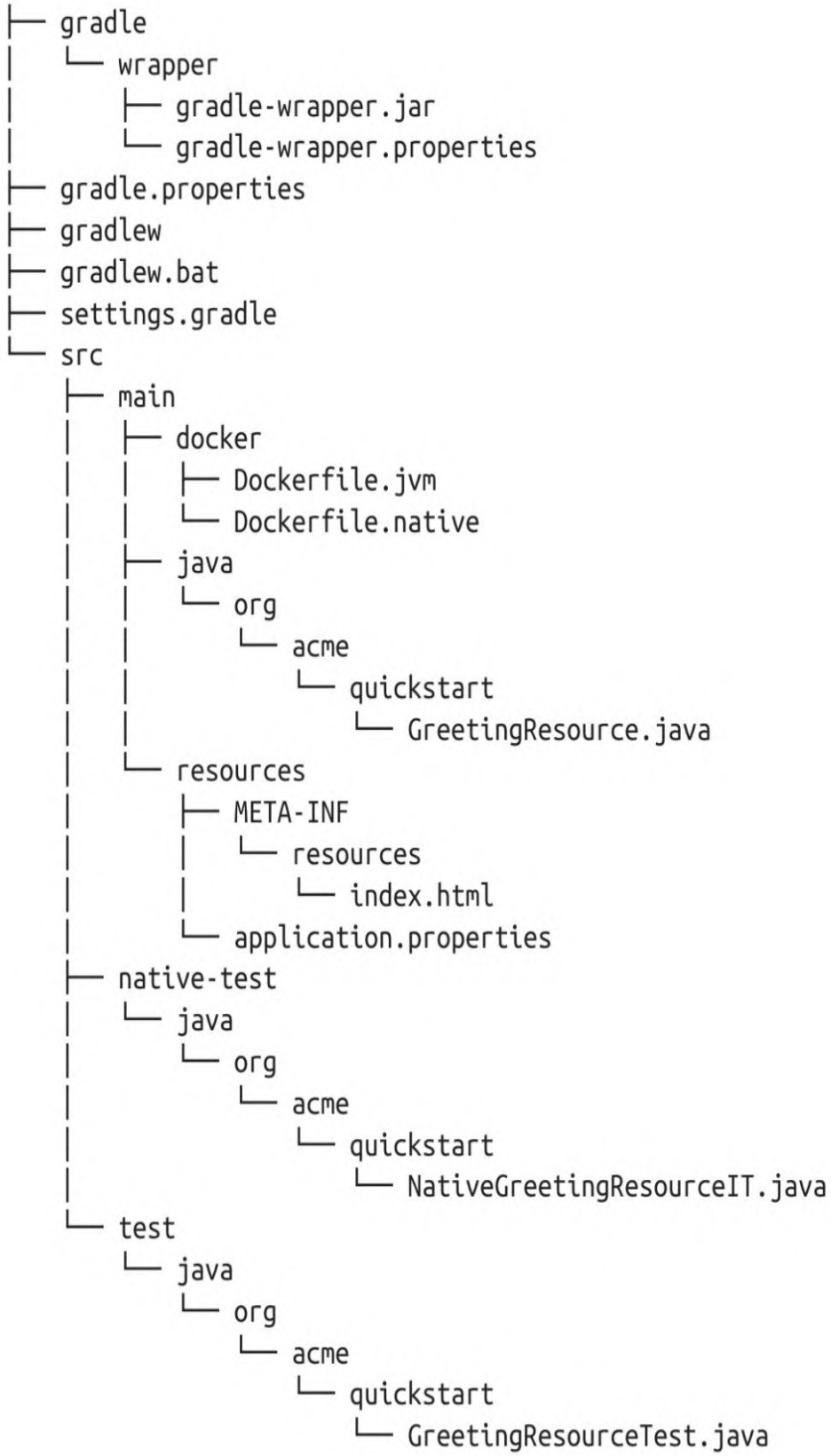
我们假设你已经安装了Apache Maven（http://maven.apache.org）。打开终端，执行以下命令：

```
mvn io.quarkus:quarkus-maven-plugin:1.4.1.Final:create \
    -DprojectGroupId=org.acme \
    -DprojectArtifactId=getting-started \
    -DclassName="org.acme.quickstart.GreetingResource" \
    -Dpath="/hello" \
    -DbuildTool=gradle
```

与Apache Maven不同的是，该命令将在current目录下创建结构。

产生的项目有如下结构：

```
.
├── README.md
├── build.gradle
```


gradle
wrapper
gradle-wrapper.jar
gradle-wrapper.properties
gradle.properties
gradlew
gradlew.bat
settings.gradle
src
main
docker
Dockerfile.jvm
Dockerfile.native
java
org
acme
quickstart
GreetingResource.java
resources
META-INF
resources
index.html
application.properties
native-test
java
org
acme
quickstart
NativeGreetingResourceIT.java
test
java
org
acme
quickstart
GreetingResourceTest.java

2.3 用Quarkus Start Coding网站搭建Quarkus项目的脚手架

问题

如何在Quarkus中生成一个简单的项目来快速启动Quarkus，而无须安装Maven或Gradle？

解决方案

使用Quarkus Start Coding网站，访问https://code.quarkus.io来生成一个简单的Quarkus项目。

讨论

写作本书时，主页如图2-1所示。

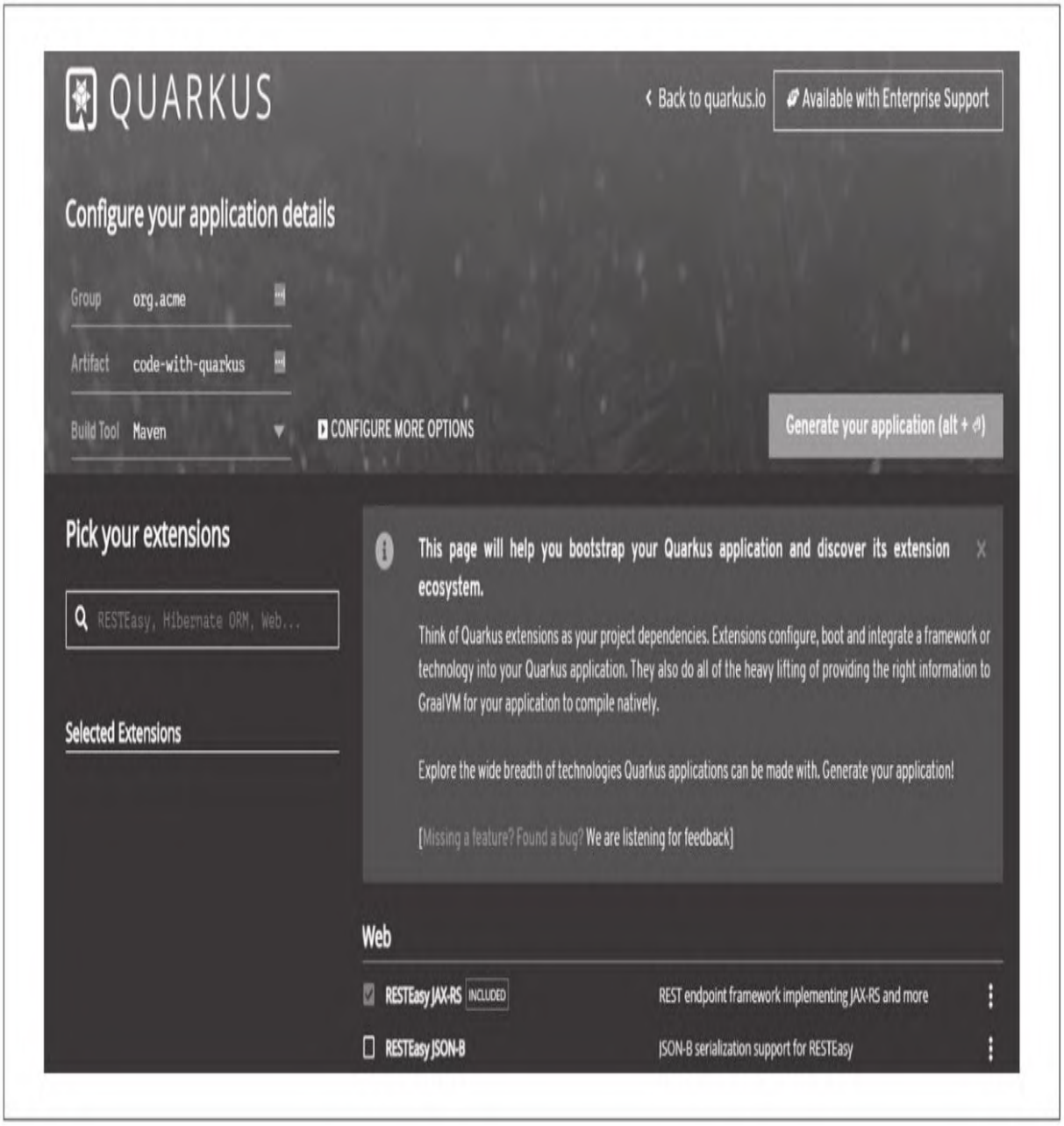


图2-1： https://code.quarkus.io主页

页面加载完毕后，按下Generate your application按钮，下载一个ZIP文件，里面有生成的项目。

打开终端并解压生成的项目：

```
unzip code-with-quarkus.zip
cd code-with-quarkus/
```

已搭建脚手架的项目与你在2.1节中生成的项目相同，带有以下元素：

- 一个包含最小Quarkus依赖的pom.xml文件
- 一个简单的JAX-RS资源
- 一个JAX-RS资源的测试
- 原生测试
- 用于生成容器的Dockerfile
- 一个空的配置文件

另见

我们还没有讨论过Quarkus扩展，但请注意，你可以使用任何已注册的Quarkus扩展生成一个项目。你将在接下来的章节中看到关于扩展的更多内容。

可以在页面中添加扩展，可以在每个扩展的复选框中选择具体的扩展或者使用搜索框来添加。

2.4 用Visual Studio Code搭建Quarkus项目的脚手架

问题

如何在Visual Studio (VS) Code中生成一个简单的项目来快速启动Quarkus？

解决方案

使用Quarkus VS Code扩展。

讨论

我们为Visual Studio Code IDE开发了一个Quarkus扩展，将Quarkus的一些功能集成到IDE中。以下是其中的一些功能：

- 搭建项目脚手架的命令
- 添加扩展的命令
- 自动完成配置文件（属性和YAML格式）Snippet

要安装插件，需打开VS Code，按Extensions按钮，如图2-2所示。

图2-2：单击VS Code中的扩展按钮，可以安装Quarkus扩展

搜索quarkus并选择Quarkus Tools for Visual Studio Code by Red Hat。你的搜索应该会弹出类似于图2-3的结果。

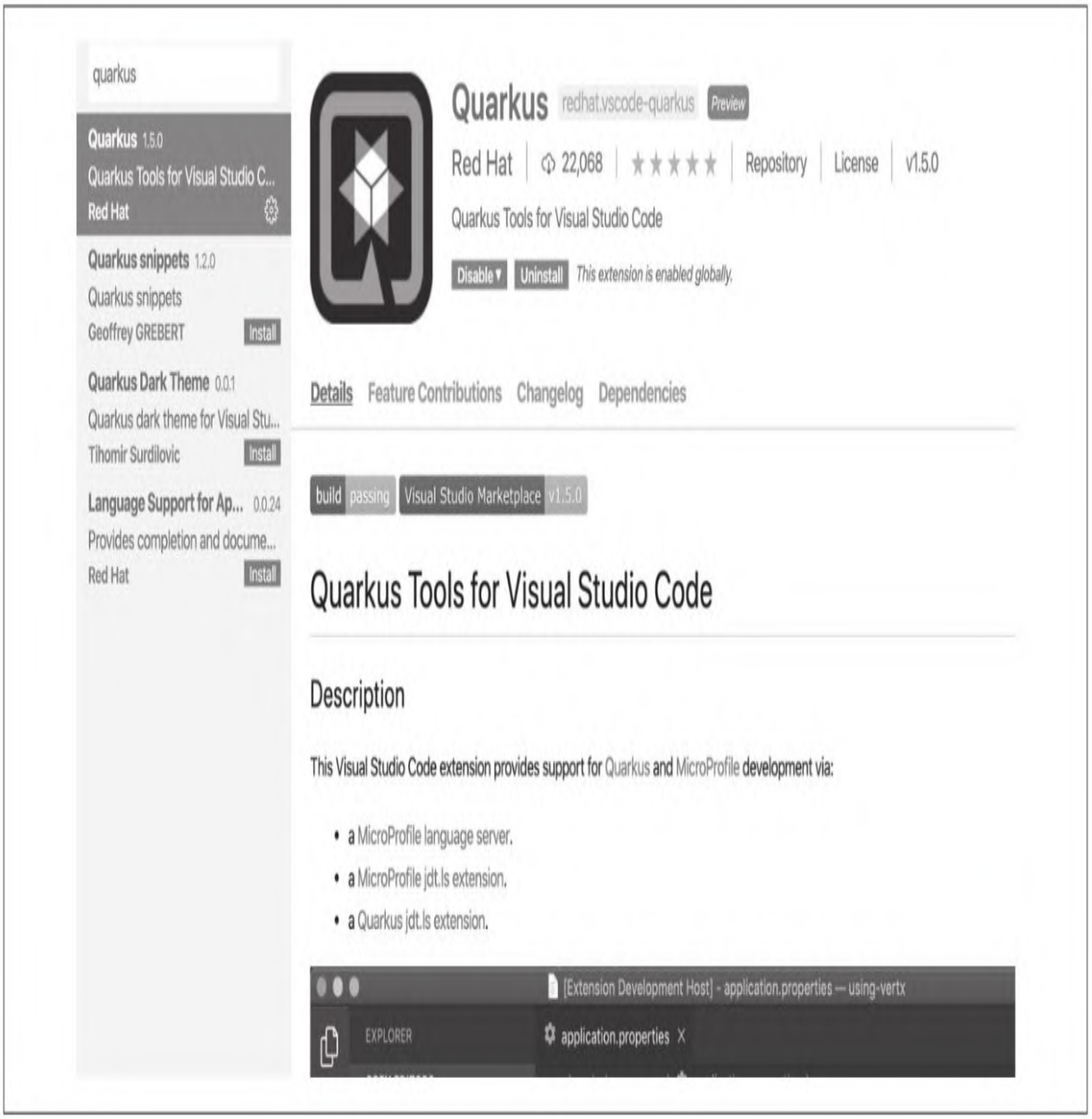


图2-3：Quarkus扩展可从VS Code市场免费获得

安装过程结束后，重新启动IDE，就可以开始使用扩展程序了。

要生成新的Quarkus项目，需打开Command Palette，选择Generate a Quarkus Project。在撰写本书时，可用的选项如图2-4所

示。

图2-4：使用Command Palette生成一个新的Quarkus项目

如图2-5所示，下一步会询问你使用哪个构建工具。还有一些关于groupId、artifactId等的问题。

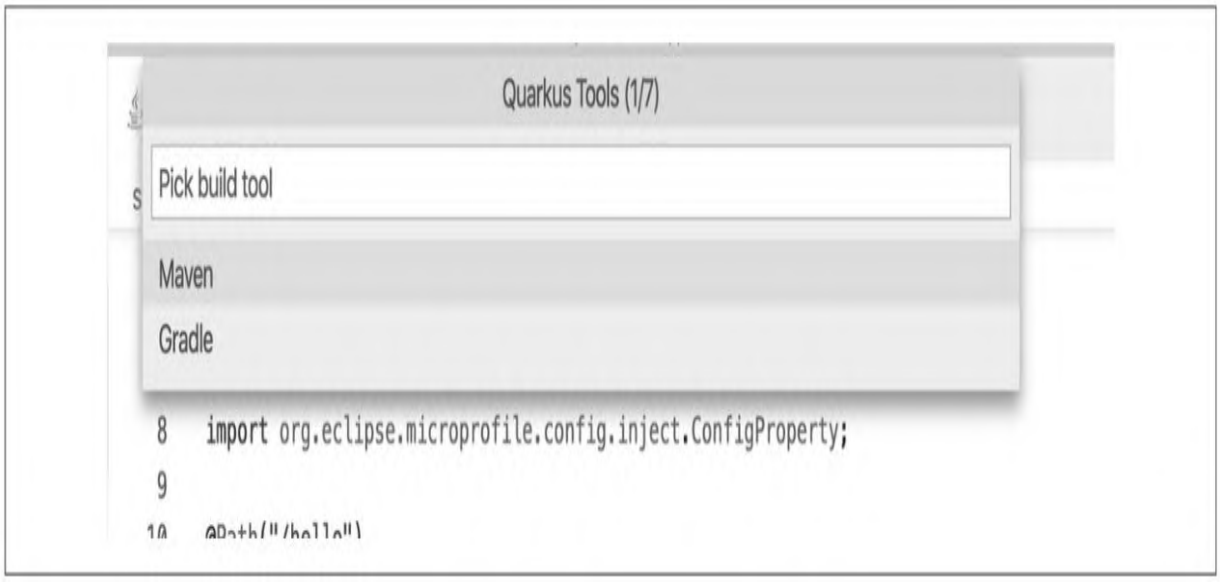


图2-5：选择构建工具

可以通过控制台中的Add extension命令来添加扩展，如图2-6所示。

图2-6：Add extension命令

选择任何你想添加到项目中的Quarkus扩展。图2-7是可用扩展的例子。

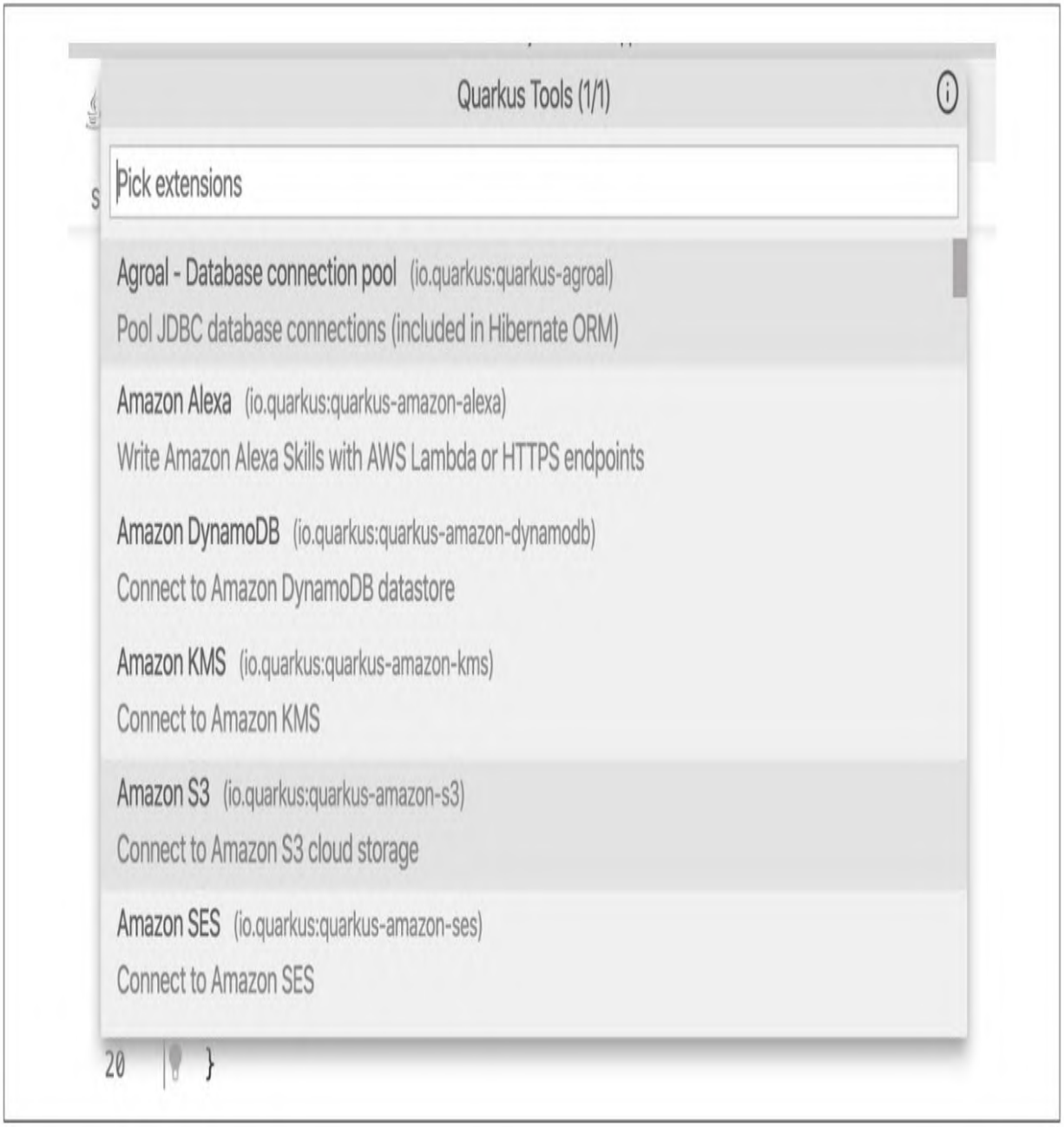


图2-7：应用程序可以安装的可用扩展列表

在图2-8中，你可以看到Quarkus扩展所提供的一些功能。图2-8还显示了自动完成配置属性，帮助你正确配置应用程序。

图2-8：配置属性键的自动完成和类型信息

图2-9显示了配置属性的具体描述。

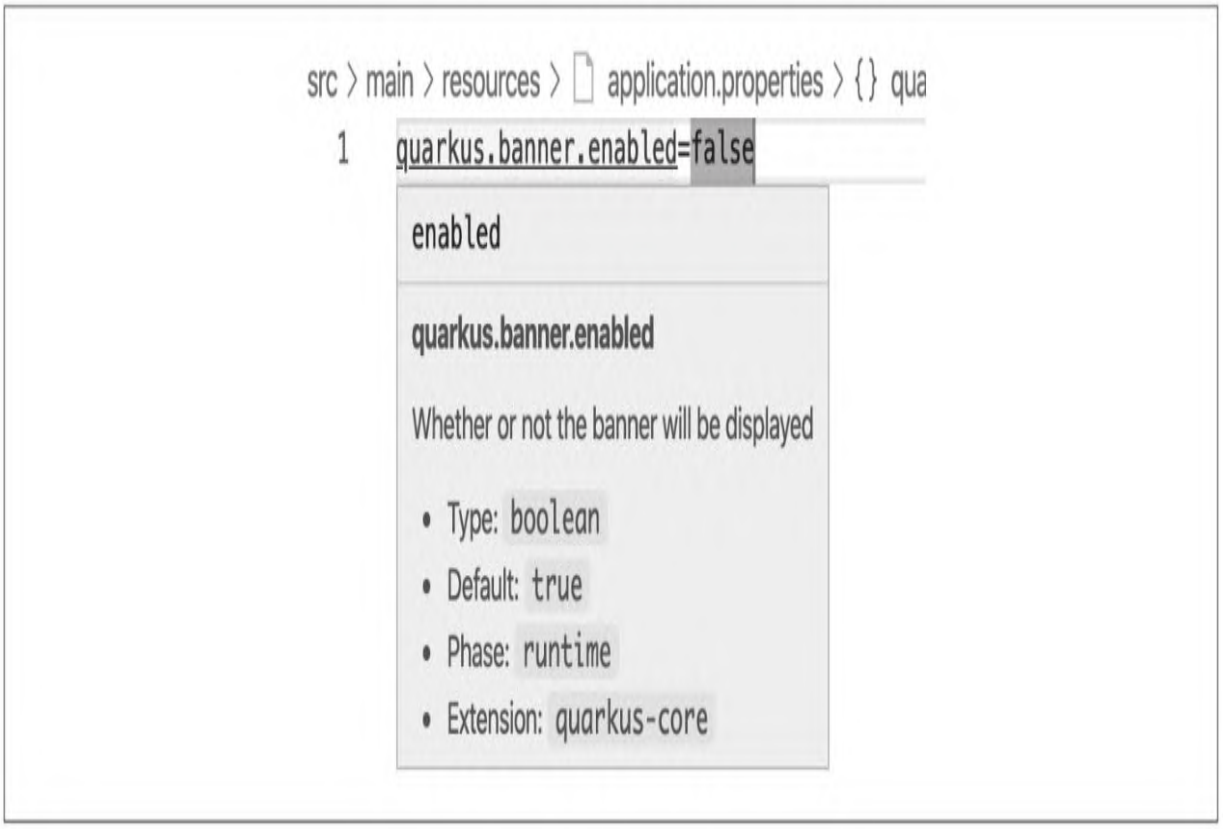


图2-9：将鼠标悬停在配置属性上，可获得有关该属性的更多信息

图2-10演示了如何检测应用程序中未使用的属性。

图2-10：检测未使用的配置属性

图2-11显示了代表端点的URL。单击它会打开一个新的浏览器窗口，指向给定的URL。

图2-11：VS Code扩展为每个端点方法生成一个可点击的URL端点

另见

Quarkus扩展可用于多种IDE：

- Visual Studio Code（https://oreil.ly/rL1Md）
- Eclipse（https://oreil.ly/3Ais）
- IntelliJ（https://oreil.ly/Whvdj）

2.5　使用开发模式实时重载

问题

如何检测项目中的更改结果，而不需要重新打包和重新部署完整的应用程序？

解决方案

使用开发模式，可以通过后台编译实现热部署，这意味着当你修改你的Java文件或资源文件并刷新浏览器时，这些更改将自动生效。

每个搭建脚手架的方法都默认为Java 11。你需要一个Java 11虚拟机，或者修改项目以使用不同版本的JVM。

讨论

要在开发模式下启动应用程序，需在项目的根目录下运行quarkus:dev命令。以下的例子使用的是在2.1节中创建的项目：

```
./mvnw compile quarkus:dev

[INFO] Scanning for projects...

....
[INFO] --- quarkus-maven-plugin:1.4.1.Final:dev (default-cli) @
       getting-started ---
Listening for transport dt_socket at address: 5005
INFO  [io.qua.dep.QuarkusAugmentor] (main) Beginning quarkus augmentation
INFO  [io.qua.dep.QuarkusAugmentor] (main) Quarkus augmentation
   completed in 946ms
INFO  [io.quarkus] (main) Quarkus 1.4.1.Final started in 1.445s.
   Listening on: http://[::]:8080
INFO  [io.quarkus] (main) Installed features: [cdi, resteasy]
```

当应用程序启动并运行后，打开一个新的终端窗口并运行以下命令：

```
curl http://localhost:8080/hello

hello
```

现在在不停止quarkus:dev模式的情况下，在org.acme.quickstart.GreetingResource.java中做如下修改。

```
@GET
@Produces(MediaType.TEXT_PLAIN)
public String hello() {
    return "hola";
}
```

然后运行以下命令：

```
curl http://localhost:8080/hello

hola
```

需要注意，你已经对源代码做了一个更改，而且不需要再重新编译、重新打包或重新部署，这个更改已经被自动填充到正在运行的实例中——不需要任何特殊的设置。

现在，你已将编码→编译→部署→刷新→重复这一开发工作流程简化为编码→刷新→重复。

开发模式会检测Java文件、应用程序配置和静态资源的更改。

要停止开发模式，需进入quarkus:dev端点，按Ctrl+C。

要在Gradle项目中运行开发模式，可以运行一个quarkusDev任务：

```
./gradlew quarkusDev
...
```

2.6　提供静态资源

问题

如何提供HTML、JavaScript、CSS和图像等静态资源？

解决方案

在Quarkus中，任何复制到*src/main/resources/META-INF/resources*中的资源都是由根目录提供的。

在某些情况下，你可能希望向调用者提供静态资源，这些资源可以是静态的可下载内容或HTML页面。

默认情况下，Quarkus有一个*index.html*文件作为静态资源。

启动应用程序：

```
./mvnw compile quarkus:dev
```

打开浏览器，输入以下网址：http://localhost:8080/index.html。

你会看到如图2-12所示的界面。

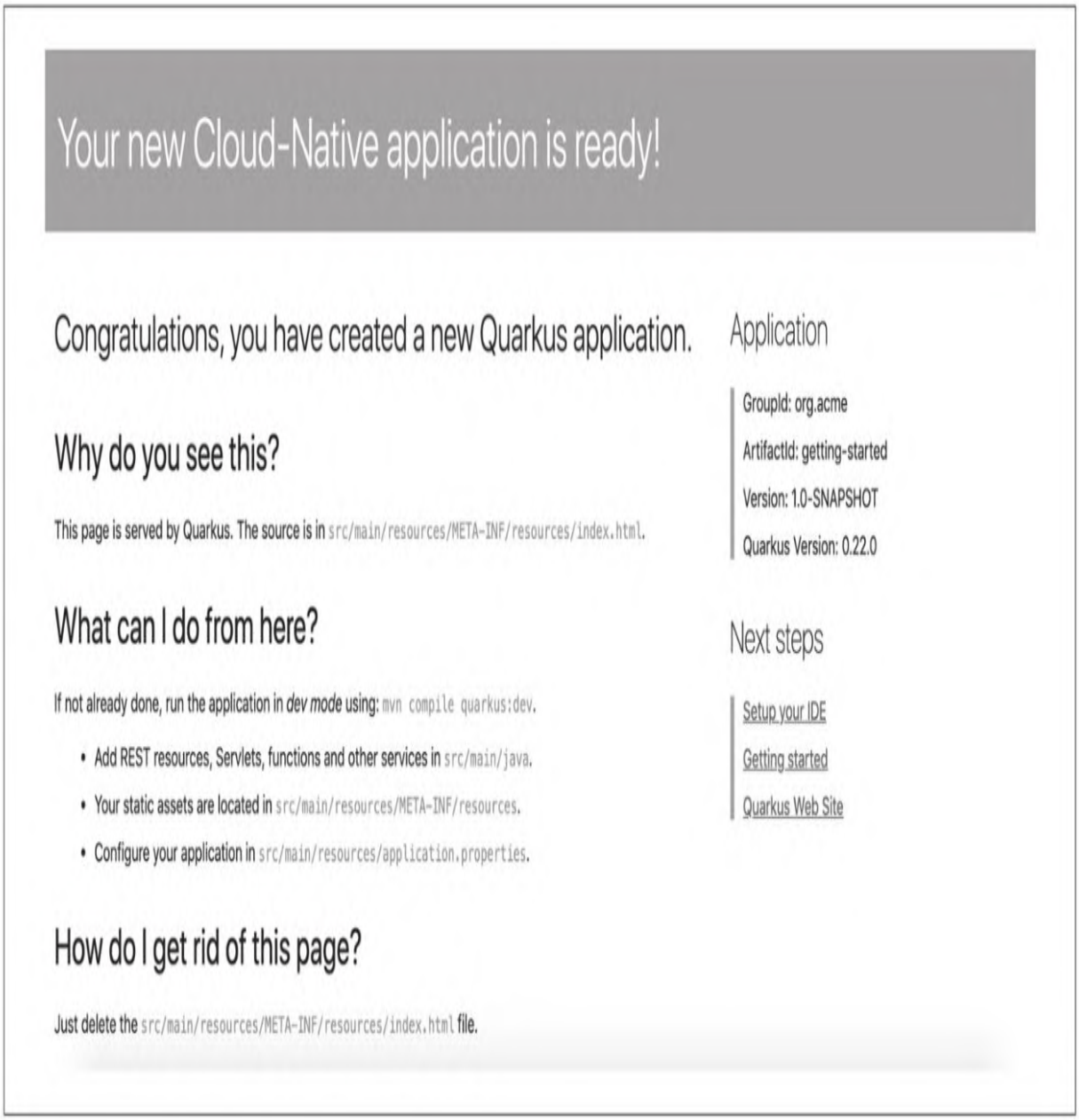


图2-12：Quarkus为你的应用程序创建了一个占位索引，其中包含基本信息和下一步，以帮助你进行创建好脚手架之后的工作

实时重载也适用于静态资源。

第3章　开发RESTful服务

Quarkus集成了RESTEasy，这是一种用于定义REST API的JAX-RS实现。在本章中，你将学习如何在Quarkus中开发RESTful Web服务。本章涵盖以下主题：

- 如何使用JAX-RS来创建CRUD服务
- 如何启用CORS向其他域请求资源
- 如何实现响应式路由
- 如何实现过滤器来操作请求和响应

3.1 创建一个简单的REST API端点

问题

如何创建一个带有CRUD操作的REST API端点？

解决方案

使用之前生成的JAX-RS GreetingResource资源，并用JAX-RS注解来填充它。

JAX-RS是Quarkus中用来定义REST端点的默认框架。所有的JAX-RS注解都已经正确地位于类路径上。你需要使用HTTP动词注解（@GET、@POST、@PUT、@DELETE）来声明端点方法将监听的HTTP动词。当然，你需要使用@Path注解来定义你的端点相对于应用程序其他部分的URI。

打开org.acme.quickstart.GreetingResource.java：

```
package org.acme.quickstart;
import javax.ws.rs.GET;
import javax.ws.rs.Path;
import javax.ws.rs.Produces;
import javax.ws.rs.core.MediaType;

@Path("/hello") ❶
public class GreetingResource {
    @GET ❷
    @Produces(MediaType.TEXT_PLAIN) ❸
    public String hello() {
        return "hello"; ❹
    }
}
```

❶确定当前资源的URI路径

❷响应HTTP GET请求

❸定义返回的媒体类型

❹返回纯文本

让我们创建其余的方法来创建、更新和删除message。

```
@POST ❶
@Consumes(MediaType.TEXT_PLAIN) ❷
public void create(String message) { ❸
    System.out.println("Create");
}

@PUT ❹
@Consumes(MediaType.TEXT_PLAIN)
@Produces(MediaType.TEXT_PLAIN)
public String update(String message) {
    System.out.println("Update");
    return message;
}

@DELETE ❺
public void delete() {
    System.out.println("Delete");
}
```

❶响应HTTP POST请求

❷定义可接受的媒体类型

❸请求的正文内容

❹响应HTTP PUT请求

❺响应HTTP DELETE请求

以下是有效的HTTP方法：@GET、@POST、@PUT、@DELETE、@PATCH、@HEAD和@OPTIONS。

3.2　提取请求参数

问题

如何使用JAX-RS提取请求参数？

解决方案

使用JAX-RS规范提供的一些内置注解。

打开org.acme.quickstart.GreetingResource.java类，并将带有请求参数的hello方法修改为如下的内容：

```
public static enum Order {
    desc, asc;
}

@GET
@Produces(MediaType.TEXT_PLAIN)
public String hello(
            @Context UriInfo uriInfo, ❶
            @QueryParam("order") Order order, ❷
            @NotBlank @HeaderParam("authorization") String authorization ❸
            ) {


    return String.format("URI: %s - Order %s - Authorization: %s",
                    uriInfo.getAbsolutePath(), order, authorization);
}
```

❶获取请求的UriInfo。UriInfo是JAX-RS的一部分，允许你获取应用和请求的URI信息

❷获取Enum类型为order的查询参数

❸获取与bean验证集成的名为authorization的头参数

打开一个新的终端窗口，启动Quarkus应用程序，并向GET方法发送一个请求：

```
./mvnw clean compile quarkus:dev

curl -X GET "http://localhost:8080/hello?order=asc" \
    -H "accept: text/plain" -H "authorization: XYZ"
URI: http://localhost:8080/hello - Order asc - Authorization: XYZ

curl -X GET "http://localhost:8080/hello?order=asc" \
    -H "accept: text/plain" -v
HTTP/1.1 400 Bad Request
```

其他请求参数可以使用注解来提取，例如表单参数（@FormParam）、矩阵参数（@MatrixParam）或cookie值（@CookieParam）。此外，使用@Context注解，你还可以注入其他与JAX-RS相关的元素，如javax.ws.rs.core.SecurityContext、javax.ws.rs.sse.SseEvent Sink或javax.ws.rs.sse.Sse。

讨论

在3.1节中，你看到了如何使用JAX-RS创建REST API端点，但是你通常需要从一个请求中提取更多的信息，而不仅仅是主体内容。

在使用Quarkus和JAX-RS时，需要重点注意，默认情况下，Quarkus使用RESTEasy直接与Vert.x一起工作，而不使用任何与Servlet规范相关的东西。

一般来说，开发REST API端点可能需要的一切都得到了很好的支持，当你需要实现自定义Servlet过滤器或者直接在代码中获取HTTP请求时，Quarkus提供了替代方案。

但是，如果需要的话，你可以通过添加quarkus-undertow扩展，从而配置Quarkus在使用Servlet规范时使用RESTEasy而不是Vert.x。

```
./mvnw quarkus:add-extension -Dextensions="quarkus-undertow"

./gradlew addExtension --extensions="quarkus-undertow"
```

另见

要了解有关JAX-RS的更多信息，请访问以下网站。

・Eclipse基金会：Jakarta RESTful Web Services（https://oreil.ly/Tgn5d）

・RESTEasy（https://oreil.ly/WpJ3x）

3.3　使用HTTP响应状态码

问题

如何使用HTTP响应状态码来正确反映请求的结果？

解决方案

JAX-RS规范使用javax.ws.rs.core.Response接口来返回正确的HTTP响应状态码，并设置任何其他所需的信息，如响应内容、cookie或header。

```
package org.acme.quickstart;

import javax.ws.rs.Consumes;
import javax.ws.rs.POST;
import javax.ws.rs.Path;
import javax.ws.rs.Produces;
import javax.ws.rs.core.MediaType;
import javax.ws.rs.core.Response;
import javax.ws.rs.core.UriBuilder;

@Path("/developer")
public class DeveloperResource {

    @POST
    @Produces(MediaType.APPLICATION_JSON)
    @Consumes(MediaType.APPLICATION_JSON)
    public Response createDeveloper(Developer developer) {
        developer.persist();
        return Response.created( ❶
            UriBuilder
                .fromResource(DeveloperResource.class) ❷
                .path(Long.toString(developer.getId())) ❸
                .build()
            )
            .entity(developer) ❹
            .build(); ❺
    }

    public static class Developer {

        static long counter = 1;

        private long id;
        private String name;

        public long getId() {
            return id;
        }

        public void setName(String name) {
            this.name = name;
        }

        public String getName() {
            return name;
        }

        public void persist() {
            this.id = counter++;
        }
    }
}
```

❶将响应状态码设置为201，以Location头作为URI进行创建

❷设置资源类的路径

❸在Location头中设置开发者ID

❹将创建的开发者设置为响应内容

❺构建Response对象

如果要从端点返回JSON，需要在项目中使用quarkus-resteasy-jsonb或quarkus-resteasy-jackson扩展。

打开一个新的终端窗口，启动Quarkus应用程序，并向GET方法发送一个请求：

```
./mvnw clean compile quarkus:dev

curl -d '{"name":"Ada"}' -H "Content-Type: application/json" \
  -X POST http://localhost:8080/developer -v

< HTTP/1.1 201 Created
< Content-Length: 21
< Content-Type: application/json
< Location: http://localhost:8080/developer/1
<
{"id":1,"name":"Ada"}
```

请注意，Location头包含一个有效的URI来访问创建的资源。

讨论

当定义一个RESTful Web API时，遵循一些约定真的很重要，这些约定是由使用的底层技术提供的。对于RESTful Web服务来说，它是HTTP层。

定义API的另一个关键是使用正确的响应状态码，这些状态码被发回给客户端，以表明请求是否已经完成。有5类状态码：

- 信息响应（100～199）
- 成功的答复（200～299）
- 重定向（300～399）
- 客户端错误（400～499）
- 服务器错误（500～599）

默认情况下，Quarkus试图用正确的HTTP状态码提供开箱即用的响应。例如，它在违反约束的情况下提供400 Bad Request，在服务器异常的情况下提供500 Internal Server Error。但有一个用例是它默认不覆盖的：创建一个资源，在这个资源中，HTTP 201 Created状态响应码应该被发回给客户端，在消息的主体中包含新资源，在Location头中设置新资源的URL。

另见

完整的HTTP响应状态码汇总在以下网站。

- MDN Web文档：HTTP响应状态代码（https://oreil.ly/Gq02d）

3.4　绑定HTTP方法

问题

如何将方法绑定到没有JAX-RS规范提供的专用注解的HTTP动词上？

解决方案

使用javax.ws.rs.HttpMethod注解来创建HTTP方法注解。

JAX-RS规范提供了7个注解@GET、@POST、@PUT、@DELETE、@PATCH、@HEAD和@OPTIONS来指定一个方法应该响应的HTTP方法。但是HTTP方法还有很多，JAX-RS提供了javax.ws.rs.HttpMethod注解来支持它们。

首先要做的是创建一个元注解。我们将使用LOCK动词，它定义在RFC-4918（https://tools.ietf.org/html/rfc4918#section-9.10）中。LOCK动词锁定对资源的访问或刷新现有的锁。我们的注解将被命名为LOCK，它的注解是@javax.ws.rs.HttpMethod：

```
package org.acme.quickstart;

import java.lang.annotation.Documented;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

import javax.ws.rs.HttpMethod;

@Target({ElementType.METHOD})
@Retention(RetentionPolicy.RUNTIME)
@HttpMethod("LOCK") ❶
@Documented
public @interface LOCK {
}
```

❶将LOCK HTTP方法绑定到注解上

最后，在一个资源方法中使用这个注解将其与LOCK HTTP动词绑定。

打开org.acme.quickstart.GreetingResource.java类，创建一个LOCK方法。

```
@LOCK ❶
@Produces(MediaType.TEXT_PLAIN)
@Path("{id}")
public String lockResource(@PathParam("id") long id) {
    return id + " locked";
}
```

❶绑定到LOCK HTTP方法

打开一个新的终端窗口，启动Quarkus应用程序，然后向LOCK方法发送一个请求：

```
./mvnw clean compile quarkus:dev

curl -X LOCK http://localhost:8080/hello/1
1 locked
```

另见

完整的HTTP方法列表可以在下面的GitHub页面找到：

· KNOW YOUR HTTP methods WELL（https://oreil.ly/DC9Wi）

3.5　启用跨源资源共享

问题

如何从另一个域请求受限制的资源？

解决方案

使用quarkus.http.cors配置属性来启用跨源资源共享（CORS）。

讨论

CORS是一种机制，允许从一个资源服务的域之外的另一个域请求受限制的资源。Quarkus提供了一套配置属性来配置CORS。

要在Quarkus中启用CORS，需要在*application.properties*文件中将quarkus.http.cors配置属性设置为true。

CORS配置的例子如下：

```
quarkus.http.cors=true
quarkus.http.cors.origins=http://example.com
quarkus.http.cors.methods=GET,PUT,POST,DELETE
quarkus.http.cors.headers=accept,authorization,content-type,x-requested-with
```

你可以用curl查看输出和头文件：

```
curl -d '{"name":"Ada"}' -H "Content-Type: application/json" \
    -X POST http://localhost:8080/developer \
    -H "Origin: http://example.com" --verbose
```

输出应该显示access-control-allow-origin头：

```
upload completely sent off: 14 out of 14 bytes
* Mark bundle as not supporting multiuse
< HTTP/1.1 201 Created
< access-control-allow-origin: http://example.com
< access-control-allow-credentials: true
< Content-Length: 21
< Content-Type: application/json
< Location: http://localhost:8080/developer/5
```

另见

你可以在下面的维基百科上找到关于CORS的更多信息：

- 跨源资源共享（https://oreil.ly/iSiqh）

3.6　使用响应式路由

问题

如何使用响应式路由实现HTTP端点？

解决方案

使用Vert.x的io.vertx.ext.web.Router路由器实例或io.quarkus.vertx.web.Route注解。

有两种方法可以在Quarkus中使用响应式路由。第一种方法是直接使用io.vertx.ext.web.Router类注册路由。

为了在启动时检索Router实例，你需要使用Contexts and Dependency Injection（CDI）观察对象的创建。

创建一个名为org.acme.quickstart.ApplicationRoutes.java的新类：

```
package org.acme.quickstart;

import javax.enterprise.context.ApplicationScoped;
import javax.enterprise.event.Observes;

import io.quarkus.vertx.http.runtime.filters.Filters;
import io.quarkus.vertx.web.Route;
import io.vertx.core.http.HttpMethod;   ❺
import io.vertx.ext.web.Router;         ❺
import io.vertx.ext.web.RoutingContext; ❺

@ApplicationScoped ❶
public class ApplicationRoutes {

    public void routes(@Observes Router router) { ❷

        router
            .get("/ok") ❸
            .handler(rc -> rc.response().end("OK from Route")); ❹

    }
}
```

❶将对象实例化到CDI容器中，其作用域为application

❷提供Router对象来注册路由

❸将GET HTTP方法绑定到/ok

❹处理逻辑

❺例子中稍后使用的import的对象

打开一个新的终端窗口，启动Quarkus应用程序，并向新方法发送一个请求：

```
./mvnw clean compile quarkus:dev

curl http://localhost:8080/ok
OK from Route
```

使用响应式路由的第二种方式是使用io.quarkus.vertx.web.Route注解的声明式方法。要访问这个注解，你需要添加quarkus-vertx-web扩展：

```
./mvnw quarkus:add-extension -Dextensions="quarkus-vertx-web"
```

然后你可以用@Route来注解方法。这些方法必须在CDI bean中定义。

打开org.acme.quickstart.ApplicationRoutes.java类并定义一个路由：

```
@Route(path = "/declarativeok", methods = HttpMethod.GET) ❶
public void greetings(RoutingContext routingContext) { ❷
    String name = routingContext.request().getParam("name"); ❸

    if (name == null) {
        name = "world";
    }

    routingContext.response().end("OK " + name + " you are right"); ❹
}
```

❶设置HTTP路径和方法

❷RoutingContext获取请求信息

❸获取查询参数

❹处理逻辑

打开一个新的终端窗口，启动Quarkus应用程序，并向新方法发送一个请求：

```
./mvnw clean compile quarkus:dev

curl localhost:8080/declarativeok?name=Alex
OK Alex you are right
```

讨论

Quarkus HTTP基于一个非阻塞的响应式引擎。其内部使用的是Vert.x和Netty。当接收到一个请求时，它由event loop（事件循环）管理，在servlet或JAX-RS的情况下，它可以依靠一个worker线程；在响应式路由的情况下，它可以使用I/O线程来处理调用的逻辑。

需要注意的是，响应式路由必须是非阻塞式的，或者显式声明为阻塞式的。如果不是，由于响应式事件循环的特性，你将阻塞循环，

从而阻止进一步的循环处理，直到线程解锁。

在同一个项目中，你可以将JAX-RS端点与响应式路由混合使用，不会发生任何问题。

另见

你可以在下面的网页上了解关于Vert.x中的响应式路由的更多信息：

- 基本Vert.x-Web概念（https://oreil.ly/kznp9）

3.7　拦截HTTP请求

问题

如何拦截HTTP请求来操作请求或响应？

解决方案

有时你需要在到达端点逻辑之前（例如，安全检查）或在响应被发回给调用者之前（例如，压缩响应）对请求进行操作。使用Quarkus，你可以使用Vert.x过滤器或JAX-RS过滤器接口来拦截HTTP请求。

让我们看看如何使用io.quarkus.vertx.http.runtime.filters.Filters实现过滤器。

为了在启动时检索Filters实例，你需要观察使用CDI创建对象的过程。

打开org.acme.quickstart.ApplicationRoutes.java类，添加一个名为filters的方法：

```
public void filters(@Observes Filters filters) { ❶
    filters
        .register(
            rc -> {
                rc.response() ❷
                    .putHeader("V-Header", "Header added by VertX Filter"); ❸
                rc.next(); ❹
            },
            10); ❺
}
```

❶提供Filters对象来注册过滤器

❷修改响应

❸在响应中添加一个新的头

❹继续过滤链

❺设定执行顺序

需要注意的是，这些过滤器适用于servlet、JAX-RS资源和响应式路由。

打开一个新的终端窗口，启动Quarkus应用程序，并向新方法发送请求：

```
./mvnw clean compile quarkus:dev

echo Reactive-Route
curl localhost:8080/ok -v
< V-Header: Header added by VertX Filter


< content-length: 13
OK from Route

echo JAX-RS
curl -X GET "http://localhost:8080/hello?order=asc" \
    -H "accept: text/plain" -H "authorization: XYZ" -v
< V-Header: Header added by VertX Filter
< content-length: 65
URI: http://localhost:8080/hello - Order asc - Authorization: XYZ
```

请注意，这两个请求（响应式路由和JAX-RS端点）都是通过注册的过滤器和添加一个新头来修改的。

也就是说，你也可以使用javax.ws.rs.containerRequestFilter/javax.ws.rs.container-ResponseFilter接口来实现一个过滤器。

创建一个新的类，名为org.acme.quickstart.HeaderAdditionContainerResponseFilter.java：

```
package org.acme.quickstart;

import java.io.IOException;

import javax.ws.rs.container.ContainerRequestContext;
import javax.ws.rs.container.ContainerResponseContext;
import javax.ws.rs.container.ContainerResponseFilter;
import javax.ws.rs.ext.Provider;

@Provider ❶
public class HeaderAdditionContainerResponseFilter
              implements ContainerResponseFilter { ❷

    @Override
    public void filter(ContainerRequestContext requestContext,
                       ContainerResponseContext responseContext)
            throws IOException {
                responseContext.getHeaders()
                  .add("X-Header", "Header added by JAXRS Filter"); ❸
    }
}
```

❶将该类设置为扩展接口

❷根据需要作出改变

❸在响应中添加一个新的头

这个过滤器只适用于JAX-RS资源，而不是在响应式路由中。

打开一个新的终端窗口，启动Quarkus应用程序，并向新方法发送请求来尝试。

```
./mvnw clean compile quarkus:dev

echo Reactive-Route
```

```
curl localhost:8080/ok -v
< V-Header: Header added by VertX Filter
< content-length: 13
OK from Route

echo JAX-RS
curl -X GET "http://localhost:8080/hello?order=asc" \
    -H "accept: text/plain" -H "authorization: XYZ" -v
< V-Header: Header added by VertX Filter
< Content-Length: 65
< Content-Type: text/plain;charset=UTF-8
< X-Header: Header added by JAXRS Filter
URI: http://localhost:8080/hello - Order asc - Authorization: XYZ
```

讨论

注意，在响应式路由端点的情况下，只添加了V-Header头，而没有添加X-Header头。同时，在JAX-RS端点中，请求通过添加两个HTTP头被两个过滤器修改。

另见

要了解关于JAX-RS和Vert.x的更多信息，可以访问以下网站。

- Eclipse基金会：Jakarta RESTful网络服务（https://oreil.ly/xioAv）
- Vert.x文档（https://vertx.io/docs）

3.8　使用SSL进行安全连接

问题

如何保护连接，以防止攻击者窃取敏感信息？

解决方案

启用Quarkus使用SSL来保护连接。

当要传输的信息是敏感信息（密码、账号、健康信息等）时，确保客户端和应用程序之间的通信安全是非常重要的。因此，使用SSL来保护服务之间的通信是非常重要的。

为了保证通信的安全，必须提供两个元素：证书和相关的密钥文件。这两个元素可以单独提供，也可以以keystore（密钥库）的形式提供。

配置Quarkus来使用一个包含证书条目的keystore：

```
quarkus.http.ssl-port=8443 ❶
quarkus.http.ssl.certificate.key-store-file=keystore.jks ❷
quarkus.http.ssl.certificate.key-store-file-type=jks
quarkus.http.ssl.certificate.key-store-password=changeit ❸
```

❶设置HTTPS端口

❷keystore的类型和相对于src/main/resources的位置

❸打开keystore的密码

启动应用程序并向HTTPS端点发送请求：

```
./mvnw clean compile quarkus:dev

curl --insecure https://localhost:8443/hello
hello
```

由于证书是自签的，所以提供--insecure标志来跳过证书验证。在一个证书不是自签的例子中，不应该提供insecure标志。为了简单起见，在这个例子中使用了这个标志。

在配置文件中以纯文本的形式提供密码是不好的做法。可以通过使用环境变量QUARKUS_HTTP_SSL_CERTIFICATE_KEY_STORE_PASSWORD来提供，就像你在本书开头介绍MicroProfile Config规范时读到的那样。

讨论

对于忙碌的开发人员来说，可以这样为Quarkus生成自己的密钥证书：

1. 进入*src/main/resources*。

2. 执行以下命令：

```
keytool -genkey -keyalg RSA -alias selfsigned \
    -keystore keystore.jks -storepass changeit \
    -validity 360 -keysize 2048
```

另见

要了解如何生成证书、密钥存储和信任存储，请参阅以下网页。

- Oracle:Java Platform,Standard Edition Tools

Reference:keytool（https://oreil.ly/mwOSH）

第4章　配置

本章涵盖以下主题：

- 如何配置一个Quarkus服务
- 如何在服务中注入配置参数
- 如何在不同的环境中使用配置值
- 如何正确配置日志系统
- 如何创建自定义的配置系统

4.1　使用自定义属性配置应用程序

问题

如何使用自定义属性配置Quarkus应用程序？

解决方案

Quarkus使用了Eclipse MicroProfile规范，其中之一就是配置规范，但是为了简化配置，Quarkus只使用一个文件来处理所有的配置，即*application.properties*，这个文件必须放在classpath的根目录下。

这个文件可以用来配置Quarkus的属性，比如日志或默认路径、Quarkus扩展（如数据源或Kafka），或者为应用程序定义的自定义属性。你将在本书中看到所有这些属性的配置，但本节主要讲自定义属性的配置。

打开*src/main/resources/application.properties*文件，添加以下属性：

```
greeting.message=Hello World
```

你可以通过在字段中使用org.eclipse.microprofile.config.inject.ConfigProperty注解来注入在*application.properties*中定义的属性值。

打开org.acme.quickstart.GreetingResource.java，注入greeting.message属性值：

```
@ConfigProperty(name = "greeting.message") ❶
String message; ❷

@GET
@Produces(MediaType.TEXT_PLAIN)
public String hello() {
    return message; ❸
}
```

❶注入greeting.message属性的值

❷将field放置在包保护的范围内

❸返回配置值

当使用GraalVM和反射时，出于性能方面的考虑，建议你在包保护的范围内使用，这些field的值会在运行时被注入。你可以在Quarkus CDI参考指南（https://oreil.ly/8e1Sd）中阅读关于它的更多内容。

在一个新的终端窗口中请求/hello，可以看到输出结果是*application.properties*中配置的值：

```
curl http://localhost:8080/hello

Hello World
```

如果你想给一个配置属性提供默认值，而不是要求必须设置，可以使用@ConfigProperty注解的defaultValue参数。

打开org.acme.quickstart.GreetingResource.java文件，注入greeting.upper-case属性值：

```
@ConfigProperty(name = "greeting.upper-case",
                defaultValue = "true") ❶
boolean upperCase;
@GET
@Path("/optional")
@Produces(MediaType.TEXT_PLAIN)
public String helloOptional() {
    return upperCase ? message.toUpperCase() : message;
}
```

❶将greeting.upper-case属性的默认值设置为true

在终端窗口中请求/hello/optional，可以看到输出结果的字符是大写的：

```
curl http://localhost:8080/hello/optional

HELLO WORLD
```

也支持给属性设置多个值——可以根据需要或喜好将field类型定义为Arrays、java.util.List、java.util.Set中的一种。属性值的分

隔符是逗号（,），转义符是反斜杠（\）。

打开*src/main/resources/application.properties*文件，并添加以下有三个值的属性：

```
greeting.suffix=!!, How are you???
```

打开*org.acme.quickstart.GreetingResource.java*并注入greeting.suffix属性值：

```
@ConfigProperty(name = "greeting.suffix")
List<String> suffixes;
@GET
@Path("/list")
@Produces(MediaType.TEXT_PLAIN)
public String helloList() {
    return message + suffixes.get(1);
}
```

在终端窗口中请求/hello/list，可以看到输出结果中包含greeting.suffix属性的第二个值：

```
curl http://localhost:8080/hello/list

Hello World How are you?
```

应用程序也支持YAML格式的配置。在这种情况下，配置文件的名字为*application.yaml*或*application.yml*。

要使用YAML格式的配置文件，你需要添加config-yaml扩展：

```
./mvnw quarkus:add-extension -Dextensions="config-yaml"
```

例如以下使用properties格式的配置文件：

```
greeting.message=Hello World

%staging.quarkus.http.port=8182

quarkus.http.cors=true
quarkus.http.cors.methods=GET,PUT,POST
```

等效的YAML格式是：

```
greeting:
  message: Hello World ❶
"%staging": ❷
  quarkus:
    http:
      port: 8182
quarkus:
  http:
    cors:
      ~: true ❸
      methods: GET,PUT,POST
```

❶简单属性的结构设置

❷profile（配置文件）可以使用引号包含

❸当有子键时，~用来指代没有前缀的部分

讨论

Eclipse MicroProfile Configuration自带以下内置的转换器，可将配置值映射到Java对象中：

· boolean和java.lang.Boolean，值为true、1、YES、Y和ON代表真，其他代表假

· byte和java.lang.Byte

· short和java.lang.Short

· int和java.lang.Integer

· long和java.lang.Long

· float和java.lang.Float

· double和java.lang.Double

· char和java.lang.Character

· java.lang.Class，根据调用Class.forName的结果生成

如果内置的转换器或自定义转换器存在，则使用discovered/found方法实例化转换器对象，并传递字符串参数进行转换。如果内置的转换器或自定义转换器不存在，则在目标对象中检查以下方法：

· 目标类型具有public static T of(String)方法

· 目标类型具有public static T valueOf(String)方法

· 目标类型具有一个参数为String的公有构造函数

· 目标类型具有public static T parse(CharSequence)方法

4.2　以程序化的方式访问配置属性

问题

如何使用程序化的方式访问配置属性，而不是注入org.eclipse.microprofile.config.inject.ConfigProperty注解来访问它们？

解决方案

在你希望以程序化的方式访问属性的对象中注入org.eclipse.microprofile.config.Config类。

Eclipse MicroProfile Configuration规范允许你注入org.eclipse.microprofile.config.Config来程序化地获取属性值，而不是使用ConfigProperty直接注入。

打开org.acme.quickstart.GreetingResource.java，注入Config类：

```
@Inject ❶
Config config;
@GET
@Path("/config")
@Produces(MediaType.TEXT_PLAIN)
public String helloConfig() {
    config.getPropertyNames().forEach( p -> System.out.println(p)); ❷

    return config.getValue("greeting.message", String.class); ❸
}
```

❶使用Inject CDI注解来注入配置实例

❷现在可以循环访问属性列表

❸转换为需要的类型

你也可以通过调用ConfigProvider.getConfig()方法来获取Config类（不使用CDI）。

4.3　在外部覆盖配置值

问题

如何在运行时覆盖配置值？

解决方案

你可以在运行时通过将其设置为系统属性或环境变量来覆盖任何配置属性。

Quarkus允许你通过将配置设置为系统属性（-Dproperty.name=value）或环境变量（export PROPERTY_NAME=value）来覆盖任何配置属性。系统属性比环境变量有更高的优先级。

例如数据库的URL、用户名或密码，它们只有在目标环境中才被知晓。但是你需要知道，这里有一个权衡：可用的运行时属性越多，Quarkus在构建时期的前期能做的就越少。

让我们将4.1节中使用的应用程序打包，并通过设置一个系统属性来覆盖greeting.message的值：

```
./mvnw clean package -DskipTests

java -Dgreeting.message=Aloha -jar target/getting-started-1.0-SNAPSHOT-runner.jar
```

在一个新的终端窗口中，通过运行以下命令来验证属性值是否已从Hello World被覆盖为Aloha：

```
curl localhost:8080/hello

Aloha
```

在环境变量的情况下，一个给定的属性名有三种命名惯例，这是因为有些操作系统只允许使用字母和下划线（_），而不允许使用点（.）等其他字符。为了支持所有可能的操作系统，Quarkus使用了以下规则：

1.完全匹配（greeting.message）。

2.将非字母数字字符替换为下划线（greeting_message）。

3.将非字母数字字符替换为下划线，并将其余字符转换为大写字母（GREETING_MESSAGE）。

例如下面的*application.properties*文件：

```
greeting.message=Hello World
```

你可以使用以下任何一个环境变量名覆盖它的值，因为它们都是等价的：

```
export greeting.message=Aloha
export greeting_message=Aloha
export GREETING_MESSAGE=Aloha
```

还有一个特殊的地方，你可以把*application.properties*文件放在应用程序本身之外——应用程序运行的名为config的目录中，在该文件中定义的任何运行时属性将覆盖默认配置。

*config/application.properties*在开发模式下也能使用，但你需要把它添加到你的构建工具输出目录上才行（如果是Maven，则是target目录；如果是Grandle，则是build目录），所以你要注意在运行clean任务时需要重新创建它。

除了环境变量和*application.properties*文件，还可以在当前工作目录下放置一个.env文件，按照环境变量的格式（GREETING_MESSAGE=Aloha）来覆盖配置值。

4.4 使用profile进行配置

问题

如何根据运行Quarkus的环境来覆盖配置值？

解决方案

Quarkus支持profile的配置。这允许你在同一个文件中为同一个属性设置多个配置值，并在运行Quarkus服务的不同环境中使用不同的值。

启用profile的语法是%{profile}.config.key=value。

讨论

Quarkus有三个内置的profile。

dev

在开发模式下启用（如quarkus:dev）。

test

运行测试时启用。

prod

当不在开发或测试模式下运行时的默认配置文件。它是隐式设置的，不需要在*application.properties*中设置。

打开*src/main/resources/application.properties*文件，并设置为在开发模式下以8181端口启动Quarkus：

```
%dev.quarkus.http.port=8181
```

更改后，启动服务，检查监听端口是否为8181，而不是默认的8080：

```
./mvnw compile quarkus:dev

INFO  [io.qua.dep.QuarkusAugmentor] (main) Beginning quarkus augmentation
INFO  [io.qua.dep.QuarkusAugmentor] (main) Quarkus augmentation completed
    in 671ms
INFO  [io.quarkus] (main) Quarkus 1.4.1 started in 1.385s. Listening on:
    http://0.0.0.0:8181
INFO  [io.quarkus] (main) Profile dev activated. Live Coding activated.
INFO  [io.quarkus] (main) Installed features:
    [cdi, hibernate-validator, resteasy]
```

注意，现在监听地址是http://0.0.0.0:8181，而不是默认的8080。

最后，回滚到8080端口，删除*application.properties*中%dev.quarkus.http.port=8181的配置，使其与本书其他部分使用的端口一致。

4.5　修改日志配置

问题

如何更改默认的日志配置？

解决方案

Quarkus使用了一个统一的配置模型，所有的配置属性都放在同一个文件中，即*application.properties*，你可以在那里配置日志的各种参数。

例如，如果你想改变日志级别，只需将quarkus.log.level设置为最小日志级别即可。

打开*src/main/resources/application.properties*并添加以下内容：

```
quarkus.log.level=DEBUG
```

现在启动应用程序，可以看到控制台中打印了很多新的日志信息：

```
./mvnw compile quarkus:dev

...
[INFO] --- quarkus-maven-plugin:0.22.0:dev (default-cli) @ getting-started ---
Listening for transport dt_socket at address: 5005
DEBUG [org.jbo.logging] (main) Logging Provider: \
    org.jboss.logging.JBossLogManagerProvider
INFO  [io.qua.dep.QuarkusAugmentor] (main) Beginning quarkus augmentation
DEBUG [io.qua.run.con.ConverterSupport] (main) Populate SmallRye config builder
    with converter for class java.net.InetSocketAddress of priority 200
DEBUG [io.qua.run.con.ConverterSupport] (main) Populate SmallRye config builder
    with converter for class org.wildfly.common.net.CidrAddress of priority 200
```

我们用反斜杠来格式化本书中需要跨越多行的代码。

你也可以通过使用quarkus.log.file.enable属性将日志存储在文件中。默认情况下，输出会被写入一个名为*quarkus.log*的文件中：

```
quarkus.log.file.enable=true
```

当你在src目录开发或工作时，你的日志文件将位于target目录中。

4.6　添加应用程序日志

问题

如何在应用程序中添加日志？

解决方案

大多数情况下，你的应用程序需要编写程序自身的日志信息，而不是仅仅依靠Quarkus提供的默认日志。Quarkus应用程序可以使用任何一种支持的API进行日志记录，这些日志将被合并。

Quarkus支持以下日志库：

- JDK java.util.logging
- JBoss logging
- SLF4J
- Apache Commons Logging

我们来看看如何使用JBoss Logging来记录日志。打开org.acme.quickstart.GreetingResource.java，设置当/hello/log被

调用时，打印一条日志信息：

```
private static org.jboss.logging.Logger logger =
                org.jboss.logging.Logger.getLogger(GreetingResource.class); ❶

@GET
@Path("/log") ❷

 @Produces(MediaType.TEXT_PLAIN)
 public String helloLog() {
     logger.info("I said Hello"); ❸
     return "hello";
 }
```

❶创建logger实例

❷子路径设置为/log

❸打印一条info级别的日志

现在启动应用程序：

```
./mvnw compile quarkus:dev
```

在一个新的终端窗口中，请求/hello/log：

```
curl http://localhost:8080/hello/log
```

如果你检查启动Quarkus的终端窗口，会看到下面的日志：

```
INFO  [org.acm.qui.GreetingResource] (executor-thread-1) I said Hello
```

讨论

日志记录是以每个类别为基础进行的，适用于一个类别的配置也适用于该类别的所有子类别，除非有更具体的匹配的子类别配置。

类别由类的位置（即定义它们的包或子包）来表示，例如，如果要将Undertow安全日志设置为trace级别，则需要在*application.properties*中设置quarkus.log.category."io.undertow.request.security".level=TRACE属性。

按照前面的例子，让我们限制org.acme.quickstart中的类（和子类）的日志打印级别，设置最小日志级别为WARNING：

```
quarkus.log.category."org.acme.quickstart".level=WARNING ❶
```

❶设置类别时必须使用双引号

如果你再次发请求到http://localhost:8080/hello/log，会发现之前的日志不再显示了。

4.7　高级日志配置

问题

如何集中管理所有服务的日志？

解决方案

当使用微服务架构和Kubernetes时，日志是一个重要的考虑因素，因为每个服务都有单独的日志输出。但作为开发者或操作者，你可能希望将所有的日志集中到一个地方，这样就可以将日志作为一个整体来使用。

Quarkus日志也支持JSON和GELF输出。

这些日志可以以JSON格式而不是纯文本的形式写入日志处理程序中——通过注册logging-json扩展来实现：

```
./mvnw quarkus:add-extension -Dextensions="logging-json"
```

使用GELF扩展来生成GELF格式的日志，并使用TCP或UDP发送日志。

现在使用Graylog扩展日志格式（GELF）的三个集中式的日志系统是：

· Graylog（MongoDB、Elasticsearch、Graylog）

· ELK（Elasticsearch、Logstash、Kibana）

· EFK（Elasticsearch、Fluentd、Kibana）

要以GELF格式进行日志记录，需要添加logging-gelf扩展：

```
./mvnw quarkus:add-extension -Dextensions="logging-gelf"
```

因为示例代码不变，所以使用的是相同的访问路径：

```
private static org.jboss.logging.Logger logger =
                org.jboss.logging.Logger.getLogger(GreetingResource.class); ❶
@GET
@Path("/log") ❷
@Produces(MediaType.TEXT_PLAIN)
public String helloLog() {
    logger.info("I said Hello"); ❸
    return "hello";
}
```

❶创建logger实例

❷子路径设置为/log

❸打印一条info级别的日志

GELF处理系统必须在application.properties中显式配置：

```
quarkus.log.handler.gelf.enabled=true ❶
quarkus.log.handler.gelf.host=localhost ❷
quarkus.log.handler.gelf.port=12201 ❸
```

❶启用GELF扩展

❷设置发送日志的地址

❸端点端口设置

如果使用Logstash（ELK），你需要启用能处理GELF格式的输入插件：

```
input {
  gelf {
    port => 12201
  }
}
output {
  stdout {}
  elasticsearch {
    hosts => ["http://elasticsearch:9200"]
  }
}
```

如果使用Fluentd（EFK），你需要启用能处理GELF格式的输入插件：

```
<source>
  type gelf
  tag example.gelf
  bind 0.0.0.0
  port 12201
</source>

<match example.gelf>
  @type elasticsearch
  host elasticsearch
  port 9200
  logstash_format true
</match>
```

讨论

Quarkus日志默认支持syslog格式，无须添加任何扩展。syslog格式的日志配置（在Fluentd中使用）可以用来替代上面的GELF格式的配置（在Quarkus中使用）：

```
quarkus.log.syslog.enable=true
quarkus.log.syslog.endpoint=localhost:5140
quarkus.log.syslog.protocol=udp
quarkus.log.syslog.app-name=quarkus
quarkus.log.syslog.hostname=quarkus-test
```

在Fluentd中，你需要启用能够处理syslog格式的输入插件：

```
<source>
  @type syslog
  port 5140
```

```
  bind 0.0.0.0
  message_format rfc5424
  tag system
</source>

<match **>
  @type elasticsearch
  host elasticsearch
  port 9200
  logstash_format true
</match>
```

如果你使用的是Kubernetes，最简单的方法是登录到控制台，并在集群中安装一个收集所有日志的中央日志管理器。

另见

要了解关于高级日志主题的更多信息，请访问以下网站：

· Logstash/Gelf Loggers（https://oreil.ly/Mj9Ha）

4.8 使用自定义profile进行配置

问题

如何为创建的自定义profile设置不同的配置值？

解决方案

到目前为止，你已经看到了Quarkus内置的profile，我们可以为同一个属性设置不同的配置值，并使它们适应不同的环境。使用Quarkus，你也可以设置自己的profile。

你唯一需要做的就是通过quarkus.profile系统属性或QUARKUS_PROFILE环境变量来指定你想要启用的profile。如果两者都设置了，则系统属性优先于环境变量。

之后需要创建带有profile名称的属性，并将当前的profile设置为该名称。让我们创建一个新的staging profile，它覆盖Quarkus的监听端口。

打开*src/main/resources/application.properties*文件，并设置当staging profile启用时，Quarkus的启动端口为8182：

```
%staging.quarkus.http.port=8182
```

然后启动应用程序并启用staging profile：

```
./mvnw -Dquarkus.profile=staging compile quarkus:dev

INFO  [io.qua.dep.QuarkusAugmentor] (main) Beginning quarkus augmentation
INFO  [io.qua.dep.QuarkusAugmentor] (main) Quarkus augmentation completed
    in 640ms
INFO  [io.quarkus] (main) Quarkus 0.23.2 started in 1.300s. Listening on:
    http://0.0.0.0:8182
INFO  [io.quarkus] (main) Profile staging activated. Live Coding activated.
INFO  [io.quarkus] (main) Installed features: [cdi, hibernate-validator,
    resteasy]
```

上面的实例使用了系统属性的方法启用profile，不过你也可以使用QUARKUS_PROFILE环境变量来设置它。

讨论

如果你想在测试环境中设置要使用的profile，只需要在构建脚本中把quarkus.test.profile系统属性设置为给定的profile。例如，在Maven中：

```
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-surefire-plugin</artifactId>
<version>${surefire-plugin.version}</version>
<configuration>
    <systemPropertyVariables>
        <quarkus.test.profile>foo</quarkus.test.profile>
        <buildDirectory>${project.build.directory}</buildDirectory>
    </systemPropertyVariables>
</configuration>
```

或者，在Gradle中：

```
test {
    useJUnitPlatform()
    systemProperty "quarkus.test.profile", "foo"
}
```

另外，你也可以调整默认的生产profile。Quarkus内置的profile是prod，所以在没有指定profile的情况下运行应用程序时，prod就是默认的profile。但是你可以在构建时改变默认的profile，这样在没有指定任何profile的情况下，你设置的profile就是应用程序运行时的默认profile。

你需要做的就是使用quarkus.profile系统属性构建应用程序，并将你想要的profile值设置为默认值：

```
./mvnw package -Pnative -Dquarkus.profile=prod-kubernetes`
./target/getting-started-1.0-runner ❶
```

❶该命令设置默认使用prod-kubernetes profile

4.9 创建自定义配置源

问题

如何从其他配置源而不是从*application.properties*文件加载配置参数？

解决方案

Quarkus使用Eclipse MicroProfile Configuration规范来实现所有关于配置的逻辑。该规范提供了org.eclipse.microprofile.config.spi.ConfigSource Java SPI（https://oreil.ly/oOA51）接口，以实现一种自定义的方式来加载配置属性，而不是使用Quarkus提供的默认方式加载配置属性。

例如，你可以从数据库、XML文件或REST API中加载配置属性。

让我们创建一个简单的内存配置源，它从实例化时填充的Map中获取配置属性。创建一个名为org.acme.quick-start.InMemoryConfigSource.java的新类：

```
package org.acme.quickstart;

import java.util.HashMap;
import java.util.Map;

import org.eclipse.microprofile.config.spi.ConfigSource;

public class InMemoryConfigSource implements ConfigSource {

    private Map<String, String> prop = new HashMap<>();

    public InMemoryConfigSource() { ❶
        prop.put("greeting.color", "red");
    }

    @Override
    public int getOrdinal() { ❷
        return 500;
    }

    @Override
    public Map<String, String> getProperties() { ❸
        return prop;
    }

    @Override
    public String getValue(String propertyName) { ❹
        return prop.get(propertyName);
    }
```

```
    @Override
    public String getName() { ❺
        return "MemoryConfigSource";
    }

}
```

❶用属性填充Map

❷用于确定配置值的重要程度，数值越大，优先级越高

❸以Map的形式获取所有属性，本例为直接获取

❹获取单个属性的值

❺返回此配置源的名称

你需要将其注册为一个Java SPI。在*src/main/resources/META-INF*中创建*services*文件夹，然后在services文件夹里创建一个名为*org.eclipse.microprofile.config.spi.ConfigSource*的文件，内容如下：

```
org.acme.quickstart.InMemoryConfigSource
```

最后，你可以修改org.acme.quickstart.GreetingResource.java类来注入此属性：

```
@ConfigProperty(name = "greeting.color") ❶
String color;

@GET
@Path("/color")
@Produces(MediaType.TEXT_PLAIN)
public String color() {
    return color;
}
```

❶注入InMemoryConfigSource中定义的属性值

在终端窗口中请求/hello/color，可以看到输出的结果是自定义配置源中的值：

```
curl http://localhost:8080/hello/color

red
```

讨论

每个ConfigSource都有一个指定的序数值，当同一应用程序定义了多个ConfigSource时，该序数用于设置从ConfigSource中获取值的重要程度。序数值较高的ConfigSource会比序数值较低的ConfigSource优先使用。以下面列表中的默认值作为参考，在所有的配置源上都会使用系统属性，如果没有找到其他的ConfigSource，则使用*src/main/resources*目录下的*application.properties*文件：

- 系统属性为400
- 环境变量为300
- config目录下的*application.properties*为260
- 项目中的*application.properties*为250

4.10 创建自定义转换器

问题

如何实现一个自定义转换器？

解决方案

你可以通过实现org.eclipse.microprofile.config.spi.Converter Java SPI将一个属性从String转换为任何类型的对象。

Quarkus使用Eclipse MicroProfile Configuration规范来实现所有关于配置的逻辑，该规范提供了org.eclipse.microprofile.config.spi.Converter Java SPI（https://oreil.ly/kcqQw）接口来实现配置值到自定义类型的转换。

例如，你可以将一个百分比值（即15%）转换为Percentage类型，将百分比包装为double类型。

创建一个新的POJO类：org.acme.quickstart.Percentor.java：

```
package org.acme.quickstart;

public class Percentage {

    private double percentage;

    public Percentage(double percentage) {
        this.percentage = percentage;
    }

    public double getPercentage() {
        return percentage;
    }

}
```

然后创建一个类 org.acme.quickstart.percentorConverter.java，将字符串表示转换为百分比表示：

```
package org.acme.quickstart;

import javax.annotation.Priority;

import org.eclipse.microprofile.config.spi.Converter;

@Priority(300) ❶
public class PercentageConverter implements Converter<Percentage> { ❷

    @Override
    public Percentage convert(String value) {

        String numeric = value.substring(0, value.length() - 1);
        return new Percentage (Double.parseDouble(numeric) / 100);

    }

}
```

❶设置优先级。在这个示例中，它是可选的

❷设置通用的转换类型

你需要将其注册为一个Java SPI。在*src/main/resources/META-INF*创建*services*文件夹，然后在*services*文件夹中创建一个名为*org.eclipse.microprofile.config.spi.Converter*的文件，内容如下：

```
org.acme.quickstart.PercentageConverter
```

修改org.acme.quickstart.GreetingResource.java类注入此属性：

```
@ConfigProperty(name = "greeting.vat")
Percentage vat;

@GET
@Path("/vat")
@Produces(MediaType.TEXT_PLAIN)
public String vat() {
    return Double.toString(vat.getPercentage());
}
```

在*src/main/resources*目录下的*application.properties*文件中添加一个新属性：

```
greeting.vat = 21%
```

在终端窗口中，请求/hello/vat，输出结果是把vat的值转成的double类型：

```
curl http://localhost:8080/hello/vat

0.21
```

讨论

默认情况下，如果在一个转换器上找不到@Priority注解，那么它的优先级为100。Quarkus转换器的优先级为200，所以如果你想替换一个Quarkus转换器，应该使用更大的数值；如果你不需要替换Quarkus转换器，使用默认的值即可。

4.1节中已经列出了Quarkus核心的转换器。

4.11　配置值的分组

问题

如何避免重复设置配置属性的通用前缀？

解决方案

可以使用@io.quarkus.arc.config.ConfigProperties注解来分组共同的属性（那些具有相同前缀的属性）。

当在你的应用程序中创建特别的配置属性时，通常这些属性会有相同的前缀（例如，greetings）。要注入这些属性值，可以使用@ConfigProperty注解（如4.1节所示），也可以使用io.quarkus.arc.config.ConfigProperties注解将属性分组。

使用*application.properties*文件：

```
greeting.message=Hello World
greeting.suffix=!!, How are you???
```

让我们使用io.quarkus.arc.config.ConfigProperties注解实现一个将配置属性映射到Java对象的类。创建一个新的类org.acme.quickstart.GreetingConfiguration.java：

```
package org.acme.quickstart;

import java.util.List;
import java.util.Optional;

import javax.validation.constraints.Max;
import javax.validation.constraints.Min;

import io.quarkus.arc.config.ConfigProperties;

@ConfigProperties(prefix = "greeting") ❶
public class GreetingConfiguration {


    public String message; ❷
    public String suffix = "!"; ❸
}
```

❶配置POJO的共同前缀

❷映射greeting.message属性

❸如果没有设置该属性，greeting.suffix的默认值

注意，prefix属性不是强制性的。如果它没有被设置，那么要使用的前缀将由类名来决定（去掉后缀部分Configuration），在这种情况下，前缀属性可以自动解析为greeting。

然后你可以注入这个配置POJO来使用配置值。

可以修改org.acme.quickstart.GreetingResource.java类来注入这个类：

```
@Inject ❶
GreetingConfiguration greetingConfiguration;

@GET
@Path("/configurations")
@Produces(MediaType.TEXT_PLAIN)
public String helloConfigurations() {
    return greetingConfiguration.message + greetingConfiguration.suffix;
}
```

❶用CDI @Inject注解注入配置

在终端窗口中请求/hello/configurations，可以看到配置值构成了输出结果，如下所示：

```
curl http://localhost:8080/hello/configurations

Hello World!!, How are you???
```

正如所见，你不需要通过使用@ConfigProperty来注解每一个field，只需要利用类定义来获取属性名或默认值。

讨论

此外，Quarkus支持嵌套的对象配置，因此你也可以通过使用内部类来映射。

假设我们在*application.properties*中添加一个名为greeting.output.recipients的新属性：

```
greeting.output.recipients=Ada,Alexandra
```

你可以使用一个内部类将其映射到配置对象中。修改类org.acme.quickstart.GreetingConfiguration.java，然后添加一个新的内部类，代表子类输出，并将其注册为一个field：

```
public OutputConfiguration output; ❶

public static class OutputConfiguration {
    public List<String> recipients;
}
```

❶子类别的名称field名（output）

然后你可以访问greetingConfiguration.output.recipients来获取对应的值，也可以用Bean Validation来注解这些field，以便在启动时验证所有配置值是否有效。如果它们无效，应用程序将无法启动，并将在日志中显示错误信息。

4.12　验证配置值

问题

如何验证配置值是否合法？

解决方案

当使用@ConfigProperty注解在类中注入一个属性值时，可以使用Bean Validation规范来验证该属性值是否有效。

Bean Validation规范允许你使用注解对对象设置约束。Quarkus集成了Eclipse MicroProfile Configuration规范和Bean Validation规范，你可以一起使用它们来校验一个配置值是否符合某些标准。这个校验是在启动时执行的，如果有任何不合法行为，控制台中都会显示一个错误信息，并中止启动程序。

你需要做的第一件事是注册Quarkus Bean Validation依赖。可以通过编辑pom.xml或者在项目的根目录下运行下面的Maven command来手动完成：

```
./mvnw quarkus:add-extension -Dextensions="quarkus-hibernate-validator"
```

之后，你需要创建一个配置对象，这在前面已经学习过。在接下来的例子中，对greeting.repeat配置属性设置了一个约束，因此不能设置范围1～3（含）以外的数值。为了验证整数范围，使用了以下Bean Validation注解：javax.validation.constraces.Max和javax.validation.constraces.Min。打开org.acme.quickstart.GreetingConfiguration.java并添加Bean Validation注解：

```
@Min(1) ❶
@Max(3) ❷
public Integer repeat;
```

❶接受的最小值

❷接受的最大值

打开src/main/resources/application.properties文件，将greeting.repeat配置属性设置为7：

```
greeting.repeat=7
```

启动应用程序，你会看到一个错误消息，告知其中一个配置值违反了定义的约束条件：

```
./mvnw compile quarkus:dev
```

讨论

在这个例子中，你已经看到了Bean Validation规范的简单介绍，以及一些可以用来验证字段的注解。然而，Hibernate Validation和所使用的Bean Validation实现支持很多约束条件，比如@Digits、@Email、@NotNull和@NotBlank。

第5章　编程模型

在本章中，你将会了解到与Quarkus编程模型相关的话题。Quarkus所遵循的编程模型是独特的，与其他框架不同的是，Quarkus允许你混合使用CDI命令式模型和SmallRye Mutiny响应式模型。第15章专门介绍了使用响应式模型的方法。有时，你可能会发现自己需要这两种模型，所以知道如何使用每种模型是件好事。

本章涵盖以下主题：

· 如何对JSON和XML文档进行编组（marshal）和解组（unmarshal）

· 如何校验请求参数

· 如何使用CDI规范作为上下文和依赖注入的解决方案

· 如何为Quarkus服务编写测试

5.1　对JSON进行编组和解组

问题

如何把JSON文档编组到Java对象，把Java对象解组到JSON文档？

解决方案

使用JSON-B规范或Jackson项目。

当创建REST API时，你通常使用JSON数据格式进行信息传递。到目前为止，你已经看到了只返回简单的纯文本响应的例子。在本节中，你将学习如何开始使用JSON作为请求主体和响应的数据格式。

首先，在*pom.xml*中添加JSON-B扩展。打开一个终端窗口，在项目的根目录运行以下命令：

```
./mvnw quarkus:add-extension -Dextensions="quarkus-resteasy-jsonb"

[INFO] --- quarkus-maven-plugin:1.4.1.Final:add-extension (default-cli)
    @ custom-config ---
✅ Adding extension io.quarkus:quarkus-resteasy-jsonb
```

这可以快速将io.quarkus:quarkus-resteasy-jsonb添加到构建工具中。

在Gradle中，可以使用./gradlew addExtension --extensions="quarkus-resteasy-jsonb"来添加扩展。

下一步是创建一个名为Developer的类，该类将在端点中被编程和解组。创建一个名为org.acme.quickstart.Developer.java的新类：

```
package org.acme.quickstart;

public class Developer {

    private String name;
    private String favoriteLanguage;
    private int age;

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getFavoriteLanguage() {
        return favoriteLanguage;
    }

    public void setFavoriteLanguage(String favoriteLanguage) {
        this.favoriteLanguage = favoriteLanguage;
    }

    public int getAge() {
        return age;
    }
    public void setAge(int age) {
        this.age = age;
    }

}
```

最后，你可以创建一个REST API来实现developer类的操作。创建一个名为org.acme.quickstart.DeveloperResource.java的新类：

```
package org.acme.quickstart;

import java.util.ArrayList;
import java.util.List;

import javax.ws.rs.Consumes;
import javax.ws.rs.GET;
import javax.ws.rs.POST;
import javax.ws.rs.Path;
import javax.ws.rs.Produces;
import javax.ws.rs.core.MediaType;
import javax.ws.rs.core.Response;

@Path("/developer")
public class DeveloperResource {

    private static final List<Developer> developers = new ArrayList<>();

    @POST
    @Consumes(MediaType.APPLICATION_JSON)
    public Response addDeveloper(Developer developer) {
        developers.add(developer);
        return Response.ok().build();
    }

    @GET
    @Produces(MediaType.APPLICATION_JSON)
    public List<Developer> getDevelopers() {
        return developers;
    }
}
```

打开一个新的终端窗口，启动Quarkus应用程序，尝试发送POST和GET方法的请求：

```
./mvnw clean compile quarkus:dev

curl -d '{"name":"Alex","age":39, "favoriteLanguage":"java"}' \
  -H "Content-Type: application/json" -X POST http://localhost:8080/developer

curl localhost:8080/developer
[{"age":39,"favoriteLanguage":"java","name":"Alex"}]
```

注意，每个Java字段都直接映射到JSON字段。如果你想改变这一点，可以使用javax.json.bind.annotation.JsonbProperty注解来设置不同的映射名：

```
@JsonbProperty("favorite-language")
String favoriteLanguage;
```

讨论

除了使用JSON-B，你也可以使用Jackson项目实现JSON文档与Java对象的编组和解组。你需要添加Jackson-Extension才能将其作为JSON解决方案使用：

```
./mvnw quarkus:add-extension -Dextensions="quarkus-resteasy-jackson"
```

默认情况下，它提供了一个com.fasterxml.jackson.databind.ObjectMapper，但你也可以使用CDI的方式提供一个自定义的ObjectMapper：

```
package org.acme.quickstart;

import com.fasterxml.jackson.databind.ObjectMapper;
import io.quarkus.jackson.ObjectMapperCustomizer;
import javax.inject.Singleton;

@Singleton
public class RegisterCustomModuleCustomizer
    implements ObjectMapperCustomizer {

    public void customize(ObjectMapper mapper) {
        objectMapper.configure(
            DeserializationFeature.FAIL_ON_NULL_FOR_PRIMITIVES, false);
    }
}
```

另见

你可以在以下网页中了解关于JSON-B和Jackson的更多信息：

· Jakarta JSON Binding （JSON-B） （http://json-b.net）

· Jackson Project Home @GitHub（https://oreil.ly/U3hwH）

5.2　对XML进行编组和解组

问题

如何把XML文档编组到Java对象，把Java对象解组到XML文档？

解决方案

使用JAX-B规范。

当创建REST API时，你可能希望使用XML作为数据格式进行信息传递。到目前为止，你已经看到了如何使用JSON格式来实现，但在本节中，你将学习如何开始使用XML作为请求主体和响应的数据格式。

首先在*pom.xml*中添加JAX-B扩展。打开一个终端窗口，在项目的根目录运行以下命令：

```
./mvnw quarkus:add-extension -Dextensions="quarkus-resteasy-jaxb"

[INFO] --- quarkus-maven-plugin:1.4.1.Final:add-extension (default-cli)
    @ custom-config ---
✅ Adding extension io.quarkus:quarkus-resteasy-jaxb
```

这可以快速将io.quarkus:quarkus-resteasy-jaxb添加到构建工具中。

在Gradle中，可以使用./gradlew addExtension --extensions = "quarkus-resteasy-jaxb "来添加扩展。

下一步是创建一个名为Computer的类，该类将在端点中被编组和解组。创建一个名为org.acme.quickstart.Computer.java的新类：

```
package org.acme.quickstart;

import javax.xml.bind.annotation.XmlRootElement;

@XmlRootElement ❶
public class Computer {

    private String brand;
    private String serialNumber;

    public String getBrand() {
        return brand;
    }

    public void setBrand(String brand) {
        this.brand = brand;
    }

    public String getSerialNumber() {
        return serialNumber;
    }
    public void setSerialNumber(String serialNumber) {
        this.serialNumber = serialNumber;
    }

}
```

❶XmlRootElement将其设置为XML文档

最后，你可以创建一个REST API来实现Computer类的操作。创建一个名为org.acme.quickstart.ComputerResource.java的新类：

```
package org.acme.quickstart;

import java.util.ArrayList;
import java.util.List;

import javax.ws.rs.Consumes;
import javax.ws.rs.GET;
import javax.ws.rs.POST;
import javax.ws.rs.Path;
import javax.ws.rs.Produces;
import javax.ws.rs.core.MediaType;
import javax.ws.rs.core.Response;

@Path("/computer")
public class ComputerResource {

    private static final List<Computer> computers = new ArrayList<>();

    @POST
    @Consumes(MediaType.APPLICATION_XML)
    public Response addComputer(Computer computer) {
        computers.add(computer);
        return Response.ok().build();
    }

    @GET
    @Produces(MediaType.APPLICATION_XML)
    public List<Computer> getComputers() {
        return computers;
    }

}
```

打开一个新的终端窗口，启动Quarkus应用程序，尝试发送POST和GET方法的请求：

```
./mvnw clean compile quarkus:dev

curl \
    -d '<computer><brand>iMac</brand>
        <serialNumber>111-111-111</serialNumber></computer>'
    -H "Content-Type: application/xml" -X POST http://localhost:8080/computer

curl localhost:8080/computer
<?xml version="1.0" encoding="UTF-8" standalone="yes"?><collection><computer>
<brand>iMac</brand><serialNumber>111-111-111</serialNumber>
</computer></collection>
```

讨论

除了@XmlRootElement，JAX-B规范中还有其他重要的注解：

@XmlRootElement

设置XML文档的根目录，也可以用它来设置元素的名称或命名空间。

@XmlType

定义字段写入的顺序。

@XmlElement

定义实际的XML元素名称，以及其他属性，如namespace、nillable或required。

@XmlAttribute

定义字段要被映射为属性而不是元素。

@XmlTransient

表示不包含在XML中的字段。

另见

你可以在以下网页中了解关于JAX-B的更多信息：

· Oracle：Lesson:Introduction to JAXB（The Java Tutorials）（https://oreil.ly/r9FKb）

5.3　校验输入值和输出值

问题

如何校验REST服务或项目服务的输入值和输出值的合法性？

解决方案

使用Bean Validation规范来为你的模型添加校验。

通常，你的模型中可能包含一些约束来校验模型是否在语义上有效。例如，名字不能为空，或者一个电子邮件是一个有效的电子邮件。Quarkus与Bean Validation集成，通过注解来表达对象模型上的约束。

首先，在pom.xml中添加Bean Validation扩展。打开一个终端窗口，在项目的根目录下运行以下命令：

```
./mvnw quarkus:add-extension -Dextensions="quarkus-hibernate-validator"

[INFO] --- quarkus-maven-plugin:1.4.1.Final:add-extension (default-cli)
    @ custom-config ---
✅ Adding extension io.quarkus:quarkus-resteasy-jsonb
```

这可以快速将io.quarkus:quarkus-hibernate-validator添加到构建工具中。

在Gradle中，你可以使用./gradlew addExtension --extensions="quarkus-hibernate-validator"来添加扩展。

下一步是更新developer类，添加一些约束条件的注解。打开org.acme.quickstart.Developer.java类，并对一些字段添加注解：

```
@Size(min = 4) ❶
private String name;

@NotBlank ❷
private String favoriteLanguage;
```

❶字符串的最小长度为4

❷字段是强制性的

任何一个可用的包都能完成这样的功能，但如果你只想使用规范的API，请使用javax包。

最后，你需要用javax.validation.Valid注解来标识某一个参数需要校验。打开org.acme.quickstart.DeveloperResource.java类，并对develop参数添加注解：

```
@POST
@Consumes(MediaType.APPLICATION_JSON)
public Response addDeveloper(@Valid Developer developer) { ❶
    developers.add(developer);
    return Response.ok().build();
}
```

❶@Valid是校验对象的强制参数

打开一个新的终端窗口，启动Quarkus应用程序，然后执行POST方法请求：

```
./mvnw clean compile quarkus:dev

curl -d '{"name":"Ada","age":7, "favoriteLanguage":"java"}' \
     -H "Content-Type: application/json" \
     -X POST http://localhost:8080/developer -v

< HTTP/1.1 400 Bad Request
< Content-Length: 89
< validation-exception: true
< Content-Type: text/plain;charset=UTF-8


curl -d '{"name":"Alexandra","age":5, "favoriteLanguage":"java"}' \
     -H "Content-Type: application/json" \
     -X POST http://localhost:8080/developer -v

< HTTP/1.1 200 OK
< Content-Length: 0
```

注意，在第一个请求中，name的长度不合法，因此返回了一个400 Bad Request HTTP状态码。在第二个请求中，请求体是合法的，所以

返回是符合预期的。

但是请注意，如果出现错误，响应体中并不包含任何关于失败的信息。这样做的好处是不会直接显示内部结构，而是以一种可控的方式显示。

讨论

如果你想提供一个更友好的响应体，可以提供一个ExceptionMapper的实现。

创建一个名为org.acme.quickstart.BeanValidationExceptionMapper.java的新类：

```
package org.acme.quickstart;

import javax.json.Json;
import javax.json.JsonArray;
import javax.json.JsonArrayBuilder;
import javax.validation.ConstraintViolation;
import javax.validation.ConstraintViolationException;
import javax.ws.rs.core.MediaType;
import javax.ws.rs.core.Response;
import javax.ws.rs.ext.ExceptionMapper;
import javax.ws.rs.ext.Provider;

@Provider ❶
public class BeanValidationExceptionMapper
  implements ExceptionMapper<ConstraintViolationException> { ❷

    @Override
    public Response toResponse(ConstraintViolationException exception) {
      return Response.status(Response.Status.BAD_REQUEST)
        .entity(createErrorMessage(exception))
        .type(MediaType.APPLICATION_JSON)
        .build();
    }

    private JsonArray createErrorMessage(ConstraintViolationException exc) {
      JsonArrayBuilder errors = Json.createArrayBuilder(); ❸
      for (ConstraintViolation<?> violation : exc.getConstraintViolations()) { ❹
        errors.add(
            Json.createObjectBuilder() ❺
            .add("path", violation.getPropertyPath().toString())
            .add("message", violation.getMessage())
```

```
            );
        }
        return errors.build();
    }
}
```

❶@Provider设置了一个可由JAX-RS运行时发现的扩展接口实现

❷javax.ws.rs.ext.ExceptionMapper用于将异常转化为一个javax.ws.rs.core.Response

❸创建违反约束的数组

❹对每一个违反约束条件进行迭代

❺创建JSON对象

现在你可以再次发送POST方法的请求：

```
curl -d '{"name":"Ada","age":7, "favoriteLanguage":"java"}' \
     -H "Content-Type: application/json" \
     -X POST http://localhost:8080/developer -v

< HTTP/1.1 400 Bad Request
< Content-Length: 90
< Content-Type: application/json

[{"path":"addDeveloper.developer.name",
  "message":"size must be between 4 and 2147483647"}]%
```

现在的输出略有不同，但错误代码还是一样的：400 Bad Request，但是响应体的内容包含了我们在异常映射器中创建的JSON文档。

你也可以通过在返回类型中添加@Valid注解来验证输出参数（你发回给调用者的参数）：

```
@GET
@Produces(MediaType.APPLICATION_JSON)
public @Valid List<Developer> getDevelopers() {
    return developers;
}
```

此外，有时你不想在请求接口中添加校验规则，而想在业务服务层中添加。如果你使用CDI的方式，可以在业务服务中使用Bean Validation。请看下面的例子：

```
@ApplicationScoped
public class DeveloperService {
    public void promoteDeveloper(@Valid Developer developer) {
    }
}
```

另见

如果你想了解关于Bean Validation的更多信息，以及默认情况下实现了哪些约束条件（如@Min、@Max、@AssertTrue、@Email等），可以访问以下网站。

· Jakarta Bean Validation（https://oreil.ly/YHR_X）

5.4　创建自定义校验

问题

如何创建自定义校验？

解决方案

通过实现javax.validation.ConstraintValidator接口来使用Bean Validation扩展模型。

有时，Bean Validation规范提供的默认约束是不够的，需要实现与业务模型更紧密结合的约束。Bean Validation允许通过创建一个类来实现此目的，该类实现javax.validation.ConstraintValidator接口和注解——用于注解要验证的字段。

让我们看一个例子，假如你最喜欢的语言只能是基于JVM的语言。首先，你需要创建注解。创建一个名为org.acme.quick start.JvmLanguage.java的新类：

```
package org.acme.quickstart;

import java.lang.annotation.Documented;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

import javax.validation.Constraint;
import javax.validation.Payload;

@Target({ ElementType.METHOD, ElementType.FIELD, ElementType.ANNOTATION_TYPE,
         ElementType.CONSTRUCTOR, ElementType.PARAMETER, ElementType.TYPE_USE })
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Constraint(validatedBy = { JvmLanguageValidator.class}) ❶
public @interface JvmLanguage {
    String message() default "You need to provide a Jvm based-language";
    Class<?>[] groups() default { };
    Class<? extends Payload>[] payload() default { };
}
```

❶将约束条件作为正常的编译错误抛出

然后你需要创建逻辑来检测约束与否违反。这个新的类必须实现javax.validation.ConstraintValidator接口。

接下来，创建一个名为org.acme.quickstart.JvmLanguageValidator.java的类：

```
package org.acme.quickstart;

import java.util.Arrays;
import java.util.List;

import javax.validation.ConstraintValidator;
import javax.validation.ConstraintValidatorContext;

public class JvmLanguageValidator
    implements ConstraintValidator<JvmLanguage, String> { ❶ ❷

        private List<String> favoriteLanguages = Arrays.asList("java",

    @Override
    public boolean isValid(String value, ConstraintValidatorContext context) {
        return favoriteLanguages.stream()
            .anyMatch(l -> l.equalsIgnoreCase(value)); ❸
    }
}
```

❶上一步定义的注解

❷适用于验证的对象类型

❸检查所提供的最喜欢的语言（value）是否是基于JVM的语言

最后，你需要注解org.acme.quick start.Developer中的favoriteLanguage字段：

```
@JvmLanguage
@NotBlank
private String favoriteLanguage;
```

打开一个新的终端窗口，启动Quarkus应用程序，然后尝试用POST方法发送一些请求：

```
./mvnw clean compile quarkus:dev

curl -d '{"name":"Alexadra","age":7, "favoriteLanguage":"python"}' \
     -H "Content-Type: application/json"
     -X POST http://localhost:8080/developer -v

< HTTP/1.1 400 Bad Request
< Content-Length: 89
< validation-exception: true
< Content-Type: text/plain;charset=UTF-8

curl -d '{"name":"Alexandra","age":5, "favoriteLanguage":"java"}' \
     -H "Content-Type: application/json"
     -X POST http://localhost:8080/developer -v
```

```
< HTTP/1.1 200 OK
< Content-Length: 106
< Content-Type: application/json
<

[{"path":"addDeveloper.developer.favoriteLanguage",
  "message":"You need to provide a Jvm based-language"}]
```

讨论

REST端点、服务方法以及任何CDI作用域内对象上，任何遵循Bean Validation规范的验证都将在应用程序运行期间自动执行。如果你需要更多的控制，请看5.5节，该节介绍了校验对象的其他方法。

同样值得一提的是，默认情况下，违反约束的信息将使用系统语言返回。如果你想改变这个设置，可以在*application.properties*文件中通过设置quarkus.default-locale来实现：

```
quarkus.default-locale=es-ES
```

对于REST端点，本地化将基于Accept-Language HTTP头实现。你可以在application.properties文件中指定支持的语言列表：

```
quarkus.locales=en-US, es-ES
```

另见

关于更多信息，请访问以下网站：

· Jakarta Bean Validation（https://oreil.ly/R6L4d）

· Hibernate Validator（https://oreil.ly/O7BNR）

5.5　以程序化的方式校验对象

问题

如何以程序化的方式校验对象？

解决方案

使用Bean Validation javax.validation.Validator类。

在某些情况下（例如，在non-CDI bean中），你可能想控制校验过程何时被执行。为此，我们提供了javax.validation.Validator类。

让我们创建一个端点，使用javax.validation.Validator校验输入，而不是使用@Valid注解的方式。打开org.acme.quickstart.DeveloperResource.java类并注入Validator实例：

```
@Inject
Validator validator; ❶

@POST
@Path("/programmaticvalidation")
@Consumes(MediaType.APPLICATION_JSON)
@Produces(MediaType.APPLICATION_JSON)
public Response addProgrammaticValidation(Developer developer) { ❷
    Set<ConstraintViolation<Developer>> violations =
      validator.validate(developer); ❸

    if (violations.isEmpty()) { ❹
        developers.add(developer);
        return Response.ok().build();
    } else {
        JsonArrayBuilder errors = Json.createArrayBuilder();
        for (ConstraintViolation<Developer> violation : violations) { ❺
            errors.add(
                Json.createObjectBuilder()
                .add("path", violation.getPropertyPath().toString())
                .add("message", violation.getMessage())
                );
        }

        return Response.status(Response.Status.BAD_REQUEST)
                       .entity(errors.build())
                       .build();
    }
}
```

❶以Bean Validation规范注入Validator类

❷@Valid是不需要的

❸用编程的方式校验对象

❹如果没有错误，则继续

❺如果有错误，则构建错误输出

打开一个新的终端窗口，启动Quarkus应用程序，然后尝试发送一个新的POST方法的请求：

```
./mvnw clean compile quarkus:dev

curl -d '{"name":"Ada","age":7, "favoriteLanguage":"java"}' \
```

```
    -H "Content-Type: application/json" \
    -X POST http://localhost:8080/developer/programmaticvalidation -v

< HTTP/1.1 400 Bad Request
< Content-Length: 89
< validation-exception: true
< Content-Type: text/plain;charset=UTF-8
```

讨论

Quarkus会自动创建一个javax.validation.ValidatorFactory的实例，你可以通过创建自己的bean来替换这个实例。在应用程序中，以下类型的实例将被自动注入ValidatorFactory中：

- javax.validation.ClockProvider
- javax.validation.ConstraintValidator
- javax.validation.ConstraintValidatorFactory
- javax.validation.MessageInterpolator
- javax.validation.ParameterNameProvider

· javax.validation.TraversableResolver

· org.hibernate.validator.spi.properties.GetterPropertySelectionStrategy

· org.hibernate.validator.spi.scripting.ScriptEvaluatorFactory

在前面的列表中，可能只有一个特定类型的实例，而且类应该被声明为@ApplicationScoped。

5.6 依赖注入

问题

如何在类中注入依赖？

解决方案

使用上下文和依赖注入（Contexts and Dependency Injection，CDI）。

讨论

Quarkus中的依赖注入（DI）基于标准的Contexts and Dependency Injection 2.0规范（https://oreil.ly/VcDnN），对基本的用例做了一些简单的修改。

Quarkus实现了大部分的规范，除了一些不会影响你的代码实现的边缘情况。Quarkus网站维护了一个支持的功能和限制的列表，包括本书没有涉及的更高级的功能。你可以在Quarkus CDI参考指南（https://oreil.ly/-LPAd）中找到这些列表。

注入的发生与你在其他使用CDI的应用程序中预期的一样：

```
package org.acme.quickstart;

import javax.inject.Inject;
import javax.ws.rs.GET;
import javax.ws.rs.Path;
import javax.ws.rs.Produces;
import javax.ws.rs.core.MediaType;

@Path("/hello")
public class GreetingResource {
    @Inject                          ❶
    GreetingService service;         ❷
    @GET
    @Produces(MediaType.TEXT_PLAIN)
    public String hello() {
        return service.getGreeting();
    }
```

❶需要使用@Inject注解

❷由于反射的限制，首选包私有注入字段

注入的服务很标准，没有任何特殊的地方：

```
package org.acme.quickstart;

import java.util.Locale;

import javax.enterprise.context.ApplicationScoped;
import javax.inject.Inject;
import javax.inject.Named;

@ApplicationScoped                    ❶
public class GreetingService {
    public String getGreeting() {
        return "Hello";
    }
}
```

❶下面会提到，为了让这个类能被找到，你应该使用bean定义的注解Quarkus中的bean发现遵循标准CDI的简化过程。简而言之，如果你

的应用程序类中没有一个bean定义的注解（https://oreil.ly/jm4QF），Quarkus将不会发现它们。

另见

要了解更多信息，请参见以下网页。

· JBoss:JSR 365:Contexts and Dependency Injection for Java 2.0（https://oreil.ly/clOD4）

· GitHub:GraalVM Native Image Compatibility and Optimization Guide（https://oreil.ly/7sgPm）

5.7　创建工厂类

问题

如何为一个对象创建一个工厂类？

解决方案

使用CDI中的javax.enterise.inject.Produces概念。

CDI有一个叫作producer的概念，它允许你创建任何类型的对象，需要在可解析的实例列表中添加一个新的bean或类，如下所示：

```
package org.acme.quickstart;

import java.util.Locale;

import javax.enterprise.context.ApplicationScoped;
import javax.enterprise.inject.Produces;
import javax.inject.Named;

@ApplicationScoped
public class LocaleProducer {
    @Produces
    public Locale getDefaultLocale() {
        return Locale.getDefault();
    }
}
```

讨论

Quarkus通过添加@io.quarkus.arc.DefaultBean注解将producer的概念又扩展了一步。对CDI来说，这就像一个已启用的默认替代方案，因为Quarkus不允许使用其他替代方案，所以使用DefaultBean注

解的类为你提供了一种创建bean默认实例的方法。下面是从Quarkus网站上截取的代码示例：

```
@Dependent
public class TracerConfiguration {

    @Produces
    public Tracer tracer(Reporter reporter, Configuration configuration) {
        return new Tracer(reporter, configuration);
    }

    @Produces
    @DefaultBean
    public Configuration configuration() {
        // create a Configuration
    }

    @Produces
    @DefaultBean
    public Reporter reporter(){
        // create a Reporter
    }
}
```

下面的代码片段允许你的应用程序或库在任何需要的地方注入一个tracer。你也可以通过创建一个新的producer来定制它：

```
@Dependent
public class CustomTracerConfiguration {

    @Produces
    public Reporter reporter(){
        // create a custom Reporter
    }
}
```

在你的应用程序中使用这段代码后，从CustomTracerConfiguration创建的Reporter就会在你的应用程序中被使用，而不会使用默认的。

另见

要了解更多信息，请访问以下网页。

· JBoss:JSR 365:Contexts and Dependency Injection for Java 2.0（https://oreil.ly/4-OrV）

5.8　执行对象生命周期事件

问题

如何在创建或销毁对象前后执行一些逻辑？

解决方案

CDI使用@javax.annotation.PostConstruct和@javax.annotation.PreDestroy注解进行对象生命周期管理。对于PostConstruct来说，使用这些注解的方法将在对象创建之后被调用；对于PreDestroy来说，使用这些注解的方法在对象被销毁之前被调用：

```
package org.acme.quickstart;

import java.util.Arrays;
import java.util.List;

import javax.annotation.PostConstruct;
import javax.annotation.PreDestroy;
import javax.enterprise.context.ApplicationScoped;

@ApplicationScoped
public class RecommendationService {
    List<String> products;

    @PostConstruct
    public void init() {
        products = Arrays.asList("Orange", "Apple", "Mango");
        System.out.println("Products initialized");
    }

    @PreDestroy
    public void cleanup() {
        products = null;
        System.out.println("Products cleaned up");
    }

    public List<String> getProducts() {
        return products;
    }
}
```

讨论

如果一段逻辑代码需要在调用构造函数后，并且在所有的注入发生后执行，那么它应该放在一个用@PostConstruct注解的方法中，这样保证了在一个对象实例的生命中只被调用一次。

同样，如果需要在对象被销毁之前执行逻辑代码，则将其放入一个用@PreDestroy注解的方法中。可能包括关闭连接，清理资源，以及完成最终的日志打印。

另见

要了解更多信息，请参见GitHub上的页面：

· Common Annotations API:PostConstruct.java（https://oreil.ly/UxdG2）

· Common Annotations API:PreDestroy.java（https://oreil.ly/qsZUC）

5.9　执行应用程序生命周期事件

问题

如何在应用程序启动或关闭后执行一些逻辑？

解决方案

观察io.quarkus.runtime.StartupEvent和io.quarkus.runtime.ShutdownEvent事件：在应用程序启动时，Quarkus会产生StartupEvent事件；而在关闭时，会产生ShutdownEvent事件，例如：

```
package org.acme.quickstart;

import javax.enterprise.context.ApplicationScoped;
import javax.enterprise.event.Observes;
import io.quarkus.runtime.ShutdownEvent;
import io.quarkus.runtime.StartupEvent;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

@ApplicationScoped                                          ❶
public class ApplicationEventListener {
    private static final Logger LOGGER =
            LoggerFactory.getLogger(ApplicationEventListener.class);

    void onStart(@Observes StartupEvent event) {            ❷
        LOGGER.info("Application starting...");
    }

    void onStop(@Observes ShutdownEvent event) {            ❸
        LOGGER.info("Application shutting down...");
    }

}
```

❶必须添加一个bean定义的注解

❷启动事件

❸关闭事件

这两个事件对象中都没有携带任何额外的信息，所以没有任何其他的内容显示。

讨论

在Quarkus（和其他CDI框架）中，事件观察是一种非常强大的、用最小的开销来解耦逻辑的方式。

另见

详见5.8节。

5.10　使用命名限定符

问题

如何用名字限定一个注解？

解决方案

使用@javax.inspit.Named注解。

在CDI中，限定符（qualifier）是任何定义为@Retention(RUNTIME)并使用@javax.inject.Qualifier的注解。限定符通常被定义在需要使用它们的地方，如@Target({METHOD,FIELD,PARAMETER,TYPE})。

CDI自带了一个有用的限定符：@javax.inject.Named，它的值并不是必需的，但是在没有实际名字的情况下使用@Named是没有意义的。当解析一个注解时，CDI将寻找任何同样包含相同限定符的正确类型的bean。在有@Named的情况下，注解的值部分也必须匹配。

如果你有多个类型实例，但它们不是同一个对象，这个时候@Named就非常有用。CDI并不会考虑对象的实际实例，因为在对象被创

建之前，它是不知道的，而且每次都会有所不同。为了解决这个问题，CDI使用了限定符：

```
@Inject
@Named("en_US")
Locale en_US;


@Inject
@Named("es_ES")
Locale es_ES;

public String getGreeting(String locale) {
    if (locale.startsWith("en"))
        return "Hello from " + en_US.getDisplayCountry();

    if (locale.startsWith("es"))
        return "Hola desde " + es_ES.getDisplayCountry();

    return "Unknown locale";
}
```

讨论

一个完整的生成命名bean的示例：

```
package org.acme.quickstart;

import java.util.Locale;

import javax.enterprise.context.ApplicationScoped;
import javax.enterprise.inject.Produces;
import javax.inject.Named;

@ApplicationScoped
public class LocaleProducer {
    @Produces
    public Locale getDefaultLocale() {
        return Locale.getDefault();
    }
    @Produces
    @Named("en_US")
    public Locale getEnUSLocale() {
        return Locale.US;
    }

    @Produces
    @Named("es_ES")
    public Locale getEsESLocale() {
        return new Locale("es", "ES");
    }
}
```

限定符@Named，虽然功能较弱——这也是CDI试图避免的事情之一——但在集成过程中是一个有用的技巧。我们建议在可能的情况下使用强类型的注解。

另见

更多的信息，请访问以下网页。

· JBoss：注入点的限定符&Named（https://oreil.ly/5NydQ）

5.11　使用自定义限定符

问题

如何用一些其他的限定符来限定一个注解？

解决方案

开发自定义的限定符注解。

在5.10节中，我们介绍了限定符的概念：

```
package org.acme.quickstart;

import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

import javax.inject.Qualifier;

import static java.lang.annotation.ElementType.FIELD;
import static java.lang.annotation.ElementType.METHOD;
import static java.lang.annotation.ElementType.PARAMETER;
import static java.lang.annotation.ElementType.TYPE;

@Qualifier
@Retention(RetentionPolicy.RUNTIME)
@Target({METHOD, FIELD, PARAMETER, TYPE})
public @interface SpainLocale {
}
```

bean的生成如你所想：

```
@Produces
@SpainLocale
public Locale getSpainLocale() {
    return new Locale("es", "ES");
}
```

注入新的限定实例一样简单：

```
@Inject
@SpainLocale
Locale spain;
```

讨论

使用限定符注解是在普通CDI应用程序和Quarkus中使用限定CDI注入的首选方式。

另见

更多的信息，请访问以下网页：

· JBoss:Qualifiers（https://oreil.ly/MOfwa）

5.12　限定和配置注解

问题

如何使用注解来限定和配置依赖？

解决方案

使用producer中的InjectionPoint和限定符注解上的非绑定属性的组合，可以同时限定和配置一个bean。

对于限定符和producer来说，下面是个有趣的例子：

```
package org.acme.quickstart;

import java.lang.annotation.Retention;
import java.lang.annotation.Target;

import javax.enterprise.util.Nonbinding;
import javax.inject.Qualifier;

import static java.lang.annotation.ElementType.TYPE;
import static java.lang.annotation.ElementType.FIELD;
import static java.lang.annotation.ElementType.METHOD;
import static java.lang.annotation.ElementType.PARAMETER;
import static java.lang.annotation.RetentionPolicy.RUNTIME;

@Qualifier
@Retention(RUNTIME)
@Target({TYPE, METHOD, FIELD, PARAMETER})
public @interface Quote {
    @Nonbinding String msg() default "";          (1)
    @Nonbinding String source() default "";
}
```

❶属性被设置为非绑定，因此注入实际上是有效的

一般情况下，限定符的属性在注入时是被考虑的，所以如果属性不匹配，限定对象是不会被注入的：

```
@Produces
@Quote                                                          ❶
Message getQuote(InjectionPoint msg) {
    Quote q = msg.getAnnotated().getAnnotation(Quote.class);    ❷
    return new Message(q.msg(), q.source());                    ❸
}
```

❶producer的默认属性

❷从属性中获取限定实例，以获取配置

❸返回新的对象

使用方法与其他限定符完全相同：

```
@Quote(msg = "Good-bye and hello, as always.", source = "Roger Zelazny")
Message myQuote;
```

另见

更多信息，请访问以下网页：

- JBoss：注入点元数据（https://oreil.ly/BVmV2）

5.13　创建拦截器

问题

如何实现拦截器的功能？

解决方案

跨不同关注点的拦截器是指影响程序运行的一个切面，常用的例子是事务控制，它是一个影响程序中数据使用的操作，必须总是以相同或类似的方式来处理。

创建@javax.init.AroundInvoke和@javax.init.AroundConstruct两个具有相应拦截绑定功能的拦截器。你还能够创建CDI，以便将拦截器编译组合到一个注解中。

首先，使用@javax.interceptor.InterceptorBinding创建一个注解，这将被用来关联实际的拦截器代码，并对任何你希望被拦截的方法或类进行注解：

```
package org.acme.quickstart;
```

```
import java.lang.annotation.Inherited;
import java.lang.annotation.Retention;
import java.lang.annotation.Target;

import javax.interceptor.InterceptorBinding;

import static java.lang.annotation.ElementType.METHOD;
import static java.lang.annotation.ElementType.TYPE;
import static java.lang.annotation.RetentionPolicy.RUNTIME;

@Inherited
@InterceptorBinding
@Retention(RUNTIME)
@Target({METHOD, TYPE})
public @interface LogEvent {
}
```

创建拦截器：

```
package org.acme.quickstart;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

import javax.interceptor.AroundInvoke;
import javax.interceptor.Interceptor;
import javax.interceptor.InvocationContext;

@LogEvent
@Interceptor
public class LogEventInterceptor {
    static List<Event> events = new ArrayList<>();

    @AroundInvoke
    public Object logEvent(InvocationContext ctx) throws Exception {
        events.add(new Event(ctx.getMethod().getName(),
                             Arrays.deepToString(ctx.getParameters())));
        return ctx.proceed();
    }
}
```

这个例子很不实用，但很容易理解。用上面的拦截器注解一个方法或类：

```
@LogEvent
public void executeOrder(Order order) {
    // ...
}
```

每次调用executeOrder方法时，在实际的executeOrder方法被调用之前，执行拦截器中被注解为@javax.interceptor.AroundInvoke的方法，在本例中为logEvent。

讨论

拦截器在Quarkus中使用标准的CDI机制很容易实现，它提供了一种简单的方法来定义和利用应用程序中的重复性动作。

面向切面编程（AOP）已经存在了相当长的时间，确切地说是从1997年开始的，由Gregor Kiczales领导的Xerox PARC的团队创建了cross-cutting和面向切面的编程。有人声称微软的Transaction Server是第一个被广泛采用的AOP实例。最终，Enterprise JavaBeans

发展了AOP切面功能，Java生态系统中的Spring和AspectJ也采用了这种方式。

然而，我们现在讨论的是CDI和Quarkus。Quarkus ArC（Quarkus中的依赖注入风格）使用了相同的概念。

另见

更多信息，请查看以下内容：

· *Essential.NET,Volume 1:The Common Language Runtime*，作者为Don Box和Chris Sells（Addison-Wesley Professional）

· JBoss:Interceptor bindings（https://oreil.ly/QlAGP）

· Stack Overflow:What does ArC mean?（https://oreil.ly/0BpNz）

5.14 行为测试

问题

如何编写行为测试来验证服务连接的正确性，而不是验证服务内部逻辑？

解决方案

Quarkus的测试解决方案基于JUnit 5（https://oreil.ly/bh494），它是Java生态系统中标准的测试工具，并与REST-Assured（http://rest-assured.io）测试框架紧密集成，用于验证RESTful Web API。

使用REST-Assured并不是强制性的，只是一个建议，你也可以使用任何其他框架。

Quarkus测试框架中最重要的部分是一个名为QuarkusTest的注解，当你用它来注解一个测试类时，你就有效地将该测试标记为在Quarkus测试框架内执行，它指示测试遵循以下生命周期：

1.Quarkus应用程序自启动一次。当应用程序已经启动并准备好开始服务请求时，测试执行就开始了。

2.每个测试都是针对这个运行中的实例执行的。

3.Quarkus应用程序停止。

为了将运行测试对性能的影响降到最低，Quarkus应用程序只启动一次，然后所有在测试计划中定义的测试类都会针对这个运行的实例执行，所以应用程序不会因为每个测试类的执行而重新启动。

打开位于*src/test/java*目录下的org.acme.quickstart.GreetingResourceTest.java类：

```
package org.acme.quickstart;

import io.quarkus.test.junit.QuarkusTest;
import org.junit.jupiter.api.Test;
import static io.restassured.RestAssured.given;
import static org.hamcrest.CoreMatchers.is;

@QuarkusTest ❶
public class GreetingResourceTest {

    @Test
    public void testHelloEndpoint() {
        given() ❷
          .when()
          .get("/hello") ❸
          .then() ❹
             .statusCode(200)
             .body(is("hello"));
    }
}
```

❶设置为Quarkus测试

❷REST-Assured静态方法开始测试

❸使用HTTP GET方法向/hello发送请求

❹启动断言部分

你也可以从IDE中运行测试，如图5-1所示：

```
2
3   import io.quarkus.test.junit.QuarkusTest;
4   import org.junit.jupiter.api.Test;
5
6   import static io.restassured.RestAssured.given;
7   import static org.hamcrest.CoreMatchers.is;
8
9   @QuarkusTest
    Run Test | Debug Test | ✓
10  public class HelloResourceTest {
11
12      @Test
        Run Test | Debug Test | ✓
13      public void testHelloEndpoint() {
14          given()
15            .when().get("/hello")
16            .then()
17               .statusCode(200)
18               .body(is("hello"));
19      }
20
21  }
```

图5-1：Visual Studio Code与Java集成

如果你想在终端窗口中运行测试，请执行以下内容：

```
./mvnw clean compile test

[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running org.acme.quickstart.GreetingResourceTest
  INFO  [io.qua.dep.QuarkusAugmentor] (main) Beginning quarkus augmentation
  INFO  [io.qua.resteasy] (build-13) Resteasy running without servlet container.
  INFO  [io.qua.resteasy] (build-13) - Add quarkus-undertow to run Resteasy
                                       within a servlet container
  INFO  [io.qua.dep.QuarkusAugmentor] (main) Quarkus augmentation completed
                                             in 803ms
  INFO  [io.quarkus] (main) Quarkus 1.4.1.Final started in 0.427s.
                            Listening on: http://0.0.0.0:8081 ❶
  INFO  [io.quarkus] (main) Profile test activated. ❷
  INFO  [io.quarkus] (main) Installed features: [cdi, resteasy]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 3.586 s
    - in org.acme.quickstart.GreetingResourceTest
2019-11-06 13:02:43,431 INFO  [io.quarkus] (main) Quarkus stopped in 0.053s
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
```

❶Quarkus在运行测试时，监听8081端口

❷test配置文件被激活

8081端口是测试时使用的默认端口。

讨论

你可以通过将quarkus.http.test-port属性设置为不同的值来改变测试使用的端口：

```
quarkus.http.test-port=8083
```

因为Quarkus提供了与REST-Assured良好的集成，它会自动更新使用的端口，所以这部分不需要额外的配置。

Quarkus也支持在一个随机的端口而不是特定的端口中运行测试，只需将quarkus.http.test-port属性设置为0即可。

```
quarkus.http.test-port=0
```

```
./mvnw clean compile test
```

```
INFO  [io.quarkus] (main) Quarkus 1.4.1.Final started in 0.442s.
                          Listening on: http://0.0.0.0:49661
INFO  [io.quarkus] (main) Profile test activated.
INFO  [io.quarkus] (main) Installed features: [cdi, resteasy]
```

Quarkus支持编写行为测试，行为测试是在不了解服务或验证服务内部的情况下进行的。图5-2显示了行为测试的本质。

在REST API和微服务的情况下，你可以将行为测试理解为一种测试形式，它遵循的模式是：向正在运行的服务实例发送一个请求，并验证响应是否符合预期。

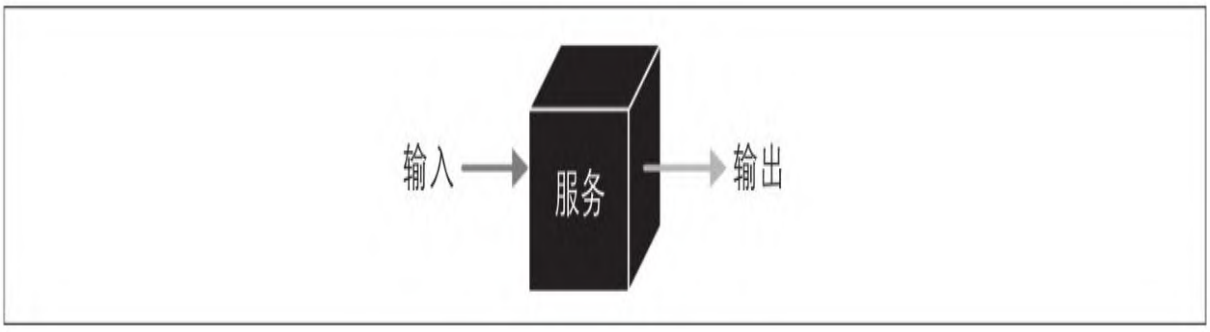


图5-2：行为测试

如果你已经使用第1章中所介绍的方法对项目进行了脚手架开发，应该已经有了一个完整的行为测试，也包括在你的构建工具脚本中注册的依赖。

另见

如果你想了解更多关于Quarkus测试框架使用的底层技术，可以访问以下网站。

- JBoss:JUnit 5 User Guide（https://oreil.ly/oahZK）
- REST-Assured（http://rest-assured.io）

5.15　单元测试

问题

如何用单元测试来验证服务内部的正确性？

解决方案

使用基于JUnit 5（https://oreil.ly/oahZK）的Quarkus测试解决方案及其与CDI的集成方案。

Quarkus允许你通过@Inject注解将CDI bean注入你的测试用例。事实上，在Quarkus中的测试只是一个CDI bean，所有在bean中有效的东西在测试中也是有效的。

让我们创建一个Greeting Service bean，使用Bean Validation来验证它的输入参数。记得添加quarkus-hibernate-validator扩展。创建一个新类org.acme.quickstart.GreetingService.java：

```
package org.acme.quickstart;

import javax.enterprise.context.ApplicationScoped;
import javax.validation.constraints.Min;
@ApplicationScoped ❶
public class GreetingService {

    public String greetingMessage(@Min(value = 16) int age) { ❷
        if (age < 19) {
            return "Hey boys and girls";
        } else {
            return "Hey ladies and gentlemen";
        }
    }

}
```

❶设置为CDI bean

❷在方法中添加验证

现在，在下面的三种情况中测试Greeting Service是否能按预期工作：

- 当用户年龄小于16岁时，会抛出一个异常
- 当用户年龄在16到18岁之间时，返回青少年的结果
- 当用户年龄大于18岁时，返回成人的结果

我们推荐你使用AssertJ（https://oreil.ly/d5tI2）项目来编写断言。要使用它，你需要在构建脚本中注册AssertJ依赖项：

```
<dependency>
  <groupId>org.assertj</groupId>
  <artifactId>assertj-core</artifactId>
  <version>3.14.0</version>
  <scope>test</scope>
</dependency>
```

在src/test/java目录创建一个新类org.acme.quickstart.GreetingService.java：

```
package org.acme.quickstart;

import javax.inject.Inject;
import javax.validation.ConstraintViolationException;

import org.assertj.core.api.Assertions;
import org.junit.jupiter.api.Test;

import io.quarkus.test.junit.QuarkusTest;

@QuarkusTest ❶
public class GreetingServiceTest {

  @Inject ❷
  GreetingService greetingService;

  @Test ❸
  public void testGreetingServiceForYoungers() {

    Assertions.assertThatExceptionOfType(ConstraintViolationException.class) ❹
      .isThrownBy(() -> greetingService.greetingMessage(15));
  }

  @Test
  public void testGreetingServiceForTeenagers() {
    String message = greetingService.greetingMessage(18);
    Assertions.assertThat(message).isEqualTo("Hey boys and girls");
  }

  @Test
  public void testGreetingServiceForAdult() {
    String message = greetingService.greetingMessage(21);
    Assertions.assertThat(message).isEqualTo("Hey female and male");
  }

}
```

❶设置为Quarkus测试

❷注入GreetingService实例

❸使用CDI创建的Greeting Service实例进行测试

❹使用AssertJ断言

打开一个新的终端窗口，并尝试运行测试：

```
./mvnw clean compile test

[INFO] Running org.acme.quickstart.GreetingResourceTest
 INFO  [io.qua.dep.QuarkusAugmentor] (main) Beginning quarkus augmentation
 INFO  [io.qua.resteasy] (build-3) Resteasy running without servlet container.
 INFO  [io.qua.resteasy] (build-3) - Add quarkus-undertow to run Resteasy
                                     within a servlet container
 INFO  [io.qua.dep.QuarkusAugmentor] (main) Quarkus augmentation completed
                                       in 813ms
 INFO  [io.quarkus] (main) Quarkus 1.4.1.Final started in 0.715s.
                   Listening on: http://0.0.0.0:51581
 INFO  [io.quarkus] (main) Profile test activated.
 INFO  [io.quarkus] (main) Installed features:
                         [cdi, hibernate-validator, resteasy]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 3.614 s
         - in org.acme.quickstart.GreetingResourceTest
[INFO] Running org.acme.quickstart.GreetingServiceTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.086 s
         - in org.acme.quickstart.GreetingServiceTest
2019-11-06 16:16:11,503 INFO  [io.quarkus] (main) Quarkus stopped in 0.029s
```

请注意，Quarkus应用程序启动了一次，但两个测试类都被执行。

讨论

5.14节介绍了如何使用行为方法编写测试，在这种方法中，你只关心服务的请求和响应。但当你想验证服务内部发生了什么，或者验证一些组件在运行的实例内部（而不是模拟环境）的行为时，该怎么办呢。当你想验证一个业务对象是否能按预期工作时，通常需要编写单元测试，这包括它与Quarkus提供的功能（Bean Validation、CDI等）的集成。

另见

要了解关于AssertJ的更多信息，请访问以下网页：

· AssertJ:fluent assertions java library（https://oreil.ly/d5tI2）

5.16　创建mock对象

问题

如何测试那些需要额外处理时间或需要与外部系统通信的类？

解决方案

在Quarkus中支持使用mock（模拟）来提供CDI对象，这些对象通过替换默认对象来模拟真实对象的行为。

模拟对象是通过对方法调用提供一些预先设置的结果来模拟真实对象的行为。

让我们来模拟在5.15节中创建的Greeting Service。

在src/test/java目录创建一个新类org.acme.quickstart.MockedGreetingService.java：

```
package org.acme.quickstart;

import io.quarkus.test.Mock;

@Mock ❶

public class MockedGreetingService
    extends GreetingService { ❷

        @Override
        public String greetingMessage(int age) {
            return "Hello World"; ❸
        }

}
```

❶在CDI中把POJO标记为模拟类（替代类）

❷该类必须扩展或实现基础服务

❸预先设置的结果

讨论

创建mock不仅是绕过外部服务和较长的运行进程的好方法，也是测试特定场景的简单方法。在前面的解决方案中，可以有两个测试：一个使用mock，另一个使用实际对象。其中一个将展示服务的预期行为，另一个可以展示预期的失败。这种技术对于测试外部服务故障特别有用。

5.17　使用Mockito创建mock对象

问题

如何使用Mockito测试那些需要额外处理时间或需要与外部系统通信的类？

解决方案

使用Mockito库提供的CDI对象，通过替换默认对象来模仿真实对象的行为。

使用Mockito，让我们模拟5.15节中创建的Greeting Service：

添加Quarkus Mockito扩展：

```
<dependency>
  <groupId>io.quarkus</groupId>
  <artifactId>quarkus-junit5-mockito</artifactId>
  <scope>test</scope>
</dependency>
```

在*src/main/java*目录下创建一个新类

org.acme.quickstart.GreetingResourceTest.java：

```
import io.quarkus.test.junit.mockito.InjectMock;
import org.junit.jupiter.api.BeforeEach;
import static org.mockito.Mockito.when;

@QuarkusTest
public class GreetingResourceTest {

    @InjectMock ❶
    GreetingService greetingService;

    @BeforeEach ❷
    public void prepareMocks() {
        when(greetingService.message())
                .thenReturn("Aloha from Mockito");
    }

    @Test
    public void testHelloEndpoint() {
        given()
          .when().get("/greeting")
          .then()
             .statusCode(200)
             .body(is("Aloha from Mockito")); ❸
    }

}
```

❶InjectMock使该字段成为Mockito的模拟字段

❷每次测试执行前，记录模拟期望值

❸返回的结果是模拟的数据

5.18　用元注解将多个注解组合为一个注解

问题

如何避免在应用程序中出现大量的注解？

解决方案

使用元注解将多个注解组合为一个注解。

你可以开发一个元注解，它包含了测试或应用程序的其他部分所需的所有注解。例如，你可以创建一个TransactionalQuarkusTest注解，它包含了@QuarkusTest和@Transactional注解，如果使用了这个新创建的注解，那么Quarkus测试默认是事务性的测试。

在*src/test/java*目录下创建一个名为org.acme.quickstart.TransactionalQuarkusTest.java的新类：

```
package org.acme.quickstart;

import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

import javax.enterprise.inject.Stereotype;
import javax.transaction.Transactional;

import io.quarkus.test.junit.QuarkusTest;

@QuarkusTest ❶
@Transactional
@Stereotype ❷
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
public @interface TransactionalQuarkusTest {
}
```

❶添加该元注解可能“继承”的注解

❷将注解设置为元注解

如果你再将这个注解应用到一个类上，也就是意味着你同时应用了@QuarkusTest和@Transactional注解：

```
@TransactionalQuarkusTest
public class DeveloperDAO {
}
```

请注意，现在的测试可读行更强，注解也是可重用的。

讨论

Quarkus测试是CDI bean，因此，你可以应用CDI拦截器。例如，你可以通过使用事务拦截器interceptor让你的测试成为事务性的。这个拦截器是通过用@javax.transaction.Transactional注解测试类来启用的。因此，一个事务性测试可能看起来像下面这样：

```
@QuarkusTest
@Transactional
public class DeveloperDAO {
}
```

当然，这是完全有效的，但有两种情况，类的多个注解可能会影响测试的可读性：

1.测试需要很多注解。例如，像@TestMethodOrder这样的JUnit 5注解来定义测试的执行顺序，或者你需要为测试启用其他拦截器。在这些情况下，你最终可能会设置比代码更多的注解。

2.在大多数情况下，很多测试需要相同的注解，所以你不断地用相同的注解来注解所有的测试。

5.19　在测试代码前后执行代码

问题

如何在测试代码前后执行一些逻辑来启动/停止/配置测试的resource?

解决方案

使用Quarkus Test Resource扩展机制来定义执行测试所需的resource。

Quarkus提供了一个扩展机制，允许你在测试用例开始之前和测试用例结束之后执行Java代码。此外，它允许你以编程的方式创建和覆盖配置属性，这样resource所需的任何参数都可以在测试resource类中设置，而不用修改*application.properties*文件。

让我们写一个简单的Quarkus Test Resource，它只是打印一些消息：

在*src/test/java*目录下创建一个新类org.acme.quickstart.HelloWorldQuarkusTestResource-LifecycleManager，实现

io.quarkus.test.common.QuarkusTestResourceLifecycleMa-nager接口：

```
package org.acme.quickstart;

import java.util.Collections;
import java.util.Map;

import io.quarkus.test.common.QuarkusTestResourceLifecycleManager;

public class HelloWorldQuarkusTestResourceLifecycleManager
    implements QuarkusTestResourceLifecycleManager { ❶

    @Override
    public Map<String, String> start() { ❷
        System.out.println("Start Test Suite execution");
        return Collections.emptyMap(); ❸
    }

    @Override
    public void stop() { ❹
        System.out.println("Stop Test Suite execution");
    }

    @Override
    public void inject(Object testInstance) { ❺
        System.out.println("Executing " + testInstance.getClass().getName());
    }

    @Override
    public int order() { ❻
        return 0;
    }

}
```

❶必须实现QuarkusTestResourceLifecycleManager

❷测试用例开始之前执行的方法

❸带有系统属性的Map对象

❹测试用例执行完成后执行的方法

❺每执行一个测试类，都会调用这个方法，传递测试实例，这样就可以注入特定的字段

❻设定在定义了多个resource的情况下的执行顺序

最后，你需要注册这个扩展，以便在测试套件执行期间执行。为此，你需要在*src/test/java*目录下的所有类中使用QuarkusTestResource注解，并设置测试resource的启动。虽然它可以是任何负责注册resource的测试类，但我们建议你创建一个特定的空类来注册测试resource。

在*src/test/java*中创建一个新的类org.acme.quickstart.HelloWorldTestResource，内容如下：

```
package org.acme.quickstart;

import io.quarkus.test.common.QuarkusTestResource;

@QuarkusTestResource(HelloWorldQuarkusTestResourceLifecycleManager.class) ❶
public class HelloWorldTestResource {
}
```

❶注册测试resource

然后在终端中运行测试，你会看到终端输出以下类似内容：

```
./mvnw clean compile test

INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running org.acme.quickstart.GreetingResourceTest
Start Test Suite execution ❶
 INFO  [io.qua.dep.QuarkusAugmentor] (main) Beginning quarkus augmentation
 INFO  [io.qua.dep.QuarkusAugmentor] (main) Quarkus augmentation completed
                                            in 756ms
 INFO  [io.quarkus] (main) Quarkus 1.4.1.Final started in 0.381s.
                   Listening on: http://0.0.0.0:8081
 INFO  [io.quarkus] (main) Profile test activated.
 INFO  [io.quarkus] (main) Installed features: [cdi, resteasy]
Executing org.acme.quickstart.GreetingResourceTest ❷
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 3.058 s
       - in org.acme.quickstart.GreetingResourceTest
Stop Test Suite execution ❸
2019-11-08 16:57:01,020 INFO  [io.quarkus] (main) Quarkus stopped in 0.027s
```

❶在Quarkus启动前调用start方法

❷在GreetingResourceTest运行前调用inject方法

❸在所有测试执行完毕后调用stop方法

这个例子可以让你了解测试的生命周期。

讨论

随着继续开发，测试（集成测试、端到端测试等）的复杂性以及运行这些测试所需的依赖也会增加。例如，测试可能需要一个数据库实例、一个Kafka代理、一个JMS队列，或者一个像Keycloak这样的身份提供者。

以此为背景，让我们写一个更有趣的测试resource，使用Docker来启动MariaDB Docker容器。

在这个例子中，你将使用Testcontainers（https://oreil.ly/aEeAV）测试框架。在撰写本书时，其最新的版本是1.14.3。

关于测试容器

Testcontainers是一个支持JUnit测试的Java库，它提供了轻量级的、可随时销毁的Docker容器实例。Testcontainers管理每个容器的生命周期，因此你可以以编程的方式来启动和停止它们。

你可以在以下测试场景中使用Docker容器：

· 数据访问层集成测试，其中需要将数据库服务器（如PostgreSQL、MySQL或Oracle）容器化。

· 需要一些外部依赖性的应用集成测试，如Web服务器或Kafka等消息系统。

· UI/验收测试，在这里你可以将网络浏览器容器化，这样每个测试都可以在浏览器的新实例中运行。

在运行测试之前，你需要在机器上安装Docker，以便Testcontainers可以在本地启动MariaDB容器。

开发Testcontainers Quarkus Test Resource的第一步是在构建工具中注册Testcontainers依赖项，以便在测试中使用MariaDB Docker容器：

```
<dependency>
  <groupId>org.testcontainers</groupId>
  <artifactId>mariadb</artifactId>
  <version>${testcontainers.version}</version>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>io.quarkus</groupId>
  <artifactId>quarkus-jdbc-mariadb</artifactId>
</dependency>
```

QuarkusTestResourceLifecycleManager的实现如下：

```
package org.acme.quickstart;

import java.util.HashMap;
import java.util.Map;

import org.acme.quickstart.MariaDbTestResource.Initializer;
import org.testcontainers.containers.MariaDBContainer;

import io.quarkus.test.common.QuarkusTestResource;
```

```
import io.quarkus.test.common.QuarkusTestResourceLifecycleManager;

@QuarkusTestResource(Initializer.class) ❶
public class MariaDbTestResource {

  public static class Initializer
      implements QuarkusTestResourceLifecycleManager { ❷

    private MariaDBContainer mariaDBContainer; ❸

    @Override
    public Map<String, String> start() {

      this.mariaDBContainer = new MariaDBContainer<>("mariadb:10.4.4"); ❹
      this.mariaDBContainer.start();❺

      return getConfigurationParameters();
    }

    private Map<String, String> getConfigurationParameters() { ❻
      final Map<String, String> conf = new HashMap<>();

      conf.put("quarkus.datasource.url", this.mariaDBContainer.getJdbcUrl());
      conf.put("quarkus.datsource.username", this.mariaDBContainer
                                                 .getUsername());
      conf.put("quarkus.datasource.password", this.mariaDBContainer
                                                   .getPassword());
      conf.put("quarkus.datasource.driver", this.mariaDBContainer
                                                 .getDriverClassName());

      return conf;
    }

    @Override
    public void stop() {
      if (this.mariaDBContainer != null) {
        this.mariaDBContainer.close(); ❼
      }
    }
  }
}
```

❶注册test resource

❷定义test resource接口

❸设置MariaDB容器对象

❹用所需的Docker镜像实例化MariaDB容器

❺启动容器，并等待容器接受连接

❻覆盖Quarkus的配置，将数据库连接指向容器

❼停止容器

最后，在终端中运行测试，你会在终端输出中看到以下内容：

```
./mvnw clean test

[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running org.acme.quickstart.GreetingResourceTest
       ℹ Checking the system...
       ✓ Docker version should be at least 1.6.0
       ✓ Docker environment should have more than 2GB free disk space
Start Test Suite execution
 INFO  [org.tes.doc.DockerClientProviderStrategy] (main)
       Loaded org.testcontainers.dockerclient
                    .UnixSocketClientProviderStrategy
       from ~/.testcontainers.properties, will try it first
...
 INFO  [□□4.4]] (main) Creating container for image: mariadb:10.4.4 ❶
 INFO  [□□4.4]] (main) Starting container with ID:
       0d07d45111b1103fd7e64ac2050320ee329ca14eb46a72d525f61bc5e433dc69
 INFO  [□□4.4]] (main) Container mariadb:10.4.4 is starting:
       0d07d45111b1103fd7e64ac2050320ee329ca14eb46a72d525f61bc5e433dc69
 INFO  [□□4.4]] (main) Waiting for database connection to become available at
       jdbc:mariadb://localhost:32773/test using query 'SELECT 1'
 INFO  [□□4.4]] (main) Container is started
       (JDBC URL: jdbc:mariadb://localhost:32773/test)
 INFO  [□□4.4]] (main) Container mariadb:10.4.4 started ❷
 INFO  [io.qua.dep.QuarkusAugmentor] (main) Beginning quarkus augmentation
 INFO  [io.qua.dep.QuarkusAugmentor] (main) Quarkus augmentation completed
                                                 in 1461ms
 INFO  [io.quarkus] (main) Quarkus 1.4.1.Final started in 0.909s.
                      Listening on: http://0.0.0.0:8081
 INFO  [io.quarkus] (main) Profile test activated.
 INFO  [io.quarkus] (main) Installed features: [cdi, jdbc-mariadb, resteasy]
Executing org.acme.quickstart.GreetingResourceTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 32.666 s
       - in org.acme.quickstart.GreetingResourceTest
Stop Test Suite execution
2019-11-12 11:57:27,758 INFO  [io.quarkus] (main) Quarkus stopped in 0.043s
```

❶创建MariaDB Docker容器

❷容器已经启动并运行，可以接收传入的请求

你可以在一个单独的项目中开发Quarkus测试resources，并将它们打包成一个外部JAR文件，然后你就可以根据需要在多个项目中重复使用它们。

Quarkus提供了以下默认的Quarkus测试Resource实现：

H2DatabaseTestResource

在服务器模式下启动和停止H2数据库

DerbyDatabaseTestResource

在服务器模式下启动和停止Derby数据库

InfinispanEmbeddedTestResource

在嵌入式模式下启动和停止Infinispan

InfinispanServerTestResource

在服务器模式下启动和停止Infinispan

KeycloakTestResource

启动和停止keycloak身份提供商

ArtemisTestResource

启动和停止Embedded ActiveMQ

KafkaTestResource

使用Debezium类启动和停止Kafka集群

KubernetesMockServerTestResource

启动和停止Kubernetes Mock服务器

另见

你可以了解关于如何在本地机器上安装Docker的更多信息，并在下面找到更多关于Testcontainers测试框架的例子：

- Docker:Get Docker（https://oreil.ly/WrfY8）
- Testcontainers（https://oreil.ly/aEeAV）

5.20　测试原生可执行文件

问题

如何测试原生可执行文件是否正确？

解决方案

使用NativeImageTest注解从原生文件（而不是使用JVM）启动应用程序。

如果你计划制作一个应用程序的原生可执行文件，那么针对运行在原生可执行文件中的应用程序编写一些行为测试是一个不错的主意。

Quarkus提供了NativeImageTest注解，用于从原生文件启动应用程序。需要注意的是，你必须在运行测试之前生成原生可执行文件，或者在运行测试之前使用quarkus.package.type系统属性生成原生可执行文件。你可以在6.4节中了解关于如何生成原生可执行文件的更多信息。

如果项目是使用前面示例中的方法来创建脚手架的话，那么已经提供了一个原生可执行文件测试。

不可能在同一个测试组件中混合JVM和原生可执行文件测试。JVM测试必须在与原生测试不同的周期中运行（例如，在Maven中，对JVM测试来说是surefire，对原生可执行文件测试来说是failsafe）。

当项目通过前面示例中方法进行脚手架设计时，默认会提供这种配置：

```
<profile>
  <id>native</id>
  <activation>
    <property>
      <name>native</name>
    </property>
  </activation>
  <build>
    <plugins>
      <plugin>
        <artifactId>maven-failsafe-plugin</artifactId>
        <version>${surefire-plugin.version}</version>
        <executions>
          <execution>
            <goals>
              <goal>integration-test</goal>
              <goal>verify</goal> ❶
            </goals>
            <configuration>
              <systemProperties>
                <native.image.path>
                  ${project.build.directory}/
                  ${project.build.finalName}-runner
                </native.image.path> ❷
              </systemProperties>
            </configuration>
          </execution>
        </executions>
      </plugin>
    </plugins>
  </build>
```

```
  <properties>
    <quarkus.package.type>
        native
    </quarkus.package.type> ❸
  </properties>
</profile>
```

❶运行原生测试（./mvnw verify）

❷产生的可执行文件的位置（本行不应该在点号后面换行）

❸在运行测试之前，产生一个原生可执行文件。

打开位于*src/test/java*目录下的org.acme.quickstart.NativeGreetingResourceIT.java类：

```
package org.acme.quickstart;

import io.quarkus.test.junit.NativeImageTest;

@NativeImageTest ❶
public class NativeGreetingResourceIT
    extends GreetingResourceTest { ❷

    // Execute the same tests but in native mode.
}
```

❶启动位于native.image.path的可执行文件。

❷扩展JVM测试，使其针对可执行文件运行（这不是必需的），你可以写你的测试，但记得用@QuarkusTest注解它们。这将会对原生可执行文件运行相同的测试。

本节中显示的所有测试功能都是有效的，除了注入测试。

讨论

编写原生测试时需要考虑一些问题：

1.Quarkus会等待60秒，等待原生图片启动后才会自动失败原生测试。这个持续时间可以使用quarkus.test.native-image-wait-time属性来改变（即./mvnw verify -Pnative -Dquarkus.test.native-image-wait-time=200）。

2.原生测试运行在prod profile上，如果你想改变这个profile，可以使用quarkus.test.native-image-profile属性设置一个替代profile。

3.通过使用io.quarkus.test.junit.DisabledOnNative Image注解（即@DisabledOnNativeImage @Test public void nonNativeTest(){}），你可以禁用某些测试方法（或类），来使它们只能在原生测试中运行。

第6章　打包Quarkus应用程序

在本章中，你将学习如何将Quarkus应用程序打包为原生可执行文件或JVM可运行的文件，从而使其能够被分发和部署。如今，容器已成为应用程序发布的标准方式，所以你需要知道如何将Quarkus应用程序容器化。

本章涵盖以下主题：

- 如何将Quarkus应用程序打包为JVM可运行的文件
- 如何将Quarkus应用程序打包为原生可执行文件
- 如何将Quarkus应用程序容器化

6.1　在命令行模式下运行

问题

如何创建一个CLI应用程序？

解决方案

使用Quarkus，你可以编写在命令行模式下运行并选择何时退出的应用程序。

要在Quarkus中启用命令行模式，需要创建一个实现了io.quarkus.runtime.QuarkusApplication接口的类：

```
package org.acme.quickstart;

import io.quarkus.runtime.Quarkus;
import io.quarkus.runtime.QuarkusApplication;
```

```
public class GreetingMain implements QuarkusApplication { ❶

    @Override
    public int run(String... args) throws Exception { ❷
        System.out.println("Hello World");
        Quarkus.waitForExit(); ❸
        return 0;
    }

}
```

❶QuarkusApplication接口设置在命令行模式下运行

❷调用main方法时执行

❸等到Quarkus进程停止后退出

然后你就可以实现著名的Java main方法，其中一个要求是：有main方法的类必须用@io.quarkus.runtime.annotations.QuarkusMain来注解：

```
package org.acme.quickstart;

import io.quarkus.runtime.Quarkus;
import io.quarkus.runtime.annotations.QuarkusMain;

@QuarkusMain ❶
public class JavaMain {

    public static void main(String... args) {
        Quarkus.run(GreetingMain.class, args); ❷
    }

}
```

❶将该类设置为main

❷启动进程

如果想使用命令参数，可以使用@io.quarkus.runtime.annotations.CommandLine-Arguments注解：

```
package org.acme.quickstart;

import javax.ws.rs.GET;
import javax.ws.rs.Path;
import javax.ws.rs.Produces;
import javax.ws.rs.core.MediaType;

import io.quarkus.runtime.annotations.CommandLineArguments;

@Path("/hello")
public class GreetingResource {
```

```
    @CommandLineArguments ❶
    String[] args;

    @GET
    @Produces(MediaType.TEXT_PLAIN)
    public String hello() {
        return args[0];
    }
}
```

❶注入命令行参数

最后，可以创建项目并运行它：

```
./mvnw clean package -DskipTests

java -jar target/greeting-started-cli-1.0-SNAPSHOT-runner.jar Aloha

curl localhost:8080/hello
Aloha
```

讨论

可以使用两种不同的方法来实现退出应用程序。前文解释了第一种方法；第二种方法是用@io.quarkus.runtime.annotations.QuarkusMain annotation来注解实现io.quarkus.runtime.QuarkusApplication接口的类。

第二种方法的缺点是你不能从IDE中运行它，因此建议你使用第一种方法。

如前所示，如果想在启动时运行一些逻辑，然后像正常的应用程序一样运行它（即不退出），那么应该从主线程中调用Quarkus.waitForExit。如果不调用这个方法，那么Quarkus应用程序就会被启动，然后被终止，所以你的应用程序实际上与其他CLI程序一样。

6.2　创建一个可运行的JAR文件

问题

如何创建一个可运行的JAR文件，并将其分发到安装了JVM的机器上？

解决方案

使用Quarkus Maven插件来创建一个可运行的JAR文件。

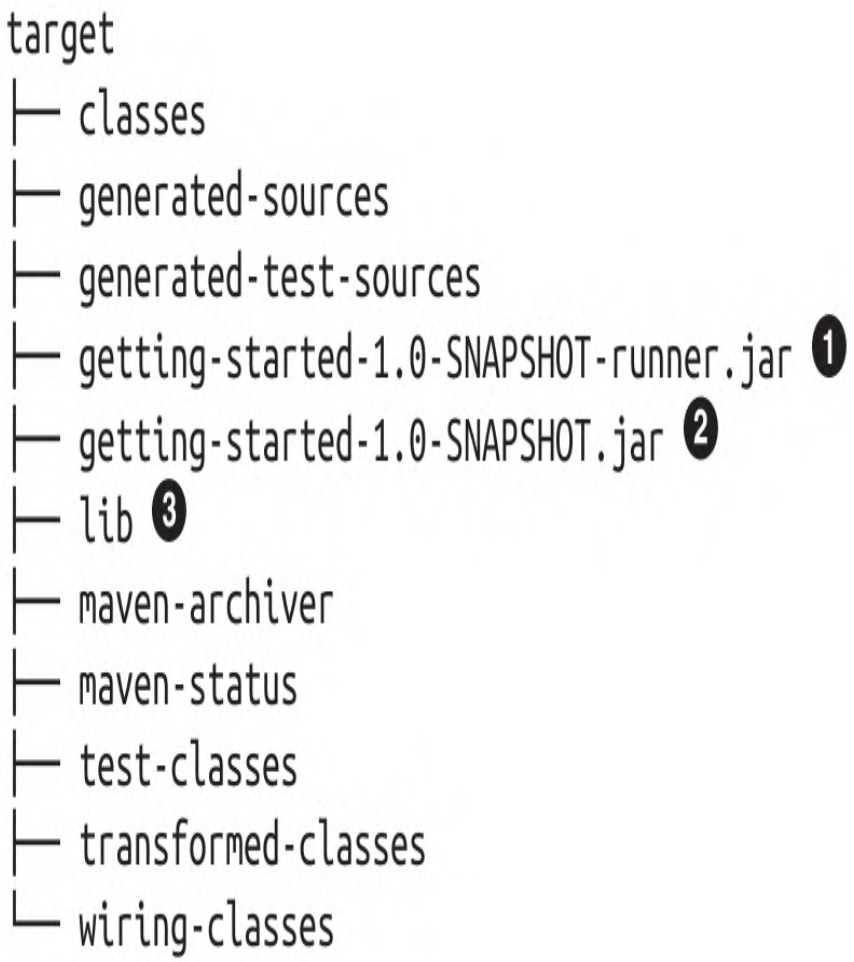
如果你已经用脚手架生成了项目，那么Quarkus Maven插件是默认安装的。

```
<plugin>
  <groupId>io.quarkus</groupId>
  <artifactId>quarkus-maven-plugin</artifactId>
  <version>${quarkus.version}</version>
  <executions>
    <execution>
      <goals>
        <goal>build</goal>
      </goals>
    </execution>
  </executions>
</plugin>
```

然后运行package来建立JAR文件：

```
./mvnw clean package
```

target目录包含以下内容：



❶可执行的JAR文件（不是über-JAR）

❷依赖项的位置

❸包含应用程序依赖项的库文件夹

如果你想部署应用程序，需要把可执行JAR文件与lib目录复制到一个地方。

然后就可以通过运行下一个命令来运行应用程序：

```
java -jar target/getting-started-1.0-SNAPSHOT-runner.jar
```

以这种方式运行Quarkus被称为在JVM模式下运行Quarkus。这意味着你并没有产生一个原生编译，而是在JVM中运行你的应用程序。

如果你想在JVM模式下将Quarkus应用程序打包到一个容器中，推荐使用这种方法，因为在容器构建阶段创建的层次会被缓存起来，以便以后重用。库通常不会改变，所以这个依赖层可能会在未来的执行中被多次重用，从而加快容器的构建速度。

讨论

要用Gradle创建一个可运行的JAR文件，可以运行quarkusBuild任务：

```
./gradlew quarkusBuild
```

另见

如果你对如何创建über-JAR或如何将Quarkus应用程序容器化感兴趣，请看6.3节。

6.3 über-JAR打包

问题

如何为Quarkus应用程序创建一个über-JAR包？

解决方案

Quarkus Maven插件支持通过在*pom.xml*中指定一个uber Jar配置选项来生成über-JAR。

要创建一个über-JAR（包含你的代码可运行类和所有必需依赖项的JAR），需要在*application.properties*文件中通过将quarkus.package.uber-jar设置为true来配置Quarkus：

```
quarkus.package.uber-jar=true
```

6.4　构建原生可执行文件

问题

如何把Quarkus应用程序构建成一个原生可执行文件？

解决方案

使用Quarkus和GraalVM构建原生可执行文件是容器和Serverless工作负载的理想选择。

原生可执行文件使Quarkus应用程序成为容器和Serverless工作负载的理想选择。Quarkus依靠GraalVM（https://www.graalvm.org）将Java应用程序构建为原生可执行文件。

在构建原生可执行文件之前，请确保GRAALVM_HOME环境变量设置为GraalVM 19.3.1或20.0.0的安装目录。

如果你使用的是macOS，这个变量应该指向*Home*子目录：export GRAALVM_HOME=<installation_dir>/Development/graalvm/Contents/Home/。

当生成一个Quarkus项目时，会注册一个默认的Maven配置文件（名称为native），可以用它来构建一个Quarkus原生可执行应用程序：

```
<profile>
  <id>native</id>
  <activation>
    <property>
      <name>native</name>
    </property>
  </activation>
  <build>
    <plugins>
      <plugin>
        <artifactId>maven-failsafe-plugin</artifactId>
        <version>${surefire-plugin.version}</version>
        <executions>
          <execution>
            <goals>
              <goal>integration-test</goal>
              <goal>verify</goal>
            </goals>
            <configuration>
              <systemProperties>
                <native.image.path>
                  ${project.build.directory}/${project.build.finalName}-runner
                </native.image.path>
              </systemProperties>
            </configuration>
          </execution>
        </executions>
      </plugin>
    </plugins>
  </build>
  <properties>
    <quarkus.package.type>native</quarkus.package.type>
  </properties>
</profile>
```

需要在构建项目时启用native profile：

```
./mvnw package -Pnative
```

之后，target目录中会出现一个原生可执行文件：

```
[INFO] --- quarkus-maven-plugin:1.4.1.Final:native-image (default) @
       getting-started ---
[INFO] [io.quarkus.creator.phase.nativeimage.NativeImagePhase] Running Quarkus
       native-image plugin on Java HotSpot(TM) 64-Bit Server VM
...
[getting-started-1.0-SNAPSHOT-runner:19]    classlist:  13,614.07 ms
[getting-started-1.0-SNAPSHOT-runner:19]        (cap):   2,306.78 ms
[getting-started-1.0-SNAPSHOT-runner:19]        setup:   4,793.43 ms
...
Printing list of used packages to
  /project/reports/used_packages_getting-started-1.0-SNAPSHOT-
                                  runner_20190927_134032.txt
[getting-started-1.0-SNAPSHOT-runner:19]    (compile):  42,452.12 ms
[getting-started-1.0-SNAPSHOT-runner:19]      compile:  62,356.07 ms
[getting-started-1.0-SNAPSHOT-runner:19]        image:   2,939.16 ms
[getting-started-1.0-SNAPSHOT-runner:19]        write:     696.65 ms
[getting-started-1.0-SNAPSHOT-runner:19]      [total]: 151,743.29 ms

target/getting-started-1.0-SNAPSHOT-runner
```

讨论

要在Gradle中构建一个原生可执行文件，可以使用buildNative任务：

```
./gradlew buildNative
```

6.5　为JAR文件构建一个Docker容器

问题

如何为6.2节中构建的JAR文件构建一个容器？

解决方案

使用提供的*Dockerfile.jvm*文件来构建容器。

生成Quarkus项目时，会在*src/main/docker*中创建两个Dockerfile：一个用于在JVM模式下使用Quarkus生成Docker容器，另一个用于原生可执行文件。

要生成一个在JVM（没有原生的）模式下运行Quarkus的容器，可以使用*Dockerfile.jvm*文件来构建容器。这个Dockerfile添加了*lib*目录和可运行的JAR，并公开了JMX。

要构建Docker镜像，需要按照6.2节所示的方法打包项目，然后构建容器：

```
./mvnw clean package
```

```
docker build -f src/main/docker/Dockerfile.jvm -t example/greetings-app
```

容器可以通过运行以下程序来启动：

```
docker run -it --rm -p 8080:8080 example/greetings-app
```

6.6　为原生可执行文件构建一个Docker容器

问题

如何建立一个原生可执行文件的容器镜像？

解决方案

要生成一个用于运行Quarkus原生可执行文件的容器，可以使用Dockerfile.native文件来构建容器。

要构建Docker镜像，需要创建一个可以在Docker容器中运行的原生文件。出于这个原因，不要使用本地GraalVM来构建原生可执行文件，因为结果文件将特定于你的操作系统，无法在容器内运行。

要创建一个能在容器中运行的可执行文件，请在终端中使用以下命令：

```
./mvnw clean package -Pnative -Dquarkus.native.container-build=true
```

该命令创建一个包含GraalVM的Docker镜像，从你的代码中生成一个64位Linux可执行文件。

需要在*pom.xml*中定义native profile，详见6.4节。

用上一步生成的原生可执行文件构建Docker镜像：

```
docker build -f src/main/docker/Dockerfile.native -t example/greetings-app
```

然后可以运行以下命令来启动容器：

```
docker run -it --rm -p 8080:8080 example/greetings-app
```

讨论

默认情况下，Quarkus使用docker来构建容器。可以使用quarkus.native.container-runtime属性来改变容器运行时。在编写本书时，支持的选项是docker和podman：

```
./mvnw package -Pnative -Dquarkus.native.container-build=true \
               -Dquarkus.native.container-runtime=podman
```

6.7　构建和容器化原生SSL应用程序

问题

如何在构建一个原生可执行文件时保证连接的安全，以防止攻击者窃取敏感信息？

解决方案

启用Quarkus，在原生可执行文件中使用SSL来确保连接安全。

如果你在JVM模式下运行Quarkus应用程序，启用SSL支持没有任何问题。但是在原生可执行文件的情况下，SSL并不是开箱即用的，必须执行一些额外的步骤（特别是在容器化应用程序时）来启用SSL支持。

我们通过在*application.properties*添加quarkus.ssl.native配置属性来启用Quarkus对原生可执行文件的SSL支持：

```
quarkus.ssl.native=true
```

启用此属性可使GraalVM native-image进程启用SSL，使用以下命令创建原生可执行文件：

```
./mvnw clean package -Pnative -DskipTests -Dquarkus.native.container-build=true

...

docker run -v \
 gretting-started/target/gretting-started-1.0-SNAPSHOT-native-image-source-jar: \
 /project:z
```

以下是启用SSL的过程中自动添加的重要标志：

```
-H:EnableURLProtocols=http,https --enable-all-security-services -H:+JNI
```

为了将这个原生可执行文件容器化，需要稍微修改Docker相关的脚本以支持SSL。

打开.dockeringnore，并将keystore.jks文件作为一个不可忽略的文件添加到生成的容器中，这是必要的，因为keystore文件需要与可执行文件一起复制：

```
*
!target/classes/keystore.jks
```

```
!target/*-runner
!target/*-runner.jar
!target/lib/*
```

为了打包以下元素，src/main/docker/Dockerfile.native文件也必须进行调整：

· SunEC库。

· 收集受信任的证书颁发机构文件，来验证应用程序中使用的证书。

```
FROM quay.io/quarkus/ubi-quarkus-native-image:19.2.1 as nativebuilder ❶
FROM quay.io/quarkus/ubi-quarkus-native-image:19.3.1-java11 as nativebuilder
RUN mkdir -p /tmp/ssl \
  && cp /opt/graalvm/lib/security/cacerts /tmp/ssl/

FROM registry.access.redhat.com/ubi8/ubi-minimal
WORKDIR /work/
COPY --from=nativebuilder /tmp/ssl/ /work/ ❷
COPY target/*-runner /work/application
RUN chmod 775 /work /work/application \ ❸
  && chown -R 1001 /work \
  && chmod -R "g+rwX" /work \
  && chown -R 1001:root /work
EXPOSE 8080 8443 ❹
USER 1001
CMD ["./application",
"-Dquarkus.http.host=0.0.0.0",
"-Djavax.net.ssl.trustStore=/work/cacerts"] ❺
```

❶从GraalVM Docker镜像中获取SunEC库和cacerts

❷复制自定义keystore.jks到根工作目录

❸设置权限

❹暴露HTTPS端口

❺运行应用程序时加载SunEC和cacerts

可以通过运行以下命令来构建容器镜像：

```
docker build -f src/main/docker/Dockerfile.native -t greeting-ssl
```

讨论

安全性和SSL现在已经很普遍了，最好让你的所有服务都使用SSL进行通信，因此，当注册了以下任何一个扩展时，Quarkus会自动支持SSL：

- Agroal连接池
- Amazon DynamoDB
- Hibernate Search Elasticsearch
- Infinispan Client

· Jaeger

· JGit

· Keycloak

· Kubernetes客户端

· Mailer

· MongoDB

· Neo4j

· OAuth2

· REST客户端

只要你的项目中有这些扩展，quarkus.native.ssl属性就默认设置为true。

第7章　持久化

你应该已经很熟悉。Quarkus所使用的持久化存储策略了，事务、数据源、Java Persistence API（JPA）等都是已经存在多年的标准了。Quarkus使用了这些标准，并且在某些情况下，在它们的基础上做了扩展，使持久化存储的工作更加容易。在本章中，你将学习如何在Quarkus中使用持久化存储。本章涵盖了传统的关系型数据库管理系统（RDBMS）和NoSQL数据库。

如果你使用的是传统的RDBMS或MongoDB，那么Quarkus还提供了一些额外的功能，比如Panache，它提供了一种entity或record类型的API。Panache实现了许多标准的JPA语法，使你的应用程序更容易阅读和维护，可以帮助你提高工作效率。

本章涵盖以下主题：

- 配置数据源
- 处理事务
- 管理数据库模式迁移
- 使用Panache API

· 与NoSQL数据库交互

7.1 配置数据源

问题

如何定义、使用一个数据源？

解决方案

使用Agroal扩展和*application.properties*。

讨论

Agroal是Quarkus中首选的数据源和连接池实现，与安全、事务管理和健康指标进行了集成。虽然它是一个扩展程序，但如果你正在使用Hibernate ORM或Panache，Agroal扩展会被顺带加载进来。之后你还需要一个数据库驱动扩展，目前，H2、PostgreSQL、MariaDB、MySQL、Microsoft SQL Server和Derby都有支持的扩展，可以通过Maven add-extension添加正确的数据库驱动扩展：

```
./mvnw quarkus:add-extension -Dextensions="jdbc-mariadb"
```

Datasource的配置就像Quarkus的其他配置一样，在*src/main/resources/application.properties*文件中完成：

```
quarkus.datasource.url=jdbc::mariadb://localhost:3306/test
quarkus.datasource.driver=org.mariadb.jdbc.Driver
quarkus.datasource.username=username-default
quarkus.datasource.min-size=3
quarkus.datasource.max-size=13
```

比较敏感的数据可以通过系统属性、环境变量、Kubernetes Secrets或Vault传递，你将在后面的章节中学习到。

如果你需要访问数据源，可以使用下面的注入方式：

```
@Inject
DataSource defaultDataSource;
```

你也可以使用AgroalDataSource，它是DataSource的一个子类型。

7.2 使用多个数据源

问题

如何让应用程序使用一个以上的数据源？

解决方案

使用命名的数据源。

Agroal允许使用多个数据源。它们的配置与默认值完全相同，但有一个值得注意的例外——名称：

```
quarkus.datasource.driver=org.h2.Driver
quarkus.datasource.url=jdbc:h2:tcp://localhost/mem:default
quarkus.datasource.username=username-default
quarkus.datasource.min-size=3
quarkus.datasource.max-size=13

quarkus.datasource.users.driver=org.h2.Driver
quarkus.datasource.users.url=jdbc:h2:tcp://localhost/mem:users
quarkus.datasource.users.username=username1
quarkus.datasource.users.min-size=1
quarkus.datasource.users.max-size=11

quarkus.datasource.inventory.driver=org.h2.Driver
quarkus.datasource.inventory.url=jdbc:h2:tcp://localhost/mem:inventory
quarkus.datasource.inventory.username=username2
quarkus.datasource.inventory.min-size=2
quarkus.datasource.inventory.max-size=12
```

配置格式如下：quarkus.datasource.[*optional name.*][*datasource property*]。

讨论

对于多个数据源，注解的工作方式是相同的，但是你需要一个限定符（关于限定符的更多信息，请参见5.10节）：

```
@Inject
AgroalDataSource defaultDataSource;

@Inject
@DataSource("users")
AgroalDataSource dataSource1;

@Inject
@DataSource("inventory")
AgroalDataSource dataSource2;
```

7.3 添加数据源的健康检查

问题

如何为数据源添加健康检查？

解决方案

使用quarkus-agroal和quarkus-smallrye-health这两个扩展。

讨论

当使用quarkus-smallrye-health扩展时，数据源的健康检查会自动添加。如果需要禁用，可以修改*application.properties*中的quarkus.datasource.health.enabled（默认为true）属性。要查看健康状态，可以访问应用程序的*/health/ready*接口，这个接口是由quarkus-smallrye-health扩展创建的。

另见

更多信息，请访问GitHub上的以下页面：

· MicroProfile Health（https://oreil.ly/CDLOd）

7.4 定义事务的边界

问题

如何使用注解来定义事务的边界？

解决方案

使用quarkus-narayana-jta扩展的@javax.transaction.Transactional注解。

讨论

quarkus-narayana-jta扩展添加了@javax.transaction.Transactional注解以及TransactionManager和UserTransaction。这个扩展会由持久化扩展自动添加，当然，如果你需要，也可以手动添加。

@Transactional注解可以被添加到任何CDI bean的方法或类中，让这些方法成为事务性的——也包括REST接口：

```
package org.acme.transaction;

import javax.inject.Inject;
import javax.transaction.Transactional;
import javax.ws.rs.GET;
import javax.ws.rs.Path;
import javax.ws.rs.Produces;
import javax.ws.rs.core.MediaType;

@Path("/tx")
@Transactional
public class Transact {
}
```

7.5　设置事务的上下文

问题

如何设置不一样的事务上下文？

解决方案

@Transactional的value属性允许为事务设置作用域。

讨论

事务上下文会传递到被注解方法中的所有嵌套调用中，除非堆栈中抛出了一个运行时异常，否则事务将在方法调用结束时提交：

```
package org.acme.transaction;

import javax.inject.Inject;
import javax.transaction.Transactional;
import javax.ws.rs.GET;
import javax.ws.rs.Path;
import javax.ws.rs.Produces;
import javax.ws.rs.core.MediaType;

@Path("/tx")
@Transactional
public class Transact {
    @Inject
    NoTransact noTx;

    @GET
    @Path("/no")
    @Produces(MediaType.TEXT_PLAIN)
    public String hi() {
        return noTx.word();
    }
}
```

```
package org.acme.transaction;

import javax.enterprise.context.ApplicationScoped;

import javax.transaction.Transactional;

import static javax.transaction.Transactional.TxType.NEVER;

@ApplicationScoped
public class NoTransact {
    @Transactional(NEVER)
    public String word() {
        return "Hi";
    }
}
```

可以通过@Transactional的dontRollbackOn或rollbackOn属性来覆盖。如果需要手动回滚一个事务，也可以使用TransactionManager注解。

下面是可用的事务上下文列表：

@Transactional(REQUIRED)（默认）

如果没有启动事务，则启动；否则，保持现有的事务。

@Transactional(REQUIRES_NEW)

如果没有启动事务，则启动；如果已经启动了一个现有的事务，则暂停该事务，并在该方法的结尾启动一个新的事务。

@Transactional(MANDATORY)

如果没有启动事务，则失败；否则，使用现有的事务。

@Transactional(SUPPORTS)

如果一个事务已经开始，则加入它；否则，以无事务的方式工作。

@Transactional(NOT_SUPPORTED)

如果一个事务被启动，则暂停它，并在方法的结尾以无事务的方式工作；否则，以无事务的方式工作。

@Transactional(NEVER)

如果一个事务被启动，则抛出一个异常；否则，以无事务的方式工作。

7.6 程序化事务控制

问题

如何对事务进行更精细的控制？

解决方案

注入UserTransaction并使用该类的方法。

UserTransaction类有一些非常简单的API：

- begin()
- commit()
- rollback()
- setRollbackOnly()
- getStatus()
- setTransactionTimeout(int)

前三个方法是经常使用的方法，getStatus()对于确定事务在其生命周期中的位置很有用，最后一个API让你能够为一个事务设置超时时间。

如果需要，你也可以通过注入javax.transaction.TransactionManager来使用它。

另见

想了解更多信息，请访问以下网页：

· Jakarta EE 8 Specification API：Interface UserTransaction（https://oreil.ly/lmjR_）

7.7　设置和修改事务超时时间

问题

如何让一个事务在一定时间后超时并回滚？

解决方案

如果想使用声明式事务，请使用@io.quarkus.narayana.jta.runtime.Transaction-Configuration注解。事务API可以用于程序化的事务控制。使用quarkus.transaction-manager.default-transaction-timeout属性（指定为java.time.Duration）可以改变事务的全局超时时间。

通过@TransactionConfiguration注解，修改事务的一次性超时时间非常简单，timeout属性用来设置超时的秒数。

讨论

如果应用程序中的每个事务都需要较长或较短的时间，可以使用*application.properties*中的quarkus.transaction-manager.default-transaction-timeout属性，该属性需要

java.time.Duration，它可以通过Duration#parse()指定为一个可解析的字符串。你也可以用一个整数设置超时时间，Quarkus会自动给该值加上PT前缀，以创建正确的时间格式。

另见

想了解更多信息，请访问以下网站：

· Oracle:Java Platform,Standard Edition 8 API Specification：parse（https://oreil.ly/8gvMZ）

7.8 使用persistence.xml

问题

如何使用*persistence.xml*文件来配置JPA？

解决方案

与使用JPA的普通方式一样，在*application.properties*中设置数据源即可。

讨论

在Quarkus中使用JPA和其他配置完全一样，所以不需要特殊的调整。

如果你使用*persistence.xml*，就不能使用quarkus.hibernate-orm.*属性了，只有在*persistence.xml*文件中定义的持久配置才会生效。

7.9 不使用persistence.xml

问题

如何不使用*persistence.xml*文件配置JPA？

解决方案

添加quarkus-hibernate-orm扩展，为你的RDBMS添加JDBC驱动，通过*application.properties*进行配置，最后用@Entity注解你的实体。

讨论

除了数据库连接之外，使用Quarkus和JPA无须特别的设置。Quarkus会做出一些有主见的选择，但是不用担心，这些选择可能是你本来就会做出的。你也能够像平常一样使用EntityManager注解。

简而言之，你可以像通常使用标准JPA那样在没有额外配置*persistence.xml*的情况下使用它。这是在Quarkus中使用JPA的首选方式。

7.10 使用不同JAR文件的实体

问题

如何从不同的JAR文件中使用实体？

解决方案

在包含实体的JAR文件中加入一个空的*META-INF/beans.xml*文件。

讨论

Quarkus依赖于编译型字节码对实体的增强：如果这些实体和应用程序的其他部分定义在同一个项目（JAR）中，那么一切都能正常工作。

然而，如果其他的类（如实体或其他CDI beans）被定义在一个外部库中，该库必须包含一个空的*META-INF/beans.xml*文件才能被导入和使用。

7.11　用Panache持久化数据

问题

如何用Hibernate和Panache来持久化数据？

解决方案

调用PanacheEntity中的persist方法。

当然，你需要添加quarkus-hibernate-orm-panache扩展，并为你的数据存储添加相应的JDBC扩展。接下来，你需要定义一个实体，也就是需要创建一个类，用@javax.persistence.Entity注解它，并基于PanacheEntity进行扩展。

讨论

Panache是一个建立在传统JPA之上的有默认设置的API，它更多的是遵循一种活动记录的方式来处理数据实体。然而，在其内部，它是使用传统的JPA来完成的。

在使用Panache的过程中，你会发现，很多功能都是通过Panache实体或PanacheEntityBase父类传递给你的实体——persist也不例

外。PanacheEntityBase包含了persist()和persistAndFlush()两个方法，虽然flush选项会立即将数据发送到数据库，但它不是使用数据持久化的推荐方式。

持久化非常简单，示例如下：

```
@POST
public Response newLibrary(Library library) {
    library.persist();
    return Response.created(URI.create("/library/" + library.encodedName()))
            .entity(library).build();
}
```

完整的Library实体如下：

```
package org.acme.panache;

import java.io.UnsupportedEncodingException;
import java.net.URLEncoder;
import java.util.List;

import javax.persistence.CascadeType;
import javax.persistence.Entity;
import javax.persistence.OneToMany;

import io.quarkus.hibernate.orm.panache.PanacheEntity;

import io.quarkus.panache.common.Parameters;

@Entity
public class Library extends PanacheEntity {
    public String name;

    @OneToMany(cascade = CascadeType.ALL, orphanRemoval = true,
               mappedBy = "library")
    public List<Inventory> inventory;
    public String encodedName() {
        String result;

        try {
            result = URLEncoder.encode(name, "UTF-8")
                    .replaceAll("\\+", "%20")
                    .replaceAll("\\%21", "!")
                    .replaceAll("\\%27", "'")
                    .replaceAll("\\%28", "(")
                    .replaceAll("\\%29", ")")
                    .replaceAll("\\%7E", "~");
        } catch (UnsupportedEncodingException e) {
```

```
            result = name;
        }

        return result;
    }
}
```

7.12 用Panache listAll方法查找所有实体条目

问题

如何找到实体的所有条目？

解决方案

使用PanacheEntityBase类中的listAll()方法。

就像前面介绍的persist()方法一样，listAll()也是PanacheEntityBase类中的一个方法，它查询数据库中实体的所有条目，以List的形式返回这些实体：

```
@GET
public List<Book> getAllBooks() {
    return Book.listAll();
}
```

这个方法实际上是findAll().list()的一个快捷方式。可以在hibernate-orm-panache代码库中找到更多关于它的信息。

7.13 用Panache findById方法查找单个实体

问题

如何根据ID信息从数据库中找到并加载一个实体？

解决方案

使用PanacheEntityBase.findById(Object)方法。

Panache通过使用findById(Object)方法简化了查找实体的过程。你所需要做的只是传递对象的ID，它将从数据库中返回正确的实例：

```
@GET
@Path("/byId/{id}")

public Book getBookById(@PathParam(value = "id") Long id) {
    Book b = Book.findById(id);
    return b;
}
```

7.14　用Panache find和list方法查找实体

问题

如何根据一个实体的特定属性从数据库中查找它？

解决方案

使用PanacheEntityBase的find和list方法。

根据需要返回结果的不同，你可以使用PanacheEntityBase的list或find方法。在内部，list使用find，所以它们本质上是一样的：

```
public static Book findByTitle(String title) {
    return find("title", title).firstResult();
}

public static List<Book> findByAuthor(String author) {
    return list("author", author);
}

public static List<Book> findByIsbn(String isbn) {
    return list("isbn", isbn);
}
```

这两种方法都有多种不同的使用方式——根据是否需要排序和传递参数而有所不同。下面的代码是使用Hibernate查询语言（HQL）（或Java Persistence查询语言（JPQL））并使用Parameters类的一个例子：

```
public static Library findByName(String name) {
    return Library
            .find("SELECT l FROM Library l " +
                  "LEFT JOIN fetch l.inventory " +
                  "WHERE l.name = :name ",
                    Parameters.with("name", name)).firstResult();
}
```

Parameters类对于find和list方法都是可用的，请参考API获取更多的信息。

你可能想知道为什么要使用完整的JPQL查询？简单来说，就是为了避免列表的序列化问题。使用left join，我们能够获取该数据库中所有的实例。

7.15　用Panache count方法获取实体计数

问题

如何统计一类资源的数量？

解决方案

使用PanacheEntityBase的count方法。

与find方法一样，Panache有不同的count方法用于获取数据库中某一类型实体的数量：

```
Book.count()
Book.count("WHERE title = ?", )
```

7.16　用Panache page方法对实体列表进行分页

问题

如何使用分页功能？

解决方案

Quarkus（特别是Panache）已经内置了分页功能。在PanacheQuery对象上有很多支持分页的方法。

讨论

分页非常容易：获取一个PanacheQuery的实例，然后就像使用find方法一样简单：

```
PanacheQuery<Book> authors = Book.find("author", author);
authors.page(Page.of(3, 25)).list();                          ❶

authors.page(Page.sizeOf(10)).list();
```

❶从第3页开始，取25条数据

当然，还有其他方法，如firstPage()、lastPage()、nextPage()和previousPage()。还有一些支持布尔的方法：hasNextPage()、hasPreviousPage()和pageCount()。

另见

更多信息，请参见GitHub上的页面：

- Page对象（https://oreil.ly/KYlsJ）
- PanacheQuery接口（https://oreil.ly/BtgHK）

7.17　用Panache stream方法对结果进行流处理

问题

如何用stream（流）来处理数据？

解决方案

在使用Panache时，所有的list方法都有相应的stream方法。下面你将看到这些方法的使用示例，它们与list方法没有任何区别：

```
Book.streamAll();
...
Book.stream("author", "Alex Soto");
```

每个stream和streamAll方法都返回一个java.util.Stream实例。

stream方法需要在事务中进行处理。

另见

更多信息，请访问以下网站：

- io.quarkus：PanacheEntityBase（https://oreil.ly/OW2fV）

7.18　测试Panache实体

问题

如何使用一个嵌入式的数据库进行测试？

解决方案

Quarkus自带的内存数据库H2和Derby，可以方便地将数据库作为一个单独的进程来启动。

确保将io.quarkus:quarkus-test-h2:1.4.1.Final或io.quarkus:quarkus-test-derby:1.4.1.Final添加到你的构建文件中。

下一步是用@QuarkusTestResource(H2DatabaseTestResource.class)或@QuarkusTest-Resource(Derby DatabaseTestResource.class)来注解任何使用嵌入式数据库的测试。最后，确保在*src/test/resources/META-INF/application.properties*中为所选数据库设置正确的数据库URL和驱动。

下面是一个使用H2的示例：

```
package my.app.integrationtests.db;

import io.quarkus.test.common.QuarkusTestResource;
import io.quarkus.test.h2.H2DatabaseTestResource;

@QuarkusTestResource(H2DatabaseTestResource.class)
public class TestResources {
}

quarkus.datasource.url=jdbc:h2:tcp://localhost/mem:test
quarkus.datasource.driver=org.h2.Driver
```

这个示例程序并没有将真实数据库添加到原生启动程序中，只是添加了客户端代码。不过，你可以在JVM或原生模式下自由地使用它来测试你的应用程序。

另见

想了解更多信息，请访问以下网站：

· H2 Database Engine（https://oreil.ly/_MMus）

- Apache Derby（https://oreil.ly/FUFeH）

7.19 使用数据访问对象和repository模式

问题

如何使用DAO（数据访问对象）或repository模式？

解决方案

Quarkus没有对Panache做限制，所以你可以使用前面介绍的Entity模式、DAO或者repository模式。

你需要了解使用repository模式的两个接口是PanacheRepository和PanacheRepository-Base。只有主键不是Long时，才需要使用base接口。所有在PanacheEntity上可用的操作在PanacheRepository上也都可以使用。repository是一个CDI bean，所以必须通过注入的方式使用它。下面是一些基础的示例：

```
package org.acme.panache;

import java.util.Collections;

import javax.enterprise.context.ApplicationScoped;

import io.quarkus.hibernate.orm.panache.PanacheQuery;
import io.quarkus.hibernate.orm.panache.PanacheRepository;
import io.quarkus.panache.common.Parameters;
import io.quarkus.panache.common.Sort;

@ApplicationScoped
public class LibraryRepository implements PanacheRepository<Library> {
    public Library findByName(String name) {
        return find("SELECT l FROM Library l " +
                    "left join fetch l.inventory where l.name = :name ",
                Parameters.with("name", name)).firstResult();
    }

    @Override
    public PanacheQuery<Library> findAll() {
        return find("from Library l left join fetch l.inventory");
    }

    @Override
    public PanacheQuery<Library> findAll(Sort sort) {
        return find("from Library l left join fetch l.inventory",
                sort, Collections.emptyMap());
    }
}
```

DAO将完全按照你的期望工作：注入一个Entity Manager和查询。在网上和其他书籍中，有许多使用Java DAO的解决方案和例子，这些例子在Quarkus中也一样可以使用。

另见

更多信息，请访问以下网站：

· io.quarkus：PanacheRepositoryBase（https://oreil.ly/H6kTU）

7.20 使用Amazon DynamoDB

问题

如何在Quarkus应用程序中使用DynamoDB？

解决方案

使用DynamoDB扩展并设置配置。DynamoDB扩展允许使用同步或异步的方式来使用Apache Amazon Web Service软件开发工具包（AWS SDK）客户端。要想启用这个功能，需要在项目中进行一些必要的设置和参数配置。首先，需配置依赖项：

```
<dependency>
    <groupId>software.amazon.awssdk</groupId>
    <artifactId>url-connection-client</artifactId>
</dependency>
```

Quarkus没有AWS客户端连接的扩展插件。

该扩展默认使用URLConnection HTTP客户端，你需要添加正确的客户端（URLConnection、Apache或Netty NIO）到构建脚本中：

```
<dependency>
    <groupId>software.amazon.awssdk</groupId>
    <artifactId>apache-client</artifactId>
    <exclusions>    ❶
        <exclusion>
            <groupId>commons-logging</groupId>
            <artifactId>commons-logging</artifactId>
        </exclusion>
    </exclusions>
</dependency>
```

❶需要排除commons-logging配置，以使客户端使用Quarkus日志

如果你使用的是Apache客户端，还需要对*application.properties*文件进行调整，因为url是默认的：

```
quarkus.dynamodb.sync-client.type=apache
```

在*application.properties*中也有针对客户端的配置（更多配置请参考资料（https://oreil.ly/HZ4A-））：

```
quarkus.dynamodb.endpoint-override=http://localhost:8000 ❶
quarkus.dynamodb.aws.region=eu-central-1 ❷
quarkus.dynamodb.aws.credentials.type=static ❸
quarkus.dynamodb.aws.credentials.static-provider.access-key-id=test-key
quarkus.dynamodb.aws.credentials.static-provider.secret-access-key=test-secret
```

❶使用非标准的地址需要这样配置，如本地DynamoDB实例

❷正确有效的region

❸static或者default

默认的认证类型将按以下顺序查找认证信息：

· 系统属性aws.accessKeyId和aws.secretKey

· 环境变量AWS_ACCESS_KEY_ID和AWS_SECRET_ACCESS_KEY

· 认证配置文件的默认位置（*$HOME/.aws/credentials*）

· 通过Amazon EC2容器服务交付的认证信息

· 通过Amazon EC2元数据服务交付的实例profile认证信息

讨论

下面的例子来自7.11节，使用DynamoDB作为持久化存储。

这里有两个类用于与DynamoDB交互，并创建一个可注入的服务，以便在REST端点中使用：

```
package org.acme.dynamodb;

import java.util.List;
import java.util.stream.Collectors;

import javax.enterprise.context.ApplicationScoped;
import javax.inject.Inject;

import software.amazon.awssdk.services.dynamodb.DynamoDbClient;

@ApplicationScoped
public class BookSyncService extends AbstractBookService {
    @Inject
    DynamoDbClient dynamoDbClient;

    public List<Book> findAll() {
        return dynamoDbClient.scanPaginator(scanRequest()).items().stream()
                .map(Book::from)
                .collect(Collectors.toList());
    }

    public List<Book> add(Book b) {
        dynamoDbClient.putItem(putRequest(b));
        return findAll();
    }

    public Book get(String isbn) {
```

```
        return Book.from(dynamoDbClient.getItem(getRequest(isbn)).item());
    }
}
```

下面的抽象类包含与DynamoDB交互、持久化和查询Book实例所需的示例代码：

```
package org.acme.dynamodb;

import java.util.HashMap;
import java.util.Map;

import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
import software.amazon.awssdk.services.dynamodb.model.GetItemRequest;
import software.amazon.awssdk.services.dynamodb.model.PutItemRequest;
import software.amazon.awssdk.services.dynamodb.model.ScanRequest;

public abstract class AbstractBookService {

    public final static String BOOK_TITLE = "title";
    public final static String BOOK_ISBN = "isbn";
    public final static String BOOK_AUTHOR = "author";

    public String getTableName() {
        return "QuarkusBook";
    }

    protected ScanRequest scanRequest() {
        return ScanRequest.builder().tableName(getTableName()).build();
    }

    protected PutItemRequest putRequest(Book book) {
        Map<String, AttributeValue> item = new HashMap<>();
        item.put(BOOK_ISBN, AttributeValue.builder()
                            .s(book.getIsbn()).build());
        item.put(BOOK_AUTHOR, AttributeValue.builder()
                              .s(book.getAuthor()).build());
        item.put(BOOK_TITLE, AttributeValue.builder()
                             .s(book.getTitle()).build());

        return PutItemRequest.builder()
                .tableName(getTableName())
                .item(item)
                .build();
    }

    protected GetItemRequest getRequest(String isbn) {
        Map<String, AttributeValue> key = new HashMap<>();
        key.put(BOOK_ISBN, AttributeValue.builder().s(isbn).build());

        return GetItemRequest.builder()
                .tableName(getTableName())
                .key(key)
```

```
                .build();
        }
}
```

最后一个是代表Book实体的类：

```
package org.acme.dynamodb;

import java.util.Map;
import java.util.Objects;

import io.quarkus.runtime.annotations.RegisterForReflection;
import software.amazon.awssdk.services.dynamodb.model.AttributeValue;

@RegisterForReflection      ❶
public class Book {
    private String isbn;
    private String author;
    private String title;

    public Book() {         ❷
    }

    public static Book from(Map<String, AttributeValue> item) {
        Book b = new Book();
        if (item != null && !item.isEmpty()) {
            b.setAuthor(item.get(AbstractBookService.BOOK_AUTHOR).s());
            b.setIsbn(item.get(AbstractBookService.BOOK_ISBN).s());
            b.setTitle(item.get(AbstractBookService.BOOK_TITLE).s());
        }
        return b;
    }

    public String getIsbn() {
        return isbn;
    }

    public void setIsbn(String isbn) {
        this.isbn = isbn;
    }

    public String getAuthor() {
        return author;
    }

    public void setAuthor(String author) {
        this.author = author;
    }

    public String getTitle() {
        return title;
    }
    public void setTitle(String title) {
        this.title = title;
    }
```

```
    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        Book book = (Book) o;
        return Objects.equals(isbn, book.isbn) &&
                Objects.equals(author, book.author) &&
                Objects.equals(title, book.title);
    }

    @Override
    public int hashCode() {
        return Objects.hash(isbn, author, title);
    }
}
```

❶原生应用程序中需要的反射注入

❷DynamoDB客户端需要

除了Quarkus需要为反射注解注册的Book类外，大部分都是标准的DynamoDB代码，只有在创建本地镜像时才需要这样做。

正如你所看到的，你之前在使用DynamoDB时已经掌握的技能也适用于Quarkus，不需要做太多的修改，这有助于你提高工作效率。

7.21 使用MongoDB

问题

如何使用MongoDB作为一个持久化存储？

解决方案

Quarkus MongoDB扩展包含了MongoDB驱动和客户端。

讨论

现在你应该已经熟悉了RESTful接口和Quarkus配置的基础知识。这里我们将展示用于与本地MongoDB实例交互的代码和配置示例。

当然，你需要在应用程序中添加连接信息：

```
quarkus.mongodb.connection-string = mongodb://localhost:27017
```

Book类是MongoDB中的资源表示：

```
package org.acme.mongodb;

import java.util.HashSet;
import java.util.Objects;
import java.util.Set;

import org.bson.Document;

public class Book {
    public String id;
    public String title;
    public String isbn;
    public Set<String> authors;

    // Needed for JSON-B
    public Book() {}

    public Book(String title) {
        this.title = title;
    }

    public Book(String title, String isbn) {
        this.title = title;
        this.isbn = isbn;
    }

    public Book(String title, String isbn, Set<String> authors) {
        this.title = title;
        this.isbn = isbn;
        this.authors = authors;
    }

    public Book(String id, String title, String isbn, Set<String> authors) {
        this.id = id;
        this.title = title;
        this.isbn = isbn;
        this.authors = authors;
    }

    public static Book from(Document doc) {
        return new Book(doc.getString("id"),
                        doc.getString("title"),
                        doc.getString("isbn"),
                        new HashSet<>(doc.getList("authors", String.class)));
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        Book book = (Book) o;
        return Objects.equals(id, book.id) &&
```

```
                Objects.equals(title, book.title) &&
                Objects.equals(isbn, book.isbn) &&
                Objects.equals(authors, book.authors);
    }

    @Override
    public int hashCode() {
        return Objects.hash(id, title, isbn, authors);
    }
}
```

service类作为一个DAO，与MongoDB实例进行交互：

```
package org.acme.mongodb;

import java.util.ArrayList;
import java.util.List;
import java.util.Objects;

import javax.enterprise.context.ApplicationScoped;
import javax.inject.Inject;

import com.mongodb.client.MongoClient;
import com.mongodb.client.MongoCollection;
import com.mongodb.client.MongoCursor;
import com.mongodb.client.model.Filters;
import org.bson.Document;

@ApplicationScoped
public class BookService {

    @Inject
    MongoClient mongoClient;

    public List<Book> list() {
        List<Book> list = new ArrayList<>();

        try (MongoCursor<Document> cursor = getCollection()
                                            .find()
                                            .iterator()) {
            cursor.forEachRemaining(doc -> list.add(Book.from(doc)));
        }

        return list;
    }

    public Book findSingle(String isbn) {
        Document document = Objects.requireNonNull(getCollection()
                .find(Filters.eq("isbn", isbn))
                .limit(1).first());
        return Book.from(document);
    }
```

```
    public void add(Book b) {
        Document doc = new Document()
                .append("isbn", b.isbn)
                .append("title", b.title)
                .append("authors", b.authors);
        getCollection().insertOne(doc);
    }

    private MongoCollection<Document> getCollection() {
        return mongoClient.getDatabase("book").getCollection("book");
    }
}
```

RESTful接口使用了前面两个类：

```
package org.acme.mongodb;

import java.util.List;

import javax.inject.Inject;
import javax.ws.rs.Consumes;
import javax.ws.rs.GET;
import javax.ws.rs.POST;
import javax.ws.rs.Path;
import javax.ws.rs.PathParam;
import javax.ws.rs.Produces;
import javax.ws.rs.core.MediaType;
import javax.ws.rs.core.Response;

@Path("/book")
@Produces(MediaType.APPLICATION_JSON)
@Consumes(MediaType.APPLICATION_JSON)
public class BookResource {
    @Inject
    BookService service;

    @GET
    public List<Book> getAll() {
        return service.list();
    }

    @GET
    @Path("{isbn}")
    public Book getSingle(@PathParam("isbn") String isbn) {
        return service.findSingle(isbn);
    }

    @POST
    public Response add(Book b) {
        service.add(b);
        return Response.status(Response.Status.CREATED)
                .entity(service.list()).build();
    }
}
```

创建和使用BSON Codec作为练习留给读者。MongoDB扩展在使用quarkus-smallrye-health扩展时会自动运行健康检查。quarkus-smallrye-health扩展将自动为你的MongoDB连接创建一个可配置的健康检查。

Quarkus MongoDB扩展还包括一个响应式客户端，将在15.12节中详细介绍。

7.22　用Panache连接MongoDB

问题

如何使用Panache来连接MongoDB？

解决方案

添加mongodb-panache扩展，使用Panache MongoEntity注解所有的Panache功能。

Panache支持的MongoDB功能与Hibernate功能原理相同，我们在7.7～7.17节中已经看到过了，它大大简化了代码：

```
package org.acme.mongodb.panache;

import java.time.LocalDate;
import java.util.List;

import io.quarkus.mongodb.panache.MongoEntity;
import io.quarkus.mongodb.panache.PanacheMongoEntity;
import org.bson.codecs.pojo.annotations.BsonProperty;

@MongoEntity(collection = "book", database = "book")     ❶
public class Book extends PanacheMongoEntity {           ❷
    public String title;
    public String isbn;
    public List<String> authors;

    @BsonProperty("pubDate")                             ❸
    public LocalDate publishDate;

    public static Book findByIsbn(String isbn) {

        return find("isbn", isbn).firstResult(); ❹
    }

    public static List<Book> findPublishedOn(LocalDate date) {
        return list("pubDate", date);
    }

}
```

❶@MongoEntity注解允许你使用自定义的collection和database参数

❷必填部分——将字段添加为公共字段

❸用@BsonProperty自定义序列化时的字段名

❹使用PanacheQL（JPQL的子集）进行查询，就像使用JPA一样

讨论

Panache MongoDB扩展使用PojoCodecProvider将实体映射到MongoDB Document。除了@BsonProperty，你还可以使用@BsonIgnore忽略字段，也能够用@BsonId设置自定义ID，并扩展PanacheMongoEntityBase。

当然，如果你需要写一个访问器方法，Panache并不会阻止你这么做，事实上，在构建时，所有的字段调用都会被相应的访问器/变体调用所替代。与Panache支持的Hibernate功能一样，MongoDB版本也支持分页、排序、流以及Panache API的其他功能。

在前面的例子中，你看到的PanacheQL查询很容易使用和理解，但也支持使用原生的MongoDB查询，只要查询以{.}开头即可。

Hibernate和MongoDB的Panache支持有一个细微差别，MongoDB能够在find()方法的返回中使用Query Projection，这允许你限制从数据库中返回哪些字段。下面是一个非常基础的例子，以Book实体为例：

```
import io.quarkus.mongodb.panache.ProjectionFor;

@ProjectionFor(Book.class)
public class BookTitle {
    public String title;
}

PanacheQuery<BookTitle> query = Book.find("isbn", "978-1-492-06265-3")
                                    .project(BookTitle.class);
```

如果你有期望一个projection类的层次结构，父类需要用@ProjectionFor来注解。

7.23 使用Neo4j

问题

如何连接并使用Neo4j?

解决方案

使用基于Neo4j Java版本驱动的Quarkus Neo4j扩展。

下面的例子使用了基于JOK的完整功能的异步编程模型，该驱动程序使用了阻塞模型（类似于JDBC）和响应式模型，响应式模型只适用于Neo4j 4+版本。

现在你已经知道如何在项目中添加额外的扩展，这里不再做介绍。下面的例子管理Book类：

```
package org.acme.neo4j;

import java.util.HashSet;
import java.util.Set;
import java.util.StringJoiner;

import org.neo4j.driver.Values;
import org.neo4j.driver.types.Node;

public class Book {
  public Long id;
  public String title;
  public String isbn;
  public Set<String> authors;

  // Needed for JSON-B
  public Book() {}

  public Book(String title) {
    this.title = title;
  }

  public Book(String title, String isbn) {
    this.title = title;
    this.isbn = isbn;
  }

  public Book(String title, String isbn, Set<String> authors) {
    this.title = title;
    this.isbn = isbn;
    this.authors = authors;
  }

  public Book(Long id, String title, String isbn, Set<String> authors) {
    this.id = id;
    this.title = title;
    this.isbn = isbn;
    this.authors = authors;
  }

  public static Book from(Node node) {
    return new Book(node.id(),
```

```
        node.get("title").asString(),
        node.get("isbn").asString(),
        new HashSet<>(
          node.get("authors")
          .asList(Values.ofString())
          )
        );
  }

  public String toJson() {
    final StringJoiner authorString =
      new StringJoiner("\",\"", "[\"", "\"]");

    authors.forEach(authorString::add);

    return "{" +
      "\"title\":\"" + this.title + "\"," +
      "\"isbn\":\"" + this.isbn + "\"," +
      "\"authors\":" + authorString.toString() +
      "}";
  }
}
```

当然，你需要配置客户端，这与设置quarkus.neo4j.uri、quarkus.neo4j.authentication.username和quarkus.neo4j.authentication.password属性一样简单。更多的属性可以参考扩展列表。

首先，你需要配置客户端的Neo4j Driver，该扩展提供了一个可注入实例：

```
@Inject
Driver driver;
```

接下来，创建一个新的REST接口，并添加Driver注入点，然后添加基本的CRUD操作：

```
@GET
public CompletionStage<Response> getAll() {
    AsyncSession session = driver.asyncSession();  ❶

    return session
            .runAsync("MATCH (b:Book) RETURN b ORDER BY b.title")  ❷
            .thenCompose(cursor -> cursor.listAsync(record ->
                    Book.from(record.get("b").asNode())))  ❸
            .thenCompose(books -> session.
                    closeAsync().thenApply(signal -> books))  ❹
            .thenApply(Response::ok)  ❺
            .thenApply(Response.ResponseBuilder::build);
}
```

❶从驱动程序中获取AsyncSession

❷执行Cypher（Neo4j的查询语言）来获取数据

❸检索cursor（光标），从节点中创建Book实例

❹完成后关闭会话

❺创建JAX-RS响应

其余的类或代码也遵循同样的模式：

```
@POST
public CompletionStage<Response> create(Book b) {
    AsyncSession session = driver.asyncSession();
    return session
            .writeTransactionAsync(tx ->
                    {
                        String query = "CREATE (b:Book " +
                        "{title: $title, isbn: $isbn, authors: $authors})" +
                        " RETURN b";
                        return tx.runAsync(query,
                                Values.parameters("title", b.title,
                                        "isbn", b.isbn,
                                        "authors", b.authors))
                                .thenCompose(ResultCursor::singleAsync);
                    }
            )
            .thenApply(record -> Book.from(record.get("b").asNode()))
            .thenCompose(persistedBook -> session.closeAsync()
                    .thenApply(signal -> persistedBook))
            .thenApply(persistedBook -> Response.created(
                    URI.create("/book/" + persistedBook.id)).build());
}

@DELETE
@Path("{id}")
public CompletionStage<Response> delete(@PathParam("id") Long id) {
    AsyncSession session = driver.asyncSession();
    return session
            .writeTransactionAsync(tx -> tx
                    .runAsync("MATCH (b:Book) WHERE id(b) = $id DELETE b",
                            Values.parameters("id", id))
                    .thenCompose(ResultCursor::consumeAsync))
            .thenCompose(resp -> session.closeAsync())
            .thenApply(signal -> Response.noContent().build());
}
```

最后，处理错误的地方有一些不同：

```
@GET
@Path("{id}")
public CompletionStage<Response> getById(@PathParam("id") Long id) {
    AsyncSession session = driver.asyncSession();
    return session.readTransactionAsync(tx ->
            tx.runAsync("MATCH (b:Book) WHERE id(b) = $id RETURN b",
                        Values.parameters("id", id))
```

```
                .thenCompose(ResultCursor::singleAsync))
        .handle(((record, err) -> {
            if (err != null) {
                Throwable source = err;
                if (err instanceof CompletionException)
                    source = ((CompletionException) err).getCause();
                Response.Status status = Response.Status.
                                                INTERNAL_SERVER_ERROR;
                if (source instanceof NoSuchRecordException)
                    status = Response.Status.NOT_FOUND;

                return Response.status(status).build();
            } else {
                return Response.ok(Book.from(record.get("b")
                                   .asNode())).build();
            }
        }))
        .thenCompose(response -> session.closeAsync()
                                        .thenApply(signal -> response))
}
```

另见

更多内容请参考Neo4j Cypher手册（https://oreil.ly/ITHPx）。

7.24 使用Flyway

问题

如何用Flyway来迁移数据库scheme？

解决方案

使用quarkus-flyway扩展。

讨论

Quarkus对使用Flyway进行scheme迁移有比较友好的支持。在应用程序启动时，你需要做5件事来让Quarkus使用Flyway：

1. 添加Flyway扩展。

2. 为你的数据库添加JDBC驱动程序。

3. 设置数据源。

4. 将迁移添加到*src/main/resources/db/migration*。

5. 将quarkus.flyway.migate-at-start设置为true。

Flyway迁移的默认命名格式是V.<version>__<description>.sql。其他的一切都会被自动处理好。

你也可以使用Flyway连接多个数据源。每个数据源需要使用命名的设置，与数据源的名称相同：quarkus.flyway.*datasource name*.setting，例如，quarkus.flyway.users.migrate-at-start。

7.25　以程序化的方式使用Flyway

问题

你想用程序化的方式使用Flyway。有时，如何控制scheme何时被迁移，而不是在应用程序启动时进行？

解决方案

使用quarkus-flyway扩展并注入Flyway实例：

```
@Inject
Flyway flyway
```

讨论

这将注入针对默认数据源的默认org.flywaydb.core.Flyway实例配置。如果你有多个数据源和Flyway实例，可以使用@FlywayDataSource或@Named注解注入特定的数据源。当使用@FlywayDataSource时，其值是数据源的名称，如果使用@Named，值应该是带有flyway_前缀的数据源的名称：

```
@Inject
@FlywayDataSource("books")
Flyway flywayBooks;

@Inject
@Named("flyway_users")
Flyway flywayUsers;
```

当然，你将能够运行所有标准的Flyway操作，如clean、migrate、validate、info、baseline和repair。

第8章　容错

在本章中，你将了解为什么需要容忍微服务架构中的故障。微服务架构严重依赖网络来运行，而网络作为一个关键的部分，并不总是运行良好，会出现网络故障、网线饱和、拓扑结构变化、下游服务更新等各种问题，因此微服务架构需要具备容错能力。

出于这个原因，构建对任何类型的问题都具有容错能力的服务，并提供优雅的解决方案，而不仅仅是传播错误，这一点非常重要。

本章涵盖以下主题：

- 实现不同的弹性策略
- 提供一些后备逻辑，以防出现错误
- 正确配置容错参数

8.1　自动重试

问题

如果发生故障，如何执行自动重试，尝试从故障中恢复？

解决方案

MicroProfile Fault Tolerance（容错）规范提供了一种在任何CDI元素上实现自动重试的方法，包括CDI bean和MicroProfile REST Client。

可以实现几种策略来防止失败，并且在最坏的情况下，提供一些默认逻辑来代替失败。假设你有一个根据读者偏好推荐书籍的服务，如果这个服务宕机了，你可以缓存一个畅销书的列表，这样至少你可以提供列表而不是失败。所以，容错策略一个重要的部分就是定义在无法恢复的情况下，执行一个回退逻辑。

MicroProfile容错集中在几个策略上，使你的代码具有容错性。让我们看看第一个策略，它就像执行自动重试一样简单。

为使用MicroProfile Fault Tolerance规范添加扩展：

```
./mvnw quarkus:add-extension -Dextensions="quarkus-smallrye-fault-tolerance"
```

从网络故障中恢复的最简单有效的方法之一是重试同一操作。如果是间歇性错误，那么可以通过重试来修复错误。

用@org.eclipse.microprofile.faulttolerance.Retry注解的类或方法会在抛出异常时执行自动重试。你可以设置不同的参数，比如最大重试次数、最大持续时间。你也可以指定重试应该执行的异常类型。

此外，你还可以通过用@org.eclipse.microprofile.faulttolerance.Fallback注解方法来实现回调逻辑。作为回调执行的逻辑可以以实现org.eclipse.microprofile.faulttolerance.FallbackHandler接口的类来实现：

```
@Retry(maxRetries = 3, ❶
       delay = 1000) ❷
@Fallback(RecoverHelloMessageFallback.class) ❸
public String getHelloWithFallback() {
    failureSimulator.failAlways();
    return "hello";
}

public static class RecoverHelloMessageFallback
    implements FallbackHandler<String> { ❹

    @Override
    public String handle(ExecutionContext executionContext) {
        return "good bye";
    }

}
```

❶将最大重试次数设置为3次

❷重试之间有1秒的间隔

❸如果3次重试后问题仍然存在，增加回退逻辑

❹FallbackHandler模板必须与recov-ering方法的返回类型相同

讨论

你可以通过配置文件覆盖这些属性中的任何一个，配置的格式如下：*fully_qualified_class_name/method_name/fault_tolerant_annotation/parameter*。

例如，你可以为一个方法或类设置特定的参数，也可以全局设置：

```
org.acme.quickstart.ServiceInvoker/getHelloWithFallback/Retry/maxDuration=30 ❶
org.acme.quickstart.ServiceInvoker/Retry/maxDuration=3000 ❷
Retry/maxDuration=3000 ❸
```

❶方法级别的覆盖

❷类级别的覆盖

❸全局覆盖

另见

有关更多信息，请访问Eclipse MicroProfile网站上的以下页面：

- Fault Tolerance（https://oreil.ly/WzhhA）
- Fault Tolerance：Retry Policy（https://oreil.ly/Kjhzj）

8.2　超时

问题

如何防止一个执行永远等待？

解决方案

MicroProfile Fault Tolerance规范提供了一种方法来实现操作的超时，防止执行永远等待。

当有一个外部服务的调用时，确保这个操作有一个与之相关的超时是一个好的做法。这样，如果出现网络延迟或故障，进程不会等待很长时间最后以失败告终，而是快速失败，这样你就可以尽快对问题做出反应。

用@org.eclipse.microprofile.faultttoler ance.Timeout注解的类或方法定义了一个超时，如果有超时，那么就会抛出org.eclipse.microprofile.faulttolerance.exceptions.TimeoutException异常：

```
@Timeout(value = 2000) ❶
public String getHelloWithTimeout() {
    failureSimulator.longMethod();

    return "hello";
 }
```

❶将超时设置为2秒

你可以通过配置文件覆盖这些属性，例如：

```
org.acme.quickstart.ServiceInvoker/getHelloWithTimeout/Timeout/value=3000 ❶
org.acme.quickstart.ServiceInvoker/Timeout/value=3000 ❷
Timeout/value=3000 ❸
```

❶方法级别的覆盖

❷类级别的覆盖

❸全局覆盖

你可以将@Timeout注解与@Fallback混合使用，以实现一些超时的恢复逻辑，或者使用@Retry在发生超时异常时执行自动重试（@Retry(retryOn=TimeoutException.class)）。

另见

要了解MicroProfile Fault Tolerance中的更多超时模式，请参见GitHub上的以下页面：

- Timeout（https://oreil.ly/af9DD）

8.3　使用舱壁模式进行过载保护

问题

如何限制一个服务接受请求的数量？

解决方案

使用MicroProfile Fault Tolerance规范提供的bulkhead pattern（舱壁模式）来实现。

bulkhead模式限制了可以同时执行的操作，让新的请求保持等待，直到当前的执行请求完成。如果等待的请求在一定时间后不能执行，则会被丢弃并抛出异常。

用@org.eclipse.microprofile.faulttolerance.Bulkhead注解的类或方法应用了一个bulkhead限制。如果有同步调用（你将在第15章学习bulkhead限制如何与异步调用一起工作），当达到并发执行的限制时，会抛出org.eclipse.microprofile.faulttolerance.exceptions.BulkheadException异常，而不是将请求排队：

```
@Bulkhead(2) ❶
public String getHelloBulkhead() {
    failureSimulator.shortMethod();
    return "hello";
}
```

❶将并发数设置为2

如果你使用siege工具模拟4个并发请求，那么输出的结果会是这样：

```
siege -r 1 -c 4 -v http://localhost:8080/hello/bulkhead

** SIEGE 4.0.4
** Preparing 4 concurrent users for battle.
The server is now under siege...
HTTP/1.1 500     0.47 secs:    2954 bytes ==> GET  /hello/bulkhead
HTTP/1.1 500     0.47 secs:    2954 bytes ==> GET  /hello/bulkhead
HTTP/1.1 200     2.46 secs:       5 bytes ==> GET  /hello/bulkhead
HTTP/1.1 200     2.46 secs:       5 bytes ==> GET  /hello/bulkhead

Transactions:                  2 hits
Availability:                  50.00 % ❶
```

❶只处理2个请求

此外，你可以通过配置文件覆盖这些属性：

```
org.acme.quickstart.ServiceInvoker/getHelloBulkhead/Bulkhead/value=10 ❶
org.acme.quickstart.ServiceInvoker/Bulkhead/value=10 ❷
Bulkhead/value=10 ❸
```

❶方法级别的覆盖

❷类级别的覆盖

❸全局覆盖

讨论

当你在处理（微）服务架构时，一个服务的调用量超载时，就会出现问题。如果继续超载，这个服务可能会过载，并在一段时间内停止处理请求。

你可以将@Bulkhead注解与任何其他之前展示的故障容忍注解混合使用，以实现一个更有弹性的策略。例如，一个bulkhead +重试与延迟。

另见

要了解关于MicroProfile容错中的bulkhead模式的更多内容，请参见GitHub上的以下页面：

· Bulkhead（https://oreil.ly/anYN5）

8.4　使用断路器模式避免故障传递

问题

如何防止服务故障传播到其他服务并消耗多个资源？

解决方案

MicroProfile Fault Tolerance规范提供了circuit breaker pattern（断路器模式），以避免在出现错误时进行不必要的调用。

让我们定义一个断路器，在4个请求窗口中出现3个错误后终止：

```
@CircuitBreaker(requestVolumeThreshold = 4, ❶
                failureRatio = 0.75,  ❷
                delay = 2000) ❸
public String getHelloCircuitBreaker() {
    failureSimulator.fail4Consecutive();
    return "hello";
}
```

❶定义滚动窗口

❷断路的阈值（4×0.75=3）

❸重新打开的时长

你可以通过配置文件覆盖这些属性：

```
org.acme.quickstart.ServiceInvoker/getHelloCircuitBreaker \
    /CircuitBreaker/failureRatio=0.75 ❶
org.acme.quickstart.ServiceInvoker/CircuitBreaker/failureRatio=3000 ❷
Timeout/value=3000 ❸
```

❶方法级别的覆盖

❷类级别的覆盖

❸全局覆盖

讨论

当你在处理（微）服务架构时，当与一个服务的通信变得不可用时，可能是因为服务宕机或延迟变高。当这种情况发生时，比较稀有的资源（如线程或文件描述符）可能会在等待其他服务响应的过程中

被消耗掉。如果这种情况继续下去，最终可能会耗尽资源，这意味着这个服务不能再处理更多的请求，这将引发整个应用程序中其他服务的一连串错误。

图8-1说明了发生在网状结构中间的服务中的故障是如何通过它的所有调用者传播的。这是一个级联故障的例子。

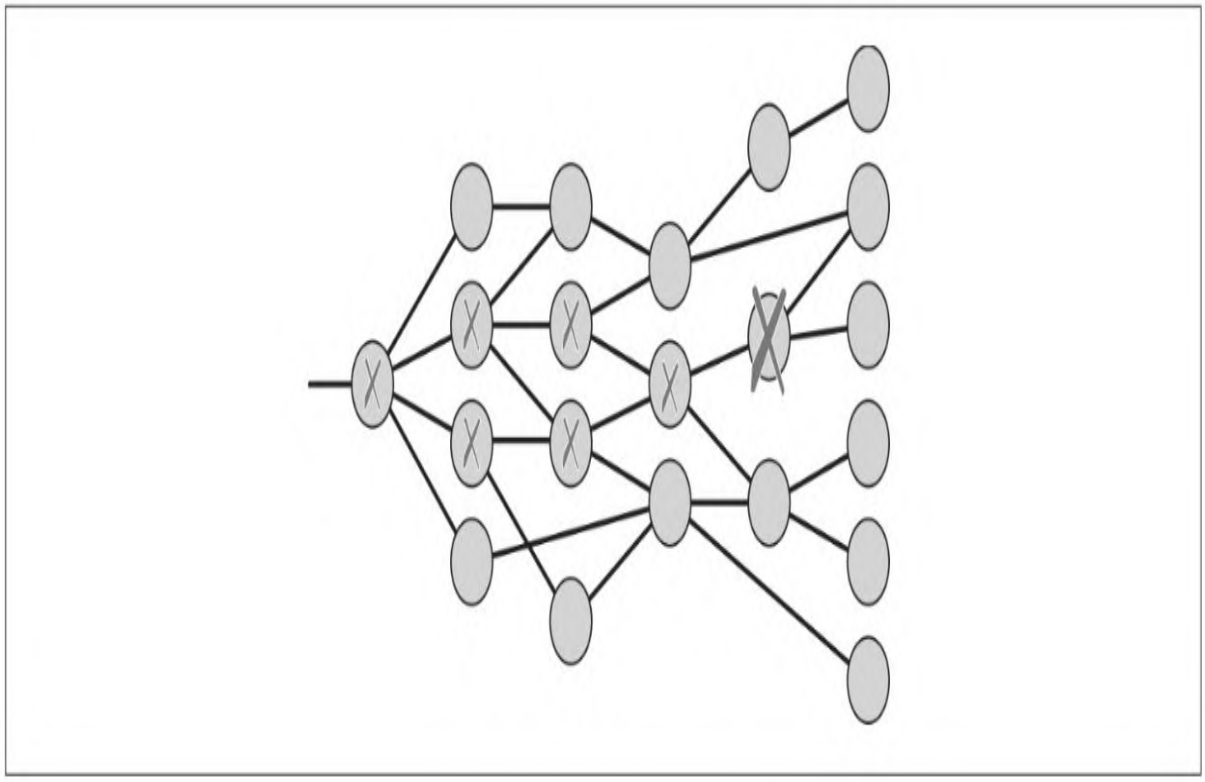


图8-1：级联故障

断路模式通过检测检测窗口内的连续故障次数来修复级联故障。如果超过了定义的错误阈值，那么电路就会跳闸，也就是说在一定时

间内，所有试图调用这个方法的尝试都会立即失败，而不会尝试执行。图8-2说明了断路器调用的示意图：

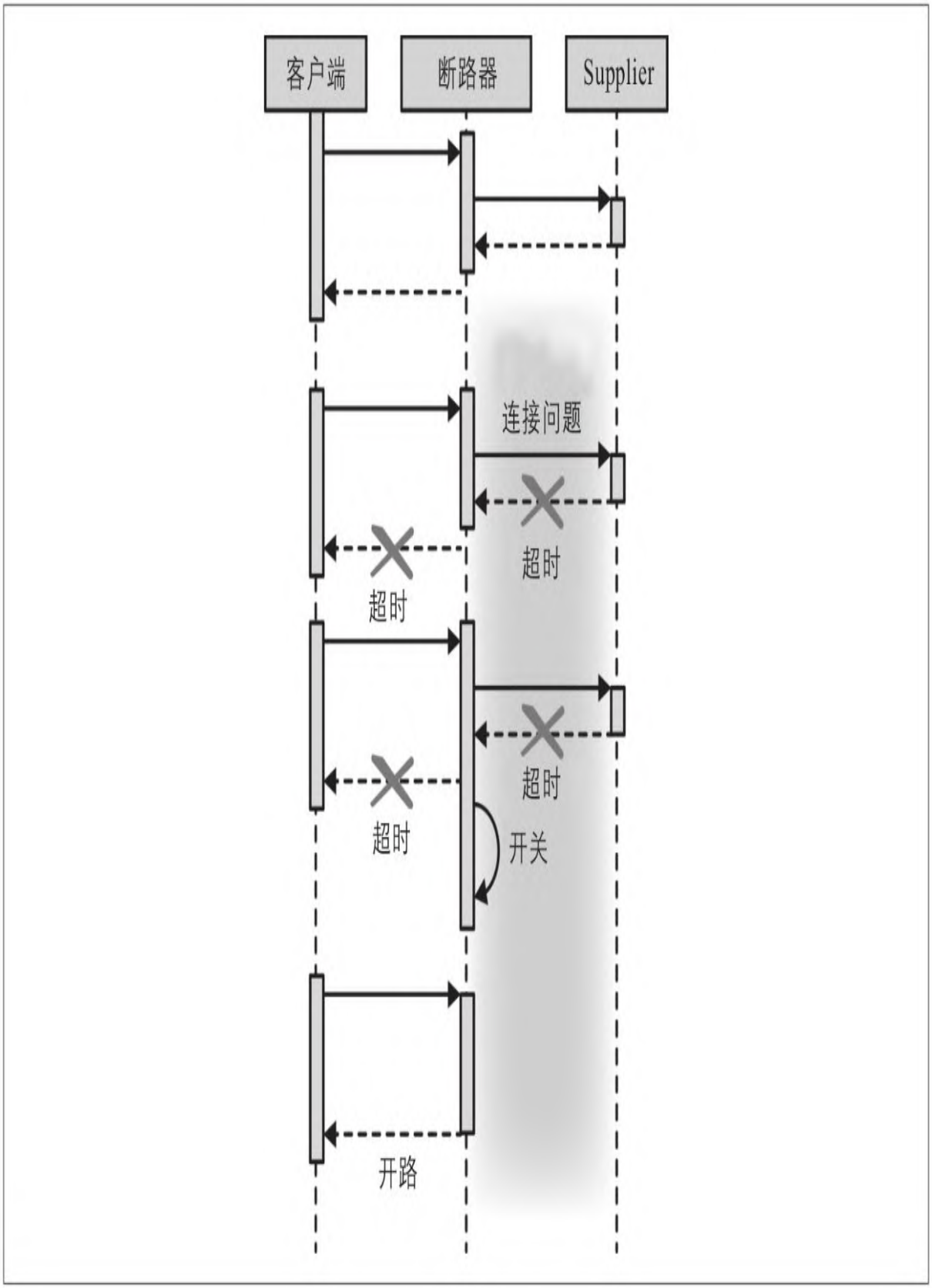

客户端
断路器
Supplier
连接问题
超时
超时
超时
超时
开关
开路

图8-2：断路器调用

一段时间后，电路会变成半开状态，这意味着下一次呼叫不会立即失败，而是会针对系统再次尝试。如果呼叫成功，则电路将被关闭；否则，电路将保持开放。断路器模式的所有可能状态如图8-3所示：

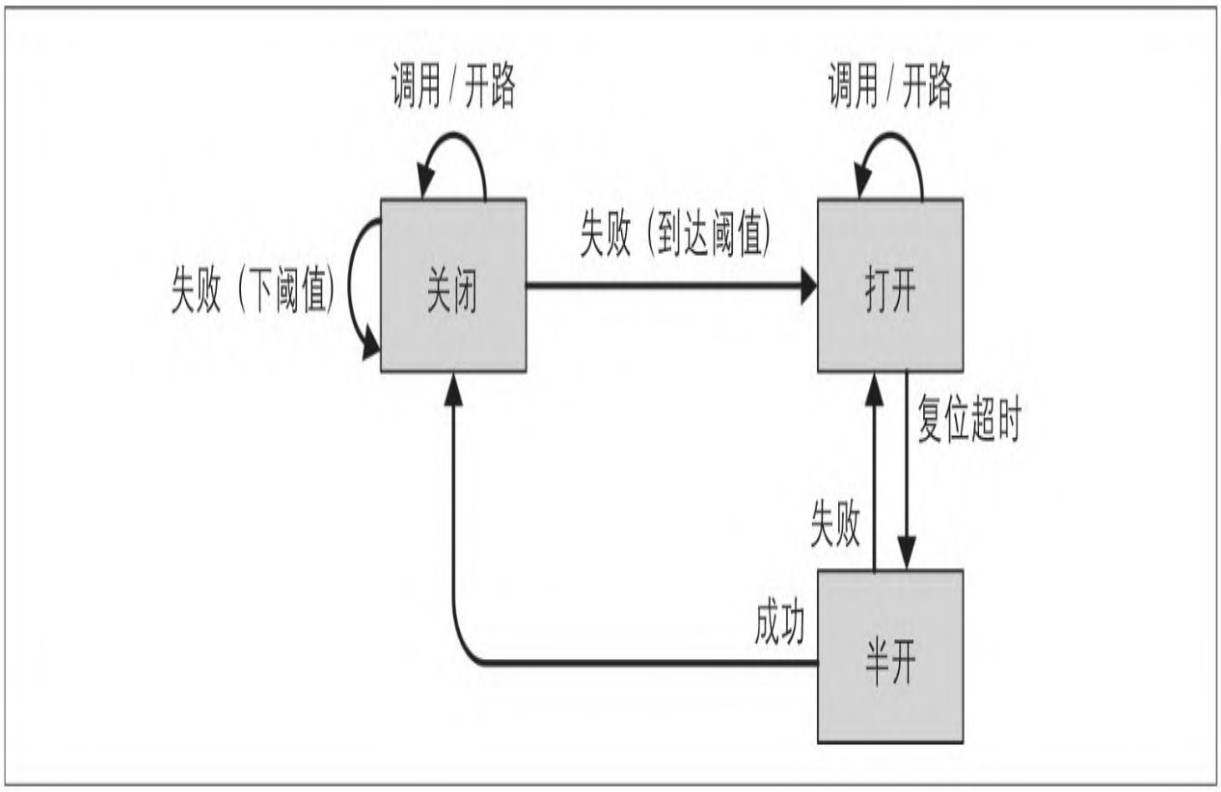


图8-3：断路器生命周期

用@org.eclipse.microprofile.faulttolerance.CircuitBreaker注解的类或方法为该操作定义了一个断路器。如果电路被打开，就会

抛出org.eclipse.microprofile.faulttoler
ance.CircuitBreakerOpenException异常。

你也可以将@CircuitBreaker与@Timeout、@Fallback、@Bulkhead或@Retry混合使用，但必须考虑到以下几点：

· 如果使用@Fallback，且CircuitBreakerOpenException被抛出，回退逻辑将被执行。

· 如果使用@Retry，每次重试都由断路器处理，并记录成功或失败。

· 如果使用@Bulkhead，则在试图进入bulkhead之前检查断路器。

另见

要了解关于MicroProfile容错中的断路器模式的更多内容，请参见GitHub上的以下页面：

· Circuit Breaker（https://oreil.ly/iOWuR）

8.5 禁用容错功能

问题

如何在某些环境中禁用容错功能？

解决方案

MicroProfile Fault Tolerance规范提供了一个特殊的参数来启用或禁用全局或单独的容错功能。

在某些情况下，你可能会想禁用容错逻辑，MicroProfile Fault Tolerance规范定义了一个叫作enabled的特殊参数，可以用来启用或禁用容错逻辑。

```
org.acme.quickstart.ServiceInvoker/getHelloCircuitBreaker/\
    CircuitBreaker/enabled=false ❶
org.acme.quickstart.ServiceInvoker/CircuitBreaker/enabled=false ❷
CircuitBreaker/enabled=false ❸
MP_Fault_Tolerance_NonFallback_Enabled=false ❹
```

❶在方法级别禁用

❷在类级别禁用

❸按类型禁用

❹禁用所有容错功能

第9章　可观察性

在本章中，你将了解可观察性，以及为什么它在微服务架构中很重要。可观察性通过观察一些参数（如错误代码、性能或任何一种业务指标）来反映系统是如何表现的。Quarkus集成了几种用于可观测性的技术。

本章涵盖以下主题：

- 定义健康检查
- 为监测系统提供衡量标准
- 配置分布式跟踪，以便在网状结构中了解请求的概况

9.1　使用自动化健康检查

问题

如何检查服务是否已经启动和运行以及是否能够正确处理请求？

解决方案

MicroProfile Health规范提供了一个API，用于从另一台机器（例如，Kubernetes控制器）探查服务的状态。

要在Quarkus应用程序中启用MicroProfile Health，只需要注册quarkus-smallrye-health扩展：

```
./mvnw quarkus:add-extension -Dextensions="quarkus-smallrye-health"
```

通过classpath中的扩展，Quarkus会自动注册默认的liveness和readiness探针，当服务启动并运行时，这两个探针都会返回UP：

```
./mvnw compile quarkus:dev

curl localhost:8080/health/live ❶

{
    "status": "UP", ❷
    "checks": [ ❸
    ]
}


curl localhost:8080/health/ready ❹

{
    "status": "UP",
    "checks": [
    ]
}
```

❶liveness的URL

❷状态是开启

❸无检查（默认设置）

❹readiness的URL

讨论

MicroProfile Health规范提供了两种健康检查方式：

liveness

如果服务已经启动，则返回200 OK的结果UP；如果服务未上线，则返回503 Service Unavailable的结果DOWN；如果无法计算健康检查，则返回500 Server Error。liveness探针endpoint默认注册在*/health/live endpoint*。

readiness

如果服务已经准备好处理请求，则返回一个200 OK的结果UP。这与liveness探针不同，因为liveness只是意味着服务已经启动，但可能还不能处理任何请求（例如，因为它正在进行数据库迁移）。如果服务还不能接受任何请求，则返回503 Service Unavailable的结果DOWN。如果无法计算健康检查，则返回500 Server Error。readiness探针endpoint默认注册在*/health/ready endpoint*。

如果你配置Quarkus使用SQL数据库（JDBC），它将自动注册一个readiness健康检查（在checks部分），以验证与数据库的连接是可能的。

以下扩展提供了自动化的readiness/liveness探针：

datasource

用于检查数据库连接状态的readiness探针。

Kafka

用于检查Kafka连接状态的readiness探针。默认情况下它是禁用的，需要通过设置quarkus.kafka.health.enabled属性为true来启用。

MongoDB

用于检查MongoDB连接状态的readiness探针。

Neo4j

用于检查Neo4j连接状态的readiness探针。

Artemis

用于检查Artemis JMS连接状态的readiness探针。

Kafka Stream

liveness（stream状态）探针和readiness（topics创建）探针。

Vault

用于检查Vault状态的readiness探针。

通过设置quarkus.*component*.health.enabled为false，可以禁止自动生成探针：

```
quarkus.kafka-streams.health.enabled=false
quarkus.mongodb.health.enabled=false
quarkus.neo4j.health.enabled=false
```

另见

要了解关于MicroProfile Health规范的更多信息，请参见GitHub上的以下页面：

· MicroProfile Health（https://oreil.ly/wZjHC）

9.2 创建自定义健康检查

问题

如何自定义检查一个服务是否启动和运行，以及它是否能够正确处理请求？

解决方案

MicroProfile Health规范使你能够创建自定义的liveness或readiness健康检查。在某些情况下，可能需要为liveness或readiness探针定制健康检查逻辑。

MicroProfile Health规范允许你通过创建一个注解为@org.eclipse.microprofile.health.Liveness和@org.eclipse.microprofile.health.Readiness的方法来创建自定义健康检查，并返回一个org.eclipse.microprofile.health.HealthCheck接口的实现。

在org.acme.quickstart.LivenessCheck.java中创建一个新的类来实现一个自定义的liveness探针：

```
@ApplicationScoped ❶
@Liveness ❷
public class LivenessCheck implements HealthCheck { ❸

    @Override
    public HealthCheckResponse call() {
        HealthCheckResponseBuilder checkResponseBuilder = HealthCheckResponse
        .named("custom liveness"); ❹

        if(isUpAndRunning()) {
            return checkResponseBuilder.up().build(); ❺
        } else {
            return checkResponseBuilder.down()
                .withData("reason", "Failed connection")
                .build(); ❻
        }

    }
}
```

❶需要作为一个CDI的类

❷设置liveness健康检查

❸实现健康检查

❹设置健康检查的名字

❺将结果设置成开启

❻将结果设置成关闭

检查liveness探针是否按预期工作：

```
./mvnw compile quarkus:dev

curl localhost:8080/health/live

{
```

```
    "status": "UP",
    "checks": [
        {
            "name": "custom liveness",
            "status": "UP"
        }
    ]
}
```

讨论

因为健康检查被注册为CDI bean，所以你也可以在工厂对象中生成健康检查，如5.7节中所解释的那样。

在本例中创建包含新的健康检查（此处是readiness检查）的新工厂类：

```
@ApplicationScoped ❶
public class CustomHealthCheck {

    @Produces ❷
    @Readiness ❸
    public HealthCheck ready() {
        if (isReady()) {
            return io.smallrye.health.HealthStatus.up("Custom readiness"); ❹
        } else {
            return io.smallrye.health.HealthStatus.down("Custom readiness");
        }
    }
}
```

❶需要作为一个CDI类

❷这个方法产生一个健康检查

❸readiness探针

❹HealthStatus是一个基础的类，其实现了HealthCheck接口

检查readiness探针是否按预期工作：

```
./mvnw compile quarkus:dev

curl localhost:8080/health/ready

{
    "checks": [
        {
            "name": "Custom readiness",
            "status": "UP"
        }
    ],
    "status": "UP"
}
```

另见

MicroProfile Health规范非常适合定义Kubernetes的liveness和readiness探针。你可以在以下网站了解它们：

· Kubernetes:Configure Liveness,Readiness,and Startup Probes（https://oreil.ly/nTaaa）

9.3 暴露指标

问题

如何通过暴露服务指标来主动检查生产中服务的当前状态，以便尽快发现任何不当行为？

解决方案

MicroProfile Metrics规范提供了一种方法，可以从你的应用程序中建立指标并向监控工具（例如，Prometheus）暴露指标。

要在Quarkus应用程序中启用MicroProfile Metrics，只需要注册quarkus-smallrye-metrics扩展：

```
./mvnw quarkus:add-extension -Dextensions="quarkus-smallrye-metrics"
```

通过在classpath中的扩展，Quarkus默认提供监控参数，并以Prometheus格式在/metrics端点上暴露它们：

```
./mvnw compile quarkus:dev

curl localhost:8080/metrics

base_cpu_processCpuLoad_percent 0.0
base_memory_maxHeap_bytes 4.294967296E9
base_cpu_systemLoadAverage 2.580078125
base_thread_daemon_count 6.0
...
vendor_memoryPool_usage_max_bytes{name="Compressed Class Space"} 3336768.0
vendor_memory_usedNonHeap_bytes 3.9182104E7
```

通过在HTTP Accept头中添加application/json类型，可以将输出格式改为JSON：

```
curl --header "Accept:application/json" localhost:8080/metrics

{
    "base": {
```

```
        "cpu.systemLoadAverage": 4.06201171875,
        "thread.count": 20,
        "classloader.loadedClasses.count": 4914,
        ...
    },
    "vendor": {
        "memoryPool.usage.max;name=G1 Survivor Space": 7340032,
        "memory.freePhysicalSize": 814391296,
        "memoryPool.usage.max;name=CodeHeap 'non-profiled nmethods'": 5773056,
        ...
    }
}
```


讨论

了解服务在微服务架构中的表现，对于防止任何可能影响到你的所有应用程序的问题至关重要。

在单体应用中，监控服务行为是相当容易的，因为你只有三四个元素需要监控。但现在有了（微）服务架构，你可能有数百个元素需要监控。

有许多值可供监控，例如：

· 内存

· 磁盘空间

· 网络

· JVM资源

· 关键方法的性能

· 业务指标（如每秒支付次数）

· 集群的总体健康状况

如果你仔细观察输出，你会发现参数的前缀是base或vendor。MicroProfile Metrics将指标分为三类：

base

服务器的核心信息。这些指标总是必需的，因为它们在规范中被指定。可在*/metrics/base*访问它们。

vendor

供应商的具体信息。每个实施方案可能会提供不同的信息。可在*/metrics/vendor*访问它们。

application

使用MicroProfile Metrics扩展机制为该服务专门开发的自定义信息。可在*/metrics/application*访问它们。

你可以通过设置quarkus.smallrye-metrics.path属性来配置指标的暴露位置。默认情况下，这个属性被设置为/metrics。

另见

要了解关于MicroProfile Metrics的更多信息，请访问GitHub上的以下页面：

- Metrics for Eclipse MicroProfile（https://oreil.ly/Q875g）

9.4　创建指标

问题

如何监控一些自定义指标，如绩效指标或业务指标？

解决方案

MicroProfile Metrics规范提供了不同的注解来注册不同类型的监控参数，如计数器、持续时间和仪表。通过这些注解，你可以创建自定义指标，这些指标可能与业务或性能参数有关，而不是像内存和CPU这样的物理值。

以下是MicroProfile Metrics的注解：

注释	描述
org.eclipse.microprofile.metrics.annotation.Counted	统计调用次数
org.eclipse.microprofile.metrics.annotation.Timed	跟踪调用的持续时间
org.eclipse.microprofile.metrics.annotation.SimplyTimed	跟踪调用的持续时间，不需要计算平均值和分布。Timed 的简化版
org.eclipse.microprofile.metrics.annotation.Metered	跟踪调用的频率
org.eclipse.microprofile.metrics.annotation.Gauge	对一个注解字段或方法的离散值进行采样
org.eclipse.microprofile.metrics.annotation.ConcurrentGauge	用于测量并行调用的计数
org.eclipse.microprofile.metrics.annotation.Metric	用于注入一个指标。有效的类型有 Meter、Timer、Counter 和 Histogram。对 Metric 执行 Gauge 只能在 CDI 生产者中使用

我们来看看如何使用指标注解以及如何创建直方图指标。

计数器

计数器递增对用@Counted注解的方法的调用，可以在方法级别或类级别使用。

下面的例子计算一个方法的调用次数：

```
@Counted( ❶
        name = "number-of-transactions", ❷
        displayName = "Transactions", ❸
        description = "How many transactions have been processed" ❹
)
@POST
@Consumes(MediaType.APPLICATION_JSON)
public Response doTransaction(Transaction transaction) {
    return Response.ok().build();
}
```

❶注册计数器

❷计数器的名字

❸设置一个显示的值

❹计数器的描述

检查计数器监控：

```
./mvnw compile quarkus:dev

curl -d '{"from":"A", "to":"B", "amount":2000}' \
     -H "Content-Type: application/json" \
     -X POST http://localhost:8080/tx

curl localhost:8080/metrics/application

application_org_acme_TransactionResource_number_of_transactions_total 1.0
```

gauge

gauge类似于汽车上的油表，是一个简单的值，可以暴露它来测量。要注册它，需要用@Gauge注解一个字段或一个方法，值或返回值将自动暴露：

```
private long highestTransaction = 0; ❶

@POST
@Consumes(MediaType.APPLICATION_JSON)
public Response doTransaction(Transaction transaction) {
    if (transaction.amount > highestTransaction) { ❷


        highestTransaction = transaction.amount;
    }
    return Response.ok().build();
}
@Gauge( ❸
        name = "highest-gross-transaction", ❹
        description = "Highest transaction so far.",
        unit= MetricUnits.NONE ❺
)
public long highestTransaction() {
    return highestTransaction;
}
```

❶初始化highestTransaction，即存储最高级事物的字段

❷如果当前事物的级别更高，更新这个字段

❸返回值设置为Gauge

❹Gauge名

❺Gauge的指标（例如，秒、百分比、每秒、字节等）

执行以下命令来运行应用程序，产生一些指标数据，并查看输出：

```
./mvnw compile quarkus:dev

curl -d '{"from":"A", "to":"B", "amount":2000}' \
    -H "Content-Type: application/json" \
    -X POST http://localhost:8080/tx

curl localhost:8080/metrics/application

application_org_acme_TransactionResource_highest_gross_transaction 2000.0
```

metered

metered（计量的）指标衡量一个方法被调用的速度。可以在方法级别或类级别上使用@Metered注解：

```
@Metered( ❶
        name = "transactions",
        unit = MetricUnits.SECONDS, ❷
        description = "Rate of transactions"
)
```

❶注册计量的指标

❷设置单位为秒

执行以下命令来运行应用程序，产生一些指标数据，并查看输出：

```
./mvnw compile quarkus:dev

curl -d '{"from":"A", "to":"B", "amount":2000}' \
     -H "Content-Type: application/json" \
     -X POST http://localhost:8080/tx

curl localhost:8080/metrics/application

application_org_acme_TransactionResource_transactions \
  _rate_per_second  0.09766473618811813
application_org_acme_TransactionResource_transactions \
  _one_min_rate_per_second  0.015991117074135343
application_org_acme_TransactionResource_transactions \
  _five_min_rate_per_second  0.0033057092356765017
application_org_acme_TransactionResource_transactions \
  _fifteen_min_rate_per_second  0.0011080303990206543
```

timed

timed（定时）的指标可以衡量一个调用的持续时间。@Timed注解可以在方法级别或类级别上使用：

```
@Timed( ❶
    name = "average-transaction",
    unit = MetricUnits.SECONDS,
    description = "Average duration of transaction"
)
```

❶注册定时的指标

执行以下命令来运行应用程序，产生一些指标数据，并查看输出：

```
./mvnw compile quarkus:dev

curl -d '{"from":"A", "to":"B", "amount":2000}' \
     -H "Content-Type: application/json" \
     -X POST http://localhost:8080/tx

curl localhost:8080/metrics/application

application_org_acme_TransactionResource_average_transaction \
  _rate_per_second 0.7080455375154214
application_org_acme_TransactionResource_average_transaction \
  _one_min_rate_per_second 0.0
application_org_acme_TransactionResource_average_transaction \
  _five_min_rate_per_second 0.0
application_org_acme_TransactionResource_average_transaction \
  _fifteen_min_rate_per_second 0.0
application_org_acme_TransactionResource_average_transaction \
  _min_seconds 1.0693E-5
application_org_acme_TransactionResource_average_transaction \
```

```
  _max_seconds 4.9597E-5
application_org_acme_TransactionResource_average_transaction \
  _mean_seconds 3.0145E-5
application_org_acme_TransactionResource_average_transaction \
  _stddev_seconds 1.9452E-5
application_org_acme_TransactionResource_average_transaction \
  _seconds_count 2.0
application_org_acme_TransactionResource_average_transaction \
  _seconds{quantile="0.5"} 4.9597E-5
application_org_acme_TransactionResource_average_transaction \
  _seconds{quantile="0.75"} 4.9597E-5
application_org_acme_TransactionResource_average_transaction \
  _seconds{quantile="0.95"} 4.9597E-5
application_org_acme_TransactionResource_average_transaction \
  _seconds{quantile="0.98"} 4.9597E-5
application_org_acme_TransactionResource_average_transaction \
  _seconds{quantile="0.99"} 4.9597E-5
application_org_acme_TransactionResource_average_transaction \
  _seconds{quantile="0.999"} 4.9597E-5
```

直方图

直方图测量值在时间上的分布，它测量的是最小值、最大值、标准差以及量子值（如中位数或第95位）。直方图没有合适的注解，但org.eclipse.microprofile.metrics.Histogram类可以用来更新指标：

```
@Metric(name = "transaction-evolution") ❶
Histogram transactionHistogram;

@POST
@Consumes(MediaType.APPLICATION_JSON)
public Response doTransaction(Transaction transaction) {
    transactionHistogram.update(transaction.amount); ❷
    return Response.ok().build();
}
```

❶注入一个具有给定名称的直方图

❷用新值更新直方图

执行以下命令来运行应用程序，输入一些指标数据，并查看输出：

```
./mvnw compile quarkus:dev

curl -d '{"from":"A", "to":"B", "amount":2000}' \
     -H "Content-Type: application/json" \
     -X POST http://localhost:8080/tx

curl localhost:8080/metrics/application
```

```
application_org_acme_TransactionResource_transaction_evolution_min 2000.0
application_org_acme_TransactionResource_transaction_evolution_max 2000.0
application_org_acme_TransactionResource_transaction_evolution_mean 2000.0
application_org_acme_TransactionResource_transaction_evolution_stddev 0.0
application_org_acme_TransactionResource_transaction_evolution_count 2.0
application_org_acme_TransactionResource_transaction_evolution \
  {quantile="0.5"}  2000.0
application_org_acme_TransactionResource_transaction_evolution \
  {quantile="0.75"}  2000.0
application_org_acme_TransactionResource_transaction_evolution \
  {quantile="0.95"}  2000.0
application_org_acme_TransactionResource_transaction_evolution \
  {quantile="0.98"}  2000.0
application_org_acme_TransactionResource_transaction_evolution \
  {quantile="0.99"}  2000.0
application_org_acme_TransactionResource_transaction_evolution \
  {quantile="0.999"}  2000.0
```

讨论

你可以通过使用OPTION HTTP方法查询到特定的endpoint，从任何指标中获取元数据信息。元数据暴露在/metrics/*scope*/*metric_name*中，其中*scope*是base、vendor或application，*metric_name*是指标的名称（如果是application，则是name属性中设置的名称）。

9.5　使用分布式跟踪

问题

如何对整个应用程序进行分析和监控？

解决方案

MicroProfile OpenTracing规范使用OpenTracing标准API（https://opentracing.io）来检测分布式跟踪的微服务。Quarkus集成了MicroProfile OpenTracing规范作为分布式跟踪的解决方案。分布式跟踪是一种用于分析和监控分布式系统的方法，可以用来检测服务之间的通信故障，确定哪些点是性能问题，或者对网络网格内发生的所有请求和响应进行日志记录。

进行分布式跟踪之前，你必须了解OpenTracing中的5个重要概念：

span（跨度）

一个命名的操作，代表一个已完成的工作单元（例如，执行的服务）。一个span可以包含更多的span，以父-子形式存在。

span context

从一个服务传播到另一个服务的跟踪信息（例如，span ID）。

baggage item

服务之间传播的自定义键/值对。

tag（标签）

由用户定义的键/值对，这些键/值对在span中设置，以便可以查询和过滤（例如http.status_code）。

log（日志）

与包含日志信息或其他重要信息的span相关的键/值对。日志用于识别span中的特定时刻。同时，标签适用于整个span，不受时间限制。

在这个例子中，Jaeger（https://www.jaegertracing.io）服务器被用来收集来自你的应用程序的所有痕迹，并使它们可以被使用或查询。图9-1显示了服务和Jaeger之间的交互。

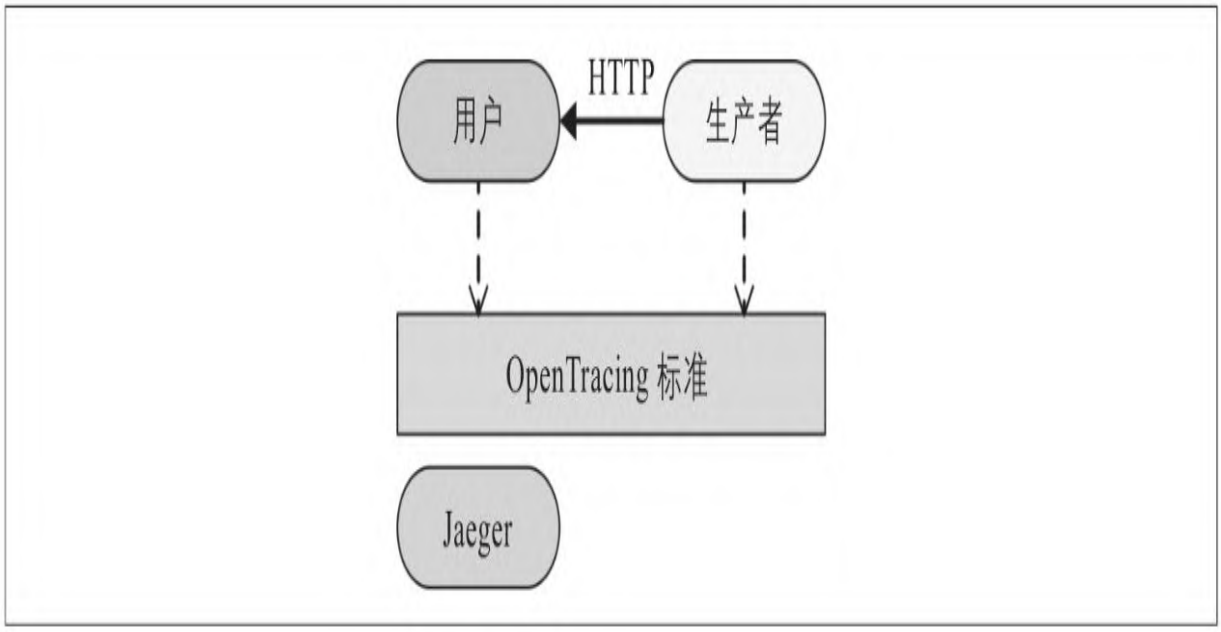


图9-1：微服务与Jaeger

图9-2显示了前面解释的Jaeger概念。

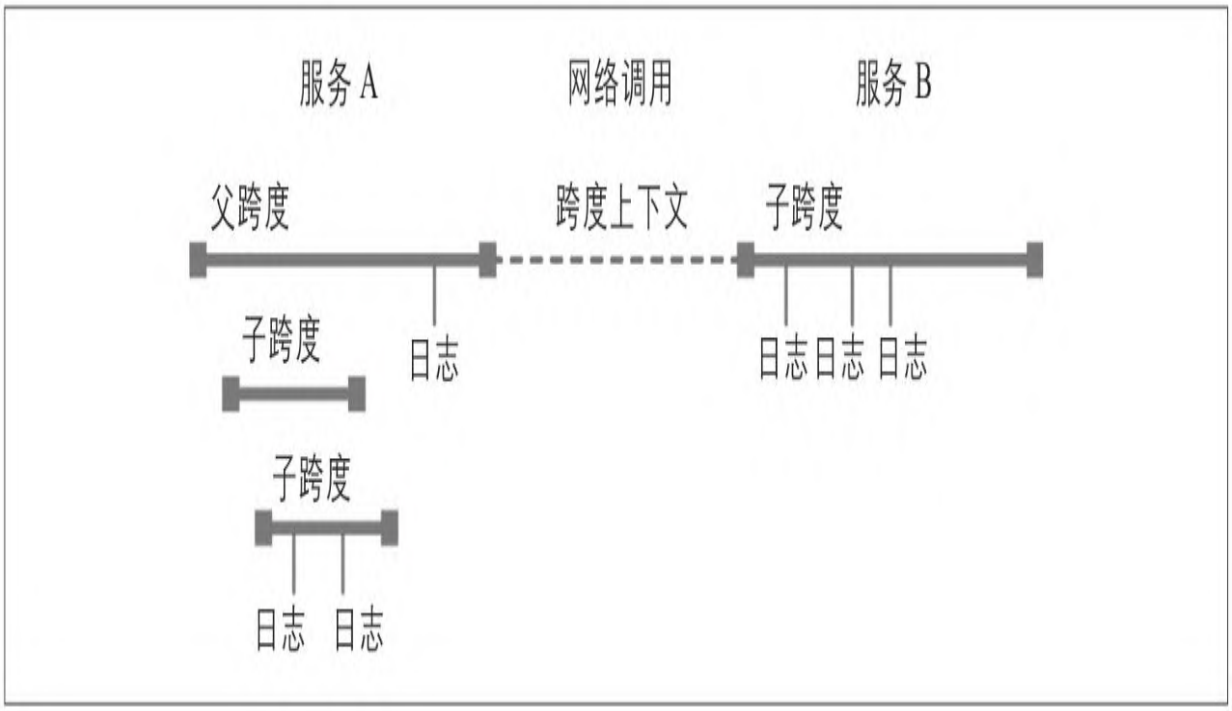


图9-2：Jaeger概念

使用jaegertracing/all-in-one容器镜像是因为它在一个镜像中包含了所有的Jaeger后台组件和UI。这并不意味着要在生产中使用，但是为了简单起见，这是9.5节中使用的镜像：

```
docker run -e COLLECTOR_ZIPKIN_HTTP_PORT=9411 -p 5775:5775/udp \
    -p 6831:6831/udp -p 6832:6832/udp -p 5778:5778 -p 16686:16686 \
    -p 14268:14268 -p 9411:9411 jaegertracing/all-in-one:1.15.1
```

要在Quarkus应用程序中启用MicroProfile OpenTracing，只需要注册quarkus-smallrye-opentracing扩展。

```
./mvnw quarkus:add-extension -Dextensions="quarkus-smallrye-opentracing"
```

通过在classpath中的扩展，Quarkus/MicroProfile OpenTracing会发送默认的跟踪信息到Jaeger服务器。你唯一需要做的就是确定Jaeger端点，在那里所有的跟踪信息都应该被发送。

默认收集的跟踪信息包括以下内容：

· 硬件指标，如CPU、内存和可用处理器。

· JVM指标，如内存堆和线程池。

MicroProfile OpenTracing为每个入站请求创建一个新的span。这个新span的默认名称是*HTTP method:package name.class name.method name*。

为输入请求创建的span将包含以下具有正确值的标签：

· Tags.SPAN_KIND=Tags.SPAN_KIND_SERVER。

· Tags.HTTP_METHOD，包含输入请求中使用的HTTP方法。

· Tags.HTTP_URL，包含输入端点的URL。

· 带有HTTP状态结果码的Tags.HTTP_STATUS。

· Tags.COMPONENT="jaxrs"。

· 如果发生服务器错误（5XX错误代码），则将Tags.ERROR改为true；如果异常有一个提供的对象，则添加两个日志，一个是event=error，另一个是error.object=<error object instance>。

在出站请求的情况下，会创建一个新的span，它是当前活动span（如果存在的话）的子span。新span的默认名称是<HTTP method>。为输出请求创建的span将包含以下具有正确值的标签：

· Tags.SPAN_KIND=Tags.SPAN_KIND_SCLIENT。

· Tags.HTTP_METHOD，包含输出请求时使用的HTTP方法。

· Tags.HTTP_URL，包含输出终端的URL。

· 带有HTTP状态结果码的Tags.HTTP_STATUS。

· Tags.COMPONENT="jaxrs"。

· 如果发生客户端错误（4XX错误代码），则将Tags.ERROR改为true；如果异常有一个提供的对象，则添加两个日志，一个是event=error，另一个是error.object=<error object instance>。

最后要做的是配置Jaeger参数：

```
quarkus.jaeger.service-name=shopping-cart ❶
quarkus.jaeger.sampler-type=const ❷
quarkus.jaeger.sampler-param=1 ❸
quarkus.jaeger.endpoint=http://localhost:14268/api/traces ❹
```

❶Jaeger中能够识别的服务名

❷设置一个sampler

❸请求的采样百分比（1就是采样全部）

❹Jaeger服务器的位置

然后启动应用程序并向服务中定义的一个终端发送一些请求。之后，通过访问Jaeger UI来检查所有分布式跟踪。打开浏览器，访问http://localhost:16686（Jaeger UI）来查看跟踪信息。

在初始页面中，可以通过几个参数进行筛选，但其中一个参数是用来选择查看完成的请求的服务。Jaeger的主页如图9-3所示。

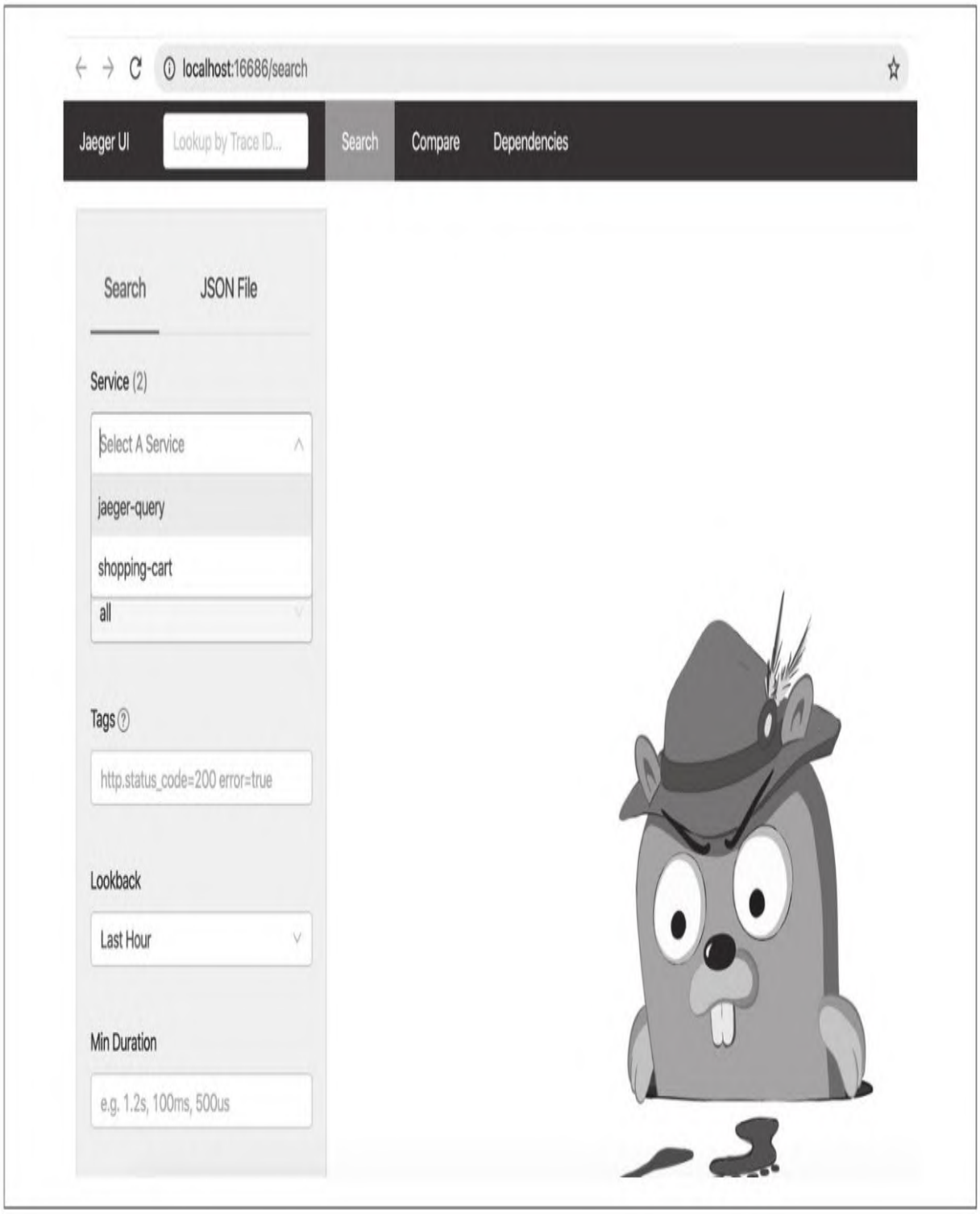


图9-3：Jaeger的主页

按下Find Traces按钮，选择所有符合给定条件的痕迹，你应该看到如图9-4所示的图像。

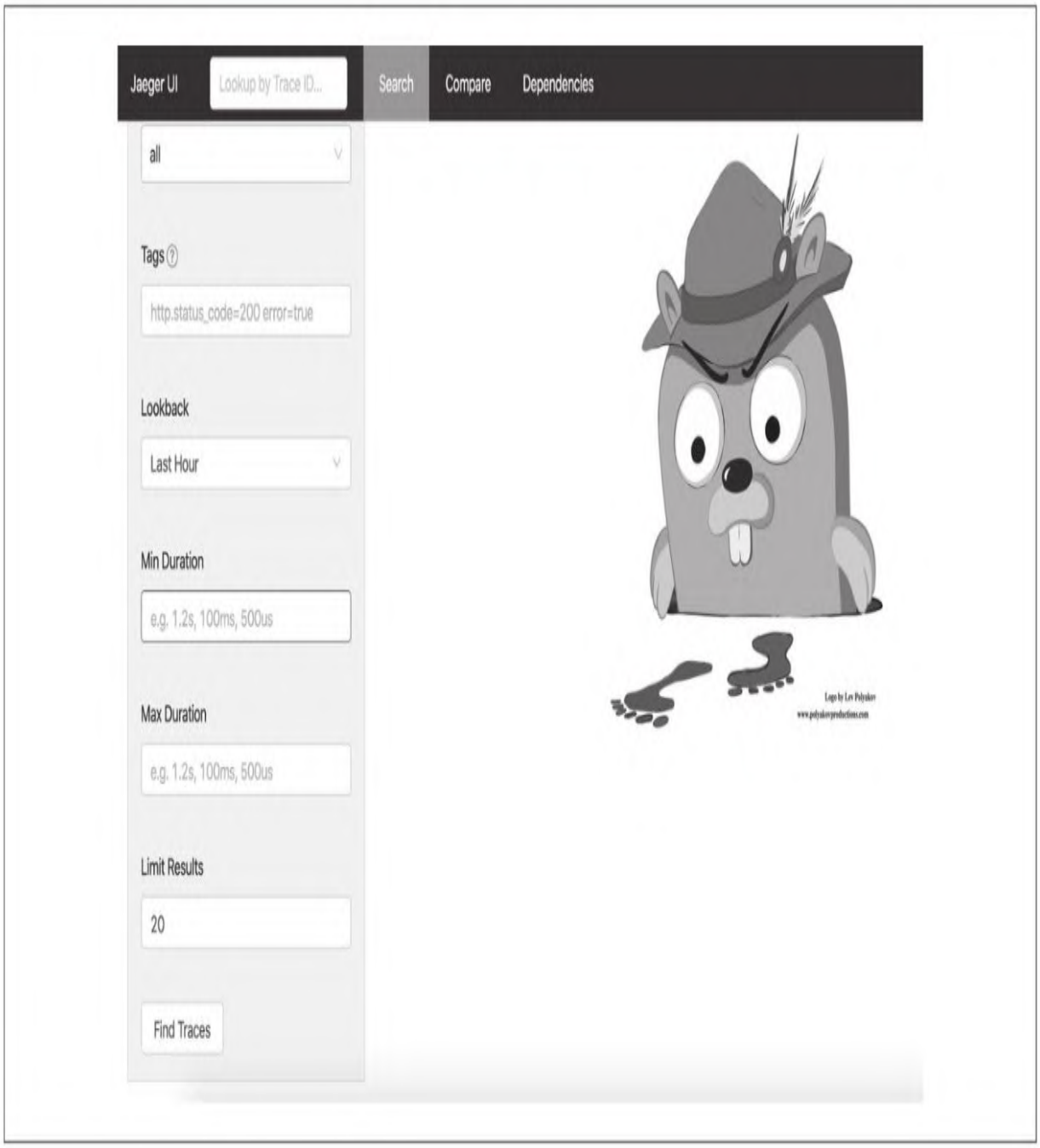


图9-4：Find traces

你会看到所有符合标准的请求。在这种情况下，这些请求都是涉及shopping-cart的痕迹，如图9-5所示。

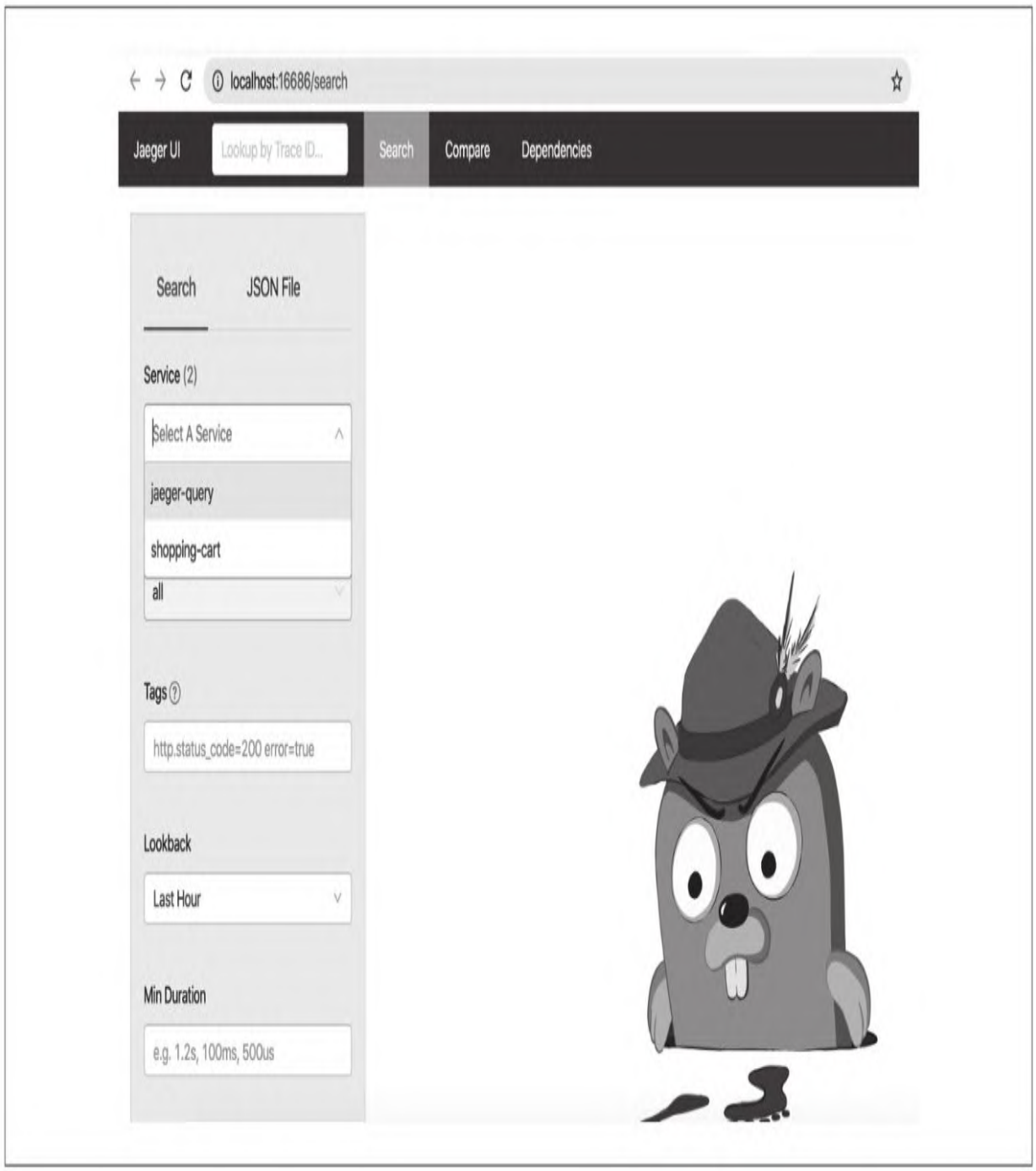


图9-5：视图痕迹

单击任何一个请求，会显示具体请求的更多细节，如图9-6所示。

图9-6：请求细节

如果出现错误，则增加一个新的日志条目，设置错误信息，如图9-7所示。

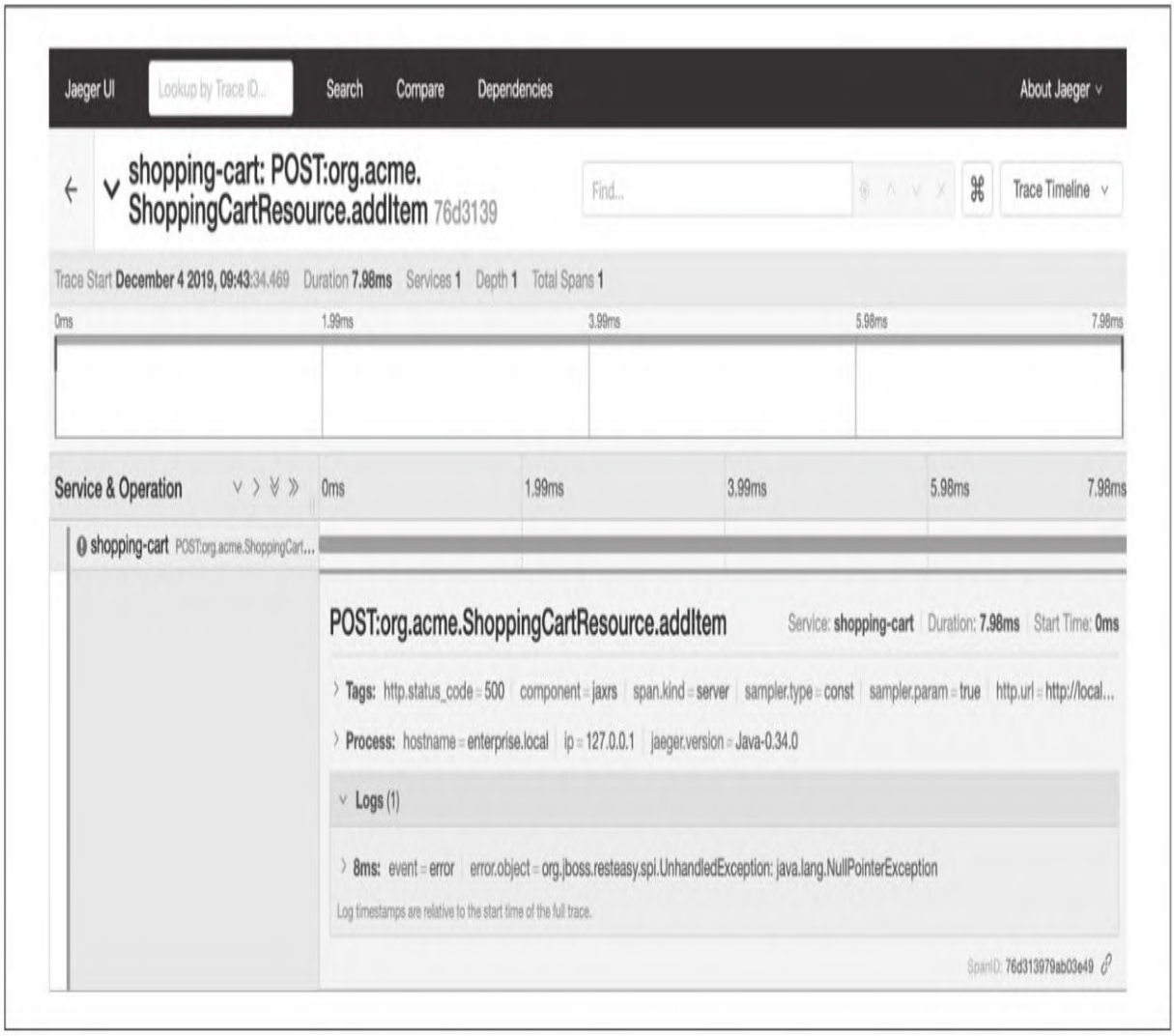


图9-7：错误日志信息

禁用跟踪

任何请求（传入或传出）都会被默认跟踪。通过使用@org.eclipse.microprofile.opentracing.Traced注释可以禁用特定类或方法的跟踪：

```
@Traced(false)
public class TransactionResource {}
```

讨论

通过设置*src/main/resources/application.properties*文件中的选项或使用9.6节中讨论的任何其他方法来配置OpenTracing。表9-1列出了一些最重要的配置属性。

表9-1：OpenTracing配置属性

属性	描述
quarkus.jaeger.enabled	定义是否启用 Jaeger 扩展（默认为 true）。它是一个构建属性，不能在运行时修改
quarkus.jaeger.endpoint	跟踪服务器终端
quarkus.jaeger.auth-token	终端的认证令牌
quarkus.jaeger.user	作为认证的一部分发送到终端的用户名
quarkus.jaeger.password	作为认证的一部分发送到终端的密码
quarkus.jaeger.sampler-type	采样器类型（const, probabilistic, ratelimiting. 或 remote）。
quarkus.jaeger.sampler-param	流量采样的百分比 (0.0-1.0)
quarkus.jaeger.service-name	服务名称
quarkus.jaeger.tags	逗号分隔的键 / 值标签列表，这些标签将被添加到所有 span 中。使用 ${environment Var:default} 支持环境变量
quarkus.jaeger.propagation	用于传播跟踪上下文的格式（默认为 jaeger）。可能的值是 jaeger 和 b3
quarkus.jaeger.sender-factory	发送者工厂的类名

另见

请参阅Quarkus的OpenTracing使用指南（https://oreil.ly/A2GJu），了解支持的全部属性列表。更多关于MicroProfile OpenTracing规范的信息可以在GitHub上找到：

- MicroProfile OpenTracing（https://oreil.ly/v7kjr）

9.6　自定义分布式跟踪

问题

如何在当前的tracing span中添加自定义信息？

解决方案

MicroProfile OpenTracing规范使用io.opentracing.Tracer类在当前span中添加新信息。

在某些情况下，需要创建一个新的子span或在当前span中添加信息，如新标签、日志信息或baggage项目。为了添加这些信息，MicroProfile OpenTracing产生一个io.opentracing.Tracer类的实例来操作当前span。

假设你想标记所有重要客户提出的请求。在这个例子中，重要客户是指那些ID以1开头的客户：

```
@Inject ❶
Tracer tracer;
@POST
@Path("/add/{customerId}")
@Transactional
@Consumes(MediaType.APPLICATION_JSON)
public Response addItem(@PathParam("customerId") String customerId, Item item) {

    if (customerId.startsWith("1")) {
        tracer.activeSpan().setTag("important.customer", true); ❷
    }
}
```

❶注入一个Tracer实例

❷在目前的span中创建一个新的tag

然后对重要客户的任何请求都会有相应的标签。

自定义标签如图9-8所示。

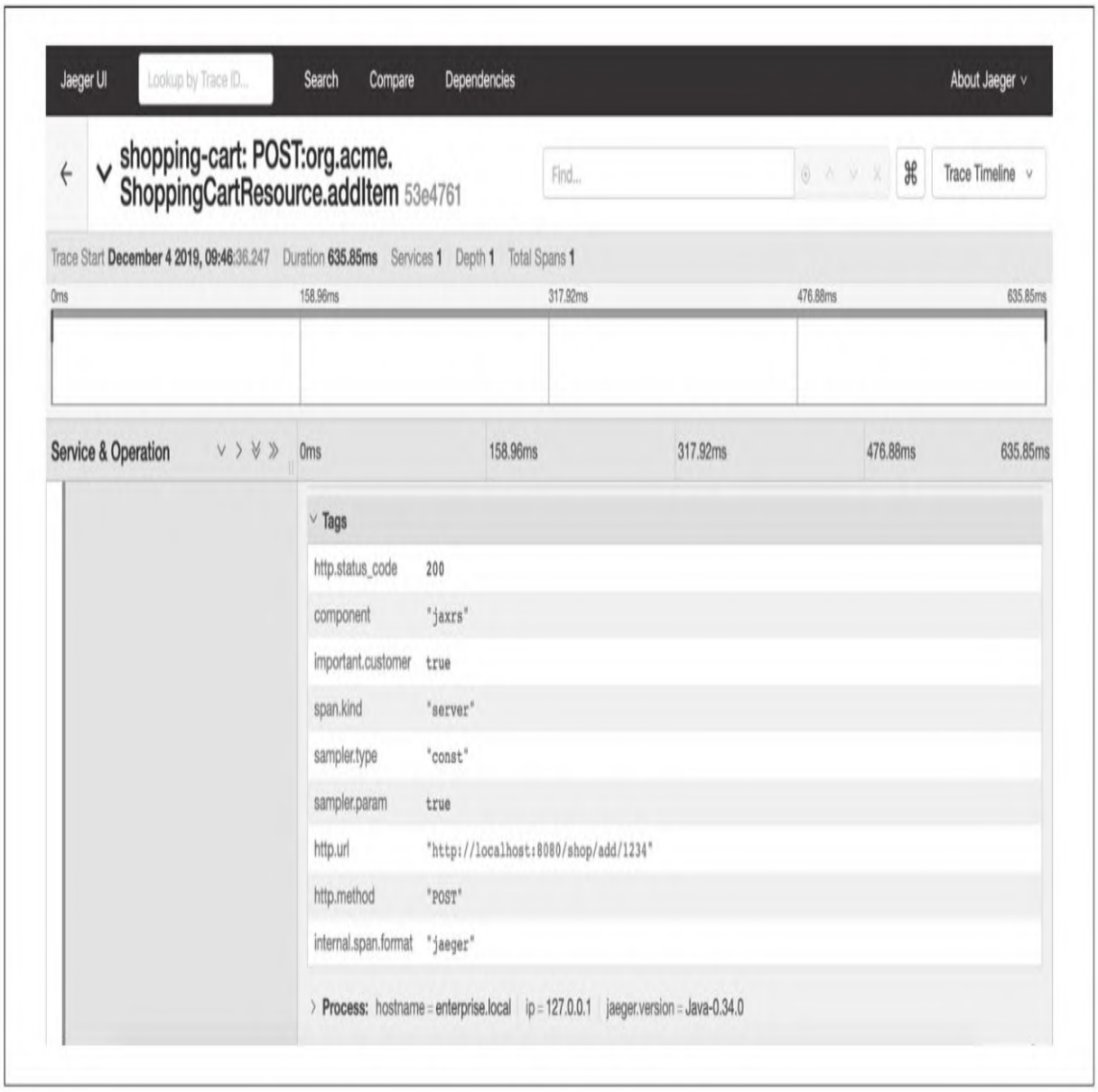


图9-8：自定义标签

讨论

Quarkus支持OpenTracing定制的其中一个工具JDBC，所以如果你想监控SQL查询，不需要自己定制当前的span，可以使用以依赖项形式

提供的集成。

在你的构建工具中注册opentracing-jdbc工具：

```
<dependency>
    <groupId>io.opentracing.contrib</groupId>
    <artifactId>opentracing-jdbc</artifactId>
</dependency>
```

然后激活JDBC连接的跟踪。这是通过在JDBC URL中添加单词tracing来实现的。因为Quarkus使用JPA，你还需要配置数据源和Hibernate来使用专用的跟踪驱动：

```
quarkus.datasource.url=jdbc:tracing:h2:mem:mydb ❶
quarkus.datasource.driver=io.opentracing.contrib.jdbc.TracingDriver ❷
quarkus.datasource.username=sa
quarkus.datasource.password=
quarkus.hibernate-orm.database.generation=update
quarkus.hibernate-orm.log.sql=true
quarkus.hibernate-orm.dialect=org.hibernate.dialect.H2Dialect ❸
```

❶使用tracing更新JDBC URL

❷设置TracingDriver取代数据库驱动

❸配置真实数据库的dialect

在请求中完成的所有查询也都反映在Jaeger UI中。

JDBC的痕迹如图9-9所示。

图9-9：JDBC痕迹

如果仔细观察，你会发现图9-9中有一个新的标签db.statement，它反映了被跟踪的查询。另外，注意到有一个购物车span，同时还包含6个span，每个对应一个查询。

要忽略特定的查询，你可以设置（多次）ignoreForTracing属性与要忽略的查询（例如，jdbc:tracing:h2:mem:test?ignoreForTracing=SELECT * FROM \ "TEST" \）。

第10章　与Kubernetes集成

到目前为止，我们一直在学习如何在裸机上开发和运行Quarkus应用，但是在Kubernetes集群中运行时Quarkus才能真正闪光。

在本章中，你将了解如何将Quarkus和Kubernetes集成以及用于在Kubernetes中开发和部署Quarkus服务的扩展。

Kubernetes成为部署应用程序的事实平台。为此，充分了解Kubernetes以及如何在其上正确开发和部署应用程序非常重要。

本章涵盖以下主题：

- 构建和推送容器镜像
- 生成Kubernetes资源
- 部署一个Quarkus服务
- 开发一个Kubernetes Operator
- 在Knative中部署一个服务

10.1　构建和推送容器镜像

问题

如何构建和推送容器镜像？

解决方案

Kubernetes中的工作单元是一个pod。一个pod代表一组容器，它们运行在同一台主机上，并共享IP和端口等资源。要将一个服务部署到Kubernetes中，需要创建一个pod。由于pod是由一个或多个容器组成的，所以需要构建服务的容器镜像。

Quarkus提供了构建和可选地推送容器镜像的扩展。在撰写本书时，支持以下容器构建策略：

Jib

Jib为Java应用构建Docker和OCI容器镜像，无须Docker守护进程（Dockerless）。这使得它非常适合在容器内运行进程时构建Docker镜像，因为这避免了Docker-in-Docker（DinD）进程的麻烦。此外，使用Quarkus的Jib可以将所有的依赖项缓存在与实际应用不同的层中，使得重建速度快、规模小。这缩减了推送时间和构建时间。

Docker

运用Docker策略，使用docker二进制文件构建容器镜像，该二进制文件安装在本地，默认使用位于*src/main/docker*下的Dockerfiles来构建镜像。

S2I

Source-to-Image（S2I）策略使用s2i二进制构建在OpenShift集群内进行容器构建。S2I构建需要创建一个BuildConfig和两个ImageStream资源。这些资源的创建由Quarkus Kubernetes扩展来实现。

在这个配置中，我们将使用Jib来构建和推送容器。这个配置的“讨论”部分将讨论Docker和S2I。

要使用Jib构建和推送容器镜像，首先需要添加Jib扩展项：

```
./mvnw quarkus:add-extensions -Dextensions="quarkus-container-image-jib"
```

然后可以自定义容器镜像的构建过程。可以在*application.properties*、系统属性或环境变量中设置这些属性，与在Quarkus中设置任何其他配置参数一样：

```
quarkus.container-image.group=lordofthejars ❶
quarkus.container-image.registry=quay.io ❷
quarkus.container-image.username=lordofthejars ❸
#quarkus.container-image.password= ❹
```

❶设置镜像的分组，这个值默认是${user.name}

❷镜像推向的仓库，默认镜像被推向docker.io

❸镜像仓库的用户名

❹镜像仓库的密码

要为项目建立并推送容器镜像，需要将quarkus.container-image.push参数设置为true，在打包阶段，创建并推送容器：

```
./mvnw clean package -Dquarkus.container-image.push=true

...
[INFO] --- maven-jar-plugin:2.4:jar (default-jar) @ greeting-jib ---
[INFO] Building jar: /greeting-jib/target/greeting-jib-1.0-SNAPSHOT.jar
[INFO]
[INFO] --- quarkus-maven-plugin:1.3.0.CR2:build (default) @ greeting-jib ---
[INFO] [org.jboss.threads] JBoss Threads version 3.0.1.Final
[INFO] [io.quarkus.deployment.pkg.steps.JarResultBuildStep] Building thin jar:
    greeting-jib/target/greeting-jib-1.0-SNAPSHOT-runner.jar
[INFO] [io.quarkus.container.image.jib.deployment.JibProcessor]
    Starting container image build ❶
[WARNING] [io.quarkus.container.image.jib.deployment.JibProcessor]
    Base image 'fabric8/java-alpine-openjdk8-jre' does not use a specific image
    digest - build may not be reproducible
[INFO] [io.quarkus.container.image.jib.deployment.JibProcessor] LogEvent
    [level=INFO, message=trying docker-credential-desktop for quay.io]
[INFO] [io.quarkus.container.image.jib.deployment.JibProcessor] LogEvent
    [level=LIFECYCLE, message=Using credentials from Docker config
    ($HOME/.docker/config.json) for
    quay.io/lordofthejars/greeting-jib:1.0-SNAPSHOT]
[INFO] [io.quarkus.container.image.jib.deployment.JibProcessor] The base image
    requires auth. Trying again for fabric8/java-alpine-openjdk8-jre...
[INFO] [io.quarkus.container.image.jib.deployment.JibProcessor] Using base
    image with digest:
    sha256:a5d31f17d618032812ae85d12426b112279f02951fa92a7ff8a9d69a6d3411b1
[INFO] [io.quarkus.container.image.jib.deployment.JibProcessor] Container
    entrypoint set to [java, -Dquarkus.http.host=0.0.0.0,
    -Djava.util.logging.manager=org.jboss.logmanager.LogManager,
    -cp, /app/resources:/app/classes:/app/libs/*,
        io.quarkus.runner.GeneratedMain]
[INFO] [io.quarkus.container.image.jib.deployment.JibProcessor] Pushed
container image quay.io/lordofthejars/greeting-jib:1.0-SNAPSHOT
(sha256:e173e0b49bd5ec1f500016f46f2cde03a055f558f72ca8ee1d6cb034a385a657)❷

[INFO] [io.quarkus.deployment.QuarkusAugmentor] Quarkus augmentation completed
    in 12980ms
```

❶这个容器镜像被构建

❷容器镜像被推向quay.io

讨论

除了Jib，还有另外两个选项可以用来构建容器镜像。要使用它们，只需要注册扩展：

Docker

quarkus-container-image-docker

S2I

quarkus-container-image-s2i

每个扩展都提供了特定的配置参数来更改构建过程。利用这些参数，可以更改用于构建容器镜像的基础镜像，并设置环境变量、传递给可执行文件的参数或Dockerfile的位置。

也可以建立镜像但不推送它到仓库。为此，需要将quarkus.container-image.build属性设置为true，而不是设置quarkus.container-image.push属性：

```
./mvnw clean package -Dquarkus.container-image.build=true
```

如果使用Jib，并且将push设置为false，扩展就会创建一个容器镜像，并将其注册到Docker守护进程中。这意味着虽然没有使用Docker来构建镜像，但它仍然是必要的。

容器镜像扩展可以从JAR包（用于JVM模式）和原生可执行文件创建容器，这取决于在*build/output*目录中找到的内容。如果你想创建一个可从Linux容器中运行的原生可执行文件，然后创建一个包含所产生的原生可执行文件的容器镜像，可以运行以下命令：

```
./mvnw clean package -Dquarkus.container-image.push=true -Pnative \
    -Dquarkus.native.container-build=true
```

将quarkus.native.container-build属性设置为true，会在Docker容器内创建原生可执行文件。

另见

更多信息，请访问GitHub上的以下页面：

· Google Container Tools:Jib（https://oreil.ly/6vrh5）

- Source-To-Image（https://oreil.ly/8PEgn）

10.2　生成Kubernetes资源

问题

如何自动化创建Kubernetes资源？

解决方案

Quarkus有一个Kubernetes扩展，能够自动生成Kubernetes资源，有合理的默认值和用户提供的可选配置。目前，该扩展可以为Kubernetes和OpenShift生成资源。

要启用Kubernetes资源的生成，需要注册quarkus-kubernetes扩展插件：

```
./mvnw quarkus:add-extension -Dextensions="quarkus-kubernetes"
```

要生成Kubernetes资源，在新的终端执行./mvnw package。然后在target目录下的构建工具生成的常规文件中，在target/kubernetes目录内创建两个新文件。这些新文件被命名为*kubernetes.json*和*kubernetes.yaml*，它们分别包含Deployment和Service的定义：

```
{
  "apiVersion" : "v1",
  "kind" : "List",
  "items" : [ {
    "apiVersion" : "v1",
    "kind" : "Service",
    "metadata" : {
      "labels" : {
        "app" : "getting-started", ❶
        "version" : "1.0-SNAPSHOT", ❷
        "group" : "alex" ❸
      },
      "name" : "getting-started"
    },
    "spec" : {
      "ports" : [ {
        "name" : "http",
        "port" : 8080,
        "targetPort" : 8080
      } ],
      "selector" : {
        "app" : "getting-started",
        "version" : "1.0-SNAPSHOT",
        "group" : "alex"
      },
      "type" : "ClusterIP"
    }
}, {
  "apiVersion" : "apps/v1",
  "kind" : "Deployment",
  "metadata" : {
    "labels" : {
      "app" : "getting-started",
      "version" : "1.0-SNAPSHOT",
      "group" : "alex"
    },
    "name" : "getting-started"
  },
  "spec" : {
    "replicas" : 1,
    "selector" : {
```

```
        "matchLabels" : {
          "app" : "getting-started",
          "version" : "1.0-SNAPSHOT",
          "group" : "alex"
        }
      },
      "template" : {
        "metadata" : {
          "labels" : {
            "app" : "getting-started",
            "version" : "1.0-SNAPSHOT",
            "group" : "alex"
          }
        },
        "spec" : {
          "containers" : [ {
            "env" : [ {
              "name" : "KUBERNETES_NAMESPACE",
              "valueFrom" : {
                "fieldRef" : {
                  "fieldPath" : "metadata.namespace"
                }
              }
            } ],
            "image" : "alex/getting-started:1.0-SNAPSHOT",
            "imagePullPolicy" : "IfNotPresent",
            "name" : "getting-started",
            "ports" : [ {
              "containerPort" : 8080,
              "name" : "http",
              "protocol" : "TCP"
            } ]
          } ]
        }
      }
    } ]
}
```

❶项目名默认值

❷版本名默认值

❸OS用户名默认值

讨论

可以通过将这些属性添加到*application.properties*中，自定义组和生成的清单中使用的名称：

```
quarkus.container-image.group=redhat
quarkus.application.name=message-app
```

Kubernetes扩展允许向清单的不同部分提供用户自定义功能：

```
quarkus.kubernetes.replicas=3 ❶

quarkus.container-image.registry=http://my.docker-registry.net ❷
quarkus.kubernetes.labels.environment=prod ❸

quarkus.kubernetes.readiness-probe.initial-delay-seconds=10 ❹
quarkus.kubernetes.readiness-probe.period-seconds=30
```

❶设置副本数量

❷增加推送镜像的Docker仓库

❸增加新的标签

❹设置readiness探针

可以通过在*application.properties*文件中设置属性为quarkus.kubernetes.deployment-target或系统属性来生成不同的资源。

此属性的默认值是kubernetes，但在撰写本书时也支持以下值：kubernetes、openshift和knative。

另见

下面的网页提供了修改生成文件的所有Kubernetes配置选项的列表：

- Quarkus:Kubernetes Extension（https://oreil.ly/oLxhT）

10.3　生成带有健康检查的Kubernetes资源

问题

如何自动生成带有liveness和readiness探针的Kubernetes资源？

解决方案

默认情况下，健康探针不会在输出文件中生成，但如果quarkus-smallrye-health扩展名存在（如9.1节中所解释的），那么liveness和readiness探针部分会自动生成：

```
"image" : "alex/getting-started:1.0-SNAPSHOT",
"imagePullPolicy" : "IfNotPresent",
"livenessProbe" : { ❶
    "failureThreshold" : 3,
    "httpGet" : {
```

```
        "path" : "/health/live", ❷
        "port" : 8080,
        "scheme" : "HTTP"
        },
    "initialDelaySeconds" : 0,
    "periodSeconds" : 30,
    "successThreshold" : 1,
    "timeoutSeconds" : 10
},
"name" : "getting-started",
"ports" : [ {
    "containerPort" : 8080,
    "name" : "http",
    "protocol" : "TCP"
    } ],
"readinessProbe" : { ❸
    "failureThreshold" : 3,
    "httpGet" : {
        "path" : "/health/ready", ❹
        "port" : 8080,
        "scheme" : "HTTP"
    },
    "initialDelaySeconds" : 0,
    "periodSeconds" : 30,
    "successThreshold" : 1,
    "timeoutSeconds" : 10
}
```

❶定义liveness探针

❷定义健康检查的path

❸定义readiness探针

❹定义健康检查的path

讨论

Kubernetes使用探针来确定服务的健康状态，并采取自动行为来解决问题。

Quarkus自动生成两个Kubernetes探针：

liveness

Kubernetes使用liveness探针来决定是否必须重启服务。如果应用程序变得无响应，可能是因为死锁或内存问题，重启容器可能是解决问题的好办法。

readiness

Kubernetes使用readiness探针来决定一个服务是否可以接受流量。有时，一个服务在接受请求之前可能需要执行一些操作。例如，

更新本地缓存系统、填充数据库模式的变化、应用批处理，或者连接到Kafka Streams等外部服务。

另见

更多信息，请见以下网站：

· Kubernetes:Configure Liveness,Readiness,and Startup Probes（https://oreil.ly/PWl_B）

10.4　在Kubernetes上部署服务

问题

如何在Kubernetes上部署服务？

解决方案

使用kubectl和Quarkus提供的功能在Kubernetes上创建和部署服务。

Quarkus通过以下操作，可以很容易在Kubernetes中创建并部署一个Java应用：

1.生成你的企业应用程序的容器原生可执行文件，如6.6节所述。

2.提供一个Dockerfile.native文件来构建Docker容器。

3.如10.2节所述，使用quarkus-kubernetes扩展生成Kubernetes资源文件。

下面介绍这些步骤。

创建一个可以在容器中运行的原生可执行文件：

```
./mvnw package -DskipTests -Pnative -Dquarkus.native.container-build=true ❶

docker build -f src/main/docker/Dockerfile.native \
    -t alex/geeting-started:1.0-SNAPSHOT . ❷
docker push docker build -f src/main/docker/Dockerfile.native \
    -t alex/getting-started:1.0-SNAPSHOT . ❸

kubectl apply -f target/kubernetes/kubernetes.json ❹

kubectl patch svc getting-started --type='json' \
    -p '[{"op":"replace","path":"/spec/type","value":"NodePort"}]' ❺

curl $(minikube service getting-started --url)/hello ❻
```

❶在Docker容器中创建原生可执行文件

❷使用之前产生的原生可执行文件创建Docker镜像

❸将镜像推向Docker镜像仓库（在minikube中，这是eval $(minikube docker-env)，无须推送）

❹在Kubernetes上部署应用

❺修改NodePort的值

❻获取访问服务的URL

请注意，步骤5和6是必需的，因为服务部署在minikube中。部署服务的Kubernetes平台不同，可能需要做不同的事情。

讨论

如果使用多阶段Docker构建功能，步骤1和步骤2可以简化为一个。在第一阶段，生成原生可执行文件，第二阶段创建运行时镜像：

```
FROM quay.io/quarkus/centos-quarkus-maven:19.2.1 AS build ❶
COPY src /usr/src/app/src
COPY pom.xml /usr/src/app
USER root
RUN chown -R quarkus /usr/src/app
USER quarkus
RUN mvn -f /usr/src/app/pom.xml -Pnative clean package

FROM registry.access.redhat.com/ubi8/ubi-minimal ❷
WORKDIR /work/
COPY --from=build /usr/src/app/target/*-runner /work/application
RUN chmod 775 /work
EXPOSE 8080
CMD ["./application", "-Dquarkus.http.host=0.0.0.0"]
```

❶生成原生可执行文件

❷从上一步生成的结果创建运行时镜像

从项目的根目录中删除.dockerignore文件（rm.docker ignore）。这是必要的，因为默认情况下，src目录是被忽略的，而要

构建原生可执行文件，需要src目录：

```
docker build -f src/main/docker/Dockerfile.multistage -t docker \
    build -f src/main/docker/Dockerfile.multistage -t ❶
```

❶创建带有原生可执行文件的运行时镜像

10.5　在OpenShift上部署服务

问题

如何在OpenShift上部署服务？

解决方案

OpenShift与之前生成的资源完美地结合在一起，所以即使你使用的是OpenShift，仍然可以使用之前提供的所有功能。但如果你想使用OpenShift提供的一些功能，需要将kubernetes.deployment.target属性设置为openshift。

生成的两个文件分别放在*target/kubernetes/openshift.json*和*target/kubernetes/openshift.yaml*中：

```
{
  "apiVersion" : "v1",
  "kind" : "List",
  "items" : [ {
    "apiVersion" : "v1",
  "kind" : "Service",
  "metadata" : {
    "labels" : {
      "app" : "getting-started",
      "version" : "1.0-SNAPSHOT",
      "group" : "alex"
    },
    "name" : "getting-started"
  },
  "spec" : {
    "ports" : [ {
      "name" : "http",
      "port" : 8080,
      "targetPort" : 8080
    } ],
    "selector" : {
      "app" : "getting-started",
      "version" : "1.0-SNAPSHOT",
      "group" : "alex"
    },
    "type" : "ClusterIP"
  }
}, {
  "apiVersion" : "image.openshift.io/v1",
  "kind" : "ImageStream",
  "metadata" : {
    "labels" : {
      "app" : "getting-started",
      "version" : "1.0-SNAPSHOT",
      "group" : "alex"
    },
    "name" : "getting-started"
  }
}, {
  "apiVersion" : "image.openshift.io/v1",
  "kind" : "ImageStream",
  "metadata" : {
```

```
    "labels" : {
      "app" : "getting-started",
      "version" : "1.0-SNAPSHOT",
      "group" : "alex"
    },
    "name" : "s2i-java"
  },
  "spec" : {
    "dockerImageRepository" : "fabric8/s2i-java"
  }
}, {
  "apiVersion" : "build.openshift.io/v1",
  "kind" : "BuildConfig",
  "metadata" : {
    "labels" : {
      "app" : "getting-started",
      "version" : "1.0-SNAPSHOT",
      "group" : "alex"
    },
    "name" : "getting-started"
  },
  "spec" : {
    "output" : {
      "to" : {
        "kind" : "ImageStreamTag",
        "name" : "getting-started:1.0-SNAPSHOT"
      }
    },
    "source" : {
      "binary" : { }
    },
    "strategy" : {
      "sourceStrategy" : {
        "from" : {
          "kind" : "ImageStreamTag",
          "name" : "s2i-java:2.3"
        }
      }
    }
  }
}, {
  "apiVersion" : "apps.openshift.io/v1",
  "kind" : "DeploymentConfig",
  "metadata" : {
    "labels" : {
      "app" : "getting-started",
      "version" : "1.0-SNAPSHOT",
      "group" : "alex"
    },
    "name" : "getting-started"
  },
  "spec" : {
    "replicas" : 1,
    "selector" : {
```

```
      "app" : "getting-started",
      "version" : "1.0-SNAPSHOT",
      "group" : "alex"
    },
    "template" : {
      "metadata" : {
        "labels" : {
          "app" : "getting-started",
          "version" : "1.0-SNAPSHOT",
          "group" : "alex"
        }
      },
      "spec" : {
          "containers" : [ {
            "env" : [ {
              "name" : "KUBERNETES_NAMESPACE",
              "valueFrom" : {
                "fieldRef" : {
                  "fieldPath" : "metadata.namespace"
                }
              }
            }, {
              "name" : "JAVA_APP_JAR",
              "value" : "/deployments/getting-started-1.0-SNAPSHOT.jar"
            } ],
            "image" : "",
            "imagePullPolicy" : "IfNotPresent",
            "name" : "getting-started",
            "ports" : [ {
              "containerPort" : 8080,
              "name" : "http",
              "protocol" : "TCP"
            } ],
          } ]
        }
      },
      "triggers" : [ {
        "imageChangeParams" : {
          "automatic" : true,
          "containerNames" : [ "getting-started" ],
          "from" : {
            "kind" : "ImageStreamTag",
            "name" : "getting-started:1.0-SNAPSHOT"
          }
        },
        "type" : "ImageChange"
      } ]
    }
  } ]
}
```

10.6　自动构建和部署容器镜像

问题

如何自动构建、推送和部署容器镜像？

解决方案

Quarkus提供了使用container-image扩展来构建和推送容器镜像的扩展，以及使用kubernetes扩展来部署到Kubernetes的扩展。

要构建、推送和部署容器镜像，首先需要添加所需的扩展：

```
./mvnw quarkus:add-extensions \
    -Dextensions="quarkus-container-image-jib, quarkus-kubernetes"
```

然后可以自定义容器映像的构建过程。可以在*application*、*properties*、系统属性或环境变量中设置这些属性，与在Quarkus中设置其他配置参数一样：

```
quarkus.container-image.group=lordofthejars ❶
quarkus.container-image.registry=quay.io ❷
quarkus.container-image.username=lordofthejars ❸
#quarkus.container-image.password= ❹
```

❶设置镜像的分组，这个值默认是${user.name}

❷镜像推向的仓库，镜像默认被推向docker.io

❸镜像仓库的用户名

❹镜像仓库的密码

最后，使用以下命令部署到Kubernetes上：

```
./mvnw clean package -Dquarkus.kubernetes.deploy=true
```

讨论

请注意，将quarkus.kubernetes.deploy设置为true会隐式地将quarkus.container-image.push属性设置为true，所以不需要手动设置它。

Kubernetes扩展使用位于 *~/.kube/config*的标准kubectl配置文件来获知应用的部署位置。

可以使用-Pnative -Dquarkus.native.container-build=true标志创建和部署原生的容器镜像。

10.7　通过Kubernetes配置应用程序

问题

如何通过Kubernetes而不是配置文件来配置应用程序？

解决方案

使用ConfigMap配置在pod内运行的应用程序。

在这个例子中，将使用ConfigMap和Kubernetes扩展来配置一个服务。为了在Pod中使用ConfigMap注入来实现Kubernetes资源的生成，需要注册quarkus-kubernetes扩展：

```
./mvnw quarkus:add-extension -Dextensions="quarkus-kubernetes"
```

当调用/hello终端时，服务会返回greeting.message配置值：

```
@ConfigProperty(name = "greeting.message")
String message;

@GET
@Produces(MediaType.TEXT_PLAIN)
public String hello() {
    return "hello " + message;
}
```

用键值对创建ConfigMap资源：

```
apiVersion: v1
kind: ConfigMap ❶
metadata:
    name: greeting-config
data:
    greeting: "Kubernetes" ❷
```

❶定义ConfigMap类型

❷给greeting键设置Kubernetes作为值

然后必须在终端窗口中运行下一条命令，将资源应用到Kubernetes集群中：

```
kubectl apply -f src/main/kubernetes/config-greeting.yaml
```

最后，在*application.properties*中设置Kubernetes扩展属性，使生成的Kubernetes部署文件中包含将configmap作为环境变量注入的字段：

```
greeting.message=local
quarkus.container-image.group=quarkus ❶
quarkus.container-image.name=greeting-app


quarkus.container-image.tag=1.0-SNAPSHOT
quarkus.kubernetes.env-vars.greeting-message.value=greeting ❷
quarkus.kubernetes.env-vars.greeting-message.configmap=greeting-config ❸
quarkus.kubernetes.image-pull-policy=if-not-present
```

❶配置Docker镜像

❷设置环境变量来覆盖greeting.message属性

❸设置ConfigMap资源用于加载

Kubernetes的生成文件将包含一个新的条目，它在容器描述中定义了一个键值对，用于获取ConfigMap的信息，这个键是configMapKeyRef。

要部署应用程序，需打开一个新的终端窗口，打包应用程序，创建Docker容器，并部署生成的Kubernetes资源：

```
./mvnw clean package -DskipTests

docker build -f src/main/docker/Dockerfile.jvm \
    -t quarkus/greeting-app:1.0-SNAPSHOT .
kubectl apply -f target/kubernetes/kubernetes.yml

kubectl patch svc greeting-app --type='json' \
    -p '[{"op":"replace","path":"/spec/type","value":"NodePort"}]'
curl $(minikube service greeting-app --url)/hello
```

讨论

ConfigMap由键值对组成，Kubernetes以文件或环境变量的形式注入pod的容器中，这样应用程序就可以读取它们并进行相应的计算。通过ConfigMap，可以将应用程序的配置与业务逻辑解耦，使其在不同环境中可移植。

ConfigMap用于存储和共享非敏感的配置属性。

MicroProfile Config规范允许使用等价的环境变量覆盖任何配置属性（大写，并将点[.]改为下划线[_]）。ConfigMap包含了配置属性。在*application.properties*中，Kubernetes扩展被配置为生成一个部署描述符，该描述符将这些属性设置为环境变量，这样当容器在Kubernetes集群内启动时，就会使用应用于该集群的特定配置。

另见

要了解Kubernetes中ConfigMap的更多信息，请访问以下网站：

· Kubernetes:Configure a Pod to Use a ConfigMap（https://oreil.ly/BPmo5）

10.8　使用配置扩展通过Kubernetes配置应用程序

问题

如何通过Kubernetes配置应用程序，而不是使用MicroProfile Config规范的配置文件？

解决方案

Quarkus有一个Kubernetes配置扩展，该配置扩展可以从Kubernetes API服务器中读取密钥和配置映射元素，并使用@ConfigProperty注解注入它们。

要启用Kubernetes资源的生成，需要注册quarkus-kubernetes-config扩展。

该扩展支持注入ConfigMap，可以是单一的键/值形式，也可以是键为文件名（其中只支持application.properties或application.yaml）、值为该文件内容。

我们用一个单一的键/值创建一个配置映射：

```
apiVersion: v1
kind: ConfigMap
metadata:
    name: my-config ❶
data:
    greeting: "Kubernetes"
```

❶配置名字对于插件非常重要

注册之前的ConfigMap资源：

```
kubectl apply -f src/main/kubernetes/my-config.yaml
```

在这个例子中，作为ConfigMap的*application.properties*文件也被注册。

添加的配置文件包含以下内容：

```
some.property1=prop1
some.property2=prop2
```

然后在名为my-file-config的ConfigMap中注册之前的文件：

```
kubectl create configmap my-file-config \
    --from-file=./src/main/kubernetes/application.properties
```

在注入这些值之前的最后一步是配置扩展以从Configmap中读取值：

```
quarkus.kubernetes-config.enabled=true ❶
quarkus.kubernetes-config.config-maps=my-config,my-file-config ❷
```

❶开启扩展

❷设置ConfigMap名

这些配置值像其他配置值一样被注入：

```
@ConfigProperty(name = "greeting") ❶
String greeting;

@ConfigProperty(name = "some.property1") ❷
String property1;

@ConfigProperty(name = "some.property2")
String property2;
```

❶简单的key被注入

❷*application.properties*文件的key也被注入

要部署应用程序，需要打开一个新的终端窗口，打包应用程序，创建Docker容器，并应用生成的Kubernetes资源：

```
./mvnw clean package -DskipTests

docker build -f src/main/docker/Dockerfile.jvm \
    -t quarkus/greeting-app:1.0-SNAPSHOT .
kubectl apply -f target/kubernetes/kubernetes.yml

kubectl patch svc greeting-app-config-ext --type='json' \
    -p '[{"op":"replace","path":"/spec/type","value":"NodePort"}]'

curl $(minikube service greeting-app-config-ext --url)/hello
Kubernetes

curl $(minikube service greeting-app-config-ext --url)/hello/p1
prop1

curl $(minikube service greeting-app-config-ext --url)/hello/p2
prop2
```

10.9　以程序化的方式与Kubernetes集群进行交互

问题

如何以程序化的方式与Kubernetes API服务器进行交互？

解决方案

使用kubernetes-client扩展来观察和响应Kubernetes资源的变化。

要添加kubernetes-extension，请运行以下命令：

```
./mvnw quarkus:add-extension -Dextensions="kubernetes-client"
```

连接到Kubernetes集群的主类是io.fabric8.kubernetes.client.KubernetesClient。扩展产生了这个实例，所以可以在代码中注入。客户端可以使用各种属性进行配置，在*application.properties*中进行设置。

这里开发的例子是一个端点（endpoint），它返回给定命名空间上所有部署的pod的名称：

```
package org.acme.quickstart;

import java.util.List;
import java.util.stream.Collectors;

import javax.inject.Inject;
import javax.ws.rs.GET;
import javax.ws.rs.Path;
import javax.ws.rs.PathParam;
import javax.ws.rs.Produces;
import javax.ws.rs.core.MediaType;

import io.fabric8.kubernetes.client.KubernetesClient;

@Path("/pod")
public class PodResource {

    @Inject ❶
    KubernetesClient kubernetesClient;

    @GET
    @Produces(MediaType.APPLICATION_JSON)
    @Path("/{namespace}")
    public List<String> getPods(@PathParam("namespace") String namespace) {
        return kubernetesClient.pods() ❷
                               .inNamespace(namespace) ❸
                               .list().getItems()
                               .stream()
                               .map(p -> p.getMetadata().getGenerateName()) ❹
                               .collect(Collectors.toList());
    }
}
```

❶KubernetesClient如其他CDI bean一样被注入

❷选择所有pod

❸来自指定的命名空间

❹只获取产生的pod的名字

建议使用代理模式下的kubectl来访问Kubernetes的REST API，因为不会发生中间人攻击。

另一种方式是直接提供位置和证书，但为了避免中间人攻击，可能需要导入根证书。

因为代理模式是推荐的方式，所以本例采用的是这种方法。

将kubectl指向应用程序必须连接的集群，打开一个新的终端窗口并运行下一条命令：

```
kubectl proxy --port=8090
```

该命令作为反向代理运行kubectl，暴露远程Kubernetes API服务器：http://localhost:8090。

通过在*application.properties*中使用quarkus.kubernetes-client.master-url属性配置KubernetesClient去访问http://localhost:8090：

```
%dev.quarkus.kubernetes-client.master-url=http://localhost:8090 ❶
```

❶设置Kubernetes API服务器的URL

最后，运行该服务，并向/pod/default端点发出请求，以获取部署在默认命名空间中的所有pod：

```
./mvnw compile quarkus:dev

curl http://localhost:8080/pod/default
["getting-started-5cd97ddd4d-"]
```

讨论

在某些情况下，需要以编程的方式创建新的Kubernetes资源，或者获取一些关于Kubernetes集群/资源的信息（部署了哪些pod、配置参数、设置密钥等）。kubernetes-client真正的闪光点是在Java中实

现Kubernetes Operator。这多亏了Quarkus具有生成原生可执行文件的能力，这是一个在Java中实现Kubernetes Operator的好方法。

在这个例子中，服务被部署在Kubernetes集群之外，可以使用Kubernetes API服务器连接它。

如果服务被部署到需要访问的Kubernetes集群中，那么quarkus.kubernetes-client.master-url属性必须设置为https://kubernetes.default.svc。

KubernetesClient的创建可以通过简单地声明一个CDI提供者工厂方法来覆盖，该方法返回配置好的KubernetesClient实例：

```
@ApplicationScoped
public class KubernetesClientProducer {

    @Produces
    public KubernetesClient kubernetesClient() {
        Config config = new ConfigBuilder()
                            .withMasterUrl("https://mymaster.com")
                            .build(); ❶
        return new DefaultKubernetesClient(config); ❷
    }
}
```

❶配置客户端

❷创建Kubernetes Client实例

在大多数情况下，要访问Kubernetes API服务器，必须要有ServiceAccount、Role和Role Binding。下面给出一个例子：

```
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: greeting-started
  namespace: default
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: greeting-started
  namespace: default
rules:
  - apiGroups: [""]
    resources: ["pods"]
    verbs: ["list"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: greeting-started
  namespace: default
roleRef:
  kind: Role
  name: greeting-started
  apiGroup: rbac.authorization.k8s.io
subjects:
  - kind: ServiceAccount
    name: greeting-started
    namespace: default
```

另见

要了解关于Fabric8 Kubernetes Client的更多信息，请访问GitHub上的以下页面：

- Kubernetes & OpenShift Java Client（https://oreil.ly/_QNSL）

10.10 测试Kubernetes Client交互

问题

如何测试Kubernetes Client代码?

解决方案

Quarkus实现了一个Quarkus Test Resource，它可以启动Kubernetes API服务器的模拟，并设置正确的配置，使Kubernetes Client使用模拟服务器实例，而不是*application.properties*中提供的值。此外，还可以设置模拟服务器以响应任何特定测试所需的任何封装请求：

```
package org.acme.quickstart;

import io.fabric8.kubernetes.api.model.Pod;
import io.fabric8.kubernetes.api.model.PodBuilder;
import io.fabric8.kubernetes.api.model.PodListBuilder;
import io.fabric8.kubernetes.client.server.mock.KubernetesMockServer;
import io.quarkus.test.common.QuarkusTestResource;
import io.quarkus.test.junit.QuarkusTest;
import io.quarkus.test.kubernetes.client.KubernetesMockServerTestResource;
import io.quarkus.test.kubernetes.client.MockServer;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import static io.restassured.RestAssured.given;
import static org.hamcrest.CoreMatchers.is;

@QuarkusTest
@QuarkusTestResource(KubernetesMockServerTestResource.class) ❶
public class PodResourceTest {

    @MockServer ❷
    KubernetesMockServer mockServer;

    @BeforeEach ❸
    public void prepareKubernetesServerAPI() {
        final Pod pod1 = new PodBuilder()
                .withNewMetadata()
```

```
                .withName("pod1")
                .withNamespace("test")
                .withGenerateName("pod1-12345")
                .and()
                .build(); ❹

        mockServer
                .expect()
                  .get()
                    .withPath("/api/v1/namespaces/test/pods")
                    .andReturn(200, new PodListBuilder()
                        .withNewMetadata()
                        .withResourceVersion("1")
                        .endMetadata()
                        .withItems(pod1).build()) ❺
                .always();

    }

    @Test
    public void testHelloEndpoint() {
        given()
          .when().get("/pod/test")
          .then()
             .statusCode(200)
             .body(is("[\"pod1-12345\"]"));
    }

}
```

❶设置Kubernetes Test Resource的模拟服务器

❷注入Kubernetes模拟服务器实例去记录所有交互

❸在每次测试前，重新进行一次交互，以保持测试隔离性

❹创建一个pod并返回

❺这个pod作为查询test命名空间下所有pod的返回值

10.11 实现一个Kubernetes Operator

问题

如何实现一个Kubernetes Operator来扩展Kubernetes，以使用自定义资源来管理Java应用程序？

解决方案

使用kubernetes-client扩展和Quarkus在Java中实现Kubernetes Operator，并将其编译成原生可执行文件。

Operator的一个用例是创建一个模板（自定义资源），在创建时设置一些值。文件模板和操作符最大的区别在于，模板的公共内容是静态的，而在操作符中，它是以编程的方式设置的，这意味着你可以自由地动态更改公共部分的定义。这就是所谓的自定义资源，在这种情况下，不使用知名的Kubernetes资源，而是用你自己的字段实现自定义Kubernetes资源。

另一个用例可能是在集群内部发生一些事情时做出响应或执行操作。假设你在集群上部署了一些内存数据网格，其中一个实例死亡。

也许在这种情况下，你想通知所有活着的实例，数据网格集群的一个元素已经停止。

正如你所看到的，它不仅是关于资源的创建，而且还应用了一些特定于你的应用的任务，这些任务需要在Kubernetes已经在执行的一个任务之上完成。

Kubernetes Operator使用Kubernetes API来决定何时以及如何运行其中的一些定制。

下面这个简单的例子从它所实现的逻辑来看并没有什么意义，但它将帮助你理解编写Kubernetes Operator的基础知识。你可以把它作为实现自己的Kubernetes Operator的起点。

要编写一个Kubernetes Operator，需要以下元素：

1.解析自定义资源的类。

2.注册并生成客户端的工厂方法，以使用自定义资源进行操作。

3.一个监听，当一个自定义资源被应用到集群时，它就会做出反应。你可以把它看作是操作者控制器或操作者实现。

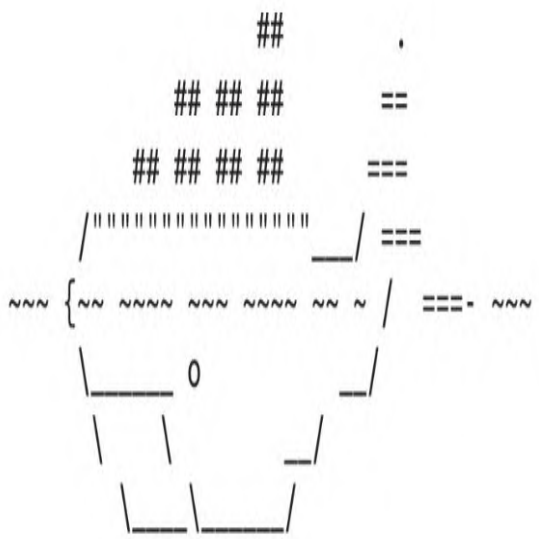
4.包含之前所有代码的Docker镜像。

5.YAML/JSON文件，用来定义自定义资源（CustomResourceDefinition）。

6.部署文件，用于部署自定义操作符。

让我们实现一个简单的Kubernetes Operator，配置命令在容器中运行，并通过这个配置实例化pod。本例中使用的基础图像是Whalesay（https://oreil.ly/98T7t），它基本在容器控制台中打印你在运行命令中作为参数传递的消息，就像下面这样：

```
docker run docker/whalesay cowsay boo
 _____
< boo >
 -----
    \
     \
      \
```



使用该图像的一个pod资源的例子如下所示：

```
apiVersion: v1
kind: Pod
metadata:
  name: whalesay
spec:
  containers:
  - name: whalesay
    image: docker/whalesay
    imagePullPolicy: "IfNotPresent"
    command: ["cowsay","Hello Alex"] ❶
```

❶设置输出信息

这个Operator的目标是，必须只提供要打印的消息。其余内容（如Docker镜像、容器配置等）由Kubernetes Operator自动设置。

创建一个自定义的Operator，需要Kubernetes客户端和Jackson依赖项：

```
./mvnw quarkus:add-extension \
-Dextensions="io.quarkus:quarkus-kubernetes-client, io.quarkus:quarkus-jackson"
```

首先要做的是定义自定义资源的样式。在这个例子中，它可以如下所示：

```
apiVersion: acme.org/v1alpha1
kind: Hello ❶
metadata:
  name: example-hello
spec:
  message: Hello Alex ❷
```

❶使用自定义资源类型

❷设置message用于打印

需要一个对象模型来解析自定义资源。在本例中，使用Jackson库从YAML映射到Java Object。需要三个类：一个用于整个资源，另一个用于规格部分，还有一个用于状态部分（这个类是空的，但它是必需的，因为它可能由集群自动填充）。

在包org.acme.quickstart.cr内的*src/main/java*处创建所有这些文件：

```
package org.acme.quickstart.cr;

import com.fasterxml.jackson.databind.annotation.JsonDeserialize;

import io.fabric8.kubernetes.client.CustomResource;

@JsonDeserialize ❶
public class HelloResource extends CustomResource { ❷

    private HelloResourceSpec spec; ❸
    private HelloResourceStatus status; ❹

    public HelloResourceStatus getStatus() {
        return status;
    }

    public void setStatus(HelloResourceStatus status) {
        this.status = status;
    }

    public HelloResourceSpec getSpec() {
        return spec;
    }

    public void setSpec(HelloResourceSpec spec) {
        this.spec = spec;
    }

    @Override
    public String toString() {
        return "name=" + getMetadata().getName()
                + ", version=" + getMetadata().getResourceVersion()
                + ", spec=" + spec;
    }
}
```

❶设置序列化的POJO

❷继承通用自定义资源的字段，例如kind、apiVersion和metadata

❸自定义spec部分

❹状态部分（status）

spec部分的映射如下所示：

```
package org.acme.quickstart.cr;

import com.fasterxml.jackson.annotation.JsonProperty;
import com.fasterxml.jackson.databind.annotation.JsonDeserialize;
```

```
@JsonDeserialize
public class HelloResourceSpec {

    @JsonProperty("message") ❶
    private String message;

    public String getMessage() {
        return message;
    }

    public void setMessage(String message) {
        this.message = message;
    }

    @Override
    public String toString() {
        return "HelloResourceSpec [message=" + message + "]";
    }

}
```

❶自定义的定义仅包含一个message字段

而空status部分的映射如下所示：

```
package org.acme.quickstart.cr;

import com.fasterxml.jackson.databind.annotation.JsonDeserialize;

@JsonDeserialize
public class HelloResourceStatus {
}
```

但从模型的角度来看，还是需要两个类。

一个类是在以下情况下使用的：不是将单个自定义资源（如前文所示）应用于集群，而是提供自定义资源的列表（使用items数组）。

```
package org.acme.quickstart.cr;

import com.fasterxml.jackson.databind.annotation.JsonDeserialize;

import io.fabric8.kubernetes.client.CustomResourceList;

@JsonDeserialize
public class HelloResourceList extends CustomResourceList<HelloResource> { ❶
}
```

❶CustomResourceList继承支持自定义资源列表所需的所有字段

另一个类用来使自定义资源可以从Operator实现中编辑：

```
package org.acme.quickstart.cr;

import io.fabric8.kubernetes.api.builder.Function;
import io.fabric8.kubernetes.client.CustomResourceDoneable;

public class HelloResourceDoneable
    extends CustomResourceDoneable<HelloResource> { ❶
    public HelloResourceDoneable(HelloResource resource, Function<HelloResource,
                                 HelloResource> function) {
        super(resource, function);
    }
}
```

❶CustomResourceDoneable类使资源可编辑

CDI工厂bean不可缺少，它提供了Operator需要的所有机器。在包org.acme.quickstart内的*src/main/java*创建这个类：

```
package org.acme.quickstart;

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Paths;

import javax.enterprise.inject.Produces;
import javax.inject.Named;
import javax.inject.Singleton;

import org.acme.quickstart.cr.HelloResource;
import org.acme.quickstart.cr.HelloResourceDoneable;
import org.acme.quickstart.cr.HelloResourceList;

import io.fabric8.kubernetes.api.model.apiextensions.CustomResourceDefinition;
import io.fabric8.kubernetes.client.DefaultKubernetesClient;
import io.fabric8.kubernetes.client.KubernetesClient;
import io.fabric8.kubernetes.client.dsl.MixedOperation;
import io.fabric8.kubernetes.client.dsl.Resource;
import io.fabric8.kubernetes.internal.KubernetesDeserializer;

public class KubernetesProducer {

  @Produces
  @Singleton
  @Named("namespace")
  String findMyCurrentNamespace() throws IOException { (1)
    return new String(Files.readAllBytes(
          Paths
            .get("/var/run/secrets/kubernetes.io/serviceaccount/namespace")));
  }

  @Produces
  @Singleton
```

```
  KubernetesClient makeDefaultClient(@Named("namespace") String namespace) {
    return new DefaultKubernetesClient().inNamespace(namespace); ❷
  }

  @Produces
  @Singleton
  MixedOperation<HelloResource,
                 HelloResourceList,
                 HelloResourceDoneable,
                 Resource<HelloResource, HelloResourceDoneable>>
  makeCustomHelloResourceClient(KubernetesClient defaultClient) { ❸
    KubernetesDeserializer
        .registerCustomKind("acme.org/v1alpha1",
                            "Hello", HelloResource.class); ❹
    CustomResourceDefinition crd = defaultClient.customResourceDefinitions()
                                      .list()
                                      .getItems()
                                      .stream()
                                      .findFirst()
      .orElseThrow(RuntimeException::new); ❺
    return defaultClient.customResources(crd, HelloResource.class,
        HelloResourceList.class,
        HelloResourceDoneable.class); ❻
  }
}
```

❶获取Operator运行所在的命名空间

❷在当前命名空间下，配置KubernetesClient

❸MixOperation用于监听自定义资源的事件（例如，应用新的自定义资源）

❹注册apiVersion和Kind，这个值截取自org.acme.quickstart.cr.HelloResource

❺获取自定义资源的定义，因为只有一个（正在开发的那个），所以可以使用findFirst

❻注册自定义资源、解析器、列表解析和doneable类

最后一个要实现的Java类是控制器。控制器（或称观察者/操作者）负责检查集群内部发生的事情，并对订阅的事件做出反应。例如，一个新的pod被创建或销毁，或者一个Hello类型的自定义资源被应用。

在这个实现中，控制器在观察何时一个新的Hello种类的资源被添加。当应用自定义资源时，则从模型中检索消息，并使用Kubernetes客户端API提供的所有构建器创建pod定义。最后，将pod部署到Kubernetes集群中。

在包org.acme.quickstart中的*src/main/java*创建这个类：

```
package org.acme.quickstart;
```

```
import java.util.HashMap;
import java.util.Map;

import javax.enterprise.event.Observes;
import javax.inject.Inject;

import org.acme.quickstart.cr.HelloResource;
import org.acme.quickstart.cr.HelloResourceDoneable;
import org.acme.quickstart.cr.HelloResourceList;

import io.fabric8.kubernetes.api.model.ContainerBuilder;
import io.fabric8.kubernetes.api.model.HasMetadata;
import io.fabric8.kubernetes.api.model.ObjectMetaBuilder;
import io.fabric8.kubernetes.api.model.Pod;
import io.fabric8.kubernetes.api.model.PodBuilder;
import io.fabric8.kubernetes.api.model.PodSpecBuilder;
import io.fabric8.kubernetes.client.KubernetesClient;
import io.fabric8.kubernetes.client.KubernetesClientException;
import io.fabric8.kubernetes.client.Watcher;
import io.fabric8.kubernetes.client.dsl.MixedOperation;
import io.fabric8.kubernetes.client.dsl.Resource;
import io.quarkus.runtime.StartupEvent;

public class HelloResourceWatcher {

  @Inject
  KubernetesClient defaultClient; ❶

  @Inject
  MixedOperation<HelloResource,
    HelloResourceList,
    HelloResourceDoneable,
    Resource<HelloResource,
    HelloResourceDoneable>> crClient; ❷

  void onStartup(@Observes StartupEvent event) { ❸
    crClient.watch(new Watcher<HelloResource>() { ❹
      @Override
      public void eventReceived(Action action, HelloResource resource) {
        System.out.println("Received " + action
            + " event for resource " + resource);
        if (action == Action.ADDED) {
          final String app = resource.getMetadata().getName(); ❺
          final String message = resource.getSpec().getMessage();

          final Map<String, String> labels = new HashMap<>(); ❻
          labels.put("app", app);

          final ObjectMetaBuilder objectMetaBuilder =
            new ObjectMetaBuilder().withName(app + "-pod")
            .withNamespace(resource.getMetadata()
                .getNamespace())
```

```
            .withLabels(labels);

          final ContainerBuilder containerBuilder =
            new ContainerBuilder().withName("whalesay")
            .withImage("docker/whalesay")
            .withCommand("cowsay", message); ❼

          final PodSpecBuilder podSpecBuilder =
            new PodSpecBuilder()
            .withContainers(containerBuilder.build())
            .withRestartPolicy("Never");

          final PodBuilder podBuilder =
            new PodBuilder()
            .withMetadata(objectMetaBuilder.build())
            .withSpec(podSpecBuilder.build());

          final Pod pod = podBuilder.build(); ❽
          HasMetadata result = defaultClient
            .resource(pod)
            .createOrReplace(); ❾

          if (result == null) {
            System.out.println("Pod " + pod
                + " couldn't be created");
          } else {
            System.out.println("Pod " + pod + " created");
          }
        }
      }

      @Override
      public void onClose(KubernetesClientException e) { ❿
        if (e != null) {
          e.printStackTrace();
          System.exit(-1);
        }
      }
    });
  }
}
```

❶注入Kubenetes Client

❷注入自定义资源开发的Operator

❸应用启动时的执行逻辑

❹监听HelloResource的任何动作

❺获取自定义资源的信息

❻根据pod的定义开始创建

❼设置自定义资源提供的消息

❽构建pod

❾将pod添加到集群

❿如果存在致命错误，停止容器

讨论

在Java方面，这就是你需要做的所有事情。但是，还有一些剩余的部分，比如包装和容器化Operator，或者在集群内部定义自定义Operator。

在开发Kubernetes Operator时，与Kubernetes API服务器的通信是通过HTTPS完成的，这就意味着如果Docker镜像中没有默认提供加密库，则必须提供加密库。

在撰写本书时，Quarkus提供的*Dockerfile.jvm*文件不包含与Kubernetes服务器通信所需的加密库。要解决这个问题，只需打开*src/main/docker/Dockerfile.jvm*并添加nss（网络安全服务）包即可：

```
FROM fabric8/java-alpine-openjdk8-jre

RUN apk add --no-cache nss
```

运行Maven和Docker将Operator容器化：

```
./mvnw clean package

docker build -f src/main/docker/Dockerfile.jvm \
  -t lordofthejars/quarkus-operator-example:1.0.0 .
```

将自定义资源定义注册到Kubernetes集群中，使其知道新的种类、自定义资源的范围或组名等。在*src/main/kubernetes*中创建一个新文件，命名为*custom-resource-definition.yaml*，定义集群注册新资源所需的所有信息：

```
apiVersion: apiextensions.k8s.io/v1beta1
kind: CustomResourceDefinition
metadata:
  name: hellos.acme.org ❶
spec:
group: acme.org ❷
names:
  kind: Hello ❸
  listKind: HelloList ❹
  plural: hellos ❺
  singular: hello ❻
scope: Namespaced ❼
subresources:
  status: {}
version: v1alpha1 ❽
```

❶合起来的plural名和group名

❷设置自定义资源的组名（在apiVersion字段中使用）

❸Kind的名字

❹种类是这个自定义资源的列表时的名字

❺plural名

❻sigular名

❼资源的范围

❽资源的版本（在apiVersion字段中使用）

而最后要创建的是一个部署文件，部署Operator。创建一个在*src/main/kubernetes*中新建一个名为*deploy.yaml*的文件：

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole ❶
metadata:
  name: quarkus-operator-example
rules:
- apiGroups:
  - ''
  resources:
  - pods ❷
  verbs:
  - get
  - list
  - watch
  - create
  - update
  - delete
  - patch
- apiGroups:
  - apiextensions.k8s.io
  resources:
  - customresourcedefinitions
  verbs:
  - list
- apiGroups:
  - acme.org ❸
  resources:
  - hellos
  verbs:
  - list
  - watch
---
apiVersion: v1
kind: ServiceAccount
metadata:
```

```
  name: quarkus-operator-example
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: quarkus-operator-example
subjects:
- kind: ServiceAccount
  name: quarkus-operator-example
  namespace: default
roleRef:
  kind: ClusterRole
  name: quarkus-operator-example
  apiGroup: rbac.authorization.k8s.io
---
apiVersion: apps/v1
kind: Deployment ❹
metadata:
  name: quarkus-operator-example
spec:
  selector:
    matchLabels:
      app: quarkus-operator-example
  replicas: 1
  template:
    metadata:
      labels:
        app: quarkus-operator-example
    spec:
      serviceAccountName: quarkus-operator-example ❺
      containers:
      - image: lordofthejars/quarkus-operator-example:1.0.0 ❻
        name: quarkus-operator-example
        imagePullPolicy: IfNotPresent
```

❶为Kubernetes资源的基于角色的访问控制（RBAC）定义集群角色

❷增加对pod的操作权限：get、list、watch、create、update、delete和patch

❸对于自定义资源（hellos.acme.org），操作权限是list和watch

❹使用deployment部署Operator

❺设置service account（链接到文件中定义的集群角色）

❻设置包含Operator的容器镜像

启动和运行Operator前的最后一步是应用所有这些创建的资源：

```
kubectl apply -f src/main/kubernetes/custom-resource-definition.yaml
kubectl apply -f src/main/kubernetes/deploy.yaml

kubectl get pods
```

```
NAME                                    READY   STATUS    RESTARTS   AGE
quarkus-operator-example-fb77dc468-8v9xk   1/1     Running   0          5s
```

Operator现在已经安装并运行。要测试Operator，只需创建一个自定义的Hello类型的资源，并显示信息：

```
apiVersion: acme.org/v1alpha1
kind: Hello ❶
metadata:
  name: example-hello
spec:
  message: Hello Alex ❷
```

❶使用自定义的kind

❷设置打印信息

并按以下方式应用于Kubernetes：

```
kubectl apply -f src/main/kubernetes/custom-resource.yaml

kubectl get pods

NAME                                    READY   STATUS      RESTARTS   AGE
example-hello-pod                       0/1     Completed   0          2m57s
quarkus-operator-example-fb77dc468-8v9xk   1/1     Running     0          3m24s
```

当它完成时，检查pod日志，以验证该消息已被打印在控制台上：

```
kubectl logs example-hello-pod


 ___________
< Hello Alex >
 -----------
    \
     \
      \
                    ##        .
              ## ## ##       ==
           ## ## ## ##      ===
       /""""""""""""""""___/ ===
  ~~~ {~~ ~~~~ ~~~ ~~~~ ~~ ~ /  ===- ~~~
       \_____ o           __/
        \    \         __/
          \___\_____/
```

虽然Operator和自定义资源通常是相关的，但也有例外。例如，创建一个观察者类来拦截任何影响pod的事件并应用一些逻辑。

另见

要了解关于Operator的更多信息，请查看以下网站：

- CoreOS:Operators（https://oreil.ly/NV2dN）
- Kubernetes:Operator pattern（https://oreil.ly/6Z77K）

10.12　用Knative部署和管理无服务器工作负载

问题

如何部署和管理无服务器工作负载？

解决方案

使用基于Kubernetes的平台Knative来部署和管理现代无服务器工作负载。

quarkus-kubernetes扩展提供了对自动生成Knative资源的支持，支持合理的默认值和用户提供的可选配置。

要实现Kubernetes资源的生成，需要注册quarkus-kubernetes扩展：

```
./mvnw quarkus:add-extension \
  -Dextensions="quarkus-kubernetes, quarkus-container-image-docker"
```

本例中，quarkus-container-image-docker扩展是使用docker二进制来构建容器镜像的，所以镜像是直接在minikube集群内部构建的，并在内部仓库内注册，所以无须外部仓库。

你需要运行eval $(minikube docker-env)来配置docker以使用minikube docker主机。

然后你需要将quarkus.kubernetes.deployment-target属性设置为knative，并将其设置为在包阶段构建一个Docker容器，以及其他关于容器镜像创建的配置属性：

```
quarkus.kubernetes.deployment-target=knative ❶
quarkus.container-image.build=true ❷
quarkus.container-image.group=lordofthejars
quarkus.container-image.registry=dev.local ❸
```

❶将目标部署设置为Knative

❷构建容器镜像时，此变量设置为lordofthejars

❸当部署本地容器镜像时，将此变量设置为dev.local

Knative控制器将镜像的标签解析为digest（摘要），以保证修改的不可更改性。当使用普通的仓库时，这很好用；但是，当使用minikube和本地镜像时，可能会引起问题。

默认情况下，Knative控制器会跳过解析带有dev.local或ko.local前缀的镜像digest。如果你在minikube中运行这个例子，必须将仓库属性设置为这两个选项中的一个，以使Knative找到要部署的镜像。

要生成Kubernetes资源，需要在新的终端中执行./mvnw包。然后，在target目录下的构建工具生成的常规文件中，在*target/kubernetes*目录下创建两个新文件，分别命名为*knative.json*和*knative.yaml*，其中包含Knative服务定义：

```
{
  "apiVersion" : "v1",
  "kind" : "ServiceAccount",
  "metadata" : {
    "annotations" : {
      "app.quarkus.io/vcs-url" :
       "https://github.com/lordofthejars/quarkus-cookbook.git",
      "app.quarkus.io/build-timestamp" : "2020-03-10 - 22:55:08 +0000",
      "app.quarkus.io/commit-id" : "17b19a409c41cc933770b20009f635a65f69440e"
    },
    "labels" : {
      "app.kubernetes.io/name" : "greeting-knative",
      "app.kubernetes.io/version" : "1.0-SNAPSHOT"
    },
    "name" : "greeting-knative"
  }
}{
  "apiVersion" : "serving.knative.dev/v1alpha1",
  "kind" : "Service",
  "metadata" : {
    "annotations" : {
      "app.quarkus.io/vcs-url" :
        "https://github.com/lordofthejars/quarkus-cookbook.git",
      "app.quarkus.io/build-timestamp" : "2020-03-10 - 22:55:08 +0000",
      "app.quarkus.io/commit-id" : "17b19a409c41cc933770b20009f635a65f69440e"
    },
    "labels" : {
      "app.kubernetes.io/name" : "greeting-knative",
      "app.kubernetes.io/version" : "1.0-SNAPSHOT"
    },
    "name" : "greeting-knative"
  },
  "spec" : {
    "runLatest" : {
      "configuration" : {
        "revisionTemplate" : {
          "spec" : {
            "container" : {
              "image" :"dev.local/lordofthejars/greeting-knative:1.0-SNAPSHOT",
```

```
            "imagePullPolicy" : "IfNotPresent"
          }
        }
      }
    }
  }
}
```

部署生成的Knative服务：

```
kubectl apply -f target/kubernetes/knative.json

serviceaccount/greeting-knative created
service.serving.knative.dev/greeting-knative created

kubectl get ksvc
NAME             URL                                                \
greeting-knative   http://greeting-knative.default.127.0.0.1.nip.io   \
LATESTCREATED          LATESTREADY            READY   REASON
greeting-knative-j8n76   greeting-knative-j8n76   True
```

将ready状态从Unknown转为到true状态可能需要几秒钟的时间。如果出现故障，即ready状态保持在false，你可以通过运行以下命令来检查原因和事件的顺序：

```
kubectl get events --sort-by=.metadata.creationTimestamp
```

为了测试服务的部署是否正确，需要打开一个新的终端窗口，在本地机器和Knative网关之间进行端口转发：

```
kubectl port-forward --namespace kourier-system $(kubectl get pod \
  -n kourier-system -l "app=3scale-kourier-gateway" \
  --output=jsonpath="{.items[0].metadata.name}") \
  8080:8080 19000:19000 8443:8443

Forwarding from 127.0.0.1:8080 -> 8080
Forwarding from [::1]:8080 -> 8080
Forwarding from 127.0.0.1:19000 -> 19000
Forwarding from [::1]:19000 -> 19000
Forwarding from 127.0.0.1:8443 -> 8443
Forwarding from [::1]:8443 -> 8443
Handling connection for 8080
Handling connection for 8080
```

注意，这只是因为服务部署在minikube中才需要这样做。根据部署服务的Kubernetes平台，你可能需要做不同的事情。

最后，向这个服务发送一个请求：

```
curl -v -H "Host: greeting-knative.default.127.0.0.1.nip.io" \
  http://localhost:8080/greeting

hello
```

要取消部署这个例子，需要运行下面的程序：

```
kubectl delete -f target/kubernetes/knative.json

serviceaccount "greeting-knative" deleted
service.serving.knative.dev "greeting-knative" deleted
```

讨论

你可以结合container-image和kubernetes扩展来构建容器镜像并自动推送到Kubernetes，如10.6节所示，不需要手动步骤。

另见

要了解更多信息，请访问以下网页：

- Knative Serving（https://oreil.ly/RBv52）

- GitHub:Kourier（https://oreil.ly/3bSDL）

第11章　认证与授权

在本章中，你将会了解到应用程序安全的支柱——授权和认证，如何在Quarkus应用程序中工作。本章涵盖以下主题：

- 文件支持的认证和授权方案
- 数据库支持的认证和授权方案
- 外部服务支持的认证和授权方案

在开始本章内容之前，本节将向你展示Quarkus安全基础知识、用于加载验证源的安全扩展，以及如何使用基于角色的访问控制（RBAC）方法保护资源。

本节中显示的示例并不意味着可以运行，但它们是其他章节的基础，我们将在其中看到安全扩展的作用。

以下是关于安全的两个主要概念。

认证

验证你的凭证（即用户名和密码）以验证你的身份，使系统知道你是谁。

授权

授予你对受保护资源的访问权限。这发生在认证过程之后。

认证

Quarkus为HTTP提供了两种认证机制，即著名的BASIC和FORM方法。这些机制可以通过任何Quarkus扩展来提供一个自定义的认证方法。例如，以Quarkus扩展的形式来对OpenID Connect服务器（比如Keycloak）进行认证。我们将在本节中探讨如何做到这一点。

要使用身份认证，需要一个身份提供者来认证用户提供的凭证（即用户名和密码）。Quarkus提供了以下身份提供者，但你也可以实现自己的身份提供者。

Elytron属性文件

以属性文件的形式提供用户/密码/角色之间的映射。该信息可以嵌入application.properties文件中，也可以嵌入一个特定的文件中。

Elytron JDBC

提供基于JDBC查询的用户/密码/角色之间的映射。

JPA

提供对通过JPA进行认证的支持。

SmallRye JWT

使用JSON Web Tokens(JWT)规范提供认证。

OIDC

使用OpenID Connect(OIDC)提供商（如Keycloak）提供认证。

Keycloak授权

为使用Keycloak授权服务的策略执行者提供支持。

Basic authentication

要使用基本访问验证，quarkus.http.auth.basic配置属性必须设置为true。

基于Form的authentication

quarkus.http.auth.form.enabled配置属性必须设置为true，才能使用表单访问验证。

Quarkus不将已认证的用户存储在HTTP会话中，因为没有集群的HTTP会话支持。取而代之的是，认证信息被存储在一个加密的cookie中。

加密密钥可以使用quarkus.http.auth.session.encryption-key属性来设置，它必须至少有16个字符长。该密钥使用SHA-256进行散列，其结果被用作AES-256加密cookie值的密钥。该cookie包含一个到期时间作为加密值的一部分，如果会话正在使用，则每隔一分钟生成一个新的cookie，并更新到期时间。

授权

Quarkus集成了Java EE Security注解（https://oreil.ly/ATPpq），以定义RBAC在RESTful Web端点和CDI bean上的应用。

此外，你可以使用配置文件（*application.properties*）代替注解来定义RESTful Web Endpoint的授权。

两种方法可以在同一应用程序中共存，但配置文件检查在任何注解检查之前执行，并且不相互排斥，这意味着在重叠的情况下，两种检查必须通过。

下面的代码段显示了如何使用Java EE Security注释来确保JAX-RS端点的安全：

```
package org.acme.quickstart;

import javax.annotation.security.DenyAll;
import javax.annotation.security.PermitAll;
import javax.annotation.security.RolesAllowed;

import io.quarkus.security.Authenticated;

import javax.ws.rs.GET;
import javax.ws.rs.Path;

@Path("/hello")
public class GreetingResource {

    @GET
    @Path("/secured")
    @RolesAllowed("Tester") ❶
    public String greetingSecured() {}

    @GET
    @Path("/unsecured")
    @PermitAll ❷
    public String greetingUnsecured() {}

    @GET
    @Path("/denied")
    @DenyAll ❸
    public String greetingDenied() {}

    @GET
    @Path("/authenticated")
    @Authenticated ❹
    public String greetingAuthenticated() {}
}
```

❶需要一个角色为Tester的认证用户

❷未经认证的用户可以访问该方法

❸无论是否经过认证，用户都不能访问

❹允许任何经过认证的用户访问。它是@RolesAllowed ("*")的别名，是由Quarkus（而不是规范）提供的。

javax.ws.rs.core.Context注解可以用来注入javax.ws.rs.core.SecurityContext实例，以获取被认证的用户的信息：

```
@GET
@Path("/secured")
@RolesAllowed("Tester")
public String greetingSecured(@Context SecurityContext sec) { ❶
    Principal user = sec.getUserPrincipal(); ❷
    String name = user != null ? user.getName() : "anonymous";
    return name;
}
```

❶为当前请求注入SecurityContext

❷获取当前登录的用户

安全性（Security）注解不仅限于JAX-RS资源，也可以在CDI bean中使用，以保护方法调用。

Quarkus支持使用配置文件而不是注解来配置RESTful Web端点。这相当于安全注解的例子可以用配置文件来表达：

```
quarkus.http.auth.policy.role-policy1.roles-allowed=Tester ❶

quarkus.http.auth.permission.roles1.paths=/hello/secured   ❷
quarkus.http.auth.permission.roles1.policy=role-policy1    ❸
quarkus.http.auth.permission.roles1.methods=GET  ❹

quarkus.http.auth.permission.deny1.paths=/hello/denied
quarkus.http.auth.permission.deny1.policy=deny ❺

quarkus.http.auth.permission.permit1.paths=/hello/unsecured
quarkus.http.auth.permission.permit1.policy=permit ❻
quarkus.http.auth.permission.permit1.methods=GET

quarkus.http.auth.permission.roles2.paths=/hello/authenticated
quarkus.http.auth.permission.roles2.policy=authenticated
quarkus.http.auth.permission.roles2.methods=GET
```

❶定义应用程序的角色。role-policy1作为引用值

❷设置资源的权限。roles1是一个任意名称，以避免重复键

❸设置角色策略

❹将权限限制在GET方法上

❺拒绝访问

❻允许进入

需要注意的是，paths属性支持用逗号分隔的多个值，也支持*通配符来匹配任意子路径。例如，quarkus.http.auth.permission.permit1.paths=/public/_,/robots.txt设置了对位于*/public*及其任意子路径以及*/robots.txt*文件的任何资源的许可。

同样，methods属性也允许用逗号分隔多个值。

有两个会影响RBAC行为的配置属性：

quarkus.security.jaxrs.deny-unannotated-endpoints

如果将其设置为true，则默认情况下拒绝所有未使用安全注解的JAX-RS端点。此属性默认为false。

quarkus.security.deny-unannotated-members

如果将其设置为true，则默认情况下拒绝所有未使用安全注解的JAX-RS端点和CDI方法。此属性默认为false。

到目前为止，你已经看到在Quarkus中，可以设置授权程序（基本的、形式的，或其他由扩展提供的），并使用安全注解或在配置文件中指定它们来定义认证角色。

本章的后续章节将探索不同的Quarkus扩展，以提供认证和授权身份提供者。

11.1　使用Elytron属性文件配置进行认证和授权

问题

如何通过在文件中存储身份信息来保证应用程序的安全？

解决方案

Quarkus安全提供了支持，使用Elytron属性文件配置作为身份提供者在文件中存储身份。

你已经了解了如何定义身份认证机制，以及如何使用RBAC来保护资源（可以使用安全注释，也可以在*application.properties*中使用），但你还不知道如何注册一个身份提供者，以及如何将用户信息（如用户名、密码或角色）存储在其所属的地方。

我们来看看如何使用Elytron属性文件配置扩展来定义身份信息。这个扩展基于属性文件来定义所有的身份信息，主要用于开发和测试。不建议在生产中使用，因为密码只能用纯文本或MD5哈希表示。

要启用Elytron属性文件配置，需要注册quarkus-elytron-security-properties-file扩展：

```
./mvnw quarkus:add-extension \
      -Dextensions="quarkus-elytron-security-properties-file"
```

该扩展支持用户与密码和用户与角色的映射，并结合属性文件。

保护端点，只允许Tester角色访问资源：

```
@GET
@Produces(MediaType.TEXT_PLAIN)
@RolesAllowed("Tester")
public String hello() {
    return "hello";
}
```

要注册身份，需要两个属性文件：一个用于映射用户和密码，另一个用于映射用户及其所属角色列表。

用户配置属性文件为每一行定义了系统中注册的一对用户和密码：

```
alex=soto
```

在用户的属性文件中，键部分是用户名，值部分是密码。

请注意，密码是纯文本的。你可以按照以下模式用MD5对密码进行哈希：HEX(MD5(username:realm:password)。

角色配置文件为每一行定义了用户名和用户所属的角色（用逗号隔开）：

```
alex=Tester
```

在角色属性文件中，键部分是用户名，值是分配给用户的角色。

Elytron安全属性文件扩展需要配置用户和角色属性文件的classpath位置：

```
quarkus.http.auth.basic=true ❶

quarkus.security.users.file.enabled=true ❷
quarkus.security.users.file.plain-text=true ❸
quarkus.security.users.file.users=users.properties ❹
quarkus.security.users.file.roles=roles.properties
```

❶启用基本认证方法

❷启用属性文件扩展的安全功能

❸设置不使用MD5哈希的密码

❹设置用户和角色属性文件的classpath位置

运行生成的测试来验证端点的保护：

```
./mvnw clean test

...
INFO  [io.quarkus] (main) Installed features:
       [cdi, resteasy, security, security-properties-file]
[ERROR] Tests run: 2, Failures: 1, Errors: 0, Skipped: 0, Time elapsed: 8.485 s
       <<< FAILURE! - in org.acme.quickstart.GreetingResourceTest
[ERROR] testHelloEndpoint  Time elapsed: 0.076 s  <<< FAILURE!
java.lang.AssertionError:
1 expectation failed.
Expected status code <200> but was <401>.
       at org.acme.quickstart.GreetingResourceTest.testHelloEndpoint
              (GreetingResourceTest.java:17)
```

测试以HTTP 401 Unauthorized错误宣告失败，因为测试没有使用Basic认证方法提供任何身份。修改测试，用配置的用户名和密码进行验证：

```
@Test
public void testSecuredHelloEndpoint() {
    given()
            .auth() ❶
            .basic("alex", "soto") ❷
            .when()
            .get("/hello")
            .then()
            .statusCode(200)
            .body(is("hello"));
}
```

❶设置认证部分

❷用给定的用户名和密码进行Basic认证

现在，在有效的认证参数下，测试通过。

讨论

Elytron属性文件配置扩展也支持将用户/密码/角色之间的映射嵌入Quarkus配置文件（*application.properties*）中，而不是使用不同的文件。

```
quarkus.security.users.embedded.enabled=true
quarkus.security.users.embedded.plain-text=true
quarkus.security.users.embedded.users.alex=soto
quarkus.security.users.embedded.roles.alex=Admin,Tester
```

存储在文件中的密码可以使用公式HEX(MD5(username":"realm":"password))进行哈希。

嵌入的Elytron属性文件配置可以使用表11-1中列出的属性进行配置。

表11-1：嵌入的Elytron属性

属性	描述
quarkus.security.users.embedded.realm-name	生成哈希密码时使用的 realm 名（默认为 Quarkus）
quarkus.security.users.embedded.enabled	启用属性文件扩展名的安全（默认为 false）
quarkus.security.users.embedded.plain-text	设置密码是否经过哈希处理。如果为 true，则散列的密码必须是 HEX(MD5(username:realm:password) 的形式（默认为 false）
quarkus.security.users.embedded.users.<user>	用户信息。键部分是用户名，值部分是密码
quarkus.security.users.embedded.roles.<user>	角色信息。键部分是用户名，值部分是密码

11.2 使用Elytron Security JDBC配置进行认证和授权

问题

如何保护应用程序的安全，并将用户身份存储在数据库中？

解决方案

Quarkus安全提供支持，使用Elytron Security JDBC配置作为身份提供者在数据源中存储用户身份。

在11.1节中，你已经看到了如何在属性文件中使用Elytron属性文件配置扩展来定义身份。然而，这种方法更多的是用于测试和开发目的，不应该在生产环境中使用。

Elytron Security JDBC扩展可以用来存储数据库中的用户身份，支持使用bcrypt password mapper（密码映射器）进行加密，而且通用性强，不会将你锁定在任何预定义的数据库模式中。

要启用Elytron Security JDBC扩展，你需要注册quarkus-elytron-security-jdbc扩展，用于连接到数据库的JDBC驱动程序以及可选的Flyway，来填充模式和一些默认用户：

```
./mvnw quarkus:add-extension \
    -Dextensions="quarkus-elytron-security-jdbc,quarkus-jdbc-h2,quarkus-flyway"
```

保护端点，只允许Tester角色访问资源：

```
@GET
@RolesAllowed("Tester")
@Produces(MediaType.TEXT_PLAIN)
public String hello() {
    return "hello";
}
```

下一步是定义数据库模式来存储所有RBAC信息。为了简单起见，在这个例子中使用了一个简单的表，包含用户、密码和角色：

```
CREATE TABLE test_user (
  id INT,
  username VARCHAR(255),
  password VARCHAR(255),
  role VARCHAR(255)
);

INSERT INTO test_user (id, username, password, role)
  VALUES (1, 'alex', 'soto', 'Tester');
```

最后，必须配置扩展来指定执行哪个查询来验证用户并检索他们所属的角色：

```
quarkus.datasource.url=jdbc:h2:mem:mydb
quarkus.datasource.driver=org.h2.Driver
quarkus.datasource.username=sa
quarkus.datasource.password=

quarkus.flyway.migrate-at-start=true

quarkus.security.jdbc.enabled=true ❶
quarkus.security.jdbc.principal-query.sql=\
  SELECT u.password, u.role FROM test_user u WHERE u.username=? ❷
quarkus.security.jdbc.principal-query.clear-password-mapper.enabled=true ❸
quarkus.security.jdbc.principal-query.clear-password-mapper\
  .password-index=1❹
quarkus.security.jdbc.principal-query.attribute-mappings.0.index=2 ❺
quarkus.security.jdbc.principal-query.attribute-mappings.0.to=groups
```

❶启用Elytron Security JDBC

❷定义了验证用户和获取角色的查询，查询必须包含一个参数（用户名），并至少返回密码，且其值应与键在同一行

❸密码以明文形式存储

❹设置密码的索引，这些都应该在同一行

❺设置角色的索引，并将该字段指定为角色

索引以1为基础。

检索密码（以及可选的角色）的查询可以根据你的模型的要求而复杂化（例如，SQL连接）。

现在，认证数据和授权数据是从数据库而不是文件中获取的。当提供用户名和密码时（例如，使用Basic认证方法），查询将被执行，以检索认证过程所需的所有信息（将所提供的密码与从数据库中检索到的密码进行匹配），并获取授权过程中的角色。

记得更新测试（如果之前没有做过），使其通过：

```
@Test
public void testSecuredHelloEndpoint() {
    given()
      .auth().basic("alex", "soto")
    .when()
      .get("/hello")
    .then()
      .statusCode(200)
      .body(is("hello"));
}
```

讨论

在本节中，你使用了一个明文密码，这显然不应该在生产环境中使用。该扩展提供了对bcrypt密码映射器的集成，因此认证过程也适用于散列密码。

关于bcrypt

bcrypt是由Niels Provos和David Mazières设计的密码散列函数，它包含了多种保护措施，比如用盐来防止彩虹表攻击，用迭代计数来防止暴力搜索攻击。

你需要用一些额外的参数来扩展配置文件，以向Elytron Security JDBC表明密码是使用bcrypt的，不应该作为明文比较。

不配置clear-password-mapper，而是使用bcrypt-password-mapper。下面是一个使用bcrypt的配置文件的例子：

```
quarkus.security.jdbc.enabled=true
quarkus.security.jdbc.principal-query.sql=\
    SELECT u.password, u.role, u.salt, u.iteration \
    FROM test_user u WHERE u.username=?
```

```
quarkus.security.jdbc.principal-query.clear-password-mapper.enabled=false
quarkus.security.jdbc.principal-query.bcrypt-password-mapper.enabled=true ❶
quarkus.security.jdbc.principal-query.bcrypt-password-mapper.password-index=\
    1          ❷
quarkus.security.jdbc.principal-query.bcrypt-password-mapper.hash-encoding=\
    BASE64     ❸
quarkus.security.jdbc.principal-query.bcrypt-password-mapper.salt-index=\
    3          ❹
quarkus.security.jdbc.principal-query.bcrypt-password-mapper.salt-encoding=\
    BASE64     ❺
quarkus.security.jdbc.principal-query.bcrypt-password-mapper.\
    iteration-count-index=4 ❻

quarkus.security.jdbc.principal-query.attribute-mappings.0.index=2
quarkus.security.jdbc.principal-query.attribute-mappings.0.to=groups
```

❶启用bcrypt

❷设置密码索引，这应该在同一行中

❸设置密码哈希编码，这应该在同一行中

❹设置盐分指数，这应该在同一行中

❺设置盐的编码，这应该在同一行中

❻设置迭代次数指数，这应该在同一行中

经过这次更改，所提供的密码和查询到的密码之间的密码匹配并不是以明文形式进行的，而是使用bcrypt对所提供的口令字进行哈希，然后与存储的密码进行比较：

```
quarkus.security.jdbc.enabled=true

quarkus.security.jdbc.principal-query.sql=\
    SELECT u.password FROM test_user u WHERE u.username=? ❶
quarkus.security.jdbc.principal-query.clear-password-mapper.enabled=true
quarkus.security.jdbc.principal-query.clear-password-mapper.password-index=1

quarkus.security.jdbc.principal-query.roles.sql=\
    SELECT r.role_name FROM test_role r, test_user_role ur \
    WHERE ur.username=? AND ur.role_id = r.id ❷ ❸
quarkus.security.jdbc.principal-query.roles.datasource=permissions ❹
quarkus.security.jdbc.principal-query.roles.attribute-mappings.0.index=1
quarkus.security.jdbc.principal-query.roles.attribute-mappings.0.to=groups
```

❶默认数据源用于检索密码

❷roles作为一个名称来标识第二个查询，所有的查询都应该在同一行中

❸从另一个查询中获取角色

❹角色查询是针对名为Permissions的数据源执行的

11.3 使用MicroProfile JWT进行授权

问题

如何在RESTful Web服务和无状态服务中保存安全上下文？

解决方案

使用JSON网络令牌（JSON Web Token，JWT）。

JWT是RFC-7519下指定的标准，用于服务之间的信息交换。JWT的特殊性在于令牌内容的格式是JSON，而不是纯文本或任何其他二进制格式。

Quarkus与MicroProfile JWT规范（https://oreil.ly/IUO-d）集成，以使用和验证JWT令牌并检索声明。

JWT令牌由声明形成，声明是要传输的信息。例如，用户名、令牌的到期时间或用户的角色。该令牌经过数字签名，因此令牌中包含的信息可以被信任和验证。

一个JWT标记由三个部分组成。所有这些部分都以Base64格式编码。

Header

它包含一些元数据，如用于签署令牌的算法；令牌的自定义信息，如令牌类型；如果使用JSON Web加密（JWE），则包含未加密的声明。

Claims

要存储在令牌里的信息。有些声明是强制性的，有些是可选的，有些是我们的应用程序自定义的。

Signature

令牌的签名。

然后将这三个部分编码为Base64，并用句点(.)连接，所以最后的token看起来像
base64(Header).base64(Claims).base64(Signature)。

在这个例子中，使用了以下的JWT标记：

```
{ ❶
  "kid": "/privateKey.pem",
  "typ": "JWT",
  "alg": "RS256"
},
{ ❷
```

```
  "sub": "jdoe-using-jwt-rbac",
  "aud": "using-jwt-rbac",
  "upn": "jdoe@quarkus.io",
  "birthdate": "2001-07-13",
  "auth_time": 1570094171,
  "iss": "https://quarkus.io/using-jwt-rbac", ❸
  "roleMappings": {
    "group2": "Group2MappedRole",
    "group1": "Group1MappedRole"
  },
  "groups": [ ❹
    "Echoer",
    "Tester",
    "Subscriber",
    "group2"
  ],
  "preferred_username": "jdoe",
  "exp": 2200814171,
  "iat": 1570094171,
  "jti": "a-123"
}
```

❶头部分

❷声明部分

❸令牌的发行者

❹令牌所有者所属的组（或角色）

以下是同一令牌的序列化版本：

eyJraWQiOiJcL3ByaXZhdGVLZXkucGVtIiwidHlwIjoiSldUIiwiYWxnIjoiUlMyNTYifQ.
eyJzdWIiOiJqZG9lLXVzaW5nLWp3dC1yYmFjIiwiYXVkIjoidXNpbmctand0LXJiYWMiLCJ
1cG4iOiJqZG9lQHF1YXJrdXMuaW8iLCJiaXJ0aGRhdGUiOiIyMDAxLTA3LTEzIiwiYXV0aF
90aW1lIjoxNTcwMDk0MTcxLCJpc3MiOiJodHRwczpcL1wvcXVhcmt1cy5pb1wvdXNpbmcta
nd0LXJiYWMiLCJyb2xlTWFwcGluZ3MiOnsiZ3JvdXAyIjoiR3JvdXAyTWFwcGVkUm9sZSIs
Imdyb3VwMSI6Ikdyb3VwMU1hcHBlZFJvbGUifSwiZ3JvdXBzIjpbIkVjaG9lciIsIlRlc3R
lciIsIlN1YnNjcmliZXIiLCJncm91cDIiXSwicHJlZmVycmVkX3VzZXJuYW1lIjoiamRvZS
IsImV4cCI6MjIwMDgxNDE3MSwiaWF0IjoxNTcwMDk0MTcxLCJqdGki0iJhLTEyMyJ9.
Hzr41h3_uewy-g2B-son0iB0btcpkgzqmF4bT3c058v45AI0iegl7HIx7QgEZHR04PdUtR3
4x9W23VJY7NJ545ucpCuKnEV1uRlspJyQevfI-mSRg1bHlMmdDt661-V3KmQES8WX2B2uqi
ryk05fCeCp3womboilzCq4VtxbmM2qgf6ag8rUNnTCLuCgEoulGwTn0F5lCrom-7dJ0TryW
1KI0qUWHMMwl4TX5cLmqJLgBzJapzc5_yEfgQZ9qXzvsT8ze0WSKKPLm7LFVt2YihkXa80l
WcjewwT61rfQkpmqSzAHL0QIs7CsM9GfnoYc0j9po83-P3GJiBMMFmn-vg

请注意这些部分是如何用句号划分的。

当收到请求时，MicroProfile JWT规范会执行以下操作：

1.从请求中提取安全令牌，通常是从Authorization头中提取。

2.验证令牌，确保令牌有效。这些检查可能包括验证签名以信任令牌，或验证令牌是否过期。

3.提取令牌信息。

4.创建一个包含身份信息的安全上下文，以便在授权（RBAC）时使用。

此外，MicroProfile JWT规范还设置了一系列强制性的要求，每个令牌都必须提供以下内容：

声明	描述
typ	令牌格式，必须是 JWT
alg	标识用于保护令牌的加密算法，必须是 RS256
kid	标识用于保护令牌的密钥
iss	令牌发行者
sub	标识该令牌的主体
aud	标识该令牌的接收者
exp	设置到期时间
iat	提供令牌的发行时间
jti	令牌的唯一标识符
upn	java.security.Principal 接口中使用的用户 - 委托人名称
groups	已分配给令牌主体的组名列表，它们是用户所属的角色

这些都是MicroProfile JWT规范的最低要求，但也可以添加额外的要求，如Preferred_username或你的应用程序可能需要在服务之间传输的任何其他信息。

注册quarkus-smallrye-jwt扩展来开始使用MicroProfile JWT规范：

```
./mvnw quarkus:add-extension -Dextensions="quarkus-smallrye-jwt"
```

配置扩展，以设置用于验证令牌未被修改的公钥，以及服务器接受的令牌的发行人（iss）声明是否有效。

以下是本规范支持的公钥格式：

· 公钥密码标准#8（PKCS#8）隐私增强邮件（PEM）

· JSON网络密钥（JWK）

· JSON万维网密钥集（JWKS）

· JSON网络密钥（JWK） Base64 URL编码

· JSON万维网密钥集（JWKS） Base64 URL编码

在本例中，我们选择JWKS格式来指定用于验证令牌的公钥。

包含公钥的JWKS文件被放置在项目目录内：

```
{
    "keys": [
        {
            "kty": "RSA",
            "kid": "/privateKey.pem",
            "e": "AQAB",
            "n": "livFI8qB4D0y2jy0CfEqFyy46R0o7S8TKpsx5xbHKoU1VWg6QkQm-ntyIv1
                 p4kE1sPEQO73-HY8-Bzs75XwRTYL1BmR1w8J5hmjVWjc6R2BTBGAYRPFRho
                 r3kpM6ni2SPmNNhurEAHw7TaqszP5eUF_F9-KEBWkwVta-PZ37bwqSE4sCb
                 1soZFrVz_UT_LF4tYpuVYt3YbqToZ3pZOZ9AX2o1GCG3xwOjkc4x0W7ezbQ
                 ZdC9iftPxVHR8irOijJRRjcPDtA6vPKpzLl6CyYnsIYPd99ltwxTHjr3npf
                 v_3Lw50bAkbT4HeLFxTx4flEoZLKO_g0bAoV2uqBhkA9xnQ"
        }
    ]
}
```

指向这些数据的配置文件如下：

```
mp.jwt.verify.publickey.location=quarkus.pub.jwk.json ❶
mp.jwt.verify.issuer=https://quarkus.io/using-jwt-rbac ❷
```

❶公钥的位置

❷服务接受的发行人

除了处理令牌的验证过程，MicroProfile JWT还与现有的Java EE安全API相结合，提供令牌的数据。这种整合发生在以下注解中：

```
javax.ws.rs.core.SecurityContext.getUserPrincipal()
javax.ws.rs.core.SecurityContext.isUserInRole(String)
javax.servlet.http.HttpServletRequest.getUserPrincipal()
javax.servlet.http.HttpServletRequest.isUserInRole(String)
javax.ejb.SessionContext.getCallerPrincipal()
javax.ejb.SessionContext.isCallerInRole(String)
javax.security.jacc.PolicyContext
  .getContext("javax.security.auth.Subject.container")
javax.security.enterprise.identitystore.IdentityStore
  .getCallerGroups(CredentialValidationResult)
@javax.annotation.security.RolesAllowed
```

此外，MicroProfile JWT规范提供了两个类来容纳CDI或JAX-RS类内的JWT数据：

org.eclipse.microprofile.jwt.JsonWebToken

暴露原始令牌的接口，并提供获取声明的方法

@org.eclipse.microprofile.jwt.Claim

附加说明，以便将声明注入类中

例如：

```
package org.acme.quickstart;
```

```
import javax.annotation.security.RolesAllowed;
import javax.enterprise.context.RequestScoped;
import javax.inject.Inject;
import javax.ws.rs.GET;
import javax.ws.rs.Path;
import javax.ws.rs.Produces;
import javax.ws.rs.core.MediaType;

import org.eclipse.microprofile.jwt.Claim;
import org.eclipse.microprofile.jwt.Claims;
import org.eclipse.microprofile.jwt.JsonWebToken;

@Path("/hello")
@RequestScoped ❶
public class GreetingResource {

    @Inject
    JsonWebToken callerPrincipal; ❷

    @Claim(standard = Claims.preferred_username) ❸
    String username;

    @GET
    @Produces(MediaType.TEXT_PLAIN)
    public String hello() {
        return "hello " + username;
    }
}
```

❶JWT令牌本质上是请求范围的。如果你希望使用该令牌，该类必须是RequestScoped，以避免在类中混合令牌

❷注入JsonWebToken接口，代表完整的JWT令牌

❸注入preferred_username声明

声明注解还支持注入私人声明名称。这些声明不是RFC提供的官方声明名称，而是服务特有的声明（自定义声明）。要注入私有声明，请使用注解值作为声明的名称：@Claim(*"my_claim"*)。此外，在非强制性声明的情况下，可以使用java.util.Optional类来指示该声明是可空的。

```
@Claim(standard = Claims.birthdate)
Optional<String> birthdate;
```

ClaimValue接口

为了支持不是@RequestScoped的beans/resources，MicroProfile JWT规范引入了org.eclipse.microprofile.jwt.ClaimValue接口，该接口与@Claim一起使用，使得注入的值不受并发请求的影响。

```
@Claim(standard = Claims.exp) ❶
ClaimValue<Long> username;

@Claim(standard = Claims.groups) ❷
ClaimValue<Set<String>> groups;

@Claim("raw_token") ❸
ClaimValue<String> rawToken;
```

❶注入的到期时间

❷组是以java.util.Set的形式注入的

❸raw_token是一个特殊的声明，它以原始格式表示JWT标记。原始的令牌可以用来在其他服务中传播它

更新测试以发送一个bearer JWT令牌到定义的端点：

```
@Test
public void testHelloEndpoint() {
    given().header("Authorization", "Bearer " + validToken) ❶
           .when().get("/hello").then().statusCode(200).body(is("hello jdoe"));
}
```

❶JWT令牌在Authorization头中作为bearer令牌发送

在当前的解决方案中，以下假设都是真的。

· 如果提供了有效的令牌，则提取preferred_username。

· 如果提供了无效的令牌（过期、签名无效、被第三方修改等），则会向调用者发送401 Unauthorized Error错误代码。

· 如果没有提供任何标记，那么请求将被处理，但preferred_username字段为空。

MicroProfile JWT规范还通过与@RolesAllowed注解集成为授权过程提供了支持。当isCallerInRole()方法被调用时，groups声明的值会在任何时候被使用，这实际上意味着groups中的任意值都可以作为应用程序中的角色使用。

本例中使用的JWT标记中的groups claim包含以下值："groups"：["Echoer"，"Tester"，"Subscriber"，"group2"]。通过使用@RolesAllowed保护对/hello的调用，并使用标记中的一个组值：

```
@GET
@Produces(MediaType.TEXT_PLAIN)
@RolesAllowed("Tester")
public String hello() {}
```

现在，你可以假设以下几点。

· 如果提供了一个有效的令牌，并且groups声明中包含了Tester群组，那么就会提取preferred_username。

· 如果提供了有效的令牌，而groups声明中不包含Tester组，则会向调用者发送403 Forbidden错误代码。

· 如果提供了一个无效的令牌（过期、签名无效、被第三方修改等），那么会向调用者发送401 Unauthorized Error错误代码。

· 如果没有提供令牌，则会向呼叫者发送401 Unauthorizpd Error错误代码。

讨论

在过去，安全上下文被保存在HTTP会话中。开始扩展服务时，事情变得越来越复杂。为了避免这个问题，一个可能的解决方案是在所有的调用中使用令牌（尤其是JSON令牌）来传递这些信息。

需要注意的是，该令牌是有签名的，而不是加密的，这意味着信息可以被任何人看到，但不能被修改。可以使用JSON Web加密技术添加一个加密层，这样声明就不是明文的，而是加密的。

本节的目的不是让你掌握JWT，而是让你学习如何在Quarkus中使用它，所以我们假设你已经对JWT有了一些了解。

另见

要了解有关JWT的更多信息，请访问以下网页：

- JSON Web Tokens（https://jwt.io）
- GitHub:JWT RBAC for MicroProfile（https://oreil.ly/tXP9d）
- IETF:JSON Web Token（https://oreil.ly/p9jUC）

11.4　使用OpenId Connect进行授权和认证

问题

如何用OpenId Connect保护你的RESTful Web API？

解决方案

使用bearing令牌授权，令牌由OpenId Connect发行。

在上一节中，你学习了如何使用JWT令牌来保护资源，但没有涉及令牌的生成，因为令牌是在前面生成的，并在文本文件中提供。

在真实的应用中，你需要一个发行令牌的身份提供者。分布式服务的事实协议是OpenId Connect和OAuth 2.0，以及符合该协议的授权服务器，如Keycloak（https://www.keycloak.org）。

注册quarkus-oidc扩展，以保护OpenId Conect的资源：

```
./mvnw quarkus:add-extension -Dextensions="quarkus-oidc"
```

配置OpenId Connect服务器的位置以验证令牌：

```
quarkus.oidc.auth-server-url=http://localhost:8180/auth/realms/quarkus ❶
quarkus.oidc.client-id=backend-service ❷
```

❶OpenID Connect服务器的基本URL

❷每个应用程序都有一个客户端ID，用于识别应用程序

使用@RolesAllowed注解保护端点：

```
@Inject
io.quarkus.security.identity.SecurityIdentity securityIdentity; ❶

@GET

@RolesAllowed("user")
@Produces(MediaType.TEXT_PLAIN)
public String hello() {
    return "hello " + securityIdentity.getPrincipal().getName();
}
```

❶代表当前登录用户的Quarkus接口

测试必须更新，以便从OpenId Connect获取访问令牌，并提供它作为bearer令牌：

```
@Test
public void testHelloEndpoint() {
    System.out.println(accessToken);
    given()
      .auth().oauth2(accessToken)
      .when().get("/hello")
      .then()
         .statusCode(200)
         .body(is("hello alice"));
}
```

访问令牌（access token）是在OpenId Connect服务器中生成的。要生成它，必须提供一些参数，如用户名和密码，以访问服务器并生成代表用户的令牌：

```
package org.acme.quickstart;

import java.net.URI;
import java.net.URISyntaxException;

import io.restassured.RestAssured;
import io.restassured.builder.RequestSpecBuilder;
import io.restassured.response.Response;
import io.restassured.response.ResponseOptions;
import io.restassured.specification.RequestSpecification;

public class RestAssuredExtension {

  public static ResponseOptions<Response> getAccessToken(String url,
                                                          String clientId,
                                                          String clientIdPwd,
                                                          String username,
                                                          String password) {
    final RequestSpecification request = prepareRequest(url);
    try {
      return request
        .auth()
        .preemptive()
        .basic(clientId, clientIdPwd)
        .contentType("application/x-www-form-urlencoded; charset=UTF-8")
        .urlEncodingEnabled(true)
        .formParam("username", username)
        .and()
        .formParam("password", password)
        .and()
        .formParam("grant_type", "password")
        .post(new URI(url));
    } catch (URISyntaxException e) {
      throw new IllegalArgumentException(e);
    }
  }

  private static RequestSpecification prepareRequest(String url) {
    final RequestSpecBuilder builder = new RequestSpecBuilder();
    final RequestSpecification requestSpec = builder.build();
    return RestAssured.given().spec(requestSpec);
  }
}
```

这段代码本质上是下一个curl命令的实现，但使用了REST-Assured：

```
curl -X POST \
    http://localhost:8180/auth/realms/quarkus/protocol/openid-connect/token \
    --user backend-service:secret \
    -H 'content-type: application/x-www-form-urlencoded' \
    -d 'username=alice&password=alice&grant_type=password'
```

现在，当运行测试时，显示出与11.3节中完全不同的东西。

首先，令牌（JWT令牌）不是静态的，而是由OpenID Connect（Keycloak）为alice用户名发出的。

以下是为alice签发的令牌的例子：

```
{
  "alg": "RS256",
  "typ": "JWT",
  "kid": "cfIADN_xxCJmVkWyN-PNXEEvMUWs2r68CxtmhEDNzXU"
},
{
  "jti": "cc54b9db-5f2f-4609-8a6b-4f76026e63ae",
  "exp": 1578935775,
  "nbf": 0,
  "iat": 1578935475,
  "iss": "http://localhost:8180/auth/realms/quarkus",
  "sub": "eb4123a3-b722-4798-9af5-8957f823657a",
  "typ": "Bearer",
  "azp": "backend-service",
  "auth_time": 0,
  "session_state": "5b674175-a2a9-4a45-a3da-394923125e55",
  "acr": "1",
  "realm_access": {
    "roles": [
      "user"
    ]
  },
  "scope": "email profile",
  "email_verified": false,
  "preferred_username": "alice"
}
```

其次，OpenID Connect负责提供一切验证令牌的工作。公钥不是手动配置的。

当提供令牌时，Keycloak会执行以下验证。

· 如果提供了一个有效的令牌，并且roles声明中包含user组，则提取preferred_username。

· 如果提供了一个有效的令牌，而roles声明不包含user组，那么会返回403 Forbidden错误代码。

· 如果提供了无效的令牌（过期、签名无效、被第三方修改等），那么会返回403 Forbidden错误代码。

· 如果没有提供任何令牌，则会返回401 Unauthorized Error错误代码。

另见

要了解关于OpenId Connect协议的更多信息，请看以下网站：

· OpenID Connect（https://openid.net/connect）

· Keycloak（https://www.keycloak.org）

11.5　使用OpenId Connect保护网络资源

问题

如何保护你的网络资源？

解决方案

使用OpenId Connect和基于文件的角色定义来保护网络资源。

网络资源可以使用OpenId Connect协议和Quarkus来保护。OpenId Connect扩展通过实现众所周知的授权代码流来实现对网络资源的认证，任何试图访问受保护资源的未经认证的用户都会被重定向到OpenId Connect Prorider网站进行认证。认证过程完成后，用户会被送回应用程序。

注册quarkus-oidc扩展来保护OpenId Connect的资源：

```
./mvnw quarkus:add-extension -Dextensions="quarkus-oidc"
```

配置OpenId Connect服务器的位置以验证令牌：

```
quarkus.oidc.auth-server-url=http://localhost:8180/auth/realms/quarkus ❶
quarkus.oidc.client-id=frontend ❷
quarkus.oidc.application-type=web-app ❸
quarkus.http.auth.permission.authenticated.paths=/* ❹
quarkus.http.auth.permission.authenticated.policy=authenticated
```

❶OpenID连接服务器的基本URL

❷每个应用程序都有一个客户端ID，用于识别应用程序

❸启用OpenID Connect授权码流

❹设置网络资源的权限

启动应用程序，打开浏览器，输入以下网址：http://localhost:8080：

```
./mvnw clean compile quarkus:dev
```

默认的index.html页面没有显示，但你会被重定向到Keycloak的认证页面。输入以下有效凭证（登录名：alice，密码：alice），即

可访问网站资源。按下Login（登录）按钮后，页面会被重定向回登录页面。

第12章　应用程序密钥管理

每一个应用程序都有需要保密的信息，这些信息可能包括数据库凭证、外部服务认证，甚至某些资源的位置。所有这些都统称为密钥。应用程序需要一个安全的地方来存储这些密钥，无论是在应用程序启动时还是在空闲时。在本章中，我们将讨论使用Kubernetes和Vault进行密钥管理。

12.1 使用Kubernetes中的secret存储数据

问题

如何将密钥以一种更安全的形式存储在Kubernetes中，而不是直接存储在pod或者容器中？

解决方案

使用Kubernetes的secret在容器上以纯文本方式存储和检索敏感数据，如密码、令牌或SSH密钥。Kubernetes中的secret对象可以用来存储敏感数据。

将敏感数据存储在secret对象中并不会自动地使其安全，因为Kubernetes不会对数据进行加密，而是默认用Base64编码。使用secret可以为你提供一些标准配置过程没有提供的功能：

· 你可以定义访问secret的授权策略。

· 你可以配置Kubernetes来加密敏感数据（这就是所谓的静态加密）。

· 你可以使用列表授予对特定容器实例的访问权。

这些功能都不是默认启用的，它们需要一些关于Kubernetes的知识。在本书中，我们只解释Quarkus如何与Kubernetes等其他工具集成，并不解释工具的操作层面。

secret可以作为环境变量或以卷挂载的方式注入容器中。环境变量方式的安全性较低，因为任何能够访问容器实例的人都可以轻易地转储内容。另一方面，卷方式在有大量密钥时容易变得复杂，因为Kubernetes为每个secret创建一个文件并存储在它的值里。

两种方式都会介绍，你可以选择一个更适合你的用例的方式。

这个例子涵盖了一个API令牌（如Github个人访问令牌）需要在应用中设置为secret的用例。

想要在Pod中使用secret注入生成的Kubernetes资源，需要注册quarkus-kubernetes扩展：

```
./mvnw quarkus:add-extension -Dextensions="quarkus-kubernetes"
```

通过API创建一个Secret类型的Kubernetes资源或使用kubectl CLI工具来创建一个secret。打开一个新的终端，运行下面的命令来注册一个新的secret，它有greeting-security ID和带有令牌的

github.api.key.token密钥（这个令牌是无效的，仅用于示例目的）：

```
kubectl create secret generic greeting-security \
--from-literal=github.api.key.token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.\
eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.\
SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c
```

现在这个secret已经被创建了，我们看一下如何将它作为环境变量传给容器。

配置属性可以从环境变量中获取。在这种情况下，这个属性被称为github.api.key.token，当然你也可以直接使用System.getenv()来访问它。前一种方法更好，因为它依靠MicroProfile Config规范来读取配置属性，而不是一些自定义的解决方案：

```
@ConfigProperty(name = "github.api.key.token")
String githubToken;
```

在*application.properties*中为Kubernetes扩展设置额外的属性，以便生成的Kubernetes部署文件包含将secret作为环境变量注入所需的字段：

```
quarkus.container-image.group=quarkus ❶
quarkus.container-image.name=greeting-started-kubernetes-secrets
quarkus.container-image.tag=1.0-SNAPSHOT
quarkus.kubernetes.image-pull-policy=if-not-present
quarkus.kubernetes.env-vars.github-api-key-token.name=github.api.key.token ❷
quarkus.kubernetes.env-vars.github-api-key-token.secret=greeting-security ❸
```

❶配置Docker镜像

❷设置环境变量覆盖github.api.key.token属性

❸设置要加载的密钥名称

Kubernetes文件的生成将在容器定义包含一个名为secretkeyRef的新条目，它定义了所有的键/值对。

MicroProfile配置规范允许使用等价的环境变量覆盖任何配置属性（大写并将点[.]改为下划线[_]）。Secrets包含了作为secret的配置属性。在*application.properties*中，Kubernetes扩展被配置为生成一个deployment描述符，该描述符将这些密钥设置为环境变量，这样当容器在Kubernetes集群内启动时，密钥将作为环境变量注入容器中，并被MicroProfile Config作为配置属性读取。

要部署应用程序，需要打开一个新的终端窗口，打包应用程序，创建Docker容器，并应用产生的Kubernetes资源：

```
./mvnw clean package -DskipTests

docker build -f src/main/docker/Dockerfile.jvm \
    -t quarkus/greeting-started-kubernetes-secrets:1.0-SNAPSHOT .
kubectl apply -f target/kubernetes/kubernetes.yml

kubectl patch svc greeting-started-kubernetes-secrets \
    --type='json' \
    -p '[{"op":"replace","path":"/spec/type","value":"NodePort"}]'
curl $(minikube service greeting-started-kubernetes-secrets --url)/hello
```

但是secret也可以挂载为卷，而不是设置为环境变量。在*application.properties*中设置Kubernetes扩展属性，使生成的Kubernetes部署文件中包含将secret文件挂载为卷的字段：

```
quarkus.kubernetes.mounts.github-token.path=/deployment/github ❶ ❷
quarkus.kubernetes.mounts.github-token.read-only=true ❸
quarkus.kubernetes.secret-volumes.github-token.secret-name=greeting-security ❹
quarkus.kubernetes.secret-volumes.github-token.default-mode=420 ❺
```

❶挂载带有github-token名的卷

❷设置在容器内安装卷的路径

❸将该卷设置为只读

❹设置要加载的密钥名称

❺将模式设置为可从进程中读取

最后一步是从代码中读取secret。由于secret被挂载在文件系统中，所以需要像其他文件一样读取：

```
@GET
@Path("/file")
@Produces(MediaType.TEXT_PLAIN)
```

```
public String ghTokenFile() throws IOException {
    final byte[] encodedGHToken = Files.readAllBytes(
        Paths.get("/deployment/github/github.api.key.token")); ❶
    return new String(encodedGHToken);
}
```

❶secret的位置是挂载路径加上秘钥

要部署应用程序，需要打开一个新的终端窗口，打包应用程序，创建Docker容器，并应用产生的Kubernetes资源：

```
./mvnw clean package -DskipTests

docker build -f src/main/docker/Dockerfile.jvm \
    -t quarkus/greeting-started-kubernetes-secrets:1.0-SNAPSHOT .
kubectl apply -f target/kubernetes/kubernetes.yml

kubectl patch svc greeting-started-kubernetes-secrets --type='json' \
    -p '[{"op":"replace","path":"/spec/type","value":"NodePort"}]'
curl $(minikube service greeting-started-kubernetes-secrets --url=/hello/file
```

讨论

Kubernetes的secret存在一些问题，需要从外部解决，以下是其中的一些问题：

- Kubernetes的secret并没有被加密，只是默认用Base64编码。
- 你需要使用SSL与etcd进行通信。etcd是存储secret的地方。
- 磁盘需要加密，因为etcd会将数据存储在磁盘上。
- 你需要正确定义RBAC去防止其他人访问secret。

另见

要了解关于Kubernetes secret的更多信息，请访问以下关于Kubernetes的网页：

- Secrets（https://oreil.ly/dFTgh）
- Encrypting Secret Data at Rest（https://oreil.ly/_auK1）
- Authorization Overview（https://oreil.ly/ctlyn）
- Using RBAC Authorization（https://oreil.ly/WrcaP）

12.2 使用Vault安全地存储配置的secret

问题

如何安全地存储配置密钥？

解决方案

使用Quarkus Vault（https://oreil.ly/UMKuH）扩展插件去查找密钥。

在处理密钥时，最关键的是存储它们，使被禁止的用户无法读取，保护对密钥的访问，仅让需要这些密钥的服务可以访问它们。

Vault是一个为存储和使用密钥提供统一接口，并简化用例的工具

关于Vault

Vault（https://oreil.ly/UMKuH）是一个安全访问密钥的开源工具。如何在生产中运行Vault不在本书的讨论范围之内。

为了简化Vault的安装，使用Vault Docker容器：

```
docker run --rm --cap-add=IPC_LOCK -e VAULT_ADDR=http://localhost:8200 \
  -p 8200:8200 --name=dev-vault vault:1.2.2
```

你可能需要设置以下环境变量：

```
$ export VAULT_ADDR='http://0.0.0.0:8200'
```

解封密钥和根令牌显示在下面，以备你想封存/解封Vault或重新认证：

```
Unseal Key: s7WbMScSOh02ERK6XEfl6ep6BReRQZzl9VekrrnyKE8=
Root Token: s.ty3QS2uNaxPdiFsSZpCQfjpc ❶
```

❶令牌，用于初始化访问Vault

在Vault容器内打开shell，配置Vault并添加一个secret：

```
docker exec -it dev-vault sh
export VAULT_TOKEN=s.ty3QS2uNaxPdiFsSZpCQfjpc ❶

vault kv put secret/myapps/vault-service/config foo=secretbar ❷
```

❶设置要访问的令牌

❷在路径secret/myapps/vault-service/config处用foo作为键创建一个新的secret创建一个允许对密钥进行读取访问的策略：

```
cat <<EOF | vault policy write vault-service-policy -
path "secret/data/myapps/vault-service/*" {
  capabilities = ["read"]
}
EOF
```

最后一步是启用凭证（userpass引擎），以便从服务中访问secret：

```
vault auth enable userpass
vault write auth/userpass/users/alex password=alex \
  policies=vault-service-policy ❶
```

❶创建一个ID为alex，密码为alex的用户

Vault支持多种认证方式对Vault服务进行认证，然后再开始使用密钥。在撰写本书时，Quarkus Vault扩展插件支持以下认证方式：

token

直接传递用户令牌，绕过认证过程。

user/password

使用用户名和密码凭证进行Vault认证。

approle

使用一个role_id和secret_id进行认证。这个方法面向自动化的工作流（机器和服务）。role_id通常被嵌入到Docker容器中，secret_id则由Kubernetes集群以cubby-hole响应的方式获取，将其封装（单次使用）并交付给目标服务。

kubernetes

使用Kubernetes服务账户令牌对Vault进行认证。

开始使用Vault前，需要先注册quarkus-vault扩展插件：

```
./mvnw quarkus:add-extension -Dextensions="quarkus-vault"
```

Quarkus Vault扩展插件集成了MicroProfile Configuration规范，因此可以使用@Config-Property注解注入一个密钥。配置应用程序使用用户名和密码作为Vault的认证方法，并设置存储密钥的基本路径：

```
quarkus.vault.url=http://localhost:8200 ❶
quarkus.vault.authentication.userpass.username=alex ❷
quarkus.vault.authentication.userpass.password=alex

quarkus.vault.kv-secret-engine-version=2 ❸
quarkus.vault.secret-config-kv-path=myapps/vault-service/config
```

❶Vault服务器的基本URL

❷认证的凭证

❸存储secret的路径

使用@org.eclipse.microprofile.config.inject.ConfigProperty注解访问密钥中foo键的值：

```
@ConfigProperty(name = "foo") ❶
String foo;

@GET
@Produces(MediaType.TEXT_PLAIN)
public String hello() {
    return foo;
}
```

❶注入的键foo的secret值

启动应用并向端点发送一个请求：

```
./mvnw clean compile quarkus:dev

curl http://localhost:8080/hello
secretbar
```

讨论

如果只有在运行时才知道路径，也可以通过注入io.quarkus.vault.VaultKVSecretEngine接口，以编程方式检索密钥。

```
@Inject
VaultKVSecretEngine kvSecretEngine;

final Map<String, String> secrets = kvSecretEngine
  .readSecret("myapps/vault-service/config"); ❶
final String fooSecret = secrets.get("foo");
```

❶提供存储在Vault键/值密钥引擎中的值

另见

要了解更多Vault知识，请访问以下网站：

· Vault:Documentation（https://oreil.ly/ke_Q5）

12.3　密码即服务

问题

如何避免将所有的加密操作分散在所有的服务上？

解决方案

使用Vault的transit引擎，让所有的加密操作在同一个地方执行。

在前面创建的Vault容器内打开一个shell，配置Vault并添加密钥来加密和签署消息。

```
docker exec -it dev-vault sh
export VAULT_TOKEN=s.ty3QS2uNaxPdiFsSZpCQfjpc ❶
vault secrets enable transit ❷

vault write -f transit/keys/my_encryption ❸
vault write transit/keys/my-sign-key type=ecdsa-p256 ❹
```

❶设置要访问的令牌

❷启用transit引擎

❸创建一个AES-256-GCM96类型的加密密钥

❹创建一个ECDSA-P256类型的签名密钥

制定一个策略，允许访问transit操作：

```
cat <<EOF | vault policy write vault-service-policy -
path "transit/*" {
  capabilities = [ "create", "read", "update" ]
}
EOF
```

最后一步是启用凭证（userpass引擎）来访问服务中的密钥：

```
vault auth enable userpass
vault write auth/userpass/users/alex password=alex \
  policies=vault-service-policy ❶
```

❶创建一个ID为alex，密码为alex的用户

使用Vault注册quarkus-vault扩展：

```
./mvnw quarkus:add-extension -Dextensions="quarkus-vault"
```

给应用配置用户名和密码作为访问Vault的认证方式：

```
quarkus.vault.url=http://localhost:8200 ❶

quarkus.vault.authentication.userpass.username=alex ❷
quarkus.vault.authentication.userpass.password=alex
```

❶Vault服务器的基本URL

❷认证的凭证

注入io.quarkus.vault.VaultTransitSecretEngine实例，再执行transit操作：

```
@Inject
VaultTransitSecretEngine transit; ❶

@GET
@Path("/encrypt")
@Produces(MediaType.TEXT_PLAIN)
```

```
public String encrypt(@QueryParam("text") String text) {
    return transit.encrypt("my_encryption", text); ❷
}

@GET
@Path("/decrypt")
@Produces(MediaType.TEXT_PLAIN)
public String decrypt(@QueryParam("text") String text) {
    return transit.decrypt("my_encryption", text).asString(); ❸

}

@GET
@Path("/sign")
@Produces(MediaType.TEXT_PLAIN)
public String sign(@QueryParam("text") String text) {
    return transit.sign("my-sign-key", text); ❹
}
```

❶Transit操作接口

❷使用加密密钥进行加密

❸使用加密密钥解密

❹用给定的签名在文本上签名

启动应用并向端点发送请求：

```
./mvnw clean compile quarkus:dev

curl http://localhost:8080/hello/encrypt?text=Ada
vault:v1:iIunGAElLpbaNWWqZq1yf4cctkEUOFdJE1oRTaSI2g==

curl http://localhost:8080/hello/decrypt? \
    text=vault:v1:iIunGAElLpbaNWWqZq1yf4cctkEUOFdJE1oRTaSI2g==
Ada

curl http://localhost:8080/hello/sign?text=Alexandra
vault:v1:MEUCIGkgS5VY5KEU2yHqnIn9qwzgfBUv3O2H4bgNAFVrYCK3AiEAnQznfdEZI6b\
  /Xtko/wEl8WhZLuKZQ/arOYkfsnwBH3M=
```

讨论

服务中通常需要执行加密、解密、签名或基于哈希的数据消息验证码（HMAC）等加密操作。这些操作通常是在每个服务中实现的，这意味着你在每个服务中重复了这种敏感的逻辑以及密钥的管理。

Vault Transit引擎为你处理所有的加密功能，而不存储结果数据。你可以将Vault看作是一种加密即服务模式，在这种模式下，数据的发送、操作和返回都不需要内部存储。

一切都由Vault内部管理，使开发人员能够专注于实现重要的业务逻辑。

Vault扩展支持以下操作：

encrypt

使用transit secret engine中配置的Vault秘钥对常规字符串进行加密。

decrypt

用指定的密钥解密被加密的数据，并返回未加密的数据。

rewrap

将使用旧密钥版本与最后一个密钥版本加密得到的密文重新加密成一个新的密文。

`sign`

用指定的键签署一个输入字符串。

`verifySignature`

检查签名是否是用指定的密钥对输入进行签名获得的。

另见

要获取更多信息，请访问以下网站：

- Vault:Transit Secrets Engine（https://oreil.ly/rloOa）

12.4　生成数据库密码并存入secret

问题

如何安全地存储数据库密码？

解决方案

读取数据库密码，将其存入secret。

数据库密码需要保护，不应该直接设置到配置文件中。Quarkus Vault扩展集成了持久性配置，可以从Vault中将数据库密码作为secret来读取。

在前面创建的Vault容器内打开一个shell，配置Vault，并将数据库密码添加为secret：

```
docker exec -it dev-vault sh
export VAULT_TOKEN=s.ty3QS2uNaxPdiFsSZpCQfjpc ❶

vault kv put secret/myapps/vault-service/db password=alex ❷
```

❶设置要访问的令牌

❷用密钥password和值alex创建一个新的密钥

制定一个策略，给予读取secret的权限：

```
cat <<EOF | vault policy write vault-service-policy -
path "secret/data/myapps/vault-service/*" {
  capabilities = ["read"]
}
EOF
```

最后一步是启用凭证（userpass引擎）来访问服务的secret：

```
vault auth enable userpass
vault write auth/userpass/users/alex password=alex \
  policies=vault-service-policy ❶
```

❶创建ID为alex，密码为alex的用户

在本例中，使用PostgreSQL服务器作为数据库。在新的终端中运行以下命令，启动一个新的Docker实例：

```
docker run --ulimit memlock=-1:-1 -it --rm=true --memory-swappiness=0 \
  --name postgres-quarkus-hibernate -e POSTGRES_USER=alex \
  -e POSTGRES_PASSWORD=alex -e POSTGRES_DB=mydatabase \
  -p 5432:5432 postgres:10.5
```

注意，这个密码和*secret/myapps/vault-service/db*路径下的一样。

注册quarkus-vault和持久化扩展：

```
./mvnw quarkus:add-extension \
  -Dextensions="quarkus-vault, quarkus-hibernate-orm-panache, \
   quarkus-jdbc-postgresql, quarkus-resteasy-jsonb"
```

数据源配置与第7章中的配置略有不同。密码不是硬编码在配置文件中，而是作为secret从Vault中获取，并用于建立连接。

除了Vault配置参数（如URL和认证方式（即用户/密码）），你还需要定义Vault内部的键/值路径，即数据库配置的存储位置。更具体地说，它是名为password的键与数据库密码存储的路径。在下面的例子中，为了将这些信息设置到Vault中，你需要运行命令vault kv put

secret/myapps/vault-service/db password=alex，但如果你按照本节的要求，在配置Vault时就已经完成了这个操作。

另外，在建立与数据库的连接时，需要覆盖使用的凭证提供者，以表明密码来自Vault，而不是作为配置属性。这可以通过使用quarkus.datasource.credentials-provider属性来实现。

用数据源和Vault参数配置应用程序，并覆盖凭证提供者：

```
quarkus.datasource.url=jdbc:postgresql://localhost:5432/mydatabase
quarkus.datasource.driver=org.postgresql.Driver
quarkus.datasource.username=alex
quarkus.datasource.credentials-provider=mydatabase ❶
quarkus.vault.credentials-provider.mydatabase\
  .kv-path=myapps/vault-service/db ❷
quarkus.vault.url=http://localhost:8200 ❸
quarkus.vault.authentication.userpass.username=alex
quarkus.vault.authentication.userpass.password=alex
quarkus.vault.kv-secret-engine-version=2
quarkus.hibernate-orm.database.generation=drop-and-create
%dev.quarkus.hibernate-orm.sql-load-script=import.sql
%dev.quarkus.hibernate-orm.log.sql=true
```

❶将凭证提供者设置为自定义名称（mydatabase）

❷设置mydatabase提供者存储密码的键/值路径

❸配置Vault参数

需要注意的是，并没有quarkus.datasource.password这个属性，因为密码是从Vault获取的。

此时，当Quarkus应用程序启动时，会执行以下步骤。

❶服务认证到Vault服务。

❷键/值从secret/myapps/vault-service/db路径中获取。

❸密钥password的值作为数据库的密码凭证。

键名称可以从密码改为其他任何键名。使用kv-key属性来命名：quarkus.vault.credentials-provider.mydatabase.kv-key=pass

讨论

Vault可以动态生成数据库凭证，并配置数据库实例，将其作为凭证使用，而不必手动配置凭证，并在Vault或需要访问数据库的服务中进行设置。这意味着没有在任何地方对凭证进行硬编码，因为它们是从Vault请求的。生成的一对用户名和密码受Vault的租赁机制约束，在合理的时间后，凭证就会失效。

采取以下步骤，配置Vault动态生成数据库凭证：

1.启用数据库secret引擎。

2.设置数据库的连接参数，设置厂商数据库（目前大部分SQL和NoSQL数据库都支持）。

3.配置一个角色，将Vault中的名称映射到SQL语句中以创建数据库凭证：

```
vault secrets enable database

cat <<EOF | vault policy write vault-service-policy -
path "database/creds/mydbrole" {
  capabilities = [ "read" ]
}
EOF

vault write database/config/mydb
     plugin_name=postgresql-database-plugin \
     allowed_roles=mydbrole \
     connection_url=postgresql://{{username}}:{{password}}\
          @localhost:5432/mydb?sslmode=disable \
     username=alex \
     password=alex

vault write database/roles/mydbrole \
     db_name=mydb \
     creation_statements="CREATE ROLE \"{{name}}\" WITH LOGIN PASSWORD \
                          '{{password}}' VALID UNTIL '{{expiration}}'; \
                          GRANT SELECT,INSERT, UPDATE, DELETE ON ALL \
                          TABLES IN SCHEMA public TO \"{{name}}\"; \
                          GRANT USAGE, SELECT ON ALL SEQUENCES IN \
                          SCHEMA public to \"{{name}}\";" \
     default_ttl="1h" \
     revocation_statements="ALTER ROLE \"{{name}}\" NOLOGIN;" \
     renew_statements="ALTER ROLE \"{{name}}\" VALID UNTIL '{{expiration}}';" \
     max_ttl="24h"
```

Vault扩展还支持通过credentials-provider上的database-credentials-role属性使用动态数据库凭证：

```
quarkus.vault.url=https://localhost:8200
quarkus.vault.authentication.userpass.username=alex
quarkus.vault.authentication.userpass.password=alex

quarkus.datasource.driver=org.postgresql.Driver
quarkus.datasource.url=jdbc:postgresql://localhost:6543/mydb
quarkus.datasource.username=postgres

quarkus.datasource.credentials-provider=dynamic-ds ❶
quarkus.datasource.credentials-provider-type=vault-credentials-provider
quarkus.vault.credentials-provider.dynamic-ds.database-credentials-role=\
  mydbrole ❷
```

❶没有设置密码

❷配置动态凭证

另见

要了解有关Vault动态数据库凭证的更多信息，请访问以下网站：

- Vault:Databases（https://oreil.ly/RDaes）

12.5 使用Vault Kubernetes Auth进行认证服务

问题

如何在不使用用户名和密码的情况下对Vault进行认证服务？

解决方案

使用Vault Kubernetes Auth方法。

到目前为止，你已经使用凭证与用户名和密码的方法来验证Quarkus服务与Vault服务。这种方法在某些情况下可能是好的（测试目的、内部应用等），但请注意，你正在引入一个新的secret来获取更多的secret。解决这个问题的一种方法是使用Kubernetes Secrets来设置Vault密码，例如，使用approle认证方法。另一种方法是使用Vault Kubernetes Auth，该方法令它非常适合验证部署在Kubernetes集群中的服务。

Vault Kubernetes auth方法使用Kubernetes服务账户令牌和定义的角色对Vault服务进行认证。使用这种方法，Vault不存储凭证。它使用受信任的第三方（Kubernetes集群）来验证它们。当实例化了带有服务的Pod时，服务账户令牌会被安装在容器内部，因此应用程序可

以访问它。secret令牌的默认挂载点是*/var/run/secrets/kubernetes.io/serviceaccount/token*。

然后，应用程序尝试将其发送到Vault服务器，从而使用该令牌进行认证。之后，Vault会对Kubernetes API进行调用，以确保令牌的有效性。如果令牌有效，就会返回一个Vault内部的令牌，以便在未来的请求中用于获取密钥。图12-1总结了这个过程。

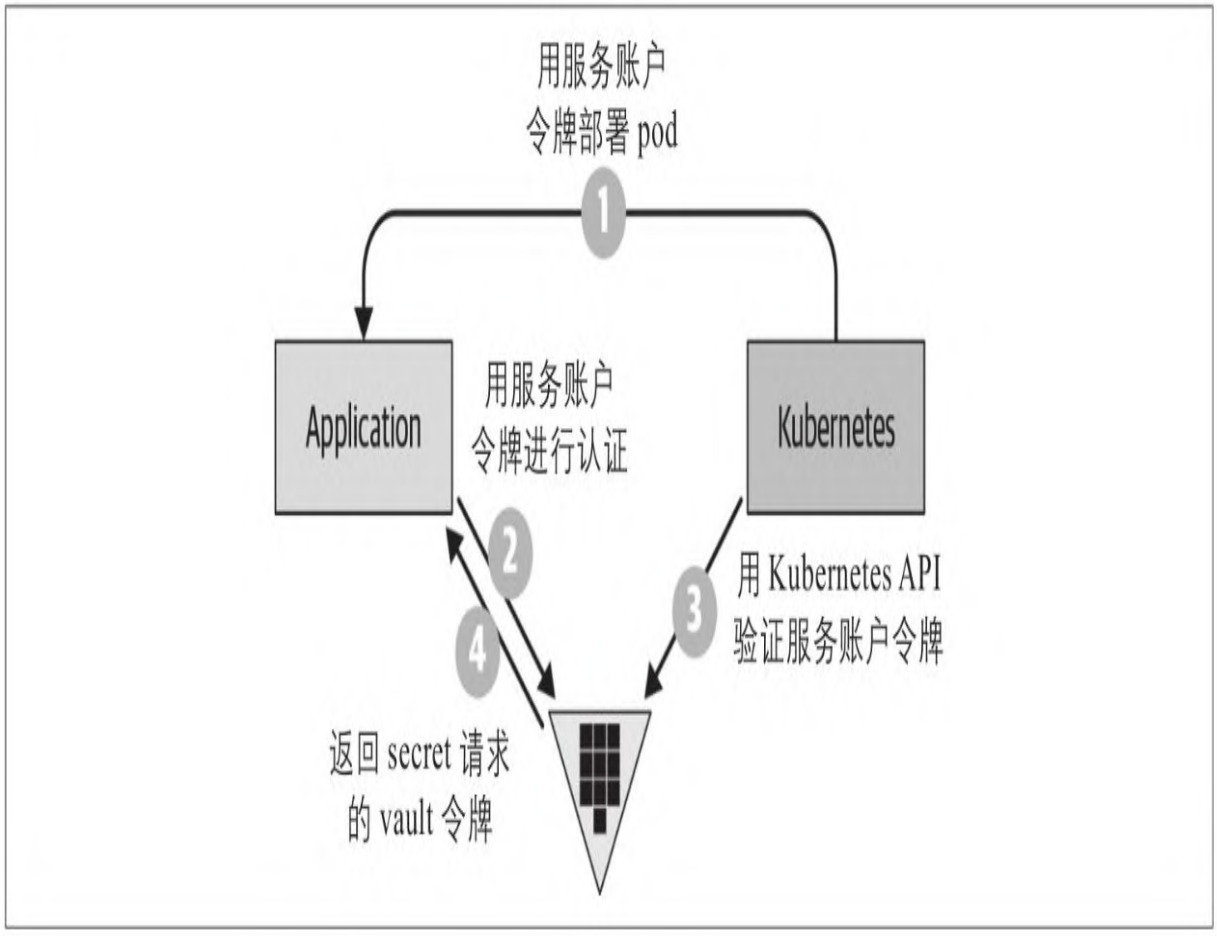


图12-1：Kubernetes认证方法

要配置Kubernetes auth模式，需要给Vault设置两个参数来连接Kubernetes API：第一个是要访问的令牌，第二个是验证Vault和Kubernetes API之间通信的证书授权。这些值都是从vault-token开头的密钥中获取的。当Vault第一次为这个例子设置时，该值是vault-token-mm5qx。

要获得令牌并将其存储在文件中，需打开终端窗口并运行以下命令：

```
kubectl get secret vault-token-mm5qx -o jsonpath='{.data.token}' \
  | base64 --decode > jwt.txt ❶ ❷

cat jwt.txt
eyJhbGciOiJSUzI1NiIsImtpZCI6Inp0WWZBcl8weW1SaTI1bjRNYVNHNmtXOUhCWDV\
yczhYandVYkVETktzRHMifQ.
```

❶将secret名称替换为你的secret名称，以vault-token开头

❷secret是以Base64存储的，所以它们需要解码

要获得证书授权并将其存储在一个文件中，需在终端中运行以下操作：

```
kubectl get secret vault-token-mm5qx -o jsonpath="{.data['ca\.crt']}" \
  | base64 --decode > ca.crt ❶ ❷

cat ca.crt

-----BEGIN CERTIFICATE-----
MIIC5zCCAc+gAwIBAgIBATANBgkqhkiG9w0BAQsFADAVMRMwEQYDVQQDEwptaW5p
-----END CERTIFICATE-----
```

❶pod被命名为Vault-0

❷一个vault-token被设置为secret

在部署应用之前，你需要启用Kubernetes auth方法并配置它，然后插入一些secret来测试它。

将Vault服务从Kubernetes集群中暴露出来，这样就可以从你的本地机器进行配置。打开一个新的终端窗口，并运行命令，将流量从localhost:8200转发到Kubernetes集群内部运行的Vault实例中：

```
kubectl port-forward svc/vault 8200:8200
```

回到运行获取令牌和证书颁发机构等命令的终端窗口，并运行以下命令来插入一个secret：

```
export VAULT_TOKEN=root ❶
export VAULT_ADDR='http://localhost:8200'

cat <<EOF | vault policy write vault-service-policy - ❷
path "secret/data/myapps/vault-service/*" {


  capabilities = ["read"]
}
EOF

vault kv put secret/myapps/vault-service/config foo=secretbar ❸
```

❶配置Vault连接参数

❷创建一个名为vault-service-policy的策略到myapps/vault-service/*secrets

❸设置一个新的密钥

最后一步是启用Kubernetes auth方法，并将其配置为使用Kubernetes API验证令牌。

执行以下命令：

```
vault auth enable kubernetes ❶

vault write auth/kubernetes/config \ ❷
    token_reviewer_jwt=@jwt.txt \ ❸
    kubernetes_host=https://kubernetes.default.svc \ ❹
    kubernetes_ca_cert=@ca.crt ❺

vault write auth/kubernetes/role/example \ ❻
        bound_service_account_names=vault \ ❼
        bound_service_account_namespaces=default \ ❽
        policies=vault-service-policy ❾
```

❶启用Kubernetes auth方法

❷配置auth方法

❸设置上一步中检索到的令牌文件

❹设置Kubernetes API主机

❺设置上一步检索到的CA文件

❻创建一个新的角色（example），以便从应用程序中进行认证

❼设置服务账户名称，此处是vault

❽设置服务运行的命名空间

❾将使用此方法进行认证的用户与所创建的策略绑定

让我们开发一个使用Vault Kubernetes auth进行验证的Quarkus服务。并获得名为foo的secret。

添加Vault和Kubernetes扩展：

```
./mvnw quarkus:add-extension -Dextensions="quarkus-vault, quarkus-kubernetes"
```

```
@ConfigProperty(name = "foo")
```

```
String foo;

@GET
@Produces(MediaType.TEXT_PLAIN)
public String hello() {
    return foo;
}
```

配置应用程序使用Kubernetes Vault auth方法和Kubernetes扩展来生成正确的部署文件：

```
quarkus.vault.url=http://vault:8200 ❶
quarkus.vault.kv-secret-engine-version=2
quarkus.vault.secret-config-kv-path=myapps/vault-service/config

quarkus.vault.authentication.kubernetes.role=example ❷
kubernetes.service-account=vault ❸

kubernetes.group=quarkus ❹
kubernetes.name=greeting-app
kubernetes.version=latest
```

❶配置Vault位置和secret

❷设置用户（在上一步创建的）要使用的example角色

❸设置要在生成的部署文件中设置的serviceaccount名称

❹设置Docker镜像的组名

注意，没有设置quarkus.vault.authentication.kubernetes.jwt-token-path属性。原因是默认值

（/var/run/secrets/kubernetes.io/serviceaccount/token）与默认值完全一致。如果密钥被挂载在不同的路径上，那么这个属性应该被设置到新的位置。

要部署应用程序，请打开一个新的终端窗口，打包应用程序，创建Docker容器，并应用生成的Kubernetes资源：

```
./mvnw clean package -DskipTests

docker build -f src/main/docker/Dockerfile.jvm \
  -t quarkus/greeting-started-vault-kubernetes-auth:1.0-SNAPSHOT .
kubectl apply -f target/kubernetes/kubernetes.yml

kubectl patch svc greeting-app --type='json' \
  -p '[{"op":"replace","path":"/spec/type","value":"NodePort"}]'
curl $(minikube service greeting-app --url)/hello
```

当部署pod时，应用会通过Vault进行认证，Vault会使用Kubernetes API验证令牌是否有效。应用程序经过认证后就可以从配置的路径中获取secret。

这个例子和之前的例子最大的不同是，在这种情况下，没有设置像Vault密码这样的secret，这意味着可以安全地访问secret，而且不需要添加任何新的secret。

讨论

我们的目的并不是为了展示如何在Kubernetes中为生产目的部署Vault。出于这个原因，我们提供了一个部署文件来部署Vault服务，以满足运行这个例子的最低要求。

这个部署文件位于*src/main/kubernetes/vault-dev-deployment.yaml*，并提供了以下元素：

- dev模式和root令牌设置为root的Vault。
- 在端口8200暴露Vault。
- 给vault设置名为vault的ServiceAccount。
- 给vault执行ClusterRoleBinding和ClusterRole操作。
- 所有资源部署在名为default的命名空间下。

通过运行以下命令部署Vault服务：

```
kubectl apply -f src/main/kubernetes/vault-dev-deployment.yaml

kubectl get pods ❶
NAME      READY   STATUS    RESTARTS   AGE
vault-0   1/1     Running   0          44s

kubectl get secrets
NAME                  TYPE                                  DATA   AGE
default-token-zdw8r   kubernetes.io/service-account-token   3      2d
greeting-security     Opaque                                1      3h9m
vault-token-mm5qx     kubernetes.io/service-account-token   3      8s ❷
```

❶pod被命名为vault-0

❷vault-token被设置为secret

要配置Vault，你需要在本地计算机上安装Vault CLI。Vault CLI是一个单一的文件，可以从Vault（https://oreil.ly/fTBOx）下载，并设置在你的PATH变量中。

假设你已经在本地安装了Vault客户端，并且它在你的PATH变量中可用，则可以配置Vault。

另见

要了解关于Vault Kubernetes auth方法的更多信息，请访问以下网站：

- Vault:Kubernetes Auth Method（https://oreil.ly/AV3D_）
- Vault Agent with Kubernetes（https://oreil.ly/WGbUx）

第13章　Quarkus REST客户端

在第3章中，你学习了如何开发RESTful服务，但在本章中，你将学习RESTful网络服务之间的通信。

使用任何基于服务的架构都不可避免地需要你与外部服务进行通信。这些服务可能是内部服务（你控制服务的生命周期，它们通常部署在同一个集群中）也可能是外部服务（第三方服务）。

如果这些服务是以RESTful Web服务的形式实现的，那么你需要一个客户端来与这些服务进行交互。Quarkus提供了两种方式来实现这一点；JAX-RS Web Client，这是标准的Java EE与RESTful服务通信的方式；MicroProfile REST Client，这是与RESTful服务通信的新方式。

本章涵盖以下主题：

- 使用JAX-RS Web客户端与其他RESTful服务进行通信
- 使用MicroProfile Rest Client与其他RESTful服务进行通信
- 确保RESTful服务之间的通信

13.1　使用JAX-RS Web客户端

问题

如何与另一个RESTful Web服务进行通信？

解决方案

使用JAX-RS Web客户端与其他RESTful Web服务进行通信。

让我们看看如何使用JAX-RS规范与其他RESTful服务进行通信。

我们要连接的外部服务World Clock API（https://oreil.ly/wl2IE），按时区返回当前日期或时间。你需要获取API（https://oreil.ly/7MOtf）所暴露的当前日期或时间。

你需要为使用REST客户端和JAX-B/Jackson添加扩展，用于编组或解组JSON和Java对象：

```
./mvnw quarkus:add-extension -Dextensions="resteasy-jsonb, rest-client"
```

如果从空目录创建，需运行以下命令：

```
mvn io.quarkus:quarkus-maven-plugin:1.4.1.Final:create \
    -DprojectGroupId=org.acme.quickstart \
    -DprojectArtifactId=clock-app \
    -DclassName="org.acme.quickstart.WorldClockResource" \
    -Dextensions="resteasy-jsonb, rest-client"
    -Dpath="/now"
```

你可以开始使用JAX-RS REST客户端与外部Rest API进行通信。让我们看看与世界时钟服务的交互是怎样的。

打开org.acme.quickstart.WorldClockResource.java并添加以下代码：

```
package org.acme.quickstart;

import javax.ws.rs.GET;
import javax.ws.rs.Path;
import javax.ws.rs.PathParam;
import javax.ws.rs.Produces;
import javax.ws.rs.client.Client;
import javax.ws.rs.client.ClientBuilder;
import javax.ws.rs.core.MediaType;
import javax.ws.rs.core.Response;

import org.eclipse.microprofile.config.inject.ConfigProperty;
import org.eclipse.microprofile.rest.client.inject.RestClient;

@Path("/now")
public class WorldClockResource {

  @ConfigProperty(name = "clock.host",
  defaultValue = "http://worldclockapi.com")
    String clockHost; ❶

  private Client client = ClientBuilder.newClient(); ❷

  @GET
  @Path("{timezone}")
  @Produces(MediaType.APPLICATION_JSON)
  public WorldClock getCurrentTime(@PathParam("timezone") String timezone) {
```

```
    WorldClock worldClock = client.target(clockHost) ❸
      .path("api/json/{timezone}/now") ❹
      .resolveTemplate("timezone", timezone) ❺
      .request(MediaType.APPLICATION_JSON)
      .get(WorldClock.class); ❻ ❼

    return worldClock;
  }
}
```

❶使服务主机可配置

❷创建一个新的REST客户端

❸设置主机

❹设置服务的路径

❺将timezone占位符解析为所提供的占位符

❻执行一个GET HTTP方法

❼将JSON输出转换为提供的POJO

打开一个新的终端窗口，启动Quarkus应用程序，然后尝试向GET方法发送请求：

```
./mvnw clean compile quarkus:dev

curl localhost:8080/now/cet
{"currentDateTime":"2019-11-13T13:29+01:00","dayOfTheWeek":"Wednesday"}%
```

讨论

同样，你也可以对其他HTTP方法进行请求。例如，要发送一个POST请求，可以调用post方法：

```
target(host)
    .request(MediaType.APPLICATION_JSON)
    .post(entity);
```

你也可以使用javax.ws.rs.core.Response来获取所有的响应细节，而不仅仅是主体：

```
@GET
@Path("{timezone}/raw")
@Produces(MediaType.APPLICATION_JSON)
public Response getCurrentTimeResponse(@PathParam("timezone")
    String timezone) {
  javax.ws.rs.core.Response responseWorldClock = client.target(clockHost)
    .path("api/json/{timezone}/now")
    .resolveTemplate("timezone", timezone)
    .request(MediaType.APPLICATION_JSON)
    .get(Response.class);


  System.out.println(responseWorldClock.getStatus());
  System.out.println(responseWorldClock.getStringHeaders());
  // ... more methods

  return responseWorldClock;
}
```

另见

你可以在Oracle网站上的以下页面进一步探索JAX-RS REST客户端。

· 使用JAX-RS客户端API访问REST资源（https://oreil.ly/7neQm）

· 在JAX-RS示例应用程序中使用客户端API（https://oreil.ly/QA8lA）

13.2 使用MicroProfile REST客户端

问题

如何与另一个RESTful Web服务进行通信，而不需要了解低级别的细节？

解决方案

使用MicroProfile REST客户端与其他RESTful Web服务进行通信。

到目前为止，你已经看到了如何使用JAX-RS Web客户端与其他REST API进行通信，但它不是类型安全的，你需要处理低级参数，而不是专注于消息通信。

MicroProfile REST客户端提供了一种类型安全的方法，通过HTTP调用RESTful服务，尽可能多地使用JAX-RS 2.0规范。REST客户端被定义为一个Java接口，使其类型安全，并使用JAX-RS注解提供网络配置。

我们在这里也将使用上一节中使用的世界时钟API。记住要获取当前的日期或时间（https://oreil.ly/7MOtf）。

创建org.acme.quickstart.WorldClockService接口，该接口负责与外部服务进行交互：

```
package org.acme.quickstart;

import javax.enterprise.context.ApplicationScoped;
import javax.ws.rs.GET;
import javax.ws.rs.Path;
import javax.ws.rs.PathParam;
import javax.ws.rs.Produces;
import javax.ws.rs.core.MediaType;

import org.eclipse.microprofile.rest.client.inject.RegisterRestClient;
```

```
@Path("/api") ❶
@ApplicationScoped
@RegisterRestClient ❷
public interface WorldClockService {

    @GET ❸
    @Path("/json/{timezone}/now") ❹
    @Produces(MediaType.APPLICATION_JSON) ❺
    WorldClock getNow(@PathParam("timezone") String timezone); ❻

}
```

❶全局路径

❷将接口设置为REST客户端

❸该请求使用GET HTTP方法

❹带path参数的子路径

❺请求的媒体类型

❻path参数通过传入的参数解析

打开org.acme.quickstart.WorldClockResource.java并添加以下代码：

```
@RestClient ❶
WorldClockService worldClockService;

@GET
@Path("{timezone}/mp")
@Produces(MediaType.APPLICATION_JSON)
public WorldClock getCurrentTimeMp(@PathParam("timezone") String timezone) {
  return worldClockService.getNow(timezone); ❷
}
```

❶注入REST客户端

❷调用外部服务

你仍然需要设置外部服务的主机。MicroProfile REST客户端有一个配置属性来设置它。

打开*application.properties*：

```
org.acme.quickstart.WorldClockService/mp-rest/url=http://worldclockapi.com
```

属性名使用以下格式。*fully_qualified_name_rest_client*/mp-rest/url，值是主机名（或URL的根）：

```
./mvnw clean compile quarkus:dev
```

```
curl localhost:8080/now/cet/mp
```

```
{"currentDateTime":"2019-11-13T16:46+01:00","dayOfTheWeek":"Wednesday"}%
```

讨论

你也可以通过实现org.eclipse.microprofile.rest.client.ex.ResponseExceptionMapper接口将状态码等于或大于400的响应转换为异常。如果注册了多个映射器，那么你需要用javax.annotation.Priority注解设置一个优先级。

创建下面的ResponseExecptionMapper类来注册它，并让应用程序在400秒内抛出IOExecptions的状态码：

```
package org.acme.quickstart;

import java.io.IOException;
import javax.ws.rs.core.MultivaluedMap;
import javax.ws.rs.core.Response;
import org.eclipse.microprofile.rest.client.ext.ResponseExceptionMapper;

public class CustomResponseExceptionMapper
             implements ResponseExceptionMapper<IOException> { ❶

    @Override
    public IOException toThrowable(Response response) { ❷
        return new IOException();
    }

    @Override
    public boolean handles(int status,
                           MultivaluedMap<String, Object> headers) { ❸
        return status >= 400 && status < 500;
    }

}
```

❶实现映射器接口

❷是否转换为异常情况

❸默认情况下是转换任何状态码≥400的响应，但你可以覆盖该方法以提供更小的范围

ResponseExceptionMapper是专门从MicroProfile REST客户端规范中获得的扩展点，但你也可以使用JAX-RS规范提供的以下扩展模型。

ClientRequestFilter

向外部服务发出请求时调用的过滤器。

ClientResponseFilter

从外部服务收到响应时调用的过滤器。

MessageBodyReader

调用后读取实体。

MessageBodyWriter

在支持主体的操作中写入请求主体。

ParamConverter

将资源中的参数转换为请求或响应中使用的格式。

ReadInterceptor

当接收到来自外部服务的响应时，监听器被启动。

WriteInterceptor

当外部服务发送请求时，监听器被启动。

你也可以通过使用@InjectMock和@RestClient来模拟WorldClockService接口：

```
@InjectMock
@RestClient
WorldClockService worldClockService;
```

另见

MicroProfile Rest Client规范可以在以下网站找到：

- Eclipse REST Client for MicroProfile（https://oreil.ly/7D0Zv）

13.3　实现CRUD客户端

问题

如何与另一个暴露了CRUD操作的RESTful Web服务进行通信？

解决方案

使用MicroProfile REST客户端和JAX-RS注解来实现一个CRUD客户端。

到目前为止，你已经看到了如何使用MicroProfile REST客户端从外部服务获取信息。当服务是一个内部服务时，你需要实现更多的操作，比如插入、删除或更新。

为了实现这些操作，你可以在MicroProfile REST客户端上使用JAX-RS注解。让我们来看一个例子：

```
package org.acme.quickstart;

import java.util.List;
```

```
import javax.ws.rs.BeanParam;
import javax.ws.rs.Consumes;
import javax.ws.rs.CookieParam;
import javax.ws.rs.DELETE;
import javax.ws.rs.GET;
import javax.ws.rs.HEAD;
import javax.ws.rs.HeaderParam;
import javax.ws.rs.POST;
import javax.ws.rs.PUT;
import javax.ws.rs.Path;
import javax.ws.rs.PathParam;
import javax.ws.rs.Produces;
import javax.ws.rs.core.Response;

import org.eclipse.microprofile.rest.client.inject.RegisterRestClient;

@Path("/developer")
@RegisterRestClient
@Consumes("application/json") ❶
@Produces("application/json")
public interface DeveloperService {


  @HEAD ❷
  Response head();

  @GET
  List<Developer> getDevelopers();


  @POST ❸
  Response createDeveloper(
      @HeaderParam("Authorization") String authorization, ❹
      Developer developer); ❺

  @DELETE ❻
  @Path("/{userId}")
  Response deleteUser(@CookieParam("AuthToken") String authorization, ❼
      @PathParam("developerId") Long developerId);
}
```

❶请求和响应是JSON格式的

❷使用HEAD HTTP方法

❸使用POST HTTP方法

❹设置Authorization头

❺开发者的内容作为正文发送

❻使用DELETE HTTP方法

❼设置AuthToken cookie

讨论

请注意JAX-RS注解是怎样用于配置如何将请求发送到其他服务的。你永远不需要在程序上做任何事情。

这种方法对开发人员很友好，有助于减少你可能最终使用JAX-RS Web客户端的模板代码。

当然，它也有一些缺点。例如，由于路径参数、要设置的头和cookie的数量，方法可能包含大量的参数。为了解决这个问题，可以传递一个POJO，其中包含所有需要的字段（而不是在方法中设置它们）。

让我们创建一个Java类来满足PUT的要求（即一个授权头和一个路径参数）：

```
package org.acme.quickstart;

import javax.ws.rs.HeaderParam;
import javax.ws.rs.PathParam;

public class PutDeveloper {

    @HeaderParam("Authorization") ❶
    private String authorization;

    @PathParam("developerId") ❷
    private String developerId;

    public String getAuthorization() {
        return authorization;
    }

    public void setAuthorization(String authorization) {
        this.authorization = authorization;
    }

    public String getDeveloperId() {
        return developerId;
    }

    public void setDeveloperId(String developerId) {
        this.developerId = developerId;
    }

}
```

❶设置头

❷设置解析参数path

使用前一个类的interface方法是这样的：

```
@PUT
@Path("/{developerId}")
Response updateUser(@BeanParam PutDeveloper putDeveloper, ❶
    Developer developer);
```

❶BeanParam用来表示这个类是一个参数聚合器。

13.4　操作header

问题

如何操作和传播从传入请求到传出服务的header信息（服务到服务认证）？

解决方案

使用MicroProfile REST客户端功能。

当你需要与其他RESTful Web服务进行通信时，你可能希望从传入的请求中传递一些header信息到传出服务或下游服务。其中一个典型的情况就是Authorization header来做服务到服务的认证。服务架构中的认证和授权通常是通过传播一个令牌（通常是一个JWT令牌），通过组成应用程序的所有服务来解决。你可以在图13-1中看到这个想法。

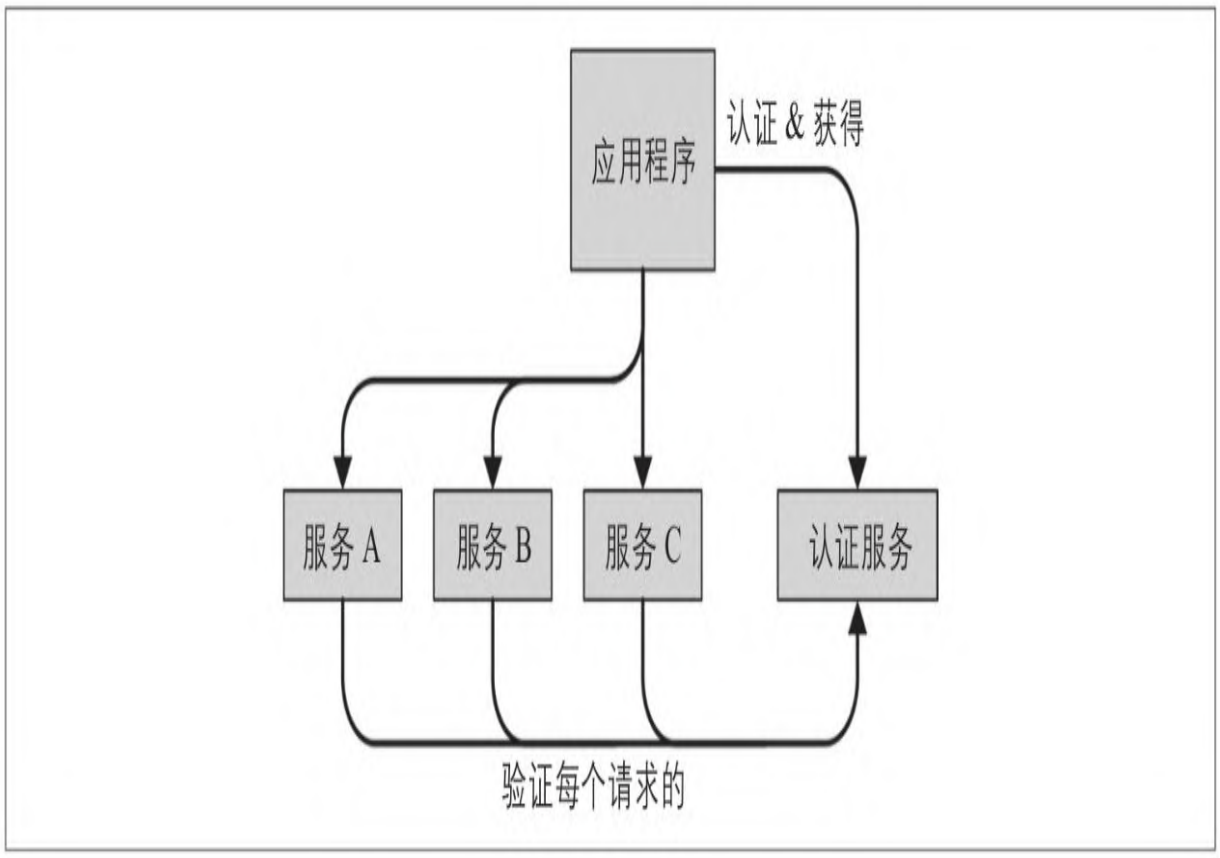


图13-1：服务到服务认证

MicroProfile REST客户端简化了所有这些操作，因为它允许你在静态层面使用注解或在程序层面实现ClientHeadersFactory接口来支持和操作header。

要在一个方法或接口中定义的所有方法上设置一个header，你可以在方法级或类级使用org.eclipse.microprofile.rest.client.annotation.ClientHeaderParam注解来设置一个有静态值的header：

```
@Path("/somePath")
@ClientHeaderParam(name="user-agent", value="curl/7.54.0") ❶
Response get();
```

❶将user_agent设置为请求

该value可以是一个方法调用，其中返回值将是header的值：

```
@ClientHeaderParam(name="user-agent", value="{determineHeaderValue}") ❶
Response otherGet();

default String determineHeaderValue(String headerName) { ❷
    return "Hi-" + headerName;
}
```

❶设置要调用的方法

❷header的名称是方法的第一个参数

这些方法提供了对header的基本操作，但无助于将header从传入请求传播到传出服务。也可以通过实现ClientHeadersFactory接口并使用RegisterClientHeaders注解来添加或传播header信息。

假设你的服务从你的上游服务接收到了名为x-auth的头文件中的认证令牌，而你的下游服务要求将这个值设置为Authorization header。让我们在MicroProfile REST客户端中实现这个header的重命名：

```
package org.acme.quickstart;

import java.util.List;

import javax.ws.rs.core.MultivaluedHashMap;
import javax.ws.rs.core.MultivaluedMap;

import org.eclipse.microprofile.rest.client.ext.ClientHeadersFactory;

public class CustomClientHeadersFactory implements ClientHeadersFactory {

  @Override
  public MultivaluedMap<String, String> update(
      MultivaluedMap<String, String> incomingHeaders, ❶
      MultivaluedMap<String, String> clientOutgoingHeaders) { ❷

    final MultivaluedMap<String, String> headers =
      new MultivaluedHashMap<String, String>(incomingHeaders);
    headers.putAll(clientOutgoingHeaders); ❸

    final List<String> auth = headers.get("x-auth"); ❹
    headers.put("Authorization", auth);
    headers.remove("x-auth");

    return headers;
  }
}
```

❶来自内联JAX-RS请求的header信息

❷在客户端接口上指定的header参数

❸添加所有header

❹重命名header值

最后，你需要通过使用RegisterClient Headers注解在客户端注册这个工厂函数：

```
@RegisterClientHeaders(CustomClientHeadersFactory.class) ❶
public interface ConfigureHeaderServices {
```

❶向这个客户端注册header工厂

讨论

如果你想只传播header，而不做任何修改，可以在不指定任何工厂的情况下，用@Regis-terClientHeaders来注解REST客户端。然后使用默认的header工厂。该默认工厂将把指定的header信息从入站JAX-RS请求传播到出站请求。要配置哪些header会被传播，需要在org.eclipse.microprofile.rest.client.propagateHeaders属性下将它们设置为逗号分隔的值：

```
org.eclipse.microprofile.rest.client.propagateHeaders=Authorization,\
                                                      MyCustomHeader
```

13.5 使用REST客户端处理多部分消息

问题

如何发送多部分内容来与需要它的REST API进行交互？

解决方案

使用RESTEasy多部分支持来处理多部分消息。

有时你需要连接的服务要求你发送多个内容体嵌入到一个消息中，通常使用multipart/form-data MIME类型实现。处理多部分MIME类型的最简单方法是使用RESTEasy multipart provider，它与MicroProfile REST客户端集成。

这个功能是RESTEasy/Quarkus所特有的，并不属于MicroProfile REST客户端规范。

在开始开发之前，你需要在你的构建工具中添加resteasy-multipart-provider依赖项：

```
<dependency>
    <groupId>org.jboss.resteasy</groupId>
    <artifactId>resteasy-multipart-provider</artifactId>
</dependency>
```

然后你需要创建定义消息有效载荷的模型对象。让我们定义一个多部分消息，它有两个部分：一个是二进制内容，另一个是字符串：

```
package org.acme.quickstart;

import java.io.InputStream;

import javax.ws.rs.FormParam;
import javax.ws.rs.core.MediaType;

import org.jboss.resteasy.annotations.providers.multipart.PartType;

public class MultipartDeveloperModel {

    @FormParam("avatar") ❶
    @PartType(MediaType.APPLICATION_OCTET_STREAM) ❷
    public InputStream file;

    @FormParam("name")
    @PartType(MediaType.TEXT_PLAIN)
    public String developerName;

}
```

❶JAX-RS注解用于定义请求中包含的表单参数

❷RESTEasy注解来定义零件的内容类型

最后，你需要使用MultipartDeveloperModel对象来声明一个新的方法，该方法作为一个参数，用org.jboss.resteasy.annotations.providers.multipart.MultipartForm进行注解：

```
@POST
@Consumes(MediaType.MULTIPART_FORM_DATA) ❶
@Produces(MediaType.TEXT_PLAIN)
String sendMultipartData(@MultipartForm ❷
                         MultipartDeveloperModel data); ❸
```

❶将输出的Mime类型设置为多部分

❷将参数定义为multipart/form-type Mime类型

❸多部分数据

13.6　使用REST客户端配置SSL

问题

如何配置REST客户端来使用SSL？

解决方案

MicroProfile REST客户端提供了一种配置SSL与其他服务进行通信的方法。

默认情况下，当使用HTTPS连接时，MicroProfile REST客户端使用JVM信任存储来验证证书。但有时，特别是在跨服务的情况下，证书不能用JVM信任存储来验证，你需要提供你的自定义信任存储。

MicroProfile REST客户端接受通过使用trustStore配置属性来设置自定义信任存储：

```
org.acme.quickstart.FruityViceService/mp-rest/trustStore= \
    classpath:/custom-truststore.jks ❶
org.acme.quickstart.FruityViceService/mp-rest/trustStorePassword=acme ❷
org.acme.quickstart.FruityViceService/mp-rest/trustStoreType=JKS ❸
```

❶trustStore设置信任存储的位置，可以是类路径资源（classpath:）或文件（file:）

❷trustStorePassword设置信任存储的密码

❸trustStoreType设置信任存储的类型

还提供了密钥存储，这在双向SSL连接中非常有用。

MicroProfile REST客户端接受通过使用keyStore配置属性来设置自定义密钥存储。

```
org.acme.quickstart.FruityViceService/mp-rest/keyStore= \
    classpath:/custom-keystore.jks ❶
org.acme.quickstart.FruityViceService/mp-rest/keyStorePassword=acme ❷
org.acme.quickstart.FruityViceService/mp-rest/keyStoreType=JKS ❸
```

❶keyStore设置密钥存储位置，可以是类路径资源（classpath:）或文件（file:）

❷keyStorePassword设置信任存储的密码

❸keyStoreType设置密钥存储的类型

最后，你可以实现javax.net.ssl.HostnameVerifier来覆盖URL的主机名和服务器的标识主机名不匹配时的行为。这个接口的实现可以决定是否应该允许这个连接。

下面是hostname verifier（主机名验证器）的一个例子：

```
package org.acme.quickstart;

import javax.net.ssl.HostnameVerifier;
import javax.net.ssl.SSLSession;

public class FruityHostnameVerifier implements HostnameVerifier {

    @Override
    public boolean verify(String hostname, SSLSession session) {
```

```
        if ("fruityvice.com".equals(hostname)) {
            return true;
        }

        return false;
    }

}
```

你需要在配置文件中启用它：

```
org.acme.quickstart.FruityViceService/mp-rest/hostnameVerifier=\
org.acme.quickstart.FruityHostnameVerifier
```

讨论

大多数时候，当你在本地运行测试时，可能没有安装连接到外部服务所需的所有信任存储或密钥存储。在这些情况下，你可能会针对服务的HTTP版本运行测试。然而，这并不总是可行的，在一些第三方服务中，只有HTTPS协议被启用。

这个问题的一个可能的解决方案是配置MicroProfile REST客户端来信任任何证书。为此，你需要配置客户端并提供一个自定义的信任管理器：

```
package org.acme.quickstart;

import java.net.Socket;
import java.net.URI;
import java.security.KeyManagementException;
import java.security.NoSuchAlgorithmException;
import java.security.cert.CertificateException;
import java.security.cert.X509Certificate;

import javax.enterprise.context.ApplicationScoped;

import javax.net.ssl.HttpsURLConnection;
import javax.net.ssl.SSLContext;
import javax.net.ssl.SSLEngine;
import javax.net.ssl.TrustManager;
import javax.net.ssl.X509ExtendedTrustManager;

import org.apache.http.conn.ssl.NoopHostnameVerifier;
import org.eclipse.microprofile.rest.client.RestClientBuilder;

@ApplicationScoped ❶
public class TrustAllFruityViceService {

  public FruityVice getFruitByName(String name) {
    FruityViceService fruityViceService = RestClientBuilder.newBuilder()
      .baseUri(URI.create("https://www.fruityvice.com/"))
      .hostnameVerifier(NoopHostnameVerifier.INSTANCE) ❷
      .sslContext(trustEverything()) ❸
      .build(FruityViceService.class);
```

```
  return fruityViceService.getFruitByName(name);
}

private static SSLContext trustEverything() { ❹

  try {
    SSLContext sc = SSLContext.getInstance("SSL");
    sc.init(null, trustAllCerts(), new java.security.SecureRandom());
    HttpsURLConnection.setDefaultSSLSocketFactory(sc.getSocketFactory());
    return sc;
  } catch (KeyManagementException | NoSuchAlgorithmException e) {
    throw new IllegalStateException(e);
  }
}

private static TrustManager[] trustAllCerts() {
  return  new TrustManager[]{
    new X509ExtendedTrustManager(){

      @Override
      public X509Certificate[] getAcceptedIssuers() {
        return null;
      }

      @Override
      public void checkServerTrusted(X509Certificate[] chain,
                                     String authType)
        throws CertificateException {
      }

      @Override
      public void checkClientTrusted(X509Certificate[] chain,
                                     String authType)

        throws CertificateException {
      }

      @Override
      public void checkServerTrusted(X509Certificate[] chain,
                                     String authType,
                                     SSLEngine sslEngine)
        throws CertificateException {
      }

      @Override
      public void checkServerTrusted(X509Certificate[] chain,
                                     String authType,
                                     Socket socket)
        throws CertificateException {
      }

      @Override
      public void checkClientTrusted(X509Certificate[] chain,
                                     String authType,
                                     SSLEngine sslEngine)
```

```
            throws CertificateException {
        }

        @Override
        public void checkServerTrusted(X509Certificate[] chain,
                                       String authType,
                                       Socket socket)
            throws CertificateException {

        }
      }
    };
  }
}
```

❶创建一个CDI bean。你需要使用@Inject代替@RestClient

❷禁用主机验证

❸不需要做任何验证就可以信任所有证书

❹自定义SSLContext与一个空的信任管理器，有效地否定所有的SSL检查

如果你注入这个实例而不是生产实例，那么每个HTTPS请求都是有效的，与外部服务使用的证书无关。

第14章　使用Spring API开发Quarkus应用程序

到目前为止，你已经看到每个例子都是使用CDI注解开发的，如@Inject、@Produces、JAX-RS注解或Java EE Security注解。但是Quarkus也为一些最常用的Spring库提供了一个兼容层，以便你可以使用所有关于Spring框架的知识来开发Quarkus应用程序。

本章涵盖以下主题：

- Spring依赖注入
- Spring REST Web
- Spring Data JPA
- Spring Security
- Spring Boot配置

14.1　使用Spring依赖注入

问题

如何使用Spring依赖注入（Dependency Injection，DI）API来开发Quarkus？

解决方案

Quarkus提供了一个API兼容层（使用扩展）来使用Spring DI注解。

虽然我们鼓励你使用CDI注解，但你也可以自由地使用Spring注解，因为最终的应用程序将以完全相同的方式运行。

要添加Spring DI扩展，请运行以下命令：

```
./mvnw quarkus:add-extension -Dextensions="spring-di"
```

你也可以通过运行以下命令来创建一个带有Spring DI扩展的项目：

```
mvn io.quarkus:quarkus-maven-plugin:1.4.1.Final:create \
    -DprojectGroupId=org.acme.quickstart \
    -DprojectArtifactId=spring-di-quickstart \
    -DclassName="org.acme.quickstart.GreeterResource" \
    -Dpath="/greeting" \
    -Dextensions="spring-di"
```

打开*application.properties*文件，添加一个新属性：

```
greetings.message=Hello World
```

要将这个配置值注入任何类中，需使用@org.springframework.beans.factory.annotation.Value Spring注解：

```
package org.acme.quickstart;

import java.util.function.Function;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration ❶
public class AppConfiguration {

    @Bean(name = "suffix") ❷
    public Function<String, String> exclamation() {
        return new Function<String, String>() { ❸
            @Override
            public String apply(String t) {
                return t + "!!";
            }
        };
    }
}
```

❶使用@Configuration注解将该类定义为配置对象

❷创建一个为给定消息添加后缀的bean

❸实现该服务

需要注意的是，在这两种情况都使用了Spring注解。

可以使用

@org.springframework.beans.factory.annotation.Autowired和

@org.springframe-work.beans.factory.annotation.Qualifier注解来注入bean：

```
package org.acme.quickstart;

import org.springframework.stereotype.Service;

@Service ❶
public class PrefixService {

    public String appendPrefix(String message) {
        return "- " + message;
    }

}
```

❶将该类设置为服务

可以用构造函数代替@Autowired来注入：

```
private PrefixService prefixService;

public GreetingResource(PrefixService prefixService) { ❶
    this.prefixService = prefixService;
}
```

❶使用构造函数注入实例。注意@Autowired不是必需的。

所有这些操作都可以组合成下面的输出：

```
@GET
@Produces(MediaType.TEXT_PLAIN)
public String hello() {
    String prefixed = prefixService.appendPrefix(message);
    return this.suffixComponent.apply(prefixed);
}
```

如果你运行这个项目，将会看到所有的对象都被正确地创建和注入，即使使用了Spring DI注解：

```
./mvnw compile quarkus:dev

curl http://localhost:8080/greeting

- Hello World!!
```

讨论

需要注意的是，Quarkus不会启动Spring Application Context实例。这是因为它的集成只在API级别（注解、返回类型等），这意味着以下几点：

· 使用其他Spring库不会有任何影响。你将在后面看到，Quarkus提供了对其他Spring库的集成。

· org.springframework.beans.factory.config.BeanPostProcessor将不会被执行。

表14-1显示了MicroProfile/CDI和Spring之间的等价注解。

表14-1：MicroProfile/CDI和Spring之间的等价注解

Spring	CDI / MicroProfile
@Autowired	@Injecct
@Qualifier	@Named
@Value	@ConfigProperty. 支持典型用例的表达式语言
@Component	@Singleton
@Service	@Singleton
@Repository	@Singleton
@Configuration	@ApplicationScoped
@Bean	@Produces

14.2 使用Spring Web

问题

如何使用Spring Web API来开发Quarkus？

解决方案

Quarkus提供了一个API兼容层（使用扩展）来使用Spring Web注解/类。

要添加Spring Web扩展，需运行以下命令：

```
./mvnw quarkus:add-extension -Dextensions="spring-web"
```

只使用Spring Web注解创建一个新资源：

```
package org.acme.quickstart;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;

import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController ❶
@RequestMapping("/greeting") ❷
```

```
public class SpringController {

  @GetMapping ❸
  public ResponseEntity<String> getMessage() { ❹
    return ResponseEntity.ok("Hello");
  }

  @GetMapping("/{name}")
  public String hello(@PathVariable(name = "name") String name) { ❺
    return "hello " + name;
  }
}
```

❶REST资源定义

❷匹配根路径

❸设置GET HTTP方法

❹返回Spring的ResponseEntity

❺获取路径信息

只有Spring注解和类被用来实现资源。没有JAX-RS API的痕迹。

讨论

你已经看到了如何在Quarkus应用程序中使用Spring依赖注入注解。Quarkus通过一个扩展与Spring Web进行集成。

虽然我们鼓励你使用JAX_RS注解，但你也可以自由使用Spring Web类和注解。无论使用哪种方式，最终的应用都将以完全相同的方式运行。

Quarkus只支持Spring Web的REST相关特性。总而言之，除了与通用的@Controller相关的功能，所有的@RestController功能都被支持。

支持以下Spring Web注解：@RestController、@RequestMapping、@GetMapping、@Post-Mapping、@PutMapping、@DeleteMapping、@PatchMapping、@RequestParam、@RequestHeader、@MatrixVariable、@PathVariable、@CookieValue、@RequestBody、@ResponseStatus、@ExceptionHandler（仅在@RestControllerAdvice类中使用）和@RestControllerAdvice（仅支持@ExceptionHandler功能）。

以下是REST控制器开箱即用支持的返回类型：基元、String（作为文字，而不是作为MVC支持）、POJO和

org.springframework.http.ResponseEntity。

以下是REST控制器开箱即用的方法参数：基元、String、POJO、javax.servlet.http.HttpServletRequest和javax.servlet.http.HttpServletResponse。

以下是异常处理程序开箱即用的返回类型：org.springframework.http.ResponseEntity和java.util.Map。

以下是异常处理程序开箱即用的方法参数：java.lang.Exception或任何其他子类型、javax.serv let.http.HttpServletRequest和javax.servlet.http.HttpServletResponse。

要使用javax.servlet类，你需要注册quarkus-undertow依赖项。

表14-2显示了JAX-RS和Spring Web之间的等价注解。

表14-2：JAX-RS和Spring Web之间的等价注解

Spring	JAX-RS
@RequestController	
@RequestMapping(path="/api")	@Path("/api")
@RequestMapping(consumes="application/json")	@Consumes("application/json")
@RequestMapping(produces="application/json")	@Produces("application/json")
@RequestParam	@QueryParam
@PathVariable	@PathParam
@RequestBody	
@RestControllerAdvice	
@ResponseStatus	Use javax.ws.rs.core.Response class
@ExceptionHandler	实现 javax.ws.rs.ext.ExceptionMapper 接口

14.3 使用Spring Data JPA

问题

如何使用Spring Data JPA API来开发Quarkus的持久层？

解决方案

Quarkus提供了一个API兼容层（使用扩展）来使用Spring Data JPA类。

要添加Spring Web扩展，需运行以下命令：

```
./mvnw quarkus:add-extension -Dextensions="spring-data-jpa"
```

使用Panache与Spring Data JPA的最大区别在于，你的repository（存储库）类必须实现Spring Data org.springframework.data.repository.CrudRepository类，而不是io.quarkus.hibernate.orm.panache.PanacheRepository。但其余部分（比如在*application.properties*中定义实体或配置数据源）是完全一样的。

创建一个新的类org.acme.quickstart.DeveloperRepository：

```
package org.acme.quickstart;

import java.util.List;

import org.springframework.data.repository.CrudRepository;

public interface DeveloperRepository extends CrudRepository<Developer, Long> { ❶
    List<Developer> findByName(String name); ❷
}
```

❶定义Spring Data JPA CRUO存储库

❷派生的查询方法

运行该项目并发送一些请求，以验证对象是否使用Spring Data接口持久化。为此，请在终端中运行以下内容：

```
./mvnw compile quarkus:dev

curl -d '{"name":"Ada"}' -H "Content-Type: application/json" \
    -X POST http://localhost:8080/developer -v

< HTTP/1.1 201 Created
< Content-Length: 0
< Location: http://localhost:8080/developer/1
<
* Connection #0 to host localhost left intact
```

讨论

在第7章中，你学习了如何使用Quarkus开发持久化代码，特别是使用Panache框架。Quarkus还通过扩展与Spring Data JPA进行集成。

虽然我们鼓励你使用Panache框架，但你也可以自由使用Spring Data JPA类，因为最终的应用程序将以完全相同的方式运行。

Quarkus支持Spring Data JPA的功能子集，基本上这是最常用的功能。

支持使用以下接口来定义一个存储库：

· org.springframework.data.repository.Repository

· org.springframework.data.repository.CrudRepository

· org.springframework.data.repository.CrudRepository

· org.springframework.data.repository.PagingAndSortingRepository

· org.springframework.data.jpa.repository.JpaRepository

更新数据库的方法会自动用@Transactional来注解，如果你使用@org.spri-ngframe work.data.jpa.repository.Query，那么你需要用@org.springframework.data.jpa.repository.Modifying来注解该方法，使其成为事务性的。

在撰写本书时，以下功能是不支持的：

· org.springframework.data.repository.queryByExampleExecutor的方法

· QueryDSL支持

· 为代码库中的所有版本库接口定制基础版本库

· java.util.concurrent.Future或扩展它的类作为repository方法的返回类型

· 使用@Query时的原生和命名查询

这些限制可能会在不久的将来得到修正。

14.4 使用Spring Security

问题

如何使用Spring Security API来保护资源？

解决方案

Quarkus提供了一个API兼容层（使用扩展）来使用Spring Security类。

要添加Spring Security扩展（和身份提供者），需运行以下命令：

```
./mvnw quarkus:add-extension \
    -Dextensions="quarkus-spring-security, quarkus-spring-web, \
                  quarkus-elytron-security-properties-file"
```

在这一点上，代码可以使用Spring Security注解（org.springframework.security.access.annotation.Secured和org.springfra mework.security.access.prepost.PreAuthorize）而

不是Java EE注解（@javax.annotation.security.RolesAllowed）来保护。

```
package org.acme.quickstart;

import org.springframework.security.access.annotation.Secured;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/hello")
public class GreetingController {
    @GetMapping
    @Secured("admin") ❶
    public String helloAdmin() {
        return "hello from admin";
    }

    @PreAuthorize("hasAnyRole('user')") ❷
    @GetMapping
    @RequestMapping("/any")
    public String helloUsers() {
        return "hello from users";
    }
}
```

❶@Secured注解

❷@PreAuthorize注解，增加对表达式的支持

在*application.properties*文件中注册有效的用户和角色，因为Elytron文件属性扩展名被注册为身份提供者：

```
quarkus.security.users.embedded.enabled=true
quarkus.security.users.embedded.plain-text=true
quarkus.security.users.embedded.users.alexandra=aixa
quarkus.security.users.embedded.roles.alexandra=admin,user
quarkus.security.users.embedded.users.ada=dev
quarkus.security.users.embedded.roles.ada=user
```

只有alexandra可以访问两个接口，而ada只能访问用户接口。

讨论

在第11章中，你学习了如何使用Java EE安全注解（@RolwsAllowed）来保护RESTful网络服务。Quarkus通过扩展与Spring Security集成，允许使用Spring Security注解。

需要注意的是，Spring Security的集成发生在API层面，仍然需要像Elytron文件属性这样的身份提供者实现。

Quarkus支持Spring Security @PreAuthorze表达式语言的一个子集，这基本上是最常用的功能的集合：

```
@PreAuthorize("hasRole('admin')")
@PreAuthorize("hasRole(@roles.USER)") ❶

@PreAuthorize("hasAnyRole(@roles.USER, 'view')")

@PreAuthorize("permitAll()")
@PreAuthorize("denyAll()")

@PreAuthorize("isAnonymous()")
@PreAuthorize("isAuthenticated()")
```

❶其中roles是用@Component注解定义的bean，USER是类的公共字段。

也支持条件表达式：

```
@PreAuthorize("#person.name == authentication.principal.username")
public void doSomethingElse(Person person){}

@PreAuthorize("@personChecker.check(#person,
                                    authentication.principal.username)")
public void doSomething(Person person){}

@Component
public class PersonChecker {
    public boolean check(Person person, String username) {
        return person.getName().equals(username);
    }
}

@PreAuthorize("hasAnyRole('user', 'admin') AND #user == principal.username")
public void allowedForUser(String user) {}
```

14.5　使用Spring Boot属性

问题

如何使用Spring Boot将配置属性映射到Java对象中？

解决方案

Quarkus提供了一个API兼容层（使用扩展）来使用Spring Boot配置属性。

Quarkus以扩展的形式与Spring Boot集成在一起。@org.springframework.boot.context.properties.ConfigurationProperties注解可用于将配置属性映射到Java对象中。

要添加Spring Boot扩展（和其他Spring集成），请运行以下命令：

```
./mvnw quarkus:add-extension \
    -Dextensions="quarkus-spring-di, quarkus-spring-web, \
                  quarkus-spring-boot-properties,
                  quarkus-hibernate-validator"
```

添加一些配置属性，绑定到一个Java对象中：

```
greeting.message=Hello World
greeting.configuration.uppercase=true
```

下一步是创建带有getter/setter的POJO，将配置属性从文件绑定到Java对象。需要注意的是，属性uppercase是在一个名为configuration的子类中定义的，这影响了POJO类的创建方式，因为每个子类都必须添加到自己的类中：

```
package org.acme.quickstart;

import javax.validation.constraints.Size;

import org.springframework.boot.context.properties.ConfigurationProperties;

@ConfigurationProperties(prefix = "greeting") ❶
public class GreetingConfiguration {

    @Size(min = 2) ❷
    private String message;
    private Configuration configuration; ❸

    public void setMessage(String message) {
        this.message = message;
    }

    public String getMessage() {
        return message;
    }

    public void setConfiguration(Configuration configuration) {
        this.configuration = configuration;
    }

    public Configuration getConfiguration() {
        return configuration;
    }

}
```

❶用ConfigurationProperties注解父类，并设置配置属性的前缀

❷支持Bean Validation注解

❸子类configuration被映射到一个同名字段中

子类POJO只是一个uppercase属性的Java类：

```
package org.acme.quickstart;

public class Configuration {

    private boolean uppercase;

    public boolean isUppercase() {
        return uppercase;
    }

    public void setUppercase(boolean uppercase) {
        this.uppercase = uppercase;
    }
}
```

配置对象可以被注入任意类中，与其他使用@Inject或@Autowired的bean一样：

```
@Autowired ❶
GreetingConfiguration greetingConfiguration;

@GetMapping
public String hello() {
    if (greetingConfiguration.getConfiguration().isUppercase()) { ❷
        return greetingConfiguration.getMessage().toUpperCase();
    }
    return greetingConfiguration.getMessage();
}
```

❶向配置对象注入与其绑定的数据

❷配置属性会自动填充到Java对象中

第15章　使用响应式编程模型

我们都熟悉几十年来在企业软件开发中占主导地位的客户端-服务器（CS）架构。然而，我们最近在架构风格上发生了转变。除了标准的CS方法，还有消息驱动的应用程序、微服务、响应式应用程序，甚至无服务器应用程序！所有这些类型的应用程序都可以使用Quarkus创建。在下面的内容中，你将学习关于响应式编程模型、消息总线和流的知识。

Quarkus（以及本书）的响应式库使用了SmallRye Mutiny。可以在SmallRye Mutiny（https://oreil.ly/nBP7H）上阅读更多关于Mutiny的信息。还有对RxJava和Reactor的支持，但它们不是首选。要使用它们，需要使用Mutiny的转换器。

15.1　创建异步HTTP端点

问题

如何创建一个异步HTTP端点？

解决方案

Quarkus与Java Streams、Eclipse MicroProfile Reactive规范和SmallRye Mutiny进行了集成，这些集成使支持异步的HTTP接口变得很容易。首先，需要做的是确定想使用哪些库。如果想使用原生的Streams或者MicroProfile Reactive规范，需要添加quarkus-smallrye-reactive-streams-operators扩展。如果想使用SmallRye Mutiny，需要添加quarkus-resteasy-mutiny扩展到你的项目中。

未来，Mutiny将成为Quarkus中所有响应式事务的首选库。

添加扩展后，需要返回一个响应式的HTTP端点：

```
@GET
@PATH("/reactive")
@Produces(MediaType.TEXT_PLAIN) ❶
public CompletionStage<String> reactiveHello() { ❷
    return ReactiveStreams.of("h", "e", "l", "l", "o")
        .map(String::toUpperCase)
        .toList()
        .run()
        .thenApply(list -> list.toString());
}
```

❶当然，任何有效的MediaType都是有效的。为了简单起见，我们使用纯文本

❷CompletionStage来自java.util.concurrent包

对于Mutiny，这个例子变成：

```
@GET
@PATH("/reactive")
@Produces(MediaType.TEXT_PLAIN)
public Multi<String> helloMutiny() {
    return Multi.createFrom().items("h", "e", "l", "l", "o");
}
```

另见

更多信息，请访问以下网站：

- SmallRye Mutiny（https://oreil.ly/nBP7H）
- SmallRye Reactive Streams Operators（https://oreil.ly/Ab8eo）
- Reactive Streams（https://oreil.ly/pgyMk）

15.2　异步流式数据传输

问题

如何以异步方式流式传输数据？

解决方案

非常类似于创建一个异步的HTTP端点，Quarkus允许你使用服务器发送的事件或服务器端事件（SSE）从应用程序中流出事件。在这种情况下，你需要做的是返回一个Publisher，并告诉JAX-RS你的端点产生了MediaType.SERVER_SENT_EVENTS。这里有一个例子，每500毫秒流转一个long类型：

```
@GET
@Path("/integers")
@Produces(MediaType.SERVER_SENT_EVENTS) ❶
public Publisher<Long> longPublisher() { ❷
    return Multi.createFrom()
            .ticks().every(Duration.ofMillis(500));
}
```

❶确保告诉JAX-RS你正在使用SSE

❷该方法的返回类型必须是Reactive Streams库中的org.reactivestream.Publisher

有了Mutiny，一个Multi就是一个Publisher，通过简单地返回一个Multi，让这一切变得更加简单。

另见

欲了解更多信息，请访问以下网站：

· Wikiwand：服务器发送的事件（SSE）（https://oreil.ly/m0cAC）

· MDN Web Docs：使用服务器发送的事件（https://oreil.ly/ZIX4X）

15.3　使用消息传递解耦组件

问题

如果使用消息传递来解耦组件？

解决方案

Vert.x是Quarkus使用的底层或捆绑框架之一。Vert.x是一个用于构建异步、事件驱动、响应式应用的框架，就像Quarkus一样！Quarkus使用Vert.x Event Bus（事件总线）来发送和接收带有解耦类的事件或消息。

要使用Vert.x，需要为应用程序添加适当的扩展。Vert.x扩展的名字是vertx。

处理事件/消息

我们先来看看监听或使用事件。在Quarkus中使用事件的最简单方法是使用io.quar-kus.vertx.ConsumeEvent注解。@ConsumeEvent有一些属性稍后会提到，首先看看它的操作：

```
package com.acme.vertx;

import javax.enterprise.context.ApplicationScoped;

import io.quarkus.vertx.ConsumeEvent;

@ApplicationScoped
public class GreetingService {
    @ConsumeEvent ❶
    public String consumeNormal(String name) { ❷
        return name.toUpperCase();
    }
}
```

❶在没有设置值的情况下，事件的地址是bean的全称。在这种情况下，它应该是com.acme.vertx.GreetingService

❷消费者的参数是消息主体。如果该方法返回任何东西，它被包装成消息响应

发送事件/信息

要发送一个事件，需要与Vert.x事件总线进行交互。可以通过注入@Inject io.vertx.axle.core.eventbus.EventBus bus获得实例。你将主要利用事件总线上的两个方法：

send

发送消息并可选择期待的回复

publish

向每个监听者发布消息

```
bus.send("address", "hello"); ❶
bus.publish("address", "hello"); ❷
bus.send("address", "hello, how are you?") ❸
    .thenAccept(msg -> {
    // do something with the message
});
```

❶向特定的地址发送消息，单个消费者收到后，就忽略回复

❷向某一特定地址发布消息，所有消费者都收到该消息

❸向一个特定的地址发送一条信息，并期望得到回复

你应该有足够的信息来使用Vert.x Eventing创建你自己的Greeting Service!

讨论

你也可以返回一个CompletionStage来以异步方式处理事件。最后，如果你希望使用io.vertx.axle.core.eventbus.Message作为方法参数，可以这样做，并在你的事件处理程序中访问其余的消息。

fire-and-forget风格的交互同样简单——只需从你的方法中返回void即可。

消费事件的方法在Vert.x事件循环中被调用，Vert.x的第一原则是永远不要阻塞事件循环。你的代码必须是非阻塞的，如果需要方法阻塞，请将@ConsumeEvent的阻塞属性设置为true。

要配置事件处理程序的名称或地址，请使用value参数：

```
@ConsumeEvent(value = "greeting")
public String consumeNamed(String name) {
    return name.toUpperCase();
}
```

另见

欲了解更多信息，请访问以下网站：

- Vert.x（https://www.vertx.io）
- Vert.x：事件总线（https://oreil.ly/TAnxk）

15.4　响应Apache Kafka消息

问题

如何对Apache Kafka消息做出响应？

解决方案

Quarkus利用Eclipse MicroProfile Reactive Messaging与Apache Kafka进行交互。

Reactive Messaging规范建立在三个主要概念之上：

1.Message

2.@Incoming

3.@Outgoing

本节主要关注Message和@Incoming，如果你需要其他方式，请查看15.5节。

消息

简而言之，Message是一个围绕有效载荷的包，该包带有可选的元数据。

@Incoming注解

这个注解（org.eclipse.microprofile.reactive.messaging.Incoming）表示该方法使用一个消息流，唯一的属性是流的名称或主题的名称。该方法在处理链的末端都会有这样的注解，也就是所谓的sink。下面是Quarkus内部的两种用法：

```
package org.acme.kafka;

import java.util.concurrent.CompletableFuture;
import java.util.concurrent.CompletionStage;

import javax.enterprise.context.ApplicationScoped;

import org.eclipse.microprofile.reactive.messaging.Incoming;
import org.eclipse.microprofile.reactive.messaging.Message;

@ApplicationScoped
public class CharacterReceiver {
  @Incoming("ascii-char")
  public CompletionStage<Void> processKafkaChar(Message<String> character) {
    return CompletableFuture.runAsync(() -> {
      System.out.println("Received a message from Kafka "
          + "using CompletableFuture: '" + character.getPayload() + "'");
    });
  }

  @Incoming("ascii-char")
  public void processCharacter(String character) {
    System.out.println("Received a String from kafka: '" + character + "'");
  }
}
```

可以看到任何一个方法都可以工作，但是，在processKafkaCharacter的情况下，它接收一个Message并返回一个CompletionStage。如果你的方法接收一个Message作为参数，它必须返回一个CompletionStage。

如果你只对有效载荷感兴趣，则不需要考虑这些，可以简单地接受有效载荷的类型并返回void，就像前面代码中的processCharacter所演示的那样。

配置

正如预期的那样，需要配置你的应用程序与Apache Kafka通信：

```
mp.messaging.incoming.ascii-char.connector=smallrye-kafka
mp.messaging.incoming.ascii-char.value.deserializer=org.apache.kafka.common\
                                                    .serialization\
                                                    .StringDeserializer
mp.messaging.incoming.ascii-char.broadcast=true
```

在前面的代码中，我们有多个订阅者，所以需要使用broadcast=true，broadcast属性让MicroProfile Reactive Messaging（SmallRye是一个实现）知道收到的消息可以派发给多个订阅者。

配置的语法如下：

```
mp.messaging.[outgoing|incoming].{channel-name}.property=value
```

channel-name段的值必须与@Incoming（还有@Outgoing，这将在下一节中介绍）的值相匹配。

在SmallRye Reactive Messaging中可以看到一些合理的默认值：Apache Kafka（https://oreil.ly/L5WHK）。

讨论

要在开发环境中快速启动和运行Apache Kafka，可以访问下面“另见”中列出的网站，或者使用以下*docker-compose.yml*文件和docker-compose：

```
version: '2'

services:

  zookeeper:
    image: strimzi/kafka:0.11.3-kafka-2.1.0
    command: [
      "sh", "-c",
      "bin/zookeeper-server-start.sh config/zookeeper.properties"
    ]
    ports:
      - "2181:2181"
    environment:
      LOG_DIR: /tmp/logs

  kafka:
    image: strimzi/kafka:0.11.3-kafka-2.1.0
    command: [
      "sh", "-c",
      "bin/kafka-server-start.sh config/server.properties
      --override listeners=$${KAFKA_LISTENERS}
      --override advertised.listeners=$${KAFKA_ADVERTISED_LISTENERS}
      --override zookeeper.connect=$${KAFKA_ZOOKEEPER_CONNECT}"
    ]
    depends_on:
      - zookeeper
    ports:
      - "9092:9092"
    environment:
      LOG_DIR: "/tmp/logs"
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
      KAFKA_LISTENERS: PLAINTEXT://0.0.0.0:9092
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
```

另见

更多信息，请访问以下内容：

- GitHub：MicroProfile Reactive Messaging（https://oreil.ly/DSu7u）
- SmallRye Reactive Messaging（https://oreil.ly/QlGHK）
- Apache Kafka（https://oreil.ly/6xsAP）
- Apache Kafka:Consumer Configs（https://oreil.ly/iE8aU）
- Vert.x Kafka客户端（https://oreil.ly/zFD5J）

15.5　向Apache Kafka发送消息

问题

如何向Apache Kafka发送消息？

解决方案

首先，需要将quarkus-smallrye-reactive-messaging-kafka扩展添加到你的项目中。在这个例子中，我们也在使用SmallRye Mutiny，所以也要添加io.smallrye.reactive:mutiny依赖项。

要向Apache Kafka发送消息，需使用Eclipse MicroProfile Reactive Messaging中的@Outgoing注解。

当生成要发送到Apache Kafka的数据时，可以用org.eclipse.microprofile.reactive.messaging.Outgoing来注解你的方法。可以发送消息流或单一消息。如果你希望发布消息流，必须返回一个org.reactivestreams.Publisher或一个org.eclipse.microprofile.reactive.streams.operator.PublisherBuilder。如果你想发布一条消息，必须返回一个org.eclipse.microprofile.reactive.messaging.Message、

java.util.concurrent.CompletionStage，或者你的消息有效载荷的相关类型。

下面是一个基本的例子，它每秒创建一个新的ASCII字符，并将其发送到“letter-out”通道：

```
package org.acme.kafka;

import java.time.Duration;
import java.util.concurrent.ThreadLocalRandom;

import io.smallrye.mutiny.Multi;
```

```
import org.eclipse.microprofile.reactive.messaging.Outgoing;
import org.reactivestreams.Publisher;

public class CharacterGenerator {
    @Outgoing("letter-out")
    public Publisher<String> generate() {
        return Multi.createFrom()
                .ticks().every(Duration.ofSeconds(1))
                .map(tick -> {
                    final int i = ThreadLocalRandom.current().nextInt(95);
                    return String.valueOf((char) (i + 32));
                });
    }
}
```

@Outgoing的值属性是必需的，是出站通道的名称。在这个例子中，我们使用了SmallRye Mutiny，但你也可以使用任何返回org.reactivestreams.Publisher实例的工具。例如，RXJava2的Flowable也可以很好地工作。

以下配置也是必要的：

```
mp.messaging.outgoing.letter-out.connector=smallrye-kafka
mp.messaging.outgoing.letter-out.topic=ascii-char
mp.messaging.outgoing.letter-out.value.serializer=org.apache.kafka.common\
                                                  .serialization\
                                                  .StringSerializer
```

讨论

如果你发现自己需要以命令的方式发送消息，可以将org.eclipse.microprofile.reactive.messaging.Emitter注入应用程序：

```
@Inject @Channel("price-create") Emitter<Double> priceEmitter;

@POST
@Consumes(MediaType.TEXT_PLAIN)
public void addPrice(Double price) {
    priceEmitter.send(price);
}
```

另见

想了解更多信息，请查看以下内容：

- 15.4节
- Apache Kafka:Producer Configs（https://oreil.ly/hZ9Bm）
- SmallRye Reactive Messaging（https://oreil.ly/QlGHK）

15.6　序列化与反序列化

问题

如何将POJO序列化或反序列化到Kafka？

解决方案

Quarkus有能力处理JSON Kafka消息。你需要选择JSONB或Jackson作为实现。所需的扩展是quarkus-resteasy-jsonb或quarkus-resteasy-jackson，这取决于你的偏好。

然后需要创建一个反序列化器，最简单的方法是扩展JSONB的JsonDeserializer或Jackson的ObjectMapperDeserializer。下面是Book类和它的反序列化器：

```
package org.acme.kafka;

public class Book {
    public String title;
    public String author;
    public Long isbn;

    public Book() {
    }

    public Book(String title, String author, Long isbn) {
        this.title = title;
        this.author = author;
        this.isbn = isbn;
    }
}
```

对于JSONB，反序列化器如下所示：

```
package org.acme.kafka;

import io.quarkus.kafka.client.serialization.JsonbDeserializer;

public class BookDeserializer extends JsonbDeserializer<Book> {
    public BookDeserializer() {
        super(Book.class);
    }
}
```

对于Jackson，反序列化器如下所示：

```
package com.acme.kafka;

import io.quarkus.kafka.client.serialization.ObjectMapperDeserializer;

public class BookDeserializer extends ObjectMapperDeserializer<Book> {
```

```
    public BookDeserializer() {
        super(Book.class);
    }
}
```

最后，需要在Quarkus配置中添加你的反序列化器和默认序列化器：

```
# Configure the Kafka source (we read from it)
mp.messaging.incoming.book-in.connector=smallrye-kafka

mp.messaging.incoming.book-in.topic=book-in
mp.messaging.incoming.book-in.value.deserializer=com.acme\
                                                 .kafka.BookDeserializer

# Configure the Kafka sink (we write to it)
mp.messaging.outgoing.book-out.connector=smallrye-kafka
mp.messaging.outgoing.book-out.topic=book-out
mp.messaging.outgoing.book-out.value.serializer=io.quarkus.kafka\
                                                .client.serialization\
                                                .JsonbSerializer
```

对于Jackson来说：

```
# Configure the Kafka source (we read from it)
mp.messaging.incoming.book-in.connector=smallrye-kafka
mp.messaging.incoming.book-in.topic=book-in
mp.messaging.incoming.book-in.value.deserializer=com.acme\
                                                 .kafka.BookDeserializer

# Configure the Kafka sink (we write to it)
mp.messaging.outgoing.book-out.connector=smallrye-kafka
mp.messaging.outgoing.book-out.topic=book-out
mp.messaging.outgoing.book-out.value.serializer=io.quarkus.kafka.client\
                                                .serialization\
                                                .ObjectMapperSerializer
```

讨论

如果你使用JSONB，并且不希望为每一个通过连线发送的POJO创建一个反序列化器，则可以使用通用的io.vertx.kafka.client.serialization.JsonObjectDeserializer，返回的对象将是一个javax.json.JsonObject。这里，我们选择使用默认的序列化器。

如果你需要比基本功能更多的东西，也可以创建自己的序列化器。

15.7 使用Kafka Streams API

问题

如何使用Kafka Streams API来查询数据？

解决方案

Quarkus中的Apache Kafka扩展（quarkus-smallrye-reactive-messagingkafka）与Apache Kafka Streams API进行了集成。这个例子有点深入，需要一些额外的部件。当然，你需要启动并运行一个Apache Kafka实例。如果你还没有可用的Apache Kafka实例，建议你使用Kubernetes设置一个Apache Kafka实例。如果你只是用于开发，可以使用下面的*docker-compose.yml*文件：

```
version: '3.5'

services:
  zookeeper:
    image: strimzi/kafka:0.11.3-kafka-2.1.0
    command: [
      "sh", "-c",
      "bin/zookeeper-server-start.sh config/zookeeper.properties"
    ]
    ports:
      - "2181:2181"
    environment:
      LOG_DIR: /tmp/logs
    networks:
      - kafkastreams-network
  kafka:
    image: strimzi/kafka:0.11.3-kafka-2.1.0
    command: [
      "sh", "-c",
      "bin/kafka-server-start.sh config/server.properties
      --override listeners=$${KAFKA_LISTENERS}
      --override advertised.listeners=$${KAFKA_ADVERTISED_LISTENERS}
      --override zookeeper.connect=$${KAFKA_ZOOKEEPER_CONNECT}
      --override num.partitions=$${KAFKA_NUM_PARTITIONS}"
    ]
    depends_on:
      - zookeeper
    ports:
      - "9092:9092"
    environment:
      LOG_DIR: "/tmp/logs"
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092
      KAFKA_LISTENERS: PLAINTEXT://0.0.0.0:9092
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_NUM_PARTITIONS: 3
    networks:
      - kafkastreams-network
```

这个解决方案的下一步是创建一个生产者，生成值并将这些生成的值发送到Kafka主题。我们将使用jukebox（点唱机）的想法来实现。我们的jukebox将包含一些歌曲和艺术家，以及一首歌被播放的次数。每一个都会被发送到不同的主题，然后由另一个服务汇总在一起：

```
package org.acme.kafka.jukebox;

import java.time.Duration;
import java.time.Instant;
import java.util.Arrays;
import java.util.Collections;
import java.util.List;
import java.util.concurrent.ThreadLocalRandom;

import javax.enterprise.context.ApplicationScoped;

import io.smallrye.mutiny.Multi;
import io.smallrye.reactive.messaging.kafka.KafkaRecord;
import org.eclipse.microprofile.reactive.messaging.Outgoing;
import org.jboss.logging.Logger;

@ApplicationScoped
public class Jukebox {
    private static final Logger LOG = Logger.getLogger(Jukebox.class);

    private ThreadLocalRandom random = ThreadLocalRandom.current();

    private List<Song> songs = Collections.unmodifiableList(
            Arrays.asList(
                new Song(1, "Confessions", "Usher"),
                new Song(2, "How Do I Live", "LeAnn Rimes"),
                new Song(3, "Physical", "Olivia Newton-John"),
                new Song(4, "You Light Up My Life", "Debby Boone"),
                new Song(5, "The Twist", "Chubby Checker"),
                new Song(6, "Mack the Knife", "Bobby Darin"),
                new Song(7, "Night Fever", "Bee Gees"),
                new Song(8, "Bette Davis Eyes", "Kim Carnes"),
                new Song(9, "Macarena (Bayside Boys Mix)", "Los Del Rio"),
                new Song(10, "Yeah!", "Usher")
            )
    );

    @Outgoing("song-values")
    public Multi<KafkaRecord<Integer, String>> generate() {
        return Multi.createFrom().ticks().every(Duration.ofMillis(500))
                .onOverflow().drop()
                .map(tick -> {
                   Song s = songs.get(random.nextInt(songs.size()));
                   int timesPlayed = random.nextInt(1, 100);

                   LOG.infov("song {0}, times played: {1,number}",
                           s.title, timesPlayed);
                   return KafkaRecord.of(s.id, Instant.now()
                                         + ";" + timesPlayed);
                });
    }

    @Outgoing("songs")
    public Multi<KafkaRecord<Integer, String>> songs() {
        return Multi.createFrom().iterable(songs)
```

```
                .map(s -> KafkaRecord.of(s.id,
                        "{\n" +
                        "\t\"id\":\""+ s.id + "\",\n" +
                        "\t\"title\":\"" + s.title + "\",\n" +
                        "\t\"artist\":\"" + s.artist + "\"\n" +
                        "}"
                        ));
    }

    private static class Song {
        int id;
        String title;
        String artist;

        public Song(int id, String title, String artist) {
            this.id = id;
            this.title = title;
            this.artist = artist;
        }
    }
}
```

每隔500毫秒，一条包含歌曲和播放次数的新消息以及时间戳就会被发送到songs主题。我们将放弃额外的配置，你可以在15.5节中查看相关步骤。

接下来，我们需要建立管道。第一步是创建一些值持有者：

```
package org.acme.kafka.jukebox;

public class Song {
    public int id;
    public String title;
    public String artist;
}
```

创建一个播放次数的持有者：

```
package org.acme.kafka.jukebox;

import java.time.Instant;

public class PlayedCount {
    public int count;
    public String title;
    public String artist;
    public int id;
    public Instant timestamp;

    public PlayedCount(int id, String title, String artist,
                       int count, Instant timestamp) {
        this.count = count;
        this.title = title;
        this.artist = artist;
        this.id = id;
```

```
        this.timestamp = timestamp;
    }
}
```

值持有者是一个对象，用于在管道中处理消息时跟踪值的汇总：

```
package org.acme.kafka.jukebox;

import java.math.BigDecimal;
import java.math.RoundingMode;

public class Aggregation {
    public int songId;
    public String songTitle;
    public String songArtist;
    public int count;
    public int sum;
    public int min;
    public int max;
    public double avg;

    public Aggregation updateFrom(PlayedCount playedCount) {
        songId = playedCount.id;
        songTitle = playedCount.title;
        songArtist = playedCount.artist;

        count++;
        sum += playedCount.count;
        avg = BigDecimal.valueOf(sum / count)
                .setScale(1, RoundingMode.HALF_UP).doubleValue();
        min = Math.min(min, playedCount.count);
        max = Math.max(max, playedCount.count);

        return this;
    }
}
```

现在，进入流式查询的实现。我们只需要定义一个方法，该方法是一个CDI生产者，返回一个Apache Kafka Stream Topology。Quarkus将负责配置，生命周期将负责Kafka流引擎：

```
package org.acme.kafka.jukebox;

import java.time.Instant;

import javax.enterprise.context.ApplicationScoped;
import javax.enterprise.inject.Produces;

import io.quarkus.kafka.client.serialization.JsonbSerde;
import org.apache.kafka.common.serialization.Serdes;
import org.apache.kafka.streams.StreamsBuilder;
import org.apache.kafka.streams.Topology;
import org.apache.kafka.streams.kstream.Consumed;
```

```
import org.apache.kafka.streams.kstream.GlobalKTable;
import org.apache.kafka.streams.kstream.Materialized;
import org.apache.kafka.streams.kstream.Produced;
import org.apache.kafka.streams.state.KeyValueBytesStoreSupplier;
import org.apache.kafka.streams.state.Stores;

@ApplicationScoped
public class TopologyProducer {
    static final String SONG_STORE = "song-store";

    private static final String SONG_TOPIC = "songs";
    private static final String SONG_VALUES_TOPIC = "song-values";
    private static final String SONG_AGG_TOPIC = "song-aggregated";

    @Produces
    public Topology buildTopology() {
        StreamsBuilder builder = new StreamsBuilder();

        JsonbSerde<Song> songSerde = new JsonbSerde<>(Song.class);
        JsonbSerde<Aggregation> aggregationSerde =
                new JsonbSerde<>(Aggregation.class);

        KeyValueBytesStoreSupplier storeSupplier =
                Stores.persistentKeyValueStore(SONG_STORE);

        GlobalKTable<Integer, Song> songs = builder.globalTable(SONG_TOPIC,
                Consumed.with(Serdes.Integer(), songSerde));

        builder.stream(SONG_VALUES_TOPIC, Consumed.with(Serdes.Integer(),
                Serdes.String()))
                .join(
                        songs,
                        (songId, timestampAndValue) -> songId,
                        (timestampAndValue, song) -> {
                            String[] parts = timestampAndValue.split(";");
                            return new PlayedCount(song.id, song.title,
                                    song.artist,
                                    Integer.parseInt(parts[1]),
                                    Instant.parse(parts[0]));
                        }
                )
                .groupByKey()
                .aggregate(
                        Aggregation::new,
                        (songId, value, aggregation) ->
                                aggregation.updateFrom(value),
                        Materialized.<Integer, Aggregation> as(storeSupplier)
                            .withKeySerde(Serdes.Integer())
                            .withValueSerde(aggregationSerde)
                )
                .toStream()
                .to(
                        SONG_AGG_TOPIC,
                        Produced.with(Serdes.Integer(), aggregationSerde)
```

```
                );
        return builder.build();
    }
}
```

解释所有发生的事情超出了本节的范围，在下面的“另见”中可以查看更多内容。简而言之，就是连接到之前的songs和song-values主题，然后根据歌曲ID将这些值合并在一起。最后对播放次数的值进行汇总，并将输出结果发送到Apache Kafka的新主题。

讨论

我们推荐使用kafkacat工具来查看发送到主题的消息。

在两个Apache Kafka的例子中，我们只连接了一个客户端和一台机器。这不是Quarkus的限制，而是我们做了一些事情来帮助简化例子。

另见

更多信息，请访问以下网站：

· Apache Kafka:Kafka Streams（https://oreil.ly/gNAof）

- Confluent:kafkacat Utility（https://oreil.ly/bgIT_）

15.8 使用AMQP

问题

如何使用AMQP（高级消息队列协议）作为消息系统？

解决方案

使用quarkus-smallrye-reactive-messaging-amqp扩展。

与Kafka集成一样，Quarkus使用Eclipse MicroProfile Reactive Messaging作为围绕所有消息交互的“门面”（facade）。通过将quarkus-smallrye-reactive-messaging-amqp扩展添加到项目中，你可以获得SmallRye AMQP连接器和相关的依赖项。这可以使@Outbound、@Inbound、@Broadcast以及其他Eclipse MicroProfile Reactive Messaging注解和概念与AMQP一起工作。

这些注解适用于AMQP 1.0，而不是0.9.x。

你还需要在*application.properties*文件中设置通道连接器为smallrye-amqp。这些配置的语法如下：

```
mp.messaging.[outgoing|incoming].[channel-name].property=value
```

也可以通过以下方法设置AMQP连接的用户名和密码：

```
amqp-username=[my-username]
amqp-password=[my-secret-password]
```

如果你需要用它们自己的凭证与不同的实例进行通信，可以在每个通道的基础上设置这些凭证。更多的属性请参考SmallRye文档。

假设通道的名称是一样的，并且AMQP的其他设置和连接信息是正确的，则15.5节中的代码在AMQP中的工作方式和在Kafka中的工作方式是完全一样的。

另见

欲了解更多信息，请访问以下网站：

· SmallRye Reactive Messaging:AMQP 1.0（https://oreil.ly/ViPyo）

15.9　使用MQTT

问题

如何使用MQTT（MQ Telemetry Transport）作为消息系统？

解决方案

使用quarkus-smallrye-reactive-messaging-mqtt扩展。

与Kafka和AMQP集成一样，Quarkus使用Eclipse MicroProfile Reactive Messaging作为所有消息交互的门面。通过添加将quarkus-smallrye-reactive-messaging-mqtt扩展添加到项目中，你可以获得SmallRye MQTT连接器和相关的依赖项。这可以使@Out bound、@Inbound、@Broadcast以及其他Eclipse MicroProfile Reactive Messaging注解和概念与MQTT一起工作。

你还需要在*application.properties*文件中设置通道连接器为smallrye-mqtt。这些配置的语法如下：

```
mp.messaging.[outgoing|incoming].[channel-name].property=value
```

连接和凭证可以按通道进行设置。更多的属性请参考SmallRye文档。

假设通道的名称相同，并且MQTT的其他设置和连接信息都是正确的，则15.4节中的代码在MQTT中的工作方式和在Kafka中的工作方式完全一样。

讨论

还支持作为一个MQTT服务器。但是，它不是一个功能齐全的MQTT服务器。例如，它将只处理发布请求和确认，不处理订阅请求。

另见

更多信息，请访问以下网站：

· SmallRye Reactive Messaging:MQTT（https://oreil.ly/QmkVY）

15.10　使用响应式SQL进行查询

问题

如何使用PostgreSQL响应式客户端查询数据？

解决方案

Quarkus集成了Vert.x Reactive SQL客户端，该客户端可以与MySQL或MariaDB以及PostgreSQL一起使用。在本节中，我们将用PostgreSQL进行演示。下面我们将使用MariaDB。

当然，你需要添加一个扩展来利用响应式SQL客户端。目前，有两个客户端：quarkus-reactive-pg-client和quarkus-reactive-mysql-client，分别针对两个数据库。你还需要确保以下扩展在你的项目中（如果你使用的是JAX-RS）：

- quarkus-resteasy
- quarkus-resteasy-jsonb或quarkus-resteasy-jackson
- quarkus-resteasy-mutiny

你需要配置访问权限：

```
quarkus.datasource.db-kind=postgresql
quarkus.datasource.username=quarkus_test
quarkus.datasource.password=quarkus_test
quarkus.datasource.reactive.url=postgresql://localhost:5432/quarkus_test
```

现在你可以使用客户端：

```
package org.acme.pg;

import io.smallrye.mutiny.Multi;
import io.smallrye.mutiny.Uni;
import io.vertx.mutiny.pgclient.PgPool;
import io.vertx.mutiny.sqlclient.Row;
import io.vertx.mutiny.sqlclient.Tuple;

public class Book {
    public Long id;
    public String title;
    public String isbn;

    public Book() {
    }

    public Book(String title, String isbn) {
        this.title = title;
        this.isbn = isbn;
    }

    public Book(Long id, String title, String isbn) {
        this.id = id;
        this.title = title;
        this.isbn = isbn;
    }

    public static Book from(Row row) {
        return new Book(row.getLong("id"),
                        row.getString("title"),
                        row.getString("isbn"));
    }

    public static Multi<Book> findAll(PgPool client) {
        return client.query("SELECT id, title, isbn " +
                            "FROM books ORDER BY title ASC") ❶
                .onItem().produceMulti(Multi.createFrom()::iterable) ❷
                .map(Book::from); ❸
    }
}
```

❶查询数据库，返回Uni<RowSet<Row>

❷查询返回后，创建Multi<Row>

❸将每一行映射到一个Book实例中

为了完成练习，你可以使用RESTful端点：

```
package org.acme.pg;

import javax.annotation.PostConstruct;
import javax.inject.Inject;
import javax.ws.rs.Consumes;
```

```
import javax.ws.rs.GET;
import javax.ws.rs.Path;
import javax.ws.rs.Produces;
import javax.ws.rs.core.MediaType;
import javax.ws.rs.core.Response;

import io.smallrye.mutiny.Uni;
import io.vertx.mutiny.pgclient.PgPool;
import org.eclipse.microprofile.config.inject.ConfigProperty;

@Path("/books")
@Produces(MediaType.APPLICATION_JSON)
@Consumes(MediaType.APPLICATION_JSON)
public class BookResource {

    @Inject
    PgPool client;
    @GET
    public Uni<Response> get() {
        return Book.findAll(client)
                .collectItems().asList()
                .map(Response::ok)
                .map(Response.ResponseBuilder::build);
    }
}
```

讨论

你也可以通过使用prepareQuery方法和Tuple类来使用准备好的查询：

```
public static Uni<Boolean> delete(PgPool client, Long id) {
    return client.preparedQuery("DELETE FROM books " +
                                "WHERE id = $1", Tuple.of(id))
            .map(rowSet -> rowSet.rowCount() == 1); ❶
}
```

❶使用元数据，从RowSet实例中返回，验证该行是否被删除

另见

底层实现详见Vert.x：Reactive PostgreSQL Client（https://oreil.ly/OnuDM）。

15.11　使用响应式SQL客户端进行插入

问题

如何使用MySQL响应式客户端插入数据？

解决方案

与前面使用PostgreSQL类似，数据插入也可以用响应式MySQL客户端来完成。同样的扩展需要将quarkus-reactive-pg-client改为quarkus-reactive-mysql-client：

- quarkus-resteasy
- quarkus-resteasy-jsonb或quarkus-resteasy-jackson
- quarkus-resteasy-mutiny

当然，你需要设置数据源：

```
quarkus.datasource.db-kind=mysql
quarkus.datasource.username=quarkus_test
quarkus.datasource.password=quarkus_test
quarkus.datasource.reactive.url=mysql://localhost:3306/quarkus_test
```

在Book.save方法中，你会看到和之前非常相似的主题：

```
public Uni<Long> save(MySQLPool client) {
    String query = "INSERT INTO books (title,isbn) VALUES (?,?)";
    return client.preparedQuery(query, Tuple.of(title, isbn))
            .map(rowSet -> rowSet
                    .property(MySQLClient.LAST_INSERTED_ID)); ❶
}
```

❶使用RowSet的属性来获取插入的ID

现在，你应该能够为BookResource端点设置适当的POST方法，在从用户接收到的新Book实例上调用save方法。

另见

欲了解更多信息，请访问以下网站：

· Vert.x:Reactive MySQL Client（https://oreil.ly/UHxfh）

15.12　使用响应式MongoDB客户端

问题

如何使用响应式MongoDB客户端？

解决方案

MongoDB Quarkus扩展还包括一个响应式MongoDB客户端。如7.21节所示，你需要添加quarkus-mongodb-client，并在你的项目中添加以下扩展。

quarkus-resteasy-mutiny

返回并与Mutiny进行交互以实现端点返回。

quarkus-smallrye-context-propagation

这使得像注入和事务这样的事件可以与异步代码一起工作。

其余的集成很简单。这里是之前MongoDB中的服务和资源类的版本，但以响应式的方式编写：

```
package org.acme.mongodb;

import java.util.List;
import java.util.Objects;

import javax.enterprise.context.ApplicationScoped;
import javax.inject.Inject;

import com.mongodb.client.model.Filters;

import io.quarkus.mongodb.reactive.ReactiveMongoClient;
import io.quarkus.mongodb.reactive.ReactiveMongoCollection;
import io.smallrye.mutiny.Uni;
import org.bson.Document;

@ApplicationScoped
public class ReactiveBookService {
    @Inject
    ReactiveMongoClient mongoClient;

    public Uni<List<Book>> list() {
        return getCollection().find()
                .map(Book::from).collectItems().asList();
    }

    public Uni<Void> add(Book b) {
        Document doc = new Document()
                .append("isbn", b.isbn)
                .append("title", b.title)
                .append("authors", b.authors);

        return getCollection().insertOne(doc);
    }

    public Uni<Book> findSingle(String isbn) {
        return Objects.requireNonNull(getCollection()
                .find(Filters.eq("isbn", isbn))
                .map(Book::from))
                .toUni();
    }

    private ReactiveMongoCollection<Document> getCollection() {
        return mongoClient.getDatabase("book")
```

```
                .getCollection("book");
    }
}
```

除了导入和用Mutiny从一个必要的方法转向一个响应的方法之外，没有任何改变。REST端点也是如此：

```
package org.acme.mongodb;

import java.util.List;

import javax.inject.Inject;
import javax.ws.rs.Consumes;
import javax.ws.rs.GET;
import javax.ws.rs.POST;
import javax.ws.rs.Path;
import javax.ws.rs.PathParam;
import javax.ws.rs.Produces;
import javax.ws.rs.core.MediaType;
import javax.ws.rs.core.Response;

import io.smallrye.mutiny.Uni;

@Path("/reactive_books")
@Produces(MediaType.APPLICATION_JSON)
@Consumes(MediaType.APPLICATION_JSON)
public class ReactiveBookResource {
    @Inject
    ReactiveBookService service;

    @GET
    public Uni<List<Book>> getAll() {
        return service.list();
    }

    @GET
    @Path("{isbn}")
    public Uni<Book> getSingle(@PathParam("isbn") String isbn) {
        return service.findSingle(isbn);
    }

    @POST
    public Uni<Response> add(Book b) {
        return service.add(b).onItem().ignore()
                .andSwitchTo(this::getAll)
                .map(books -> Response.status(Response.Status.CREATED)
                                      .entity(books).build());
    }
}
```

另见

欲了解更多信息，请访问以下网站：

- Quarkus:Context Propagation in Quarkus（https://oreil.ly/bd9hA）

15.13　使用响应式Neo4j客户端

问题

如何使用响应式Neo4j客户端？

解决方案

Neo4j Quarkus扩展支持响应式驱动。

你需要使用版本4或更高版本的Neo4j来实现完全的响应式。你还需要在项目中添加quarkus-resteasy-mutiny扩展。从7.23节开始，除了使用驱动中的RxSession和使用Mutiny之外，并没有什么变化：

```
package org.acme.neo4j;

import java.util.stream.Collectors;

import javax.inject.Inject;
import javax.ws.rs.Consumes;
import javax.ws.rs.GET;
import javax.ws.rs.POST;
import javax.ws.rs.Path;
import javax.ws.rs.Produces;
import javax.ws.rs.core.MediaType;
import javax.ws.rs.core.Response;

import io.smallrye.mutiny.Multi;
import io.smallrye.mutiny.Uni;
import org.neo4j.driver.Driver;
import org.neo4j.driver.Record;
import org.neo4j.driver.Value;
import org.neo4j.driver.Values;
import org.neo4j.driver.reactive.RxResult;
import org.reactivestreams.Publisher;

@Path("/reactivebooks")
@Produces(MediaType.APPLICATION_JSON)
@Consumes(MediaType.APPLICATION_JSON)
public class ReactiveBookResource {
    @Inject
    Driver driver;

    @GET
    @Produces(MediaType.SERVER_SENT_EVENTS) ❸
    public Publisher<Response> getAll() {
        return Multi.createFrom().resource( ❷
                driver::rxSession,
```

```
                rxSession -> rxSession.readTransaction(tx -> { ❶
                    RxResult result = tx.run("MATCH (b:Book) RETURN " +
                                             "b ORDER BY b.title");
                    return Multi.createFrom().publisher(result.records())
                            .map(Record::values)
                            .map(values -> values.stream().map(Value::asNode)
                                                  .map(Book::from)
                                                  .map(Book::toJson))
                            .map(bookStream ->
                                    Response.ok(bookStream
                                            .collect(Collectors.toList()))
                                    .build());
                }))
                .withFinalizer(rxSession -> { ❹
                    return Uni.createFrom().publisher(rxSession.close());
                });
    }

    @POST
    public Publisher<Response> create(Book b) {
        return Multi.createFrom().resource(
                driver::rxSession,
                rxSession -> rxSession.writeTransaction(tx -> {
                    String query = "CREATE " +
                                   "(b:Book {title: $title, isbn: $isbn," +
                                   " authors: $authors}) " +
                                   "RETURN b";
                    RxResult result = tx.run(query,
                            Values.parameters("title", b.title,
                                    "isbn", b.isbn, "authors", b.authors));
                    return Multi.createFrom().publisher(result.records())
                            .map(record -> Response.ok(record
                                    .asMap()).build());
                })
        ).withFinalizer(rxSession -> {
            return Uni.createFrom().publisher(rxSession.close());
        });
    }
}
```

❶从驱动程序中获取一个RxSession

❷使用Mutiny与ReactiveStreams Publisher进行交互

❸将结果反馈给用户

❹在session结束时关闭

第16章　Quarkus的附加功能

本章包含了一些不适合放在其他章节中介绍的Quarkus功能。当然，这并不意味着它们没有用！本章涵盖以下主题：

- Quarkus的模板解决方案：Qute
- OpenAPI集成
- 发送电子邮件
- 调度功能
- 应用程序数据缓存

16.1　使用Qute模板引擎创建模板

问题

如何创建模板，并用特定的数据来渲染它们？

解决方案

使用Qute模板引擎。

Qute是一个模板引擎，专门为满足Quarkus的需求而设计，可以最小化反射，并支持命令式和响应式的编码风格。

Qute可以作为一个独立的库来使用（生成报告到磁盘或生成电子邮件正文消息），也可以与JAX-RS一起使用来传递HTML内容。

要开始使用Qute和JAX-RS，请添加resteasy-qute扩展：

```
./mvnw quarkus:add-extension -Dextensions="quarkus-resteasy-qute"
```

默认情况下，模板存储在src/main/resources/templates目录及其子目录中。

下面是一个纯文本文件的简单扩展的模板：

```
Hello {name}!
```

模板是一个简单的句子，用name参数化。

要用具体的数据来渲染模板，只需要注入io.quarkus.qute.Template实例并提供模板参数即可：

```
@Inject
io.quarkus.qute.Template hello; ❶ ❷

@GET
@Produces(MediaType.TEXT_PLAIN)
public TemplateInstance hello() { ❸
    final String name = "Alex";
    return hello.data("name", name); ❹
}
```

❶Template实例定义了在模板中要执行的操作

❷默认情况下，字段名用于定位模板。在本例中，模板路径为src/main/resources/templates/hello.txt

❸渲染不是必要的，因为RESTEasy集成了TemplateInstance对象来渲染内容

❹data方法是用来设置模板参数的

运行这个项目能看到模板是如何渲染的：

```
./mvnw compile quarkus:dev

curl http://localhost:8080/hello

Hello Alex!
```

讨论

Qute支持更多的语法（例如，include和insert片段，直接注入CDI bean或变体模板），以及与其他Quarkus部件的集成，如电子邮件或日程任务。

另见

请访问以下网站了解更多关于Qute的信息：

· Quarkus:Qute Reference Guide（https://oreil.ly/R1A1S）

16.2　使用Qute渲染HTML

问题

如何使用Qute渲染HTML？

解决方案

Qute会像渲染文本一样轻松地渲染HTML。只需要让Quarkus找到与你的注入相匹配的模板即可，模板的实际内容并不太重要。

让我们在模板上渲染一个结构比较复杂的HTML页面。在这种情况下，呈现的是一个简单的HTML报告。创建一个包含报表参数的POJO类：

```
package org.acme.quickstart;

import java.util.ArrayList;
import java.util.List;

import io.quarkus.qute.TemplateData;

@TemplateData ❶
public class Movie {

    public String name;
    public int year;
    public String genre;
    public String director;
    public List<String> characters = new ArrayList<>();
    public float ratings;

    public int getStars() { ❷
        return Math.round(ratings);
    }
}
```

❶这个注解允许Quarkus避免在运行时使用反射来访问对象

❷自定义方法

讨论

下面是HTML模板的一些细节，值得解释一下。

下面是一个可选的头，你可以把它放在任何模板中，以帮助Quarkus在编译时验证所有的表达式：

```
{@org.acme.quickstart.Movie movie} ❶
<!DOCTYPE html>


 <html>
```

❶参数声明；这不是强制性的，但有助于Quarkus验证模板的类型安全

支持条件或循环等基本语法：

```
<div class="col-sm-12">
    <dl>
        {#if movie.year == 0} ❶
            <dt>Year:</dt> Not Known
        {#else} ❷
            <dt>Year:</dt> {movie.year}
        {/if}
        {#if movie.genre is 'horror'} ❸
        <dt>Genre:</dt> Buuuh
        {#else}
        <dt>Genre:</dt> {movie.genre}
        {/if}
        <dt>Director:</dt> {movie.director ?: 'Unknown'} ❹
        <dt>Main Characters:</dt>
        {#for character in movie.characters} ❺
            {character} ❻
            {#if hasNext} ❼
             -
            {/if}
        {/for}
        <dt>Rating:</dt>
        <font color="red">
        {#for i in movie.stars} ❽
            <span class="fas fa-xs fa-star"></span>
        {/for}
        </font>
    </dl>
</div>
```

❶有条件的数字型

❷else部分

❸用字符串类型的条件，使用is

❹Elvis运算符。如果参数为空，则使用默认值

❺遍历所有字符

❻显示角色信息

❼hasNext是一个特殊的属性，用于检查是否有更多的元素

❽POJO中定义的方法调用，迭代调用中定义的次数

在一个循环中，可以使用以下隐式变量：hasNext、count、index、odd和even。

此时，只能使用Iterable、Map.Entry、Set、Integer和Stream。

16.3　更改Qute模板的位置

问题

如何更改Qute来查找模板的位置？

解决方案

你可以使用io.quarkus.qute.api.ResourcePath注解来自定义模板位置（仍然在*src/main/resources/templates*内，并输出到你的应用程序可部署的*templates*目录中）：

```
@ResourcePath("movies/detail.html") ❶
Template movies;
```

❶将模板的路径设置为*src/main/resources/templates/movies/detail.html*

再次运行应用程序（如果已经运行，则让实时重载来完成它的工作），然后打开浏览器并输入这个URL：http://localhost:8080/movie。

16.4　扩展Qute数据类

问题

如何扩展一个Qute数据类的功能？

解决方案

模板扩展方法必须遵循以下规则：

· 必须是静态的。

· 必须不能返回void。

· 必须至少包含一个参数，第一个参数用于匹配基础数据对象。

当你无法访问数据对象源代码时，可以使用模板扩展来添加专门用于报告目的的方法。

你可以通过使用@io.quarkus.qute.TemplateExtension注解来实现模板扩展方法。在本例中，让我们实现一个四舍五入rating数的方法。

```
@TemplateExtension
static double roundStars(Movie movie, int decimals) { ❶ ❷
    double scale = Math.pow(10, decimals);
    return Math.round(movie.ratings * scale) / scale;
}
```

❶第一个参数是POJO数据对象

❷可设置自定义参数

从模板引擎来看，movie有一个roundStars方法，该方法有一个参数，就是要取整的小数。

在模板中，你现在可以调用以下方法：

```
({movie.roundStars(2)}) ❶
```

❶Movie类没有定义roundStars方法，但它是可以访问的，因为它是一个模板扩展再次运行应用程序（如果已经运行，则让实时重载来完成它的工作），然后打开浏览器并输入URL：http://localhost:8080/movie。

其输出应该与图16-1类似。

图16-1：HTML输出

16.5　用OpenAPI描述端点

问题

如何用OpenAPI描述你的REST API？

解决方案

使用SmallRye OpenAPI扩展。

用Quarkus创建了一个RESTful API后，你需要添加openapi扩展：

```
./mvnw quarkus:add-extension -Dextensions="openapi"
```

然后重新启动应用程序，一切就会生效：

```
./mvnw compile quarkus:dev
```

API的规范默认在*/openapi*。要改变这个位置，可以使用quarkus.smallrye-openapi.path配置：

```
quarkus.smallrye-openapi.path=/rest-api
```

你可以请求http://localhost:8080/openapi获取相关规范：

```
openapi: 3.0.1
info:
  title: Generated API
  version: "1.0"
paths:
  /task:
    get:
      responses:
        200:
          description: OK
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/SetTask'
    post:
      requestBody:
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/Task'
      responses:
        200:
          description: OK
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/SetTask'
    delete:
      requestBody:
        content:
          application/json:
            schema:
```

```
              $ref: '#/components/schemas/Task'
      responses:
        200:
          description: OK
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/SetTask'
components:
  schemas:
    Task:
      type: object
      properties:
        complete:
          type: boolean
        description:
          type: string
        reminder:
          format: date-time
          type: string
    SetTask:
      type: array
      items:
        type: object
properties:
  complete:
    type: boolean
  description:
    type: string
  reminder:
    format: date-time
    type: string
```

根据前面的规范，有GET、POST和DELETE等端点。你还可以看到，DELETE和POST需要一个任务对象，任务需要一个布尔值、字符串和一个日期时间。这很简单，也很容易理解。

讨论

在Quarkus中使用SmallRye OpenAPI扩展可以很容易地创建一个OpenAPI规范。这为你提供了一个易于使用和易于阅读的RESTful API视图。

SmallRye OpenAPI是Eclipse MicroProfile OpenAPI的一个实现。OpenAPI规范是一种标准的、语言无关的描述和覆盖RESTful API的手段，它对于人类和机器都是可读的。OpenAPI文档使用JSON或YAML定义。

参考

更多信息，请访问GitHub上的以下页面：

- Eclipse MicroProfile OpenAPI（https://oreil.ly/hczN4）
- MicroProfile OpenAPI Specification（https://oreil.ly/ufzr6）
- OpenAPI Specification（https://oreil.ly/uslyb）

在16.6节中，你将学习如何使用SmallRye OpenAPI中的注解来定制生成的规范。

16.6　自定义OpenAPI规范

问题

如何自定义生成的API规范？

解决方案

使用SmallRye OpenAPI扩展的OpenAPI注解。

重用之前创建的任务API，16.5节很容易利用OpenAPI注解来为你的API添加自定义和进一步的文档：

```
package org.acme.openapi;

import java.time.LocalDateTime;
import java.util.Collections;
import java.util.LinkedHashMap;
import java.util.Set;

import javax.ws.rs.Consumes;
import javax.ws.rs.DELETE;
import javax.ws.rs.GET;
import javax.ws.rs.POST;
import javax.ws.rs.Path;
import javax.ws.rs.Produces;
import javax.ws.rs.core.MediaType;

import org.eclipse.microprofile.openapi.annotations.Operation;
import org.eclipse.microprofile.openapi.annotations.media.Content;
import org.eclipse.microprofile.openapi.annotations.media.Schema;
import org.eclipse.microprofile.openapi.annotations.parameters.Parameter;

  @Path("/task")
  @Produces(MediaType.APPLICATION_JSON)
  @Consumes(MediaType.APPLICATION_JSON)
  public class TaskResource {

    Set<Task> tasks = Collections.newSetFromMap(
        Collections.synchronizedMap(new LinkedHashMap<>()));

    public TaskResource() {
```

```
      tasks.add(new Task("First task",
            LocalDateTime.now().plusDays(3), false));
      tasks.add(new Task("Second task",
            LocalDateTime.now().plusDays(6), false));
    }

    @GET
    @Operation(summary = "Get all tasks",
               description = "Get the full list of tasks.")
    public Set<Task> list() {
      return tasks;
    }

    @POST
    @Operation(summary = "Create a new task")
    public Set<Task> add(
        @Parameter(required = true, content =
          @Content(schema = @Schema(implementation = Task.class))) Task task) {
      tasks.add(task);
      return tasks;
    }

    @DELETE
    @Operation(summary = "Remove the specified task")
    public Set<Task> delete(
        @Parameter(required = true,
        content = @Content(schema = @Schema(implementation = Task.class)))
        Task task) {
      tasks.removeIf(existingTask -> existingTask.equals(task));
      return tasks;
    }
  }
```

```
package org.acme.openapi;

import java.time.LocalDateTime;
import java.util.Objects;

import javax.json.bind.annotation.JsonbDateFormat;

import org.eclipse.microprofile.openapi.annotations.enums.SchemaType;
import org.eclipse.microprofile.openapi.annotations.media.Schema;

public class Task {
    public String description;

    @Schema(description = "Flag indicating the task is complete")
    public Boolean complete;

    @JsonbDateFormat("yyyy-MM-dd'T'HH:mm")
    @Schema(example = "2019-12-25T06:30", type = SchemaType.STRING,
            implementation = LocalDateTime.class,
            pattern = "yyyy-MM-dd'T'HH:mm",
            description = "Date and time for the reminder.")
    public LocalDateTime reminder;
```

```
    public Task() {
    }

    public Task(String description,
                LocalDateTime reminder,
                Boolean complete) {
        this.description = description;
        this.reminder = reminder;
        this.complete = complete;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        Task task = (Task) o;
        return Objects.equals(description, task.description) &&
                Objects.equals(reminder, task.reminder) &&
                Objects.equals(complete, task.complete);
    }

    @Override
    public int hashCode() {
        return Objects.hash(description, reminder, complete);
    }
}
```

前面的代码将创建以下规范：

```yaml
---
openapi: 3.0.1
info:
  title: Generated API
  version: "1.0"
paths:
  /task:
    get:
      summary: Get all tasks
      description: Get the full list of tasks.
      responses:
        200:
          description: OK
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/SetTask'
    post:
      summary: Create a new task
      requestBody:
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/Task'
      responses:
```

```
        200:
          description: OK
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/SetTask'
    delete:
      summary: Remove the specified task
      requestBody:
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/Task'
      responses:
        200:
          description: OK
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/SetTask'
components:
  schemas:
    Task:
      type: object
      properties:
        complete:
          description: Flag indicating the task is complete
          type: boolean
        description:
          type: string
        reminder:
          format: date-time
          description: Date and time for the reminder.
          pattern: yyyy-MM-dd'T'HH:mm
          type: string
          example: 2019-12-25T06:30
    SetTask:
      type: array
      items:
        type: object
        properties:
          complete:
            description: Flag indicating the task is complete
            type: boolean
          description:
            type: string
          reminder:
            format: date-time
            description: Date and time for the reminder.
            pattern: yyyy-MM-dd'T'HH:mm
            type: string
            example: 2019-12-25T06:30
```

根据前面的规范，有GET、POST和DELETE接口。你还可以看到，DELETE和POST需要一个任务对象。任务需要一个布尔值、字符串和一个日期时间。这很简单，也很容易理解。

讨论

各种OpenAPI注解用于提供有关API的附加信息，包括描述、摘要和示例。关于这些注解的更多信息可以在规范和“另见”部分的链接中找到。

使用Quarkus可以很容易地对生成的OpenAPI规范进行进一步的定制。对于最终的自定义，Quarkus会提供静态文件OpenAPI规范。为此，你需要将有效的OpenAPI规范文件放在*META-INF/openapi.yml*或*META-INF/openapi.json*。Quarkus会将两者结合起来，并提供一个静态和动态相结合的规范。要禁用动态规范生成，只需在*applications.properties*文件中使用mp.openapi.scan.disable=true配置。

参考

更多信息，请访问GitHub上的以下页面：

· Eclipse MicroProfile OpenAPI Specification（https://oreil.ly/i47k_）

· Eclipse MicroProfile OpenAPI:Annotation Samples（https://oreil.ly/ITXQz）

· Swagger 2.X Annotations:OpenAPI Annotations（https://oreil.ly/ol6nb）

16.7　同步发送电子邮件

问题

如何同步发送电子邮件？

解决方案

利用Quarkus邮件扩展。

Quarkus可以非常直观地发送纯文本和HTML的电子邮件，并添加附件。还可以测试邮件是否被正确发送，而且无须设置自己的中继。将Email Quarkus扩展添加到现有的项目中：

```
mvn quarkus:add-extensions -Dextensions="mailer"
```

Quarkus使用Vert.x邮件客户端，其中有两个包装器：

```
@Inject
Mailer mailer;
```

```
@Inject
ReactiveMailer reactiveMailer;
```

Mailer类使用标准的阻塞和同步API调用，而ReactiveMailer使用非阻塞和异步API调用。ReactiveMailer将在下面讨论。这两个类提供了相同的功能。要发送电子邮件，只需使用withText或withHtml方法。你需要提供一个收件人、一个主题和一个正文。如果你需要添加诸如CC、BCC和附件之类的东西，可以在实际的Mail实例上进行添加。

你还需要配置SMTP提供者（在本例中，我们使用Gmail TLS）：

```
quarkus.mailer.from=quarkus-test@gmail.com
quarkus.mailer.host=smtp.gmail.com
quarkus.mailer.port=587
quarkus.mailer.start-tls=REQUIRED

❶
quarkus.mailer.username=YOUREMAIL@gmail.com
quarkus.mailer.password=YOURGENERATEDAPPLICATIONPASSWORD
```

❶这些也可以通过系统属性或环境属性来设置

通过使用MockMailbox组件，可以很容易地完成电子邮件组件的测试。它是一个由三个方法组成的简单组件：

· getMessagesSentTo

· clear

· getTotalMessagesSent

下面的测试演示了这三种方法：

```
package org.acme.email;

import java.util.List;

import javax.inject.Inject;

import io.quarkus.mailer.Mail;
import io.quarkus.mailer.Mailer;
import io.quarkus.mailer.MockMailbox;
import io.quarkus.test.junit.QuarkusTest;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import static org.assertj.core.api.Assertions.assertThat;

@QuarkusTest
public class MailerTest {
    @Inject
    Mailer mailer;
```

```
    @Inject
    MockMailbox mbox;

    @BeforeEach
    void clearMBox() {
        mbox.clear(); ❶
    }

    @Test
    public void assertBasicTextEmailSent() {
        final String mailTo = "test@example.org";
        final String testingSubject = "Testing email";
        final String testingBody = "Hello World!";

        mailer.send(Mail.withText(mailTo,
                testingSubject,
                testingBody));

        assertThat(mbox.getTotalMessagesSent()).isEqualTo(1); ❷
        List<Mail> emails = mbox.getMessagesSentTo(mailTo); ❸

        assertThat(emails).hasSize(1);
        Mail email = emails.get(0);

        assertThat(email.getSubject()).isEqualTo(testingSubject);
        assertThat(email.getText()).isEqualTo(testingBody);
    }
}
```

❶每次测试开始前，我们都会清空邮箱

❷使用getTotalMessagesSent来验证Quarkus发送了多少条信息

❸核查向特定地址发送的信息

讨论

同时支持常规附件和内联附件。下面是一个简单的内联附件的例子：

```
@Test
void attachmentTest() throws Exception {
    final String mailTo = "test@example.org";
    final String testingSubject = "email with Attachment";
    final String html = "<strong>E-mail by:</strong>" + "\n" +
            "<p><img src=\"cid:logo@quarkus.io\"/></p>";    ❶

    sendEmail(mailTo, testingSubject, html);

    Mail email = mbox.getMessagesSentTo(mailTo).get(0);
    List<Attachment> attachments = email.getAttachments();

    assertThat(email.getHtml()).isEqualTo(html);
    assertThat(attachments).hasSize(1);
```

```
    assertThat(attachments.get(0).getFile())
            .isEqualTo(new File(getAttachmentURI()));
}
private void sendEmail(String to, String subject, String body)
      throws URISyntaxException {
    final File logo = new File(getAttachmentURI());

    Mail email = Mail.withHtml(to, subject, body)
            .addInlineAttachment("quarkus-logo.svg",
                    logo,
                    "image/svg+xml",
                    "<logo@quarkus.io>");  ❷

    mailer.send(email);
}
```

❶一定要用content-id引用内联附件

❷附件的content-id

另见

更多信息，请见：

· 16.8节

· Vert.x邮件客户端（SMTP客户端实现）（https://oreil.ly/aqGZU）

16.8　以响应式方式发送电子邮件

问题

如何以非阻塞、响应式方式发送电子邮件？

解决方案

使用Quarkus邮件扩展。

上一节详细介绍了基础知识。要以响应式方式进行，只需注入ReactiveMailer组件并使用它。方法都是一样的，只是该方法返回响应式副本，而不是同步式副本：

```
package org.acme.email;

import java.util.List;
import java.util.concurrent.CountDownLatch;

import javax.inject.Inject;
```

```
import io.quarkus.mailer.Mail;
import io.quarkus.mailer.MockMailbox;
import io.quarkus.mailer.reactive.ReactiveMailer;
import io.quarkus.test.junit.QuarkusTest;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import static org.assertj.core.api.Assertions.assertThat;

@QuarkusTest
public class ReactiveMailerTest {
    @Inject
    ReactiveMailer reactiveMailer;

    @Inject
    MockMailbox mbox;

    @BeforeEach
    void clearMbox() {
        mbox.clear();
    }

    @Test
    public void testReactiveEmail() throws Exception {
        final String mailTo = "test@example.org";
        final String testingSubject = "Testing email";
        final String testingBody = "Hello World!";
        final CountDownLatch latch = new CountDownLatch(1);

        reactiveMailer.send(Mail.withText(mailTo,
                testingSubject,
                testingBody)).subscribeAsCompletionStage().join();

        assertThat(mbox.getTotalMessagesSent()).isEqualTo(1);
        List<Mail> emails = mbox.getMessagesSentTo(mailTo);

        assertThat(emails).hasSize(1);
        Mail email = emails.get(0);

        assertThat(email.getSubject()).isEqualTo(testingSubject);
        assertThat(email.getText()).isEqualTo(testingBody);
    }
}
```

这个测试和上一节的测试完全一样，唯一不同的是把CompletionStage变成了CompletableFuture，并调用join来回到测试的命令式风格。

讨论

Qute集成了Mailer扩展，所以邮件的正文内容是由模板渲染的。

这次你只需要qute扩展，因为不需要集成RESTEasy：

```
mvn quarkus:add-extensions -Dextensions="quarkus-qute"
```

主类是io.quarkus.mailer.MailTemplate，它的使用方法和io.quarkus.qute.Template一样，但第一个类包含了邮件逻辑特有的方法：

```
@ResourcePath("mail/welcome.txt") ❶
MailTemplate mailTemplate;

CompletionStage<Void> c = hello.to("to@acme.org")
    .subject("Hello from Qute template")
    .data("name", "Alex")
    .send(); ❷
```

❶模板位于*src/main/resources/templates/mail/welcome.txt*

❷从模板文件中用提供的数据发送邮件，渲染正文

响应式发送邮件的方式遵循完全相同的方法名称和用法，只不过是使用响应式类。这使得它非常容易切换和理解。

参考

更多信息，请看：

· 16.1节

· 16.7节

16.9　创建调度任务

问题

如何让一些任务按调度运行？

解决方案

在Quarkus中调度任务是快速和简单的，它提供了一个高水平的控制和定制。Quarkus有一个与Quartz集成的scheduler扩展。

创建调度任务非常简单：只需将@io.quarkus.schedule.Scheduled注解添加到一个应用范围的bean中。有两个属性可用于指定任务的调度：cron和every。

cron属性使用Quartz cron语法。如果你不熟悉Quartz，你可以访问本节“另见”中的链接了解更多信息。

every使用Duration#parse解析字符串。如果表达式以数字开头，则会自动添加PT前缀。

every和cron都会对以{开头和以}结束的表达式进行配置查找。

对delay属性取一个long，对delayUnit属性取一个TimeUnit。这两个属性一起使用会指定一个延迟，延迟之后就会启动触发器。默认情况下，触发器在被注册时就会启动。

下面是一个非常简单的用法演示：

```
package org.acme.scheduling;

import java.util.concurrent.atomic.AtomicInteger;

import javax.enterprise.context.ApplicationScoped;

import io.quarkus.scheduler.Scheduled;
import io.quarkus.scheduler.ScheduledExecution;

@ApplicationScoped
public class Scheduler {

    private AtomicInteger count = new AtomicInteger();

    int get() {
        return count.get();
    }

    @Scheduled(every = "5s")
    void fiveSeconds(ScheduledExecution execution) {
        count.incrementAndGet();
        System.out.println("Running counter: 'fiveSeconds'. Next fire: "
                + execution.getTrigger().getNextFireTime());
    }
}
```

讨论

Qute可以用来定期生成报表。

你只需要qute扩展，因为不需要集成RESTEasy：

```
mvn quarkus:add-extensions -Dextensions="quarkus-qute"
```

现在必须手动调用render()方法才能得到结果：

```
@ResourcePath("reports/report_01.html")
Template report;
```

```
@Scheduled(cron="0 30 * * * ?")
void generate() {
    final String reportContent = report
        .data("sales", listOfSales)
        .data("now", java.time.LocalDateTime.now())
        .render();
    Files.write(reportOuput, reportContent.getBytes());
}
```

另见

更多信息，请见：

- Quartz:Cron Trigger Tutorial（https://oreil.ly/XdQ7r）
- 16.1节

16.10　使用应用程序数据缓存

问题

当方法需要很长时间才能响应时，如何避免等待时间？

解决方案

使用应用程序数据缓存。

在某些情况下，一个方法的响应时间可能比预期的要长，这可能是因为它正在向外部系统发出请求，或者正在执行的逻辑需要很长的时间才能完成。

可以使用应用程序数据缓存来改善这种情况。其核心是保存一个方法调用的结果，这样以后用相同的输入对该方法调用就会返回之前计算的结果。

Quarkus集成了Caffeine（https://oreil.ly/lNjlX）作为缓存提供商。

要使用应用程序数据缓存，请添加cache扩展：

```
./mvnw quarkus:add-extension -Dextensions="cache"
```

下面是一个模拟长时间执行的方法：

```
@GET
@Produces(MediaType.TEXT_PLAIN)
public String hello() {
    long initial = System.currentTimeMillis();
    String msg = greetingProducer.getMessage(); ❶
    long end = System.currentTimeMillis();


    return msg + " " + (end - initial) + "ms";
}
```

❶这个逻辑的睡眠时间是随机的

运行项目，你将看到这个延迟：

```
./mvnw compile quarkus:dev

curl http://localhost:8080/hello
Hello World 4009ms

curl http://localhost:8080/hello
Hello World 3003ms
```

让我们使用@io.quarkus.cache.CacheResult注解来缓存getMessage()方法的调用：

```
@CacheResult(cacheName = "greeting-cache") ❶
public String getMessage() {
    try {
        TimeUnit.SECONDS.sleep(random.nextInt(4) + 1);
        return "Hello World";
    } catch (InterruptedException e) {
        throw new IllegalStateException(e);
    }

}
```

❶为这个方法调用创建一个新的缓存

再次运行应用程序（如果已经运行，则让实时重载来完成它的工作），并重复调用http://localhost:8080/hello：

```
curl http://localhost:8080/hello
Hello World 2004ms

curl http://localhost:8080/hello
Hello World 0ms
```

第二次尝试调用该方法时，不会调用该方法，而是从缓存中返回。Quarkus为每一次调用计算一个缓存键，并在缓存系统中检查是否命中。

为了计算缓存键，Quarkus默认使用所有参数值。如果没有参数方法，则从缓存名称中导出密钥。

讨论

@io.quarkus.cache.CacheKey注解可以用在方法参数中，来指定哪些参数必须被用来计算缓存密钥。例如，public String myMethod(@CacheKey String keyElement1,String notPartOfTheKey)。

@io.quarkus.cache.CacheKey注解不能用于返回void的方法。

@io.quarkus.cache.CacheInvalidate注解可以用来使缓存中的条目无效。当调用带有@CacheInvalidate注解的方法时，会计算缓存键，并用于从缓存中删除一个现有的条目。

@io.quarkus.cache.CacheInvalidateAll注解用于使所有缓存条目无效。

每个数据缓存选项都可以在*application.properties*文件中单独配置：

```
quarkus.cache.caffeine."greeting-cache".initial-capacity=10 ❶
quarkus.cache.caffeine."greeting-cache".expire-after-write=5S ❷
```

❶greeting-cache的内部数据结构的最小总大小

❷设置过期时间，在对greeting-cache进行写操作后计算

再次运行应用程序（如果已经运行，则让实时重载来完成它的工作），并重复调用http://localhost:8080/hello：

```
curl http://localhost:8080/hello
Hello World 2004ms

curl http://localhost:8080/hello
Hello World 0ms

// Wait 5 seconds

curl http://localhost:8080/hello
Hello World 1011ms
```

quarkus.cache.caffeine."greeting-cache".expire-after-access属性可以用来设置缓存的过期时间，即缓存值最近一次读或写后的时间量。

附录A　minikube

书中所有涉及Kubernetes集群的内容都在minikube中进行了测试。但是，它们也应该在任何其他Kubernetes集群中工作。

minikube是一个工具，可以让你轻松地在本地Kubernetes集群中运行Kubernetes。

本书中使用了minikube 1.7.3和Kubernetes 1.17.3。同样，任何其他版本应该都可以，因为没有使用高级技术。使用minikube需要安装一个管理程序。我们建议你使用VirtualBox hypervisor。根据我们的经验，这是最便携和最稳定的运行minikube的方式。

如何安装minikube、VirtualBox和kubectl可能取决于你运行的系统，所以我们提供了链接，你可以在这里找到安装这些组件的说明。

· VirtualBox（https://oreil.ly/KU2vk）

· Minikube（https://oreil.ly/gth-J）

· kubectl（https://oreil.ly/FpZzN）

安装完所有软件后，你可以打开一个终端窗口并运行以下内容来启动minikube：

```
minikube start --vm-driver=virtualbox --memory='8192mb' \
    --kubernetes-version='v1.17.3'

[serverless] minikube v1.7.3 on Darwin 10.15.3
Using the virtualbox driver based on user configuration
Reconfiguring existing host ...
Starting existing virtualbox VM for "default" ...
Preparing Kubernetes v1.17.3 on Docker 19.03.6 ...
Launching Kubernetes ...
Enabling addons: dashboard, default-storageclass, storage-provisioner
Done! kubectl is now configured to use "default"
```

配置docker CLI来使用minikube docker主机：

```
eval $(minikube docker-env)
```

任何用docker执行的操作，如docker build和docker run，都会在minikube集群内进行。

附录B　Keycloak

Keycloak是一个开源的身份和访问管理系统。在生产中配置和部署Keycloak不在本书的范围之内。下面的例子提供了一个包含所有用户、角色、配置等内容的领域（realm）文件，需要将其导入正在运行的Keycloak服务器中。

为了简化Keycloak的安装，使用了Keycloak Docker容器：

```
docker run --name keycloak -e KEYCLOAK_USER=admin -e KEYCLOAK_PASSWORD=admin \
    -p 8180:8080 jboss/keycloak:8.0.1
```

打开浏览器，输入以下网址：http://localhost:8180。

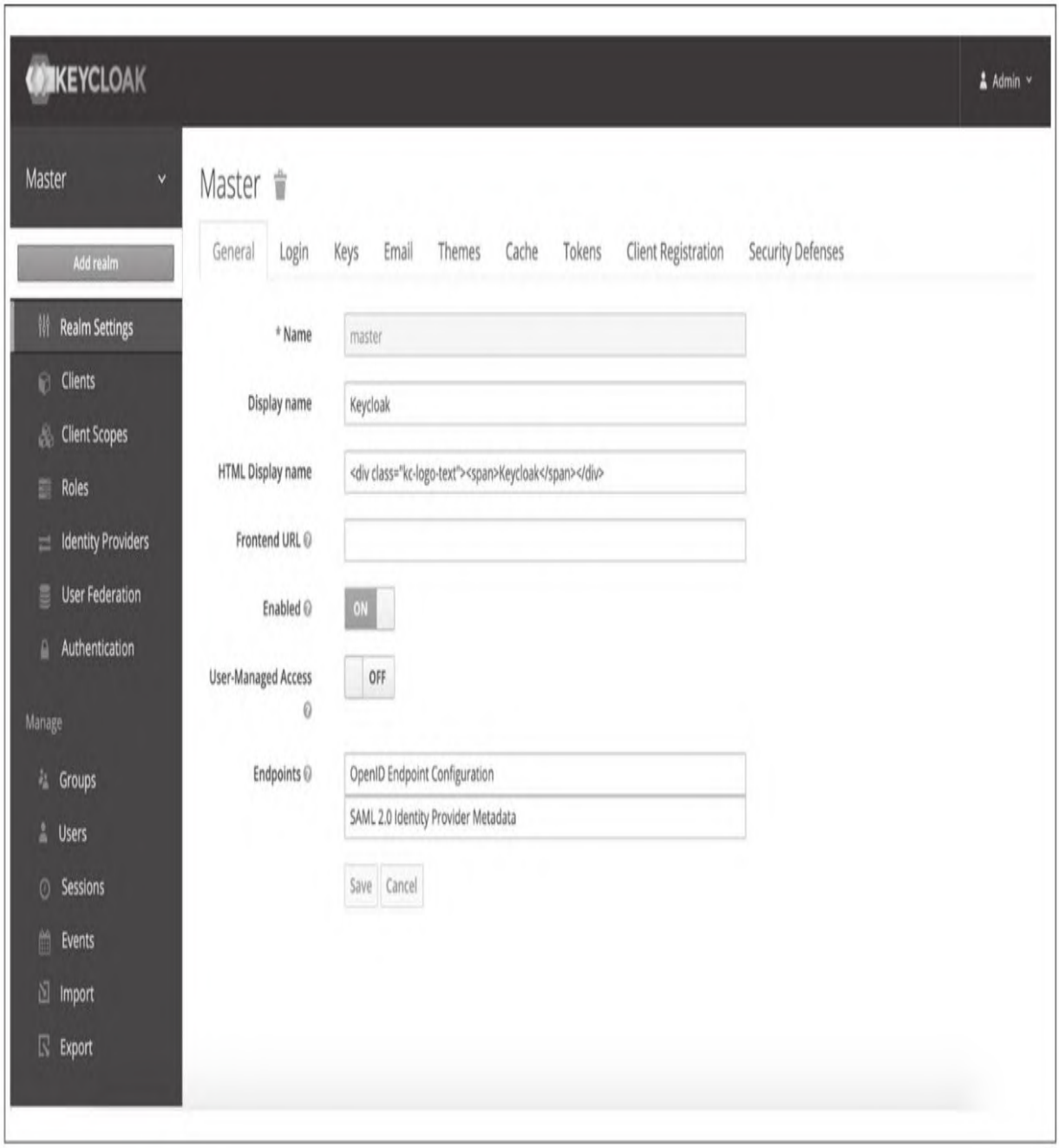
单击Administration Console，如图B-1所示。

图B-1：Keycloak主页

接下来，你会看到一个类似于图B-2中的登录选项。使用admin作为用户名和密码来登录。

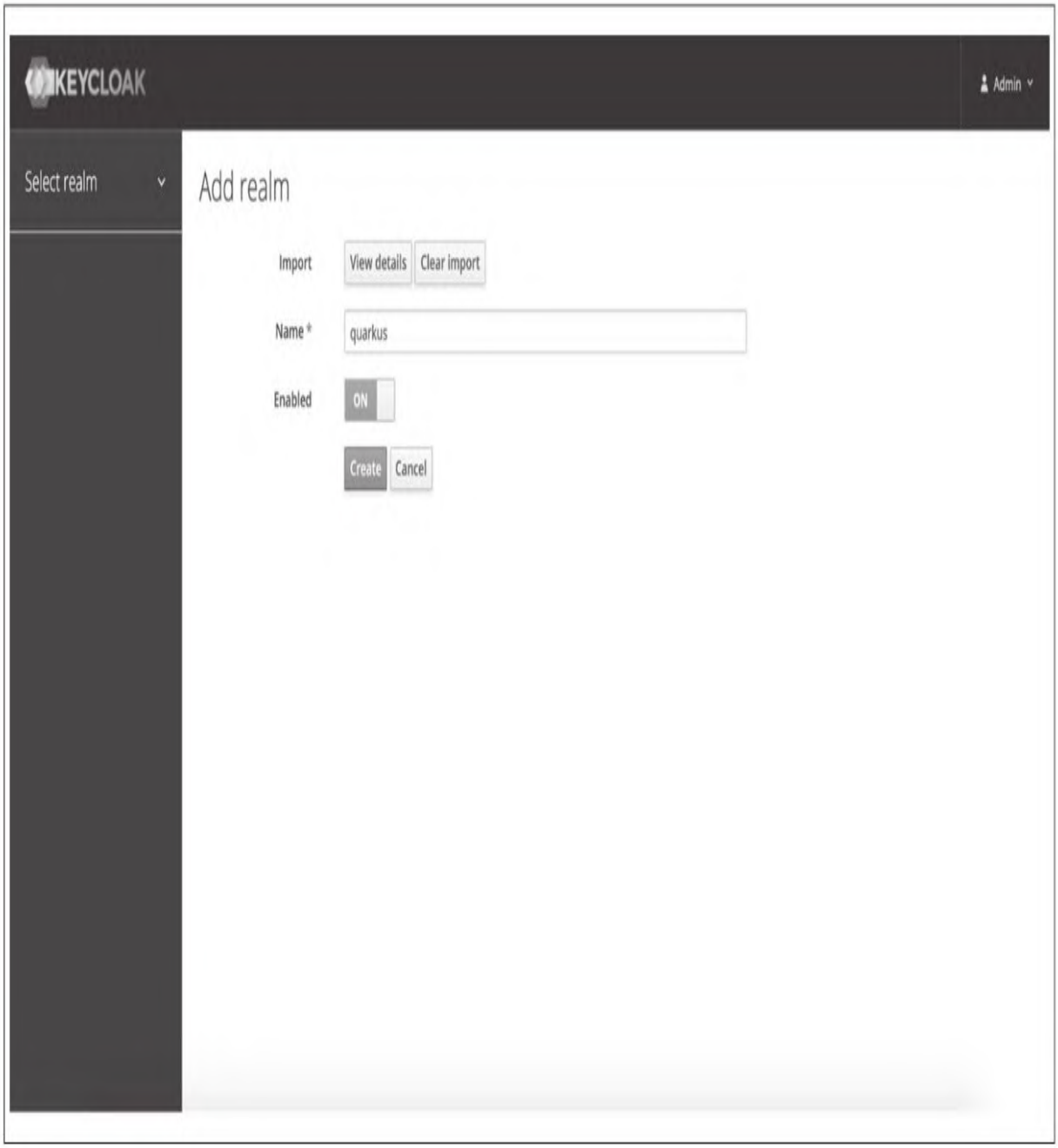
图B-2：Keycloak登录页面

在主页面中，切换主按钮，显示出Add realm按钮，然后单击该按钮，结果如图B-3所示。



图B-3：Keycloak Add realm按钮

最后，你应该看到一个如图B-4所示的屏幕。你将需要导入一个Keycloak文件。我们使用的文件在本书的代码中，网址是https://oreil.ly/quarkus-cookbook-code。


KEYCLOAK
Admin
Select realm
Add realm
Import
View details
Clear import
Name *
quarkus
Enabled
ON
Create
Cancel

图B-4：导入领域的结果

附录C　Knative

在第10章中，你需要访问一个Kubernetes集群——这可以是minikube或任何其他类型的集群。但你还需要安装Knative Serving来运行Knative。在本书中，Kourier被用作Knative的Ingress。

在本书中使用了minikube 1.7.3、Kubernetes 1.17.3、Knative 0.13.0和Kourier 0.3.12。

要安装Knative Serving，你需要运行以下命令：

```
kubectl apply -f \
 https://github.com/knative/serving/releases/download/v0.13.0/serving-core.yaml
kubectl apply -f \
 https://raw.githubusercontent.com/3scale/kourier/v0.3.12/deploy/\
 kourier-knative.yaml
```

配置Knative Serving以使用正确的ingress.class：

```
kubectl patch configmap/config-network \
  -n knative-serving \
  --type merge \
  -p '{"data":{"clusteringress.class":"kourier.ingress.networking.knative.dev",
               "ingress.class":"kourier.ingress.networking.knative.dev"}}'
```

设置你所需的域名。在本例中，域名为127.0.0.1，因为它在minikube中运行：

```
kubectl patch configmap/config-domain \
  -n knative-serving \
  --type merge \
  -p '{"data":{"127.0.0.1.nip.io":""}}'
```

现在，你可以开始部署Knative服务了。

关于作者

Alex Soto Bueno是Red Hat的Java Champion和开发者体验总监。他第一次发现Java世界的乐趣是在他开始用Java 1.2编码的时候，当时swing.jar还是一个外部库。Alex开始用ZX Spectrum编程，并拥有几台不同的计算机，如80286（他很感谢他的父母Ramon和Mili买下了它）。他是一个国际演讲者，在西班牙巴塞罗那拉曼·鲁尔大学喇沙校区任教。你甚至可以在电台上收听他的节目。

Jason Porter对软件的兴趣始于20世纪90年代初，当时他的家人给他买了人生中的第一台计算机。这一切都始于QBasic的Gorillas和Nibbles。在了解了这些游戏的源代码后，他被深深地吸引了，并有了人生规划。学完BASIC不久，他开始学习Java，并在高中和大学阶段大大提高了自己的Java和C/C++水平。在互联网泡沫破灭后，Jason开始工作并最终进入Red Hat，这是他梦寐以求的公司。他参与了许多软件项目（大部分是基于Java的），也做了多年的网络工作。他非常乐于帮助别人学习和提高工作效率。你可以在全球各地的各种会议上看到他的身影。

关于封面

本书封面上的动物是一只紫喉加勒比蜂鸟（学名Eulampis jugularis）。紫喉加勒比蜂鸟因其独特的颜色而得名，广泛栖息于委内瑞拉沿海的小安的列斯群岛的大部分岛屿。

紫喉加勒比蜂鸟是一种大型蜂鸟，有闪亮的绿色翅膀、黑色的身体和紫色喉咙。它还有一个明显下弯的喙，加上它的紫色喉咙，使其很容易与许多近亲区分。雌性紫喉加勒比蜂鸟的喙比雄性的喙更长、更弯。

它们主要分布在热带森林深处，偶尔也会出现在森林边缘。当它们在阴暗的森林栖息地时，往往身体的大部分呈黑色，仅在翅膀处保持一些明亮的颜色。这种蜂鸟以不同的方式发出尖锐、高亢的叫声。它们还与两种开花的Heliconia植物有着明显的关系，这被认为是共同进化的证据。

紫喉加勒比蜂鸟的保护状态为Least Concern，但O'Reilly系列丛书封面上的许多动物都濒临灭绝，它们对世界都很重要。

封面插图由Karen Montgomery根据*Encyclopedie D'Histoire Naturelle*中的黑白版画制作而成。